

Bonmots MAN Pages

Help with Small Telescope Spectroscopy

Wayne Green

Wednesday 26th August, 2026



Wordsalad.com

Help for the weary.

IRAF HELP Pages for Desk Reference

IRAF
Edited by Wayne Green

Wednesday 26th August, 2026

Abstract

There are more than 1,000 help pages. Here is a selection of the pages used with small telescope spectroscopy.

EMERGING DOCUMENT

Contents	17 imcopy	209
1 Overview	1 18 instruments	211
2 airmass	2 19 lcalib	215
3 apall	3 20 mk1dspec	217
4 apdefault	12 21 nhedit	223
5 apfind	14 22 observatory	230
6 aprecenter	17 23 print	236
7 apresize	20 24 response	237
8 apsum	24 25 sarith	240
9 aptrace	30 26 sensfun	248
10 calibrate	35 27 setairmass	255
11 combined	38 28 specfocus	259
12 darkcombine	188 29 specwcs	265
13 dispcor	190 30 splot	274
14 help	197 31 standard	290
15 identify	205 32 subsets	298
16 imarith	206 33 zerocombine	300

1 Overview

The command `help <command> dev=ps > command.pdf` will produce a postscript file with the help image from IRAF's help system.

The ghostscript package's `ps2pdf command.ps` will produce a `command.pdf`.

Load the ghostscript package, and add this function to a bash aliases file. This will make a complete package of all the pdfs specified, in order, on the commandline.

```
function pdfmerge { gs -dNOPAUSE -dBATCH -sDEVICE=pdfwrite -sOutputFile=combined.pdf $*;  
}
```

NAME

airmass -- compute the airmass at a given elevation above horizon

USAGE

airmass elevation

PARAMETERS**elevation**

Elevation above horizon in either degrees or radians.

scale = 750.0

Scale factor of the Earth's atmosphere.

radians= no

Input elevation in radians instead of degrees.

airmass

On output, contains the computed airmass.

EXAMPLE

Compute the airmass at an elevation of 30 degrees above the horizon

```
cl> airmass 30
airmass 1.996 at an elevation of 30. degrees (0.5236 radians)
above horizon
```

NAME

apall -- Extract one dimensional sums across the apertures

USAGE

apall input

PARAMETERS**input**

List of input images.

output = ""

List of output root names for extracted spectra. If the null string is given or the end of the output list is reached before the end of the input list then the input image name is used as the output root name. This will not conflict with the input image since an aperture number extension is added for onedspec format, the extension ".ms" for multispec format, or the extension ".ec" for echelle format.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a for example, "1, 3-5, 9-12x2".

format = "multispec" (onedspec|multispec|echelle|strip)

Format for output extracted spectra. "Onedspec" format extracts each aperture to a separate image while "multispec" and "echelle" extract multiple apertures for the same image to a single output image. The "multispec" and "echelle" format selections differ only in the extension added. The "strip" format produces a separate 2D image in which each column or line along the dispersion axis is shifted to exactly align the aperture based on the trace information.

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image. Input images without/with a database entry are skipped silently.

profiles = ""

List of profile images for variance weighting or cleaning. If variance weighting or cleaning a profile of each aperture is computed from the input image unless a profile image is specified, in which case the profile is computed from the profile image. The profile image must have the same dimensions and dispersion and it is assumed that the spectra have the same position and profile shape as in the object spectra. Use of a profile image is generally not required even for faint input spectra but the option is available for those who wish to use it.

PROCESSING PARAMETERS**interactive= yes**

Run this task interactively? If the task is not run interactively then all user queries are suppressed

and interactive aperture editing, trace fitting, and extraction review are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= yes

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a parameter must also be yes.

extract= yes

Extract the one dimensional aperture sums?

extras= yes

Extract the raw spectrum (if variance weighting is used), the sky spectrum (if background subtraction is used), and sigma spectrum (if variance weighting is used)? This information is extracted to the third dimension of the output image.

review = yes

Review the extracted spectra? The *interactive* parameter must also be yes.

line = INDEF, nsum = 10

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum selects a sum of lines and a negative selects a median of lines.

DEFAULT APERTURE PARAMETERS

lower = -5., upper= 5.

Default lower and upper aperture limits relative to the aperture center. These limits are used for apertures found with **apfind** and when defining the first aperture in **apedit**.

apidtable = ""

Aperture identification table. This may be either a image. A text file consisting of lines with an aperture number, beam number, and aperture title or identification. An image will contain the keywords SLFIBnnn with string value consisting of aperture number, beam number, optional right ascension and declination, and aperture title. This information is used to assign aperture information automatically in **apfind** and **apedit**.

DEFAULT BACKGROUND PARAMETERS

b_function = "chebyshev"

Default background fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

b_order= 1

Default background function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

b_sample = "-10:-6,6:10"

Default background sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points. Note that the background coordinates are relative to the aperture center and not image pixel coordinates so the endpoints need not be integer.

b_naverage= -3

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the fit.

b_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant fitting points and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

b_low_reject= 3., b_high_reject= 3.

Default background lower and upper rejection sigmas. If greater than zero points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

b_grow= 0.

Default reject growing radius. Points within a parameter of any rejected point are also rejected.

APERTURE CENTERING PARAMETERS

width= 5.

Width of spectrum profiles. This parameter is used for the profile centering algorithm in this and other tasks.

radius= 10.

The profile centering error radius for the centering algorithm.

threshold= 0.

Centering threshold for the centering algorithm. The range of pixel intensities near the initial centering position must exceed this threshold.

AUTOMATIC FINDING AND ORDERING PARAMETERS

nfind

Maximum number of apertures to be defined. This is a so the user is queried for a value except when given explicitly on the command line.

minsep= 5.

Minimum separation between spectra. Weaker spectra or noise within this distance of a stronger spectrum are rejected.

maxsep = 1000.

Maximum separation between adjacent spectra. This parameter is used to identify missing spectra in uniformly spaced spectra produced by fiber spectrographs. If two adjacent spectra exceed this separation then it is assumed that a spectrum is missing and the aperture identification assignments will be adjusted accordingly.

order = "increasing"

When assigning aperture identifications order the spectra "increasing" or "decreasing" with increasing pixel position (left-to-right or right-to-left in a cross-section plot of the image).

RECENTERING PARAMETERS

aprecenter = ""

List of apertures to be used in shift calculation.

npeaks = INDEF

Select the specified number of apertures with the highest peak values to be recentered. If the number is INDEF all apertures will be selected. If the value is less than 1 then the value is interpreted as a fraction of total number of apertures.

shift = yes

Use the average shift from recentering the apertures selected by the *aprecenter* parameter to apply to the apertures selected by the *apertures* parameter. The recentering is then applied to all apertures.

RESIZING PARAMETERS

llimit = INDEF, ulimit = INDEF

Lower and upper aperture size limits. If the parameter *ylevel* is INDEF then these limits are assigned to all apertures. Otherwise these parameters are used as limits to the resizing operation. A value of INDEF places the aperture limits at the image edge (for the dispersion line used).

ylevel = 0.1

Data level at which to set aperture limits. If it is INDEF then the aperture limits are set at the values given by the parameters *llimit* and *ulimit*. If it is not INDEF then it is a fraction of the peak or an actual data level depending on the parameter *peak*. It may be relative to a local background or to zero depending on the parameter *bkg*.

peak = yes

Is the data level specified by *ylevel* a

bkg = yes

Subtract a simple background when interpreting the **ylevel** parameter. The background is a slope connecting the first inflection points away from the aperture center.

r_grow = 0.

Change the lower and upper aperture limits by this fractional amount. The factor is multiplied by each limit and the result added to limit.

avglimits = no

Apply the average lower and upper aperture limits to all apertures.

TRACING PARAMETERS

t_nsum= 10

Number of dispersion lines to be summed at each step along the dispersion.

t_step= 10

Step along the dispersion axis between determination of the spectrum positions.

t_nlost= 3

Number of consecutive steps in which the profile is lost before quitting the tracing in one direction. To force tracing to continue through regions of very low signal this parameter can be made large. Note, however, that noise may drag the trace away before it recovers.

t_function="legendre"

Default trace fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

t_order= 2

Default trace function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

t_sample="*"

Default fitting sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points.

t_naverage= 1

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the

t_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant traced positions and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

t_low_reject= 3., t_high_reject= 3.

Default lower and upper rejection sigma. If greater than zero traced points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

t_grow= 0.

Default reject growing radius. Traced points within a parameter of any rejected point are also rejected.

EXTRACTION PARAMETERS

background="none" (none|average|median|minimum|fit)

Type of background subtraction. The choices are "none" for no background subtraction, "average" to average the background within the background regions, "median" to use the median in the background regions, "minimum" to use the minimum in the background regions, or "fit" to fit across the dispersion using the background within the background regions. Note that the "average" option does not do any medianing or bad pixel checking, something which is recommended. The fitting option is slower than the other options and requires additional fitting parameter.

skybox = 1

Box car smoothing length for sky background when using background subtraction. Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectral features. This is most appropriate when the sky regions are limited due to a small slit length.

weights = "none" (none|variance)

Type of extraction weighting. Note that if the *clean* parameter is set then the weights used are "variance" regardless of the weights specified by this parameter. The choices are:

"none"

The pixels are summed without weights except for partial pixels at the ends.

"variance"

The extraction is weighted by the variance based on the data values and a poisson/ccd model using the *gain* and *readnoise* parameters.

pfit = "fit1d" (fit1d|fit2d)

Profile fitting algorithm to use with variance weighting or cleaning. When determining a profile the two dimensional spectrum is divided by an estimate of the one dimensional spectrum to form a dimensional spectrum profile. This profile is then smoothed by fitting one dimensional functions, "fit1d", along the lines or columns most closely corresponding to the dispersion axis or a special two dimensional function, "fit2d", described by Marsh (see **aprofile**).

clean = no

Detect and replace deviant pixels?

saturation= INDEF

Saturation or nonlinearity level in data units. During variance weighted extractions wavelength points having any pixels above this value are excluded from the profile determination and the sigma spectrum extraction output, if selected by the *extras* parameter, flags wavelengths with saturated pixels with a negative sigma.

readnoise= 0.

Read out noise in photons. This parameter defines the minimum noise sigma. It is defined in terms of photons (or electrons) and scales to the data values through the gain parameter. A image header keyword (case insensitive) may be specified to get the value from the image.

gain = 1

Detector gain or conversion factor between photons/electrons and data values. It is specified as the number of photons per data value. A image header keyword (case insensitive) may be specified to get the value from the image.

lsigma = 4., usigma = 4.

Lower and upper rejection thresholds, given as a estimated sigma of a pixel, for cleaning.

nsubaps= 1

During extraction it is possible to equally divide the apertures into this number of subapertures. For multispec format all subapertures will be in the same file with aperture numbers of $1000 * (\text{subap} - 1) + \text{ap}$ where subap is the subaperture (1 to nsubaps) and ap is the main aperture number. For echelle format there will be a separate echelle format image containing the same subaperture from each order. The name will have the subaperture number appended. For onedspec

format each subaperture will be in a separate file with extensions and aperture numbers as in the multispec format.

ADDITIONAL PARAMETERS

Dispersion axis and I/O parameters are taken from the package parameters.

DESCRIPTION

This task provides functions for defining, modifying, tracing, and extracting apertures from two dimensional spectra. The functions desired are selected using switch parameters. When the task is run interactively queries are made at each step allowing additional control of the operations performed on each input image.

The functions, in the order in which they are generally performed, are summarized below.

- o Automatically find a specified number of spectra and assign default apertures. Apertures may also be inherited from another image or defined using an interactive graphical interface called the *aperture editor*.
- o Recenter selected reference apertures on the image spectrum profiles.
- o Resize the selected reference apertures based on spectrum profile width.
- o Interactively define or adjust aperture definitions using a interface called the *aperture editor*. All function may also be performed from this editor and, so, provides an alternative method of processing and extracting spectra.
- o Trace the positions of the selected spectra profiles from a or column to other image lines or columns and fit a smooth function. The trace function is used to shift the center of the apertures at each dispersion point in the image.
- o Extract the flux in the selected apertures into one dimensional spectra in various formats. This includes possible background subtraction, variance weighting, and bad pixel rejection.

Each of these functions has different options and parameters. In addition to selecting any of these functions in this task, they may also be selected using the aperture editor and as individual commands (which themselves allow selection of other functions). When broken down into individual tasks the parameters are also sorted by their function though there are then some mutual parameter interdependencies. This functional decomposition is what was available prior to the addition of the **apall** task. It is recommended that this task be used because it collects all the parameters in one place eliminating confusion over where a particular parameter is defined. However, documenting the various functions is better organized in terms of the separate descriptions given for each of the functions; namely under the help topics **apdefault**, **apfind**, **aprecenter**, **apresize**, **apedit**, **aptrace**, and **apsum**.

EXAMPLES

1. This example may be executed if desired. First we create an artificial spectrum with four spectra and a background. After it is created you can display or plot it. Next we define the dispersion axis and set the verbose flag to better illustrate what is happening. The task APALL is run with the default parameters except for background fitting and subtracting added. The text beginning with # are comments of things to try and do.

```
ap> artdata
ar> unlearn artdata
ar> mk1dspec apdemo1d nl=50
ar> mk2dspec apdemo2d model=STDIN
apdemo1d 1. gauss 3 0 20 .01
apdemo1d .8 gauss 3 0 40 .01
```

```

apdemo1d .6 gauss 3 0 60 .01
apdemo1d .4 gauss 3 0 80 .01
[EOF=Control D or Control Z]
ar> mknoise apdemo2d background=100. rdnoise=3. poisson+
ar> bye
# Display or plot the spectrum
ap> dispaxis=2; verbose=yes
ap> unlearn apall
ap> apall apdemo2d back=fit
Searching aperture database ...
Find apertures for apdemo2d? (yes):
Finding apertures ...
Number of apertures to be found automatically (1): 4
Jul 31 16:55: FIND - 4 apertures found for apdemo2d.
Resize apertures for apdemo2d? (yes):
Resizing apertures ...
Jul 31 16:55: RESIZE - 4 apertures resized for apdemo2d.
Edit apertures for apdemo2d? (yes):
# Get a list of commands with '?'
# See all the parameters settings with :par
# Try deleting and marking a spectrum with 'd' and 'm'
# Look at the background fitting parameters with 'b' (exit with 'q')
# Exit with 'q'
Trace apertures for apdemo2d? (yes):
Fit traced positions for apdemo2d interactively? (yes):
Tracing apertures ...
Fit curve to aperture 1 of apdemo2d interactively (yes):
# You can use ICFIT commands to adjust the fit.
Fit curve to aperture 2 of apdemo2d interactively (yes): n
Fit curve to aperture 3 of apdemo2d interactively (no):
Fit curve to aperture 4 of apdemo2d interactively (no): y
Jul 31 16:56: TRACE - 4 apertures traced in apdemo2d.
Write apertures for apdemo2d to apdemo2d (yes):
Jul 31 16:56: DATABASE - 4 apertures for apdemo2d written to database.
Extract aperture spectra for apdemo2d? (yes):
Review extracted spectra from apdemo2d? (yes):
Extracting apertures ...
Review extracted spectrum for aperture 1 from apdemo2d? (yes):
# Type 'q' to quit
Jul 31 16:56: EXTRACT - Aperture 1 from apdemo2d --> apdemo2d.ms
Review extracted spectrum for aperture 2 from apdemo2d? (yes): N
Jul 31 16:56: EXTRACT - Aperture 2 from apdemo2d --> apdemo2d.ms
Jul 31 16:56: EXTRACT - Aperture 3 from apdemo2d --> apdemo2d.ms
Jul 31 16:57: EXTRACT - Aperture 4 from apdemo2d --> apdemo2d.ms

```

2. To extract a series of similar spectra noninteractively using a reference for the aperture definitions, then recentering and resizing but not retracing:

```
ap> apall fib*.imh ref=flat inter- trace-
```

Note that the interactive flag automatically turns off the edit, fittrace, and review options and the reference image eliminates the find (find only occurs if there are no initial apertures).

REVISIONS

APALL V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

The "nsubaps" parameter now allows onedspec and echelle output formats. The echelle format is appropriate for treating each subaperture as a full echelle extraction.

APALL V2.10.3

The dispersion axis parameter was moved to purely a

As a final step when computing a weighted/cleaned spectrum the total fluxes from the weighted spectrum and the simple unweighted spectrum (excluding any deviant and saturated pixels) are computed and a "bias" factor of the ratio of the two fluxes is multiplied into the weighted spectrum and the sigma estimate. This makes the total fluxes the same. In this version the bias factor is recorded in the logfile if one is kept. Also a check is made for unusual bias factors. If the two fluxes disagree by more than a factor of two a warning is given on the standard output and the logfile with the individual total fluxes as well as the bias factor. If the bias factor is negative a warning is also given and no bias factor is applied. In the previous version a negative (inverted) spectrum would result.

SEE ALSO

apdefault, apfind, aprecenter, apresize, apedit, aptrace, apsum

NAME

apdefault -- Set default aperture parameters for the package

USAGE

apdefault

PARAMETERS

lower = -5., upper= 5.

Default lower and upper aperture limits relative to the aperture center. These limits are used for apertures found with **apfind** and when defining the first aperture in **apedit**.

apidtable = ""

Aperture identification table. This may be either a image. A text file consisting of lines with an aperture number, beam number, and aperture title or identification. An image will contain the keywords SLFIBnnn with string value consisting of aperture number, beam number, optional right ascension and declination, and aperture title. This information is used to assign aperture information automatically in **apfind** and **apedit**.

Default Background Subtraction Parameters

b_function = "chebyshev"

Default background fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

b_order= 1

Default background function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

b_sample = "-10:-6,6:10"

Default background sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points. Note that the background coordinates are relative to the aperture center and not image pixel coordinates so the endpoints need not be integer.

b_naverage= -3

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the fit.

b_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant fitting points and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

b_low_reject= 3., b_high_reject= 3.

Default background lower and upper rejection sigmas. If greater than zero points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

b_grow= 0.

Default reject growing radius. Points within a parameter of any rejected point are also rejected.

DESCRIPTION

This task sets the values of the default aperture parameters for the tasks **apedit** and **apfind** which define new apertures. For a description of the components of an aperture see the paper **The APEXTRACT Package**. In **apedit** the default aperture limits and background parameters are only used if there are no other apertures defined. The aperture identification table is used when reordering the apertures with the 'o' key. When run the parameters are displayed and modified using the **eparam** task.

The aperture limits and background fitting sample regions are defined relative to the center of the aperture. The background fitting parameters are those used by the ICFIT package. They may be modified interactively with the 'b' key in the task **apedit**. For more on background fitting and subtracting see **apbackground**.

EXAMPLES

To review and modify the default aperture parameters:

```
cl> apdefault
```

REVISIONS

APDEFAULTV2.11

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

SEE ALSO apbackground, apedit, apfind, icfit

NAME

apfind -- Find spectra and define apertures automatically

USAGE

apfind input

PARAMETERS**input**

List of input images in which spectra are to be identified and apertures defined automatically.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `for` example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database and the parameter *nfind* must be greater than zero.

recenter= no

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in finding the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value sums lines and a negative value medians lines.

nfind = 1

Maximum number of apertures to be defined. This is a so the user is queried for a value except when given explicitly on the command line.

minsep = 5.

Minimum separation between spectra. Weaker spectra or noise within this distance of a stronger spectrum are rejected.

maxsep = 1000.

Maximum separation between adjacent spectra. This parameter is used to identify missing spectra in uniformly spaced spectra produced by fiber spectrographs. If two adjacent spectra exceed this separation then it is assumed that a spectrum is missing and the aperture identification assignments will be adjusted accordingly.

order = "increasing"

When assigning aperture identifications order the spectra "increasing" or "decreasing" with increasing pixel position (left-to-right or right-to-left in a cross-section plot of the image).

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list spectra are identified and default apertures defined. The automatic aperture finding is performed only if 1) there are no apertures defined for the reference image, 2) there are no apertures defined for the input image, 3) the parameter *find* is yes, and 4) the parameter *nfind* is greater than zero.

The automatic finding algorithm uses the following steps. First, all local maxima are found. The maxima are sorted by peak value and the weaker of the peaks separated by less than the value given by the parameter *minsep* are rejected. Finally, at most the *nfind* strongest peaks are kept. **Nfind** is a query parameter, so if it is not specified explicitly on the command line, the desired number of spectra to be found is requested. After the peaks have been found the **center1d** algorithm is used to refine the centers of the profiles. Apertures having the default parameters set with the task **apdefault** are defined at each center. This algorithm is also available with the 'f' key in the task **apedit** with the change that existing apertures are kept and count toward the maximum number specified by **nfind**.

The automatic assignment of aperture numbers, beam numbers, and titles has several options. The simplest is when no aperture identification table, parameter *apidtable*, is specified and the maximum separation parameter, *maxsep*, is very large. In this case the aperture and beam numbers are sequential starting from one and numbered either from left-to-right or right-to-left depending on the *order* parameter. There are no aperture titles in this case. If two adjacent spectra are separated by more than the specified maximum then the aperture numbers jump by the integer part of the ratio of the separation to the specified maximum separation. This is used when the image is expected to have evenly spaced spectra, such as in multifiber spectrographs, in which some may be missing due to broken fibers. Finally, the aperture identification table (either a text file or an image having a set of SLFIBnnn keywords) may contain lines with aperture number, beam number, and (optional) title. The sequential numbers are then indices into this table. Note that the skipping of missing spectra and the ordering applies to entries in this table as well.

The ways in which the automatic method can fail for evenly spaced spectra with missing members are when the first spectrum is missing on the side from which the ordering begins and when the expected rather than the actual number of spectra is used. In the first case one can use the interactive 'o' key of the aperture editing facility to specify the identity of any aperture and then all other apertures will be appropriately reidentified. If more spectra are sought than actually exist then noise spikes may be mistakenly found. This problem can be eliminated by specifying the actual number of spectra or minimized by using the threshold centering parameter.

The *recenter* parameter allows recentering apertures if a reference image. Since the purpose of this task is to find new apertures it is usually the case that there are no reference images and recentering is not done. The default apertures are of fixed width. The *resize* parameter may be used to adjust the widths in a variety of ways. The aperture positions and any other parameters may also be edited with the aperture editing function if selected by the *apedit* parameter and the task is run interactively.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture finding algorithm may be selected from nearly every task in the package.

EXAMPLES

```
cl> apfind image nfind=10
```

REVISIONS

APFIND V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

SEE ALSO center1d, apdefault, aprecenter, apresize, apedit, apall

NAME

aprecenter -- Recenter apertures automatically

USAGE

aprecenter input

PARAMETERS**input**

List of input images in which apertures are to be recentered.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `*` for example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= yes

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in recentering the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value takes a sum and a negative values selects a median.

aprecenter =""

List of apertures to be used in shift calculation.

npeaks= INDEF

Select the specified number of apertures with the highest peak values to be recentered. If the number is INDEF all apertures will be selected. If the value is less than 1 then the value is interpreted as a fraction of total number of apertures.

shift = yes

Use the median shift from recentering the selected apertures to apply to all apertures. The recentering is then a constant shift for all apertures. The median is the average of the two central values for an even number of apertures.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, automatic aperture finding parameters are taken from **apfind**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the aperture center positions are redefined by centering at the specified dispersion line using the **center1d** algorithm with centering parameters from **apedit**. Normally this is done when inheriting apertures from an aperture reference image. The recentering does not change the "trace" of the aperture but simply adds a shift across the dispersion axis.

There are several recentering options. Each selected aperture may be recentered independently. However, if some or all of the spectra are relatively weak this may actually be worse than using the reference apertures defined by strong spectra or flat fields in the case of fibers or aperture masks. One may select a subset of apertures used in calculating shift. This is done with the *aprecenter* list of aperture numbers (see **ranges** for the syntax) and/or by selecting a specific number or fraction of the apertures with the strongest peak values. The list selection is done first and the strongest remaining apertures are used to satisfy the **npeaks** value. Though some or all of the apertures may be recentered independently the most common case of recentering reference apertures is to account for detector shifts. In this case one expects that any shift should be common to all apertures. The *shift* parameter allows using the new centers for all selected apertures to compute a median shift to be added to ALL apertures. Using a median shift for all apertures is the default.

The *find* parameter allows automatically finding apertures if they are defined for the image or by a reference image. Since the purpose of this task is to recenter reference apertures it is usually the case that reference images are used and apertures are not defined by this task. One case in which the apertures from the image itself might be recentered is if one wants to use a different dispersion line. The *resize* parameter may be used to adjust the widths in a variety of ways based on the spectra profiles specific to each image. The aperture positions and any other parameters may also be edited with the aperture editing function if selected by the *apedit* parameter and the task is run interactively. The recentering algorithm may be run from the aperture editor using the 'g' keystroke.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture recentering algorithm may be selected from nearly every task in the package.

EXAMPLES

```
cl> aprecenter newimage reference=flat
```

REVISIONS**APRECENTERV2.11**

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

center1d, ranges, apfind, apresize, apedit, apall

NAME

apresize -- Resize apertures automatically

USAGE

apresize input

PARAMETERS**input**

List of input images in which apertures are to be resized.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `for` example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in resizing the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value takes a sum and a negative value selects a median.

llimit = INDEF, ulimit = INDEF

Lower and upper aperture size limits. If the parameter *ylevel* is INDEF then these limits are assigned to all apertures. Otherwise these parameters are used as limits to the resizing operation. A value of INDEF places the aperture limits at the image edge (for the dispersion line used).

ylevel = 0.1

Data level at which to set aperture limits. If it is INDEF then the aperture limits are set at the values given by the parameters *llimit* and *ulimit*. If it is not INDEF then it is a fraction of the peak or an actual data level depending on the parameter *peak*. It may be relative to a local background or to zero depending on the parameter *bkg*.

peak = yes

Is the data level specified by *ylevel* a

bkg = yes

Subtract a simple background when interpreting the **ylevel** parameter. The background is a slope connecting the first minima away from the aperture center.

r_grow = 0.

Change the lower and upper aperture limits by this fractional amount. The factor is multiplied by each limit and the result added to limit.

avglimits = no

Apply the average lower and upper aperture limits to all apertures.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, automatic aperture finding parameters are taken from **apfind**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the aperture limits are redefined to be either specified values or by finding the points at which the spectrum profile, linearly interpolated, first crosses a specified value moving away from the aperture center at the specified dispersion line. In the latter case the limits may then be increased or decreased by a specified percentage, a maximum lower and upper limit, may be imposed, and the independent limits may be averaged and the single values applied to all the apertures.

The simplest resizing choice is to reset all the aperture limits to the values specified by *llimit* and *ulimit*. This option is selected if the parameter *ylevel* is INDEF.

There are several options for specifying a data level at which an aperture is sized. The most common method (the default) is to specify a fraction of the peak value since this is data independent and physically reasonable. This is done by setting the fraction with the parameter *ylevel* and the parameter *peak* to yes. If the *peak* parameter is no then the level is a data value.

The levels may be relative to zero, as might be used with fibers or high dispersion / high signal-to-noise data, or relative to a local linear background, as would be appropriate for slit data having a significant background. A background is found and used if the parameter *bkg* is set. The background determination is very simple. Starting at the peak two background points are found, one in each direction, which are inflection points; i.e. the first pixels which are less than their two neighbors. A linear slope is fit and

subtracted for the purposes of measuring the peak and setting the aperture limits. Note that if the slope is significant the actual limits may not correspond to the intercepts of a value.

Once aperture limits, a distance relative to the center, are determined they are increased or decreased by a percentage, expressed as a fraction, given by the parameter *r_grow*. To illustrate the operation, if xlow is the initial lower limit then the final lower limit will be:

$$\text{xlow final} = \text{xlow} * (1 + r_grow)$$

A value of zero leaves the aperture limits unchanged.

After the aperture limits are found, based on the above steps, a limit, given by the parameter *llimit*, is applied to the lower aperture points and, similarly, a fixed upper limit is applied to the upper aperture points. This feature protects against absurdly wide apertures.

Finally, if the parameter *avglimits* is set the individual aperture limits are averaged to form an average aperture. This average aperture is then assigned to all apertures. This option allows keeping common aperture sizes but allowing variation due to seeing changes.

The resizing algorithm is available in the interactive aperture editor. Here one may select individual apertures or all apertures using the 'a' switch. The resizing algorithm described above is selected using the 'z' key. A simple alternative is the 'y' key which resizes apertures to the y level marked by the cursor.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture resizing algorithm may be selected from nearly every task in the package with the *resize* parameter.

EXAMPLES

1. To resize all apertures to the range -4 to 4:

```
cl> apresize image llimit=-4 ulimit=4 ylevel=INDEF
```

2. To resize all aperture to a point which is 5% of the peak relative to a local background:

```
cl> apresize image ylevel=.05 peak+ bkg+
```

3. To resize all apertures to the point where the data exceeds 100 data units:

```
cl> apresize image ylevel=100 peak- bkg-
```

4. To resize all apertures to default values of the task except averaging all the results at the end:

```
cl> apresize image avg+
```

REVISIONS

APRESIZEV2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

center1d, ranges, apfind, aprecenter, apedit, apall

NAME

apsum -- Extract one dimensional sums across the apertures

USAGE

apsum input

PARAMETERS**input**

List of input images containing apertures to be extracted.

output = ""

List of output rootnames for the extracted spectra. If the null string is given or the end of the output list is reached before the end of the input list then the input image name is used as the output rootname. This will not conflict with the input image since an aperture number extension is added for onedspec format, the extension ".ms" for multispec format, or the extension ".ec" for echelle format.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a for example, "1,3-5,9-12x2".

format="multispec" (onedspec|multispec|echelle|strip)

Format for output extracted spectra. "Onedspec" format extracts each aperture to a separate image while "multispec" and "echelle" extract multiple apertures for the same image to a single output image. The "multispec" and "echelle" format selections differ only in the extension added. The "strip" format produces a separate 2D image in which each column or line along the dispersion axis is shifted to exactly align the aperture based on the trace information.

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

profiles = ""

List of profile images for variance weighting or cleaning. If variance weighting or cleaning a profile of each aperture is computed from the input image unless a profile image is specified, in which case the profile is computed from the profile image. The profile image must have the same dimensions and dispersion and it is assumed that the spectra have the same position and profile shape as in the object spectra. Use of a profile image is generally not required even for faint input spectra but the option is available for those who wish to use it.

interactive= yes

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing, trace fitting, and extraction review are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a parameter must also be yes.

extract= yes

Extract the one dimensional aperture sums?

extras= no

Extract the raw spectrum (if variance weighting is used), the sky spectrum (if background subtraction is used), and variance spectrum (if variance weighting is used)? This information is extracted to the third dimension of the output image.

review = yes

Review the extracted spectra? The *interactive* parameter must also be yes.

line = INDEF, nsum = 10

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. For tracing this is the starting line and the same number of lines are summed at each tracing point. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum takes a sum while a negative value selects a median except that tracing always uses a sum.

background = "none" (none|average|median|minimum|fit)

Type of background subtraction. The choices are "none" for no background subtraction, "average" to average the background within the background regions, "median" to use the median in the background regions, "minimum" to use the minimum in the background regions, or "fit" to fit across the dispersion using the background within the background regions. Note that the "average" option does not do any medianing or bad pixel checking, something which is recommended. The fitting option is slower than the other options and requires additional fitting parameter.

weights = "none"

Type of extraction weighting. Note that if the *clean* parameter is set then the weights used are "variance" regardless of the weights specified by this parameter. The choices are:

"none"

The pixels are summed without weights except for partial pixels at the ends.

"variance"

The extraction is weighted by the variance based on the data values and a poisson/ccd model using the *gain* and *readnoise* parameters.

pfit = "fit1d" (fit1d|fit2d)

Profile fitting algorithm to use with variance weighting or cleaning. When determining a profile the two dimensional spectrum is divided by an estimate of the one dimensional spectrum to form a dimensional spectrum profile. This profile is then smoothed by fitting one dimensional functions, "fit1d", along the lines or columns most closely corresponding to the dispersion axis or a special two dimensional function, "fit2d", described by Marsh (see **aprofile**).

clean = no

Detect and replace deviant pixels?

skybox = 1

Box car smoothing length for sky background when using background subtraction. Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectral features. This is most appropriate when the sky regions are limited due to a small slit length.

saturation= INDEF

Saturation or nonlinearity level in data units. During variance weighted extractions wavelength points having any pixels above this value are excluded from the profile determination and the sigma spectrum extraction output, if selected by the *extras* parameter, flags wavelengths with saturated pixels with a negative sigma.

readnoise= 0.

Read out noise in photons. This parameter defines the minimum noise sigma. It is defined in terms of photons (or electrons) and scales to the data values through the *gain* parameter. A image header keyword (case insensitive) may be specified to get the value from the image.

gain = 1

Detector gain or conversion factor between photons/electrons and data values. It is specified as the number of photons per data value. A image header keyword (case insensitive) may be specified to get the value from the image.

lsigma = 4., usigma = 4.

Lower and upper rejection thresholds, given as a estimated sigma of a pixel, for cleaning.

nsubaps= 1

During extraction it is possible to equally divide the apertures into this number of subapertures. For multispec format all subapertures will be in the same file with aperture numbers of $1000 * (\text{subap} - 1) + \text{ap}$ where subap is the subaperture (1 to nsubaps) and ap is the main aperture number. For echelle format there will be a separate echelle format image containing the same subaperture from each order. The name will have the subaperture number appended. For onedspec format each subaperture will be in a separate file with extensions and aperture numbers as in the multispec format.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters from **apdefault**, automatic aperture finding parameters from **apfind**, recentering parameters from **aprecenter**, resizing parameters from **apresize**, parameters used for centering and editing the apertures from **apedit**, and tracing parameters from **aptrace**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the two dimensional spectra are extracted to one dimensional spectra by summing the pixels across the dispersion axis at each wavelength along the dispersion axis within a set of defined apertures. The extraction apertures consist of an aperture number, a beam number, a title, a center, limits relative to the center, a curve describing shifts of the aperture center across the dispersion axis as a function of the wavelength, and parameters for background fitting and subtraction. See **apextract** for a detailed discussion of the aperture structures.

The extracted spectra are recorded in one, two, or three dimensional images depending on the *format* and *extras* parameters. The output image rootnames are specified by the *output* list. If list is empty or shorter than the input list the missing names are taken to be the same as the input image names. Because the rootnames have extensions added it is common to default to the input names in order to preserve a naming relation between the input two dimensional spectra and the extracted spectra.

When the parameter *extras*=no only the extracted spectra are output. If the format parameter *format*="onedspec" the output aperture extractions are one dimensional images with names formed from the output rootname and a numeric extension given by the aperture number; i.e. root.0001 for aperture 1. Note that there will be as many output images as there are apertures for each input image, all with the same output rootname but with different aperture extensions. The aperture beam number associated with each aperture is recorded in the output image under the keyword BEAM-NUM. The output image name format and the BEAM-NUM entry in the image are chosen to be compatible with the **onedspec** package.

If the format parameter is "echelle" or "multispec" the output aperture extractions are put into a two dimensional image with a name formed from the output rootname and the extension ".ech" or ".ms". Each line in the output image corresponds to one aperture. Thus in this format there is one output image for each input image. These are the preferred output formats for reasons of compactness and ease of handling. These formats are compatible with the **onedspec**, **echelle**, and **msred** packages. The relation between the line and the aperture numbers is given by the header parameter APNUMn where n the value is the aperture number and other numeric information.

If the *extras* parameter is set to yes then the above formats become three dimensional. Each plane in the third dimension contains associated information for the spectra in the first plane. If variance weighted extractions are done the unweighted spectra are recorded. If background subtraction is done the background spectra are recorded. If variance weighted extractions are done the sigma spectrum (the estimated sigma of each spectrum pixel based on the individual variances of the pixels summed) is recorded. The order of the additional information is as given above. For example, an unweighted extraction with background subtraction will have one additional plane containing the sky spectra while a variance weighted extraction with background subtractions will have the variance weighted spectra, the unweighted spectra, the background spectra, and the sigma spectra in consecutive planes.

Aperture definitions may be inherited from those of other images by specifying a reference image with the **references** parameter. Images in the reference list are matched with those in the input list in order. If the reference image list is shorter than the number of input images, the last reference image is used for all remaining input images. Thus, a single reference image may be given for all the input images or different reference images may be given for each input image. The special reference name "last" may be used to select the last set apertures used in any of the **apextract** tasks.

If an aperture reference image is not specified or no apertures are found for the specified reference image, previously defined apertures for the input image are sought in the aperture database. Note that reference apertures supersede apertures for the input image. If no apertures are defined they may be created automatically, the *find* option, or interactively in the aperture editor, if the *interactive* and *edit* options are set.

The functions performed by the task are selected by a parameters. The functions are an automatic spectrum finding and aperture defining algorithm (see **apfind**) which is ignored if apertures are already defined, automatic recentering and resizing algorithms (see **aprecenter** and **apresize**), an interactive aperture editing function (see **apedit**), a spectrum position tracing and trace function fit (see **aptrace**), and the main function of this task, one dimensional spectrum extraction.

Each function selection will produce a query for each input spectrum if the *interactive* parameter is set. The queries are answered by "yes", "no", "YES", or "NO", where the upper case responses suppress the query for following images. There are other queries associated with tracing and extracted spectrum review which first ask whether the operation is to be done interactively and, if yes, lead to queries for each aperture. The cursor keys available during spectrum review are minimal, only the CURSOR MODE keys for expanding and adjusting the graph are available and the quit key 'q'. If the *interactive* parameter is not set then aperture editing, interactive trace fitting, and spectrum review are ignored.

Background sky subtraction is done during the extraction based on background regions and parameters defined by the default parameters or changed during the interactive setting of the apertures. The background subtraction options are to do no background subtraction, subtract the average, median, or minimum of the pixels in the background regions, or to fit a function and subtract the function from under the extracted object pixels. The background regions are specified in pixels from the aperture center and follow changes in center of the spectrum along the dispersion. The syntax is colon separated ranges with multiple ranges separated by a comma or space. The background fitting uses the **icfit** routines which include medians, iterative rejection of deviant points, and a choice of function types and orders. Note that it is important to use a method which rejects cosmic rays such as using either medians over all the background regions (*background* = "median") or median samples during fitting (*b_naverage* < -1). The background subtraction algorithm and options are described in greater detail in **apsum** and **apbackground**.

Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectra features. This is most appropriate when the sky region is limited due to small slit length. The smoothing length is specified by the parameter *skybox*.

For a more extended discussion about the background determination see **apbackground**.

The aperture extractions consists of summing all the background subtracted pixel values at a given wavelength within the aperture limits. The aperture limits form a fixed width aperture but the center varies smoothly to follow changes in the position of the spectrum across the dispersion axis. At the ends of the aperture partial pixels are used.

The pixels in the sum may be weighted as specified by the *weights* parameter. If the *weights* parameter is "none" and the *clean* parameter is no then the simple sum of the pixels (with fractional endpoints) is extracted. If the *weights* parameter is "variance" or if the **clean** parameter is yes the pixels are weighted by their estimated variance derived from a noise model based on the *gain* and *readnoise* parameters and a smooth profile function. Normally the profile function is determined from the data being extracted. However, one may substitute a "profile" image as specified by the *profiles* parameter for computing the profile. This requires that the profile image have spectra of identical position and profile as the image being extracted. For example, this would likely be the case with fiber spectra and an off-telescope spectrograph and a strong flat field or object spectrum could be used for weak spectra. Note that experience has shown that even for very weak spectra there is little improvement with using a separate profile image but the user is free to experiment.

When the *clean* parameter is set pixels deviating by more than a specified number of sigma from the profile function are excluded from the variance weighted sum. Note that the *clean* parameter always selects variance weights. For a more complete discussion of the extraction sums, variance weighting, cleaning, the noise model, and profile function determination see **apvariance** and **aprofiles**.

EXAMPLES

1. To simply extract the spectra from a multislit observation:

```
cl> apsum multislit1
```

The positions of the slits are defined using either automatic finding or with the aperture editor. The positions of the slits are traced if necessary and then the apertures are extracted to the image "multislit1.ms". The steps of defining the slit positions and tracing can be done as part of this command or previously using the other tasks in the **apextract** package.

REVISIONS

APSUM V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The "nsubaps" parameter now allows onedspec and echelle output formats. The echelle format is appropriate for treating each subaperture as a full echelle extraction.

The dispersion axis parameter was moved to purely a

As a final step when computing a weighted/cleaned spectrum the total fluxes from the weighted spectrum and the simple unweighted spectrum (excluding any deviant and saturated pixels) are computed and a "bias" factor of the ratio of the two fluxes is multiplied into the weighted spectrum and the sigma estimate. This makes the total fluxes the same. In this version the bias factor is recorded in the logfile if one is kept. Also a check is made for unusual bias factors. If the two fluxes disagree by more than a factor of two a warning is given on the standard output and the logfile with the individual total fluxes as well as the bias factor. If the bias factor is negative a warning is also given and no bias factor is applied. In the previous version a negative (inverted) spectrum would result.

SEE ALSO

apbackground, apvariance, aprofile, apdefault, apfind, aprecenter, apresize, apedit, aptrace, apall

NAME

aptrace -- Trace spectra for aperture extraction

USAGE

aptrace images

PARAMETERS**input**

List of input images to be traced.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `for` example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= yes

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing and trace fitting are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a *parameter* must also be yes.

line = INDEF, nsum = 1

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. For tracing this is the starting line and the same number of lines are summed at each tracing point. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum selects the number of lines to sum while a negative value selects a median. Tracing always uses a sum.

step = 10

Step along the dispersion axis between determination of the spectrum positions.

nlost = 3

Number of consecutive steps in which the profile is lost before quitting the tracing in one direction. To force tracing to continue through regions of very low signal this parameter can be made large. Note, however, that noise may drag the trace away before it recovers.

The following parameters are the defaults used to fit the traced positions by a function of the dispersion line. These parameters are those used by the ICFIT package.

function = "legendre"

Default trace fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

order = 2

Default trace function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

sample = "*"

Default fitting sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points.

naverage = 1

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the

niterate = 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant traced positions and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

low_reject = 3., high_reject = 3.

Default lower and upper rejection sigma. If greater than zero traced points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

grow = 0.

Default reject growing radius. Traced points within a parameter of any rejected point are also rejected.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters from **apdefault**, automatic aperture finding parameters from **apfind**, recentering parameters from **aprecenter**, resizing parameters from **apresize**, and parameters used for centering and editing the apertures from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list the position of the spectrum within each aperture are determined at a number of points along the dispersion axis and a smooth function is fit to these positions. The fitted curve defines a shift to be added to the aperture center at each wavelength. Other options allow defining apertures using a reference image, defining apertures through an automatic finding algorithm (see **apfind**), automatically recentering apertures (see **aprecenter**), automatically resizing apertures (see **apresize**), and interactively editing the apertures prior to tracing (see **apedit**). Tracing is selected with the parameter *trace*. If the tracing is done interactively (the *interactive* parameter set to yes) then the user is queried whether or not to trace each image. The responses are "yes", "no", "YES", or "NO", where the upper case queries suppress this query for the following images.

The tracing begins with the specified dispersion line. A dispersion line is a line or column of the image perpendicular to the dispersion axis. The dispersion axis is defined in the image header or by the package parameter *dispaxis*. If the starting dispersion line is INDEF then the middle dispersion line of the image is used. The positions of the spectra are determined using the **center1d** algorithm and the centering parameters from the **apedit** task. (See help under **center1d** for a description of the one dimensional position measuring algorithm.) The positions are redetermined at other points along the dispersion axis by stepping from the starting line in steps specified by the user. A number of dispersion lines around each dispersion line to be measured may be summed to improve the position determinations, particularly for weak profiles. This number usually is set equal to the tracing step.

It is important to understand how to set the step size and the relationship between the step size and the centering error radius. Larger steps reduce the computational time, which is an important consideration. However, if the step is too large then the tracing may fail to follow the systematic changes in the positions of the spectrum. The centering error radius, *radius*, is used to limit the maximum position change between two successive steps. If the positions of a spectrum changes by more than the specified amount or the data contrast falls below the *threshold* parameter then the position is marked as lost.

The centering radius should be large enough to follow changes in the spectrum positions from point to point but small enough to detect an error in the tracing by a sudden abrupt change in position, such as caused by crowding with other spectra or by the disappearance of the spectrum. The *nlost* parameter determines how many consecutive steps the position may fail to be found before tracing in that direction is stopped. If parameter is small the trace will stop quickly upon loss of the profile while if it is very large it will continue to try and recover the profile.

The parameter *threshold* checks for the vanishing of a requiring a minimum range in the data used for centering. If the tracing fails when the spectra are strong and well defined the problem is usually that the step size is too large and/or the centering error radius is too small.

The traced positions of a spectrum include some measurement variation from point to point. Since the actual position of the spectrum in the image should be a smooth curve, a function of the dispersion line is to the measured points. The fitted function is stored as part of the aperture description. It is an offset to be added to the aperture's center as a function of the dispersion line. Even if the fitting is done interactively plots of the trace and the fit are recorded in the plot file or device specified by the parameter *plotfile*.

Fitting the traced spectrum positions with a performed interactively when parameters *fittrace* and

interactive are yes. This allows changing the default fitting parameters. The function fitting is done with the interactive curve fitting tools described under the help topic **icfit**. There are two levels of queries when fitting the spectrum positions interactively; prompts for each image and prompts for each aperture in an image. These prompts may be answered individually with the lower case responses "yes" or "no" or answered for all further prompts with the responses "YES" or "NO". Responding with "yes" or "YES" to image prompt allows interactive fitting of the traced positions for the spectra. Prompts are then given for each aperture in the image. When an spectrum is not fit interactively the last set of fitting parameters are used (initially the default function and order given by the task parameters). Note that answering "YES" or "NO" to a aperture prompt applies to all further aperture in the current image only. Responding with "no" or "NO" to the image prompt fits the spectrum positions for all apertures in all images with the last set of fitting parameters.

The tracing may also be done from the interactive aperture editor with the 't' key. The aperture tracing algorithm may be selected from many of the tasks in the package with the *trace* parameter.

APTRACE DATABASE COEFFICIENTS

The path of an aperture is described by a function that gives an additive offset relative to the aperture center as stored under the database keyword center. The function is saved in the database as a coefficients. The section containing the coefficients starts with the keyword "curve" and the number of coefficients.

The first four coefficients define the type of function, the order or number of spline pieces, and the range of the independent variable (the line or column coordinate along the dispersion). The first coefficient is the function type code with values:

Code	Type
1	Chebyshev polynomial
2	Legendre polynomial
3	Cubic spline
4	Linear spline

The second coefficient is the order (actually the number of terms) of the polynomial or the number of pieces in the spline.

The next two coefficients are the range of the independent variable over which the function is defined. These values are used to normalize the input variable to the range -1 to 1 in the polynomial functions. If independent variable is x and the normalized variable is n, then

$$n = (2 * x - (xmax + xmin)) / (xmax - xmin)$$

where xmin and xmax are the two coefficients.

The spline functions divide the range into the specified number of pieces. A spline coordinate s and the nearest integer below s, denoted as j, are defined by

$$s = (x - xmin) / (xmax - xmin) * npieces$$

$$j = \text{integer part of } s$$

where npieces are the number of pieces.

The remaining coefficients are those for the appropriate function. The number of coefficients is either the same as the function order for the polynomials, npieces+1 for the linear spline, or npieces + 2 for the cubic spline.

1. Chebyshev Polynomial

The polynomial can be expressed as the sum

$$y = \text{sum from } i=1 \text{ to order } \{c_i * z_i\}$$

where the c_i are the coefficients and the z_i are defined interactively as:

$$\begin{aligned} z_1 &= 1 \\ z_2 &= n \\ z_i &= 2 * n * z_{i-1} - z_{i-2} \end{aligned}$$

2. Legendre Polynomial

The polynomial can be expressed as the sum

$$y = \text{sum from } i=1 \text{ to order } \{c_i * z_i\}$$

where the c_i are the coefficients and the z_i are defined interactively as:

$$\begin{aligned} z_1 &= 1 \\ z_2 &= n \\ z_i &= ((2*i-3) * n * z_{i-1} - (i-2) * z_{i-2}) / (i - 1) \end{aligned}$$

3. Linear Spline

The linear spline is evaluated as

$$y = c_j * a + c_{j+1} * b$$

where j is as defined earlier and a and b are fractional difference between s and the nearest integers above and below

$$\begin{aligned} a &= (j + 1) - s \\ b &= s - j \end{aligned}$$

4. Cubic Spline

The cubic spline is evaluated as

$$y = \text{sum from } i=0 \text{ to } 3 \{c_{i+j} * z_i\}$$

where j is as defined earlier. The term z_i are computed from a and b , as defined earlier, as follows

$$\begin{aligned} z_0 &= a^3 \\ z_1 &= 1 + 3 * a * (1 + a * b) \\ z_2 &= 1 + 3 * b * (1 + a * b) \\ z_3 &= b^3 \end{aligned}$$

EXAMPLES

REVISIONS

APTRACEV2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

apdefault, apfind, aprecenter, apresize, apedit, apall, center1d, icfit, gtools

NAME

calibrate -- Apply extinction corrections and flux calibrations

USAGE

calibrate input output [records]

PARAMETERS**input**

List of input spectra to be calibrated. When using record format extensions the root names are specified, otherwise full image names are used.

output

List of calibrated spectra. If no output list is specified or if output name is the same as the input name then the calibrated spectra replace the input spectra. When using record format extensions the output names consist of root names to which the appropriate record number extension is added. The record number extension will be the same as the input record number extension. The output spectra are coerced to have real datatype pixels regardless of the pixel type.

records(imred.irsandimred.iidsonly)

The set of record number extensions to be applied to each input and output root name when using record number extension format. The syntax consists of comma separated numbers or ranges of numbers. A range consists of two numbers separated by a hyphen. This parameter is not queried when record number formats are not used.

extinct= yes

Apply extinction correction if a spectrum has not been previously corrected? When applying an extinction correction, an extinction file is required.

flux = yes

Apply a flux calibration if a spectrum has not been previously calibrated? When applying a flux calibration, sensitivity spectra are required.

extinction= <no default>

Extinction file for the observation. Standard extinction files are available in the "onedstds\$" directory.

observatory =")_observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to the package parameter of the same name. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters in the **observatory** task. See **observatory** for additional information.

ignoreaps= no

Ignore aperture numbers and apply a single flux calibration to all apertures? Normally multiaperture instruments have separate sensitivity functions for each aperture while long slit or Fabry-Perot data use a single sensitivity function where the apertures are to be ignored. The sensitivity spectra are obtained by adding the aperture number as an extension to the sensitivity spectrum root name. When apertures are ignored the specified sensitivity spectrum name is used without adding an extension and applied to all input apertures.

sensitivity = "sens"

The root name for the sensitivity spectra produced by **sensfunc**. Normally with multiaperture instruments, **sensfunc** will produce a spectrum appropriate to each aperture with an aperture number extension. If the apertures are ignored (*ignoreaps* = yes) then the sensitivity spectrum specified is used for all apertures and no aperture number is appended automatically.

fnu = no

The default calibration is into units of flux per unit wavelength (F-lambda). If *fnu* = yes then the calibrated spectrum will be in units of flux per unit frequency (F-nu).

airmass,exptime

If the airmass and exposure time are not in the header nor can they be determined from other keywords in the header then these query parameters are used to request the airmass and exposure time. The values are updated in the input and output images.

DESCRIPTION

The input spectra are corrected for extinction and calibrated to a scale using sensitivity spectra produced by the task **sensfunc**. One or both calibrations may be performed by selecting the appropriate parameter flags. It is an error if no calibration is specified. Normally the spectra should be extinction corrected if also flux calibrating. The image header keywords DC-FLAG (or the dispersion type field in the "multispec" world coordinate system), EX-FLAG, and CA-FLAG are checked for dispersion solution (required), previous extinction correction, and previous flux calibration. If previously calibrated the spectrum is skipped and a new output image is not created.

The input spectra are specified by a list of root names (when using record extension format) or full image names. The output calibrated spectra may replace the input spectra if no output spectra list is specified or if output name is the same as the input name. When using record number extensions the output spectra will have the same extensions applied to the root names as those used for the input spectra.

When applying an extinction correction the AIRMASS keyword is sought. If the keyword is not present then the airmass at the time defined by the other header keywords is computed using the latitude of the observatory and observation parameters in the image header. The observatory is first determined from the image under the keyword OBSERVAT. If absent the observatory specified by the task parameter "observatory" is used. See **observatory** for further details of the observatory database. If the air mass cannot be determined an error results. Currently a single airmass is used and no correction for changing extinction during the observation is made and adjustment to the middle of the exposure. The task **setairmass** provides a correction for the exposure time to compute an effective air mass. Running this task before calibration is recommended.

If the airmass is not in the header and cannot be computed then the user is queried for a value. The value entered is then recorded in both the input and output image headers. Also if the exposure time is not found then it is also queried and recorded in the image headers.

The extinction correction is given by the factor

$$10. ** (0.4 * \text{airmass} * \text{extinction})$$

where the extinction is the value interpolated from the specified extinction file for the wavelength of each pixel. After extinction correction the EX-FLAG is set to 0.

When applying a flux calibration the spectra are divided by the aperture sensitivity which is represented by a spectrum produced by the task **sensfunc**. The sensitivity spectrum is in units of:

$$2.5 * \text{Log}_{10} [\text{counts/sec/Ang} /$$

A new spectrum is created in "F-lambda" units - or "F-nu" units - ergs/cm2/sec/Hz. The sensitivity must span the range of wavelengths in the spectrum and interpolation is used if the wavelength coordinates are not identical. If some pixels in the spectrum being calibrated fall outside the wavelength range of the sensitivity function spectrum a warning message giving the number of pixels outside the range. In this case the sensitivity value for the nearest wavelength in the sensitivity function is used.

Multiaperture instruments typically have a separate aperture sensitivity function for each aperture. The appropriate sensitivity function for each input spectrum is selected based on the spectrum's aperture by appending this number to the root sensitivity function spectrum name. If the *ignoreaps* flag is set, however, the aperture number relation is ignored and the single sensitivity spectrum (without extension) is applied.

EXAMPLES

1. To flux calibrates a series of spectra replacing the input spectra by the calibrated spectra:

```
cl> calibrate nite1 ""
```

2. To only extinction correct echelle spectra:

```
cl> calibrate ccd*.ec.imh new//ccd*.ec.imh flux-
```

3. To flux calibrate a long slit spectrum:

```
cl> dispaxis = 2
cl> calibrate obj.imh fcobj.imh
```

REVISIONS

CALIBRATEV2.10.3

This task was revised to operate on 2D and 3D spatial spectra; i.e. long slit and Fabry-Perot data cubes. This task now includes the functionality previously found in **longslit.extinction** and **longslit.fluxcalib**.

A query for the airmass and exposure time is now made if the information is not in the header and cannot be computed from other header keywords.

CALIBRATEV2.10

This task was revised to operate on nonlinear dispersion corrected spectra and 3D images (the **apextract** "extras"). The aperture selection parameter was eliminated (since the header structure does not allow mixing calibrated and uncalibrated spectra) and the latitude parameter was replaced by the observatory parameter. The observatory mechanism insures that if the observatory latitude is needed for computing an airmass and the observatory is specified in the image header the correct calibration will be applied. The record format syntax is available in the **irs/iids** packages. The output spectra are coerced to have real pixel datatype.

SEE ALSO

setairmass, standard, sensfunc, observatory, continuum

NAME

airmass -- compute the airmass at a given elevation above horizon

USAGE

airmass elevation

PARAMETERS**elevation**

Elevation above horizon in either degrees or radians.

scale = 750.0

Scale factor of the Earth's atmosphere.

radians= no

Input elevation in radians instead of degrees.

airmass

On output, contains the computed airmass.

EXAMPLE

Compute the airmass at an elevation of 30 degrees above the horizon

```
cl> airmass 30
airmass 1.996 at an elevation of 30. degrees (0.5236 radians)
above horizon
```


NAME

apall -- Extract one dimensional sums across the apertures

USAGE

apall input

PARAMETERS**input**

List of input images.

output = ""

List of output root names for extracted spectra. If the null string is given or the end of the output list is reached before the end of the input list then the input image name is used as the output root name. This will not conflict with the input image since an aperture number extension is added for onedspec format, the extension ".ms" for multispec format, or the extension ".ec" for echelle format.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a for example, "1, 3-5, 9-12x2".

format = "multispec" (onedspec|multispec|echelle|strip)

Format for output extracted spectra. "Onedspec" format extracts each aperture to a separate image while "multispec" and "echelle" extract multiple apertures for the same image to a single output image. The "multispec" and "echelle" format selections differ only in the extension added. The "strip" format produces a separate 2D image in which each column or line along the dispersion axis is shifted to exactly align the aperture based on the trace information.

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image. Input images without/with a database entry are skipped silently.

profiles = ""

List of profile images for variance weighting or cleaning. If variance weighting or cleaning a profile of each aperture is computed from the input image unless a profile image is specified, in which case the profile is computed from the profile image. The profile image must have the same dimensions and dispersion and it is assumed that the spectra have the same position and profile shape as in the object spectra. Use of a profile image is generally not required even for faint input spectra but the option is available for those who wish to use it.

PROCESSING PARAMETERS**interactive= yes**

Run this task interactively? If the task is not run interactively then all user queries are suppressed

and interactive aperture editing, trace fitting, and extraction review are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= yes

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a parameter must also be yes.

extract= yes

Extract the one dimensional aperture sums?

extras= yes

Extract the raw spectrum (if variance weighting is used), the sky spectrum (if background subtraction is used), and sigma spectrum (if variance weighting is used)? This information is extracted to the third dimension of the output image.

review = yes

Review the extracted spectra? The *interactive* parameter must also be yes.

line = INDEF, nsum = 10

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum selects a sum of lines and a negative selects a median of lines.

DEFAULT APERTURE PARAMETERS

lower = -5., upper= 5.

Default lower and upper aperture limits relative to the aperture center. These limits are used for apertures found with **apfind** and when defining the first aperture in **apedit**.

apidtable = ""

Aperture identification table. This may be either a image. A text file consisting of lines with an aperture number, beam number, and aperture title or identification. An image will contain the keywords SLFIBnnn with string value consisting of aperture number, beam number, optional right ascension and declination, and aperture title. This information is used to assign aperture information automatically in **apfind** and **apedit**.

DEFAULT BACKGROUND PARAMETERS

b_function = "chebyshev"

Default background fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

b_order= 1

Default background function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

b_sample = "-10:-6,6:10"

Default background sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points. Note that the background coordinates are relative to the aperture center and not image pixel coordinates so the endpoints need not be integer.

b_naverage= -3

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the fit.

b_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant fitting points and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

b_low_reject= 3., b_high_reject= 3.

Default background lower and upper rejection sigmas. If greater than zero points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

b_grow= 0.

Default reject growing radius. Points within a parameter of any rejected point are also rejected.

APERTURE CENTERING PARAMETERS

width= 5.

Width of spectrum profiles. This parameter is used for the profile centering algorithm in this and other tasks.

radius= 10.

The profile centering error radius for the centering algorithm.

threshold= 0.

Centering threshold for the centering algorithm. The range of pixel intensities near the initial centering position must exceed this threshold.

AUTOMATIC FINDING AND ORDERING PARAMETERS

nfind

Maximum number of apertures to be defined. This is a so the user is queried for a value except when given explicitly on the command line.

minsep= 5.

Minimum separation between spectra. Weaker spectra or noise within this distance of a stronger spectrum are rejected.

maxsep = 1000.

Maximum separation between adjacent spectra. This parameter is used to identify missing spectra in uniformly spaced spectra produced by fiber spectrographs. If two adjacent spectra exceed this separation then it is assumed that a spectrum is missing and the aperture identification assignments will be adjusted accordingly.

order = "increasing"

When assigning aperture identifications order the spectra "increasing" or "decreasing" with increasing pixel position (left-to-right or right-to-left in a cross-section plot of the image).

RECENTERING PARAMETERS

aprecenter = ""

List of apertures to be used in shift calculation.

npeaks = INDEF

Select the specified number of apertures with the highest peak values to be recentered. If the number is INDEF all apertures will be selected. If the value is less than 1 then the value is interpreted as a fraction of total number of apertures.

shift = yes

Use the average shift from recentering the apertures selected by the *aprecenter* parameter to apply to the apertures selected by the *apertures* parameter. The recentering is then applied to all apertures.

RESIZING PARAMETERS

llimit = INDEF, ulimit = INDEF

Lower and upper aperture size limits. If the parameter *ylevel* is INDEF then these limits are assigned to all apertures. Otherwise these parameters are used as limits to the resizing operation. A value of INDEF places the aperture limits at the image edge (for the dispersion line used).

ylevel = 0.1

Data level at which to set aperture limits. If it is INDEF then the aperture limits are set at the values given by the parameters *llimit* and *ulimit*. If it is not INDEF then it is a fraction of the peak or an actual data level depending on the parameter *peak*. It may be relative to a local background or to zero depending on the parameter *bkg*.

peak = yes

Is the data level specified by *ylevel* a

bkg = yes

Subtract a simple background when interpreting the **ylevel** parameter. The background is a slope connecting the first inflection points away from the aperture center.

r_grow = 0.

Change the lower and upper aperture limits by this fractional amount. The factor is multiplied by each limit and the result added to limit.

avglimits = no

Apply the average lower and upper aperture limits to all apertures.

TRACING PARAMETERS

t_nsum= 10

Number of dispersion lines to be summed at each step along the dispersion.

t_step= 10

Step along the dispersion axis between determination of the spectrum positions.

t_nlost= 3

Number of consecutive steps in which the profile is lost before quitting the tracing in one direction. To force tracing to continue through regions of very low signal this parameter can be made large. Note, however, that noise may drag the trace away before it recovers.

t_function="legendre"

Default trace fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

t_order= 2

Default trace function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

t_sample="*"

Default fitting sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points.

t_naverage= 1

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the

t_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant traced positions and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

t_low_reject= 3., t_high_reject= 3.

Default lower and upper rejection sigma. If greater than zero traced points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

t_grow= 0.

Default reject growing radius. Traced points within a parameter of any rejected point are also rejected.

EXTRACTION PARAMETERS

background="none" (none|average|median|minimum|fit)

Type of background subtraction. The choices are "none" for no background subtraction, "average" to average the background within the background regions, "median" to use the median in the background regions, "minimum" to use the minimum in the background regions, or "fit" to fit across the dispersion using the background within the background regions. Note that the "average" option does not do any medianing or bad pixel checking, something which is recommended. The fitting option is slower than the other options and requires additional fitting parameter.

skybox = 1

Box car smoothing length for sky background when using background subtraction. Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectral features. This is most appropriate when the sky regions are limited due to a small slit length.

weights = "none" (none|variance)

Type of extraction weighting. Note that if the *clean* parameter is set then the weights used are "variance" regardless of the weights specified by this parameter. The choices are:

"none"

The pixels are summed without weights except for partial pixels at the ends.

"variance"

The extraction is weighted by the variance based on the data values and a poisson/ccd model using the *gain* and *readnoise* parameters.

pfit = "fit1d" (fit1d|fit2d)

Profile fitting algorithm to use with variance weighting or cleaning. When determining a profile the two dimensional spectrum is divided by an estimate of the one dimensional spectrum to form a dimensional spectrum profile. This profile is then smoothed by fitting one dimensional functions, "fit1d", along the lines or columns most closely corresponding to the dispersion axis or a special two dimensional function, "fit2d", described by Marsh (see **aprofile**).

clean = no

Detect and replace deviant pixels?

saturation= INDEF

Saturation or nonlinearity level in data units. During variance weighted extractions wavelength points having any pixels above this value are excluded from the profile determination and the sigma spectrum extraction output, if selected by the *extras* parameter, flags wavelengths with saturated pixels with a negative sigma.

readnoise= 0.

Read out noise in photons. This parameter defines the minimum noise sigma. It is defined in terms of photons (or electrons) and scales to the data values through the gain parameter. A image header keyword (case insensitive) may be specified to get the value from the image.

gain = 1

Detector gain or conversion factor between photons/electrons and data values. It is specified as the number of photons per data value. A image header keyword (case insensitive) may be specified to get the value from the image.

lsigma = 4., usigma = 4.

Lower and upper rejection thresholds, given as a estimated sigma of a pixel, for cleaning.

nsubaps= 1

During extraction it is possible to equally divide the apertures into this number of subapertures. For multispec format all subapertures will be in the same file with aperture numbers of $1000 * (\text{subap} - 1) + \text{ap}$ where subap is the subaperture (1 to nsubaps) and ap is the main aperture number. For echelle format there will be a separate echelle format image containing the same subaperture from each order. The name will have the subaperture number appended. For onedspec

format each subaperture will be in a separate file with extensions and aperture numbers as in the multispec format.

ADDITIONAL PARAMETERS

Dispersion axis and I/O parameters are taken from the package parameters.

DESCRIPTION

This task provides functions for defining, modifying, tracing, and extracting apertures from two dimensional spectra. The functions desired are selected using switch parameters. When the task is run interactively queries are made at each step allowing additional control of the operations performed on each input image.

The functions, in the order in which they are generally performed, are summarized below.

- o Automatically find a specified number of spectra and assign default apertures. Apertures may also be inherited from another image or defined using an interactive graphical interface called the *aperture editor*.
- o Recenter selected reference apertures on the image spectrum profiles.
- o Resize the selected reference apertures based on spectrum profile width.
- o Interactively define or adjust aperture definitions using a interface called the *aperture editor*. All function may also be performed from this editor and, so, provides an alternative method of processing and extracting spectra.
- o Trace the positions of the selected spectra profiles from a or column to other image lines or columns and fit a smooth function. The trace function is used to shift the center of the apertures at each dispersion point in the image.
- o Extract the flux in the selected apertures into one dimensional spectra in various formats. This includes possible background subtraction, variance weighting, and bad pixel rejection.

Each of these functions has different options and parameters. In addition to selecting any of these functions in this task, they may also be selected using the aperture editor and as individual commands (which themselves allow selection of other functions). When broken down into individual tasks the parameters are also sorted by their function though there are then some mutual parameter interdependencies. This functional decomposition is what was available prior to the addition of the **apall** task. It is recommended that this task be used because it collects all the parameters in one place eliminating confusion over where a particular parameter is defined. However, documenting the various functions is better organized in terms of the separate descriptions given for each of the functions; namely under the help topics **apdefault**, **apfind**, **aprecenter**, **apresize**, **apedit**, **aptrace**, and **apsum**.

EXAMPLES

1. This example may be executed if desired. First we create an artificial spectrum with four spectra and a background. After it is created you can display or plot it. Next we define the dispersion axis and set the verbose flag to better illustrate what is happening. The task APALL is run with the default parameters except for background fitting and subtracting added. The text beginning with # are comments of things to try and do.

```
ap> artdata
ar> unlearn artdata
ar> mk1dspec apdemo1d nl=50
ar> mk2dspec apdemo2d model=STDIN
apdemo1d 1. gauss 3 0 20 .01
apdemo1d .8 gauss 3 0 40 .01
```

```

apdemo1d .6 gauss 3 0 60 .01
apdemo1d .4 gauss 3 0 80 .01
[EOF=Control D or Control Z]
ar> mknoise apdemo2d background=100. rdnoise=3. poisson+
ar> bye
# Display or plot the spectrum
ap> dispaxis=2; verbose=yes
ap> unlearn apall
ap> apall apdemo2d back=fit
Searching aperture database ...
Find apertures for apdemo2d? (yes):
Finding apertures ...
Number of apertures to be found automatically (1): 4
Jul 31 16:55: FIND - 4 apertures found for apdemo2d.
Resize apertures for apdemo2d? (yes):
Resizing apertures ...
Jul 31 16:55: RESIZE - 4 apertures resized for apdemo2d.
Edit apertures for apdemo2d? (yes):
# Get a list of commands with '?'
# See all the parameters settings with :par
# Try deleting and marking a spectrum with 'd' and 'm'
# Look at the background fitting parameters with 'b' (exit with 'q')
# Exit with 'q'
Trace apertures for apdemo2d? (yes):
Fit traced positions for apdemo2d interactively? (yes):
Tracing apertures ...
Fit curve to aperture 1 of apdemo2d interactively (yes):
# You can use ICFIT commands to adjust the fit.
Fit curve to aperture 2 of apdemo2d interactively (yes): n
Fit curve to aperture 3 of apdemo2d interactively (no):
Fit curve to aperture 4 of apdemo2d interactively (no): y
Jul 31 16:56: TRACE - 4 apertures traced in apdemo2d.
Write apertures for apdemo2d to apdemo2d (yes):
Jul 31 16:56: DATABASE - 4 apertures for apdemo2d written to database.
Extract aperture spectra for apdemo2d? (yes):
Review extracted spectra from apdemo2d? (yes):
Extracting apertures ...
Review extracted spectrum for aperture 1 from apdemo2d? (yes):
# Type 'q' to quit
Jul 31 16:56: EXTRACT - Aperture 1 from apdemo2d --> apdemo2d.ms
Review extracted spectrum for aperture 2 from apdemo2d? (yes): N
Jul 31 16:56: EXTRACT - Aperture 2 from apdemo2d --> apdemo2d.ms
Jul 31 16:56: EXTRACT - Aperture 3 from apdemo2d --> apdemo2d.ms
Jul 31 16:57: EXTRACT - Aperture 4 from apdemo2d --> apdemo2d.ms

```

2. To extract a series of similar spectra noninteractively using a reference for the aperture definitions, then recentering and resizing but not retracing:

```
ap> apall fib*.imh ref=flat inter- trace-
```

Note that the interactive flag automatically turns off the edit, fittrace, and review options and the reference image eliminates the find (find only occurs if there are no initial apertures).

REVISIONS

APALL V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

The "nsubaps" parameter now allows onedspec and echelle output formats. The echelle format is appropriate for treating each subaperture as a full echelle extraction.

APALL V2.10.3

The dispersion axis parameter was moved to purely a

As a final step when computing a weighted/cleaned spectrum the total fluxes from the weighted spectrum and the simple unweighted spectrum (excluding any deviant and saturated pixels) are computed and a "bias" factor of the ratio of the two fluxes is multiplied into the weighted spectrum and the sigma estimate. This makes the total fluxes the same. In this version the bias factor is recorded in the logfile if one is kept. Also a check is made for unusual bias factors. If the two fluxes disagree by more than a factor of two a warning is given on the standard output and the logfile with the individual total fluxes as well as the bias factor. If the bias factor is negative a warning is also given and no bias factor is applied. In the previous version a negative (inverted) spectrum would result.

SEE ALSO

apdefault, apfind, aprecenter, apresize, apedit, aptrace, apsum

NAME

apdefault -- Set default aperture parameters for the package

USAGE

apdefault

PARAMETERS

lower = -5., upper= 5.

Default lower and upper aperture limits relative to the aperture center. These limits are used for apertures found with **apfind** and when defining the first aperture in **apedit**.

apidtable = ""

Aperture identification table. This may be either a image. A text file consisting of lines with an aperture number, beam number, and aperture title or identification. An image will contain the keywords SLFIBnnn with string value consisting of aperture number, beam number, optional right ascension and declination, and aperture title. This information is used to assign aperture information automatically in **apfind** and **apedit**.

Default Background Subtraction Parameters

b_function = "chebyshev"

Default background fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

b_order= 1

Default background function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

b_sample = "-10:-6,6:10"

Default background sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points. Note that the background coordinates are relative to the aperture center and not image pixel coordinates so the endpoints need not be integer.

b_naverage= -3

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the fit.

b_niterate= 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant fitting points and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

b_low_reject= 3., b_high_reject= 3.

Default background lower and upper rejection sigmas. If greater than zero points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

b_grow= 0.

Default reject growing radius. Points within a parameter of any rejected point are also rejected.

DESCRIPTION

This task sets the values of the default aperture parameters for the tasks **apedit** and **apfind** which define new apertures. For a description of the components of an aperture see the paper **The APEXTRACT Package**. In **apedit** the default aperture limits and background parameters are only used if there are no other apertures defined. The aperture identification table is used when reordering the apertures with the 'o' key. When run the parameters are displayed and modified using the **eparam** task.

The aperture limits and background fitting sample regions are defined relative to the center of the aperture. The background fitting parameters are those used by the ICFIT package. They may be modified interactively with the 'b' key in the task **apedit**. For more on background fitting and subtracting see **apbackground**.

EXAMPLES

To review and modify the default aperture parameters:

```
cl> apdefault
```

REVISIONS

APDEFAULTV2.11

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

SEE ALSO apbackground, apedit, apfind, icfit

NAME

apfind -- Find spectra and define apertures automatically

USAGE

apfind input

PARAMETERS**input**

List of input images in which spectra are to be identified and apertures defined automatically.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a *for* example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database and the parameter *nfind* must be greater than zero.

recenter= no

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in finding the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value sums lines and a negative value medians lines.

nfind = 1

Maximum number of apertures to be defined. This is a so the user is queried for a value except when given explicitly on the command line.

minsep = 5.

Minimum separation between spectra. Weaker spectra or noise within this distance of a stronger spectrum are rejected.

maxsep = 1000.

Maximum separation between adjacent spectra. This parameter is used to identify missing spectra in uniformly spaced spectra produced by fiber spectrographs. If two adjacent spectra exceed this separation then it is assumed that a spectrum is missing and the aperture identification assignments will be adjusted accordingly.

order = "increasing"

When assigning aperture identifications order the spectra "increasing" or "decreasing" with increasing pixel position (left-to-right or right-to-left in a cross-section plot of the image).

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list spectra are identified and default apertures defined. The automatic aperture finding is performed only if 1) there are no apertures defined for the reference image, 2) there are no apertures defined for the input image, 3) the parameter *find* is yes, and 4) the parameter *nfind* is greater than zero.

The automatic finding algorithm uses the following steps. First, all local maxima are found. The maxima are sorted by peak value and the weaker of the peaks separated by less than the value given by the parameter *minsep* are rejected. Finally, at most the *nfind* strongest peaks are kept. **Nfind** is a query parameter, so if it is not specified explicitly on the command line, the desired number of spectra to be found is requested. After the peaks have been found the **center1d** algorithm is used to refine the centers of the profiles. Apertures having the default parameters set with the task **apdefault** are defined at each center. This algorithm is also available with the 'f' key in the task **apedit** with the change that existing apertures are kept and count toward the maximum number specified by **nfind**.

The automatic assignment of aperture numbers, beam numbers, and titles has several options. The simplest is when no aperture identification table, parameter *apidtable*, is specified and the maximum separation parameter, *maxsep*, is very large. In this case the aperture and beam numbers are sequential starting from one and numbered either from left-to-right or right-to-left depending on the *order* parameter. There are no aperture titles in this case. If two adjacent spectra are separated by more than the specified maximum then the aperture numbers jump by the integer part of the ratio of the separation to the specified maximum separation. This is used when the image is expected to have evenly spaced spectra, such as in multifiber spectrographs, in which some may be missing due to broken fibers. Finally, the aperture identification table (either a text file or an image having a set of SLFIBnnn keywords) may contain lines with aperture number, beam number, and (optional) title. The sequential numbers are then indices into this table. Note that the skipping of missing spectra and the ordering applies to entries in this table as well.

The ways in which the automatic method can fail for evenly spaced spectra with missing members are when the first spectrum is missing on the side from which the ordering begins and when the expected rather than the actual number of spectra is used. In the first case one can use the interactive 'o' key of the aperture editing facility to specify the identity of any aperture and then all other apertures will be appropriately reidentified. If more spectra are sought than actually exist then noise spikes may be mistakenly found. This problem can be eliminated by specifying the actual number of spectra or minimized by using the threshold centering parameter.

The *recenter* parameter allows recentering apertures if a reference image. Since the purpose of this task is to find new apertures it is usually the case that there are no reference images and recentering is not done. The default apertures are of fixed width. The *resize* parameter may be used to adjust the widths in a variety of ways. The aperture positions and any other parameters may also be edited with the aperture editing function if selected by the *apedit* parameter and the task is run interactively.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture finding algorithm may be selected from nearly every task in the package.

EXAMPLES

```
cl> apfind image nfind=10
```

REVISIONS

APFIND V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The aperture ID table information may now be contained in the image header under the keywords SLFIBnnn.

SEE ALSO center1d, apdefault, aprecenter, apresize, apedit, apall

NAME

aprecenter -- Recenter apertures automatically

USAGE

aprecenter input

PARAMETERS**input**

List of input images in which apertures are to be recentered.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `*` for example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= yes

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in recentering the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value takes a sum and a negative values selects a median.

aprecenter =""

List of apertures to be used in shift calculation.

npeaks= INDEF

Select the specified number of apertures with the highest peak values to be recentered. If the number is INDEF all apertures will be selected. If the value is less than 1 then the value is interpreted as a fraction of total number of apertures.

shift = yes

Use the median shift from recentering the selected apertures to apply to all apertures. The recentering is then a constant shift for all apertures. The median is the average of the two central values for an even number of apertures.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, automatic aperture finding parameters are taken from **apfind**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the aperture center positions are redefined by centering at the specified dispersion line using the **center1d** algorithm with centering parameters from **apedit**. Normally this is done when inheriting apertures from an aperture reference image. The recentering does not change the "trace" of the aperture but simply adds a shift across the dispersion axis.

There are several recentering options. Each selected aperture may be recentered independently. However, if some or all of the spectra are relatively weak this may actually be worse than using the reference apertures defined by strong spectra or flat fields in the case of fibers or aperture masks. One may select a subset of apertures used in calculating shift. This is done with the *aprecenter* list of aperture numbers (see **ranges** for the syntax) and/or by selecting a specific number or fraction of the apertures with the strongest peak values. The list selection is done first and the strongest remaining apertures are used to satisfy the **npeaks** value. Though some or all of the apertures may be recentered independently the most common case of recentering reference apertures is to account for detector shifts. In this case one expects that any shift should be common to all apertures. The *shift* parameter allows using the new centers for all selected apertures to compute a median shift to be added to ALL apertures. Using a median shift for all apertures is the default.

The *find* parameter allows automatically finding apertures if they are defined for the image or by a reference image. Since the purpose of this task is to recenter reference apertures it is usually the case that reference images are used and apertures are not defined by this task. One case in which the apertures from the image itself might be recentered is if one wants to use a different dispersion line. The *resize* parameter may be used to adjust the widths in a variety of ways based on the spectra profiles specific to each image. The aperture positions and any other parameters may also be edited with the aperture editing function if selected by the *apedit* parameter and the task is run interactively. The recentering algorithm may be run from the aperture editor using the 'g' keystroke.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture recentering algorithm may be selected from nearly every task in the package.

EXAMPLES

```
cl> aprecenter newimage reference=flat
```

REVISIONS**APRECENTERV2.11**

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

center1d, ranges, apfind, apresize, apedit, apall

NAME

apresize -- Resize apertures automatically

USAGE

apresize input

PARAMETERS**input**

List of input images in which apertures are to be resized.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `for` example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= no

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing is disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

line = INDEF

The dispersion line (line or column perpendicular to the dispersion axis) to be used in resizing the spectra. A value of INDEF selects the middle of the image.

nsum = 1

Number of dispersion lines to be summed or medianed. The lines are taken around the specified dispersion line. A positive value takes a sum and a negative value selects a median.

llimit = INDEF, ulimit = INDEF

Lower and upper aperture size limits. If the parameter *ylevel* is INDEF then these limits are assigned to all apertures. Otherwise these parameters are used as limits to the resizing operation. A value of INDEF places the aperture limits at the image edge (for the dispersion line used).

ylevel = 0.1

Data level at which to set aperture limits. If it is INDEF then the aperture limits are set at the values given by the parameters *llimit* and *ulimit*. If it is not INDEF then it is a fraction of the peak or an actual data level depending on the parameter *peak*. It may be relative to a local background or to zero depending on the parameter *bkg*.

peak = yes

Is the data level specified by *ylevel* a

bkg = yes

Subtract a simple background when interpreting the **ylevel** parameter. The background is a slope connecting the first minima away from the aperture center.

r_grow = 0.

Change the lower and upper aperture limits by this fractional amount. The factor is multiplied by each limit and the result added to limit.

avglimits = no

Apply the average lower and upper aperture limits to all apertures.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters are taken from the task **apdefault**, automatic aperture finding parameters are taken from **apfind**, and parameters used for centering and editing the apertures are taken from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the aperture limits are redefined to be either specified values or by finding the points at which the spectrum profile, linearly interpolated, first crosses a specified value moving away from the aperture center at the specified dispersion line. In the latter case the limits may then be increased or decreased by a specified percentage, a maximum lower and upper limit, may be imposed, and the independent limits may be averaged and the single values applied to all the apertures.

The simplest resizing choice is to reset all the aperture limits to the values specified by *llimit* and *ulimit*. This option is selected if the parameter *ylevel* is INDEF.

There are several options for specifying a data level at which an aperture is sized. The most common method (the default) is to specify a fraction of the peak value since this is data independent and physically reasonable. This is done by setting the fraction with the parameter *ylevel* and the parameter *peak* to yes. If the *peak* parameter is no then the level is a data value.

The levels may be relative to zero, as might be used with fibers or high dispersion / high signal-to-noise data, or relative to a local linear background, as would be appropriate for slit data having a significant background. A background is found and used if the parameter *bkg* is set. The background determination is very simple. Starting at the peak two background points are found, one in each direction, which are inflection points; i.e. the first pixels which are less than their two neighbors. A linear slope is fit and

subtracted for the purposes of measuring the peak and setting the aperture limits. Note that if the slope is significant the actual limits may not correspond to the intercepts of a value.

Once aperture limits, a distance relative to the center, are determined they are increased or decreased by a percentage, expressed as a fraction, given by the parameter *r_grow*. To illustrate the operation, if xlow is the initial lower limit then the final lower limit will be:

$$\text{xlow final} = \text{xlow} * (1 + r_grow)$$

A value of zero leaves the aperture limits unchanged.

After the aperture limits are found, based on the above steps, a limit, given by the parameter *llimit*, is applied to the lower aperture points and, similarly, a fixed upper limit is applied to the upper aperture points. This feature protects against absurdly wide apertures.

Finally, if the parameter *avglimits* is set the individual aperture limits are averaged to form an average aperture. This average aperture is then assigned to all apertures. This option allows keeping common aperture sizes but allowing variation due to seeing changes.

The resizing algorithm is available in the interactive aperture editor. Here one may select individual apertures or all apertures using the 'a' switch. The resizing algorithm described above is selected using the 'z' key. A simple alternative is the 'y' key which resizes apertures to the y level marked by the cursor.

If the task is interactive the user is queried whether to perform various steps on each image. The queries may be answered with one of the four values "yes", "no", "YES" and "NO", where an upper case response suppresses all further queries to this question.

The aperture resizing algorithm may be selected from nearly every task in the package with the *resize* parameter.

EXAMPLES

1. To resize all apertures to the range -4 to 4:

```
cl> apresize image llimit=-4 ulimit=4 ylevel=INDEF
```

2. To resize all aperture to a point which is 5% of the peak relative to a local background:

```
cl> apresize image ylevel=.05 peak+ bkg+
```

3. To resize all apertures to the point where the data exceeds 100 data units:

```
cl> apresize image ylevel=100 peak- bkg-
```

4. To resize all apertures to default values of the task except averaging all the results at the end:

```
cl> apresize image avg+
```

REVISIONS

APRESIZEV2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

center1d, ranges, apfind, aprecenter, apedit, apall

NAME

apsum -- Extract one dimensional sums across the apertures

USAGE

apsum input

PARAMETERS**input**

List of input images containing apertures to be extracted.

output = ""

List of output rootnames for the extracted spectra. If the null string is given or the end of the output list is reached before the end of the input list then the input image name is used as the output rootname. This will not conflict with the input image since an aperture number extension is added for onedspec format, the extension ".ms" for multispec format, or the extension ".ec" for echelle format.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a for example, "1,3-5,9-12x2".

format="multispec" (onedspec|multispec|echelle|strip)

Format for output extracted spectra. "Onedspec" format extracts each aperture to a separate image while "multispec" and "echelle" extract multiple apertures for the same image to a single output image. The "multispec" and "echelle" format selections differ only in the extension added. The "strip" format produces a separate 2D image in which each column or line along the dispersion axis is shifted to exactly align the aperture based on the trace information.

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

profiles = ""

List of profile images for variance weighting or cleaning. If variance weighting or cleaning a profile of each aperture is computed from the input image unless a profile image is specified, in which case the profile is computed from the profile image. The profile image must have the same dimensions and dispersion and it is assumed that the spectra have the same position and profile shape as in the object spectra. Use of a profile image is generally not required even for faint input spectra but the option is available for those who wish to use it.

interactive= yes

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing, trace fitting, and extraction review are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = no

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a parameter must also be yes.

extract= yes

Extract the one dimensional aperture sums?

extras= no

Extract the raw spectrum (if variance weighting is used), the sky spectrum (if background subtraction is used), and variance spectrum (if variance weighting is used)? This information is extracted to the third dimension of the output image.

review = yes

Review the extracted spectra? The *interactive* parameter must also be yes.

line = INDEF, nsum = 10

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. For tracing this is the starting line and the same number of lines are summed at each tracing point. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum takes a sum while a negative value selects a median except that tracing always uses a sum.

background = "none" (none|average|median|minimum|fit)

Type of background subtraction. The choices are "none" for no background subtraction, "average" to average the background within the background regions, "median" to use the median in the background regions, "minimum" to use the minimum in the background regions, or "fit" to fit across the dispersion using the background within the background regions. Note that the "average" option does not do any medianing or bad pixel checking, something which is recommended. The fitting option is slower than the other options and requires additional fitting parameter.

weights = "none"

Type of extraction weighting. Note that if the *clean* parameter is set then the weights used are "variance" regardless of the weights specified by this parameter. The choices are:

"none"

The pixels are summed without weights except for partial pixels at the ends.

"variance"

The extraction is weighted by the variance based on the data values and a poisson/ccd model using the *gain* and *readnoise* parameters.

pfit = "fit1d" (fit1d|fit2d)

Profile fitting algorithm to use with variance weighting or cleaning. When determining a profile the two dimensional spectrum is divided by an estimate of the one dimensional spectrum to form a dimensional spectrum profile. This profile is then smoothed by fitting one dimensional functions, "fit1d", along the lines or columns most closely corresponding to the dispersion axis or a special two dimensional function, "fit2d", described by Marsh (see **aprofile**).

clean = no

Detect and replace deviant pixels?

skybox = 1

Box car smoothing length for sky background when using background subtraction. Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectral features. This is most appropriate when the sky regions are limited due to a small slit length.

saturation= INDEF

Saturation or nonlinearity level in data units. During variance weighted extractions wavelength points having any pixels above this value are excluded from the profile determination and the sigma spectrum extraction output, if selected by the *extras* parameter, flags wavelengths with saturated pixels with a negative sigma.

readnoise= 0.

Read out noise in photons. This parameter defines the minimum noise sigma. It is defined in terms of photons (or electrons) and scales to the data values through the *gain* parameter. A image header keyword (case insensitive) may be specified to get the value from the image.

gain = 1

Detector gain or conversion factor between photons/electrons and data values. It is specified as the number of photons per data value. A image header keyword (case insensitive) may be specified to get the value from the image.

lsigma = 4., usigma = 4.

Lower and upper rejection thresholds, given as a estimated sigma of a pixel, for cleaning.

nsubaps= 1

During extraction it is possible to equally divide the apertures into this number of subapertures. For multispec format all subapertures will be in the same file with aperture numbers of $1000 * (\text{subap} - 1) + \text{ap}$ where subap is the subaperture (1 to nsubaps) and ap is the main aperture number. For echelle format there will be a separate echelle format image containing the same subaperture from each order. The name will have the subaperture number appended. For onedspec format each subaperture will be in a separate file with extensions and aperture numbers as in the multispec format.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters from **apdefault**, automatic aperture finding parameters from **apfind**, recentering parameters from **aprecenter**, resizing parameters from **apresize**, parameters used for centering and editing the apertures from **apedit**, and tracing parameters from **aptrace**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list, the two dimensional spectra are extracted to one dimensional spectra by summing the pixels across the dispersion axis at each wavelength along the dispersion axis within a set of defined apertures. The extraction apertures consist of an aperture number, a beam number, a title, a center, limits relative to the center, a curve describing shifts of the aperture center across the dispersion axis as a function of the wavelength, and parameters for background fitting and subtraction. See **apextract** for a detailed discussion of the aperture structures.

The extracted spectra are recorded in one, two, or three dimensional images depending on the *format* and *extras* parameters. The output image rootnames are specified by the *output* list. If list is empty or shorter than the input list the missing names are taken to be the same as the input image names. Because the rootnames have extensions added it is common to default to the input names in order to preserve a naming relation between the input two dimensional spectra and the extracted spectra.

When the parameter *extras*=no only the extracted spectra are output. If the format parameter *format*="onedspec" the output aperture extractions are one dimensional images with names formed from the output rootname and a numeric extension given by the aperture number; i.e. root.0001 for aperture 1. Note that there will be as many output images as there are apertures for each input image, all with the same output rootname but with different aperture extensions. The aperture beam number associated with each aperture is recorded in the output image under the keyword BEAM-NUM. The output image name format and the BEAM-NUM entry in the image are chosen to be compatible with the **onedspec** package.

If the format parameter is "echelle" or "multispec" the output aperture extractions are put into a two dimensional image with a name formed from the output rootname and the extension ".ech" or ".ms". Each line in the output image corresponds to one aperture. Thus in this format there is one output image for each input image. These are the preferred output formats for reasons of compactness and ease of handling. These formats are compatible with the **onedspec**, **echelle**, and **msred** packages. The relation between the line and the aperture numbers is given by the header parameter APNUMn where n the value is the aperture number and other numeric information.

If the *extras* parameter is set to yes then the above formats become three dimensional. Each plane in the third dimension contains associated information for the spectra in the first plane. If variance weighted extractions are done the unweighted spectra are recorded. If background subtraction is done the background spectra are recorded. If variance weighted extractions are done the sigma spectrum (the estimated sigma of each spectrum pixel based on the individual variances of the pixels summed) is recorded. The order of the additional information is as given above. For example, an unweighted extraction with background subtraction will have one additional plane containing the sky spectra while a variance weighted extraction with background subtractions will have the variance weighted spectra, the unweighted spectra, the background spectra, and the sigma spectra in consecutive planes.

Aperture definitions may be inherited from those of other images by specifying a reference image with the **references** parameter. Images in the reference list are matched with those in the input list in order. If the reference image list is shorter than the number of input images, the last reference image is used for all remaining input images. Thus, a single reference image may be given for all the input images or different reference images may be given for each input image. The special reference name "last" may be used to select the last set apertures used in any of the **apextract** tasks.

If an aperture reference image is not specified or no apertures are found for the specified reference image, previously defined apertures for the input image are sought in the aperture database. Note that reference apertures supersede apertures for the input image. If no apertures are defined they may be created automatically, the *find* option, or interactively in the aperture editor, if the *interactive* and *edit* options are set.

The functions performed by the task are selected by a parameters. The functions are an automatic spectrum finding and aperture defining algorithm (see **apfind**) which is ignored if apertures are already defined, automatic recentering and resizing algorithms (see **aprecenter** and **apresize**), an interactive aperture editing function (see **apedit**), a spectrum position tracing and trace function fit (see **aptrace**), and the main function of this task, one dimensional spectrum extraction.

Each function selection will produce a query for each input spectrum if the *interactive* parameter is set. The queries are answered by "yes", "no", "YES", or "NO", where the upper case responses suppress the query for following images. There are other queries associated with tracing and extracted spectrum review which first ask whether the operation is to be done interactively and, if yes, lead to queries for each aperture. The cursor keys available during spectrum review are minimal, only the CURSOR MODE keys for expanding and adjusting the graph are available and the quit key 'q'. If the *interactive* parameter is not set then aperture editing, interactive trace fitting, and spectrum review are ignored.

Background sky subtraction is done during the extraction based on background regions and parameters defined by the default parameters or changed during the interactive setting of the apertures. The background subtraction options are to do no background subtraction, subtract the average, median, or minimum of the pixels in the background regions, or to fit a function and subtract the function from under the extracted object pixels. The background regions are specified in pixels from the aperture center and follow changes in center of the spectrum along the dispersion. The syntax is colon separated ranges with multiple ranges separated by a comma or space. The background fitting uses the **icfit** routines which include medians, iterative rejection of deviant points, and a choice of function types and orders. Note that it is important to use a method which rejects cosmic rays such as using either medians over all the background regions (*background* = "median") or median samples during fitting (*b_naverage* < -1). The background subtraction algorithm and options are described in greater detail in **apsum** and **apbackground**.

Since the background noise is often the limiting factor for good extraction one may box car smooth the sky to improve the statistics in smooth background regions at the expense of distorting the subtraction near spectra features. This is most appropriate when the sky region is limited due to small slit length. The smoothing length is specified by the parameter *skybox*.

For a more extended discussion about the background determination see **apbackground**.

The aperture extractions consists of summing all the background subtracted pixel values at a given wavelength within the aperture limits. The aperture limits form a fixed width aperture but the center varies smoothly to follow changes in the position of the spectrum across the dispersion axis. At the ends of the aperture partial pixels are used.

The pixels in the sum may be weighted as specified by the *weights* parameter. If the *weights* parameter is "none" and the *clean* parameter is no then the simple sum of the pixels (with fractional endpoints) is extracted. If the *weights* parameter is "variance" or if the **clean** parameter is yes the pixels are weighted by their estimated variance derived from a noise model based on the *gain* and *readnoise* parameters and a smooth profile function. Normally the profile function is determined from the data being extracted. However, one may substitute a "profile" image as specified by the *profiles* parameter for computing the profile. This requires that the profile image have spectra of identical position and profile as the image being extracted. For example, this would likely be the case with fiber spectra and an off-telescope spectrograph and a strong flat field or object spectrum could be used for weak spectra. Note that experience has shown that even for very weak spectra there is little improvement with using a separate profile image but the user is free to experiment.

When the *clean* parameter is set pixels deviating by more than a specified number of sigma from the profile function are excluded from the variance weighted sum. Note that the *clean* parameter always selects variance weights. For a more complete discussion of the extraction sums, variance weighting, cleaning, the noise model, and profile function determination see **apvariance** and **aprofiles**.

EXAMPLES

1. To simply extract the spectra from a multislit observation:

```
cl> apsum multislit1
```

The positions of the slits are defined using either automatic finding or with the aperture editor. The positions of the slits are traced if necessary and then the apertures are extracted to the image "multislit1.ms". The steps of defining the slit positions and tracing can be done as part of this command or previously using the other tasks in the **apextract** package.

REVISIONS

APSUM V2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

The "nsubaps" parameter now allows onedspec and echelle output formats. The echelle format is appropriate for treating each subaperture as a full echelle extraction.

The dispersion axis parameter was moved to purely a

As a final step when computing a weighted/cleaned spectrum the total fluxes from the weighted spectrum and the simple unweighted spectrum (excluding any deviant and saturated pixels) are computed and a "bias" factor of the ratio of the two fluxes is multiplied into the weighted spectrum and the sigma estimate. This makes the total fluxes the same. In this version the bias factor is recorded in the logfile if one is kept. Also a check is made for unusual bias factors. If the two fluxes disagree by more than a factor of two a warning is given on the standard output and the logfile with the individual total fluxes as well as the bias factor. If the bias factor is negative a warning is also given and no bias factor is applied. In the previous version a negative (inverted) spectrum would result.

SEE ALSO

apbackground, apvariance, aprofile, apdefault, apfind, aprecenter, apresize, apedit, aptrace, apall

NAME

aptrace -- Trace spectra for aperture extraction

USAGE

aptrace images

PARAMETERS**input**

List of input images to be traced.

apertures = ""

Apertures to recenter, resize, trace, and extract. This only applies to apertures read from the input or reference database. Any new apertures defined with the automatic finding algorithm or interactively are always selected. The syntax is a list comma separated ranges where a range can be a single aperture number, a hyphen separated range of aperture numbers, or a range with a `for` example, "1, 3-5, 9-12x2".

references = ""

List of reference images to be used to define apertures for the input images. When a reference image is given it supersedes apertures previously defined for the input image. The list may be null, "", or any number of images less than or equal to the list of input images. There are three special words which may be used in place of an image name. The word "last" refers to the last set of apertures written to the database. The word "OLD" requires that an entry exist and the word "NEW" requires that the entry not exist for each input image.

interactive= yes

Run this task interactively? If the task is not run interactively then all user queries are suppressed and interactive aperture editing and trace fitting are disabled.

find = yes

Find the spectra and define apertures automatically? In order for spectra to be found automatically there must be no apertures for the input image or reference image defined in the database.

recenter= no

Recenter the apertures?

resize = yes

Resize the apertures?

edit = yes

Edit the apertures? The *interactive* parameter must also be yes.

trace= yes

Trace the apertures?

fittrace= yes

Interactively fit the traced positions by a *parameter* must also be yes.

line = INDEF, nsum = 1

The dispersion line (line or column perpendicular to the dispersion axis) and number of adjacent lines (half before and half after unless at the end of the image) used in finding, recentering, resizing, and editing operations. For tracing this is the starting line and the same number of lines are summed at each tracing point. A line of INDEF selects the middle of the image along the dispersion axis. A positive nsum selects the number of lines to sum while a negative value selects a median. Tracing always uses a sum.

step = 10

Step along the dispersion axis between determination of the spectrum positions.

nlost = 3

Number of consecutive steps in which the profile is lost before quitting the tracing in one direction. To force tracing to continue through regions of very low signal this parameter can be made large. Note, however, that noise may drag the trace away before it recovers.

The following parameters are the defaults used to fit the traced positions by a function of the dispersion line. These parameters are those used by the ICFIT package.

function = "legendre"

Default trace fitting function. The fitting function types are "chebyshev" polynomial, "legendre" polynomial, "spline1" linear spline, and "spline3" cubic spline.

order = 2

Default trace function order. The order refers to the number of terms in the polynomial functions or the number of spline pieces in the spline functions.

sample = "*"

Default fitting sample. The sample is given by a ranges each separated by either whitespace or commas. The string "*" refers to all points.

naverage = 1

Default number of points to average or median. Positive numbers average that number of sequential points to form a fitting point. Negative numbers median that number, in absolute value, of sequential points. A value of 1 does no averaging and each data point is used in the

niterate = 0

Default number of rejection iterations. If greater than zero the fit is used to detect deviant traced positions and reject them before repeating the fit. The number of iterations of this process is given by this parameter.

low_reject = 3., high_reject = 3.

Default lower and upper rejection sigma. If greater than zero traced points deviating from the fit below and above the fit by more than this number of times the sigma of the residuals are rejected before refitting.

grow = 0.

Default reject growing radius. Traced points within a parameter of any rejected point are also rejected.

ADDITIONAL PARAMETERS

I/O parameters and the default dispersion axis are taken from the package parameters, the default aperture parameters from **apdefault**, automatic aperture finding parameters from **apfind**, recentering parameters from **aprecenter**, resizing parameters from **apresize**, and parameters used for centering and editing the apertures from **apedit**.

When this operation is performed from the task **apall** all parameters except the package parameters are included in that task.

DESCRIPTION

For each image in the input image list the position of the spectrum within each aperture are determined at a number of points along the dispersion axis and a smooth function is fit to these positions. The fitted curve defines a shift to be added to the aperture center at each wavelength. Other options allow defining apertures using a reference image, defining apertures through an automatic finding algorithm (see **apfind**), automatically recentering apertures (see **aprecenter**), automatically resizing apertures (see **apresize**), and interactively editing the apertures prior to tracing (see **apedit**). Tracing is selected with the parameter *trace*. If the tracing is done interactively (the *interactive* parameter set to yes) then the user is queried whether or not to trace each image. The responses are "yes", "no", "YES", or "NO", where the upper case queries suppress this query for the following images.

The tracing begins with the specified dispersion line. A dispersion line is a line or column of the image perpendicular to the dispersion axis. The dispersion axis is defined in the image header or by the package parameter *dispaxis*. If the starting dispersion line is INDEF then the middle dispersion line of the image is used. The positions of the spectra are determined using the **center1d** algorithm and the centering parameters from the **apedit** task. (See help under **center1d** for a description of the one dimensional position measuring algorithm.) The positions are redetermined at other points along the dispersion axis by stepping from the starting line in steps specified by the user. A number of dispersion lines around each dispersion line to be measured may be summed to improve the position determinations, particularly for weak profiles. This number usually is set equal to the tracing step.

It is important to understand how to set the step size and the relationship between the step size and the centering error radius. Larger steps reduce the computational time, which is an important consideration. However, if the step is too large then the tracing may fail to follow the systematic changes in the positions of the spectrum. The centering error radius, *radius*, is used to limit the maximum position change between two successive steps. If the positions of a spectrum changes by more than the specified amount or the data contrast falls below the *threshold* parameter then the position is marked as lost.

The centering radius should be large enough to follow changes in the spectrum positions from point to point but small enough to detect an error in the tracing by a sudden abrupt change in position, such as caused by crowding with other spectra or by the disappearance of the spectrum. The *nlost* parameter determines how many consecutive steps the position may fail to be found before tracing in that direction is stopped. If parameter is small the trace will stop quickly upon loss of the profile while if it is very large it will continue to try and recover the profile.

The parameter *threshold* checks for the vanishing of a requiring a minimum range in the data used for centering. If the tracing fails when the spectra are strong and well defined the problem is usually that the step size is too large and/or the centering error radius is too small.

The traced positions of a spectrum include some measurement variation from point to point. Since the actual position of the spectrum in the image should be a smooth curve, a function of the dispersion line is to the measured points. The fitted function is stored as part of the aperture description. It is an offset to be added to the aperture's center as a function of the dispersion line. Even if the fitting is done interactively plots of the trace and the fit are recorded in the plot file or device specified by the parameter *plotfile*.

Fitting the traced spectrum positions with a performed interactively when parameters *fittrace* and

interactive are yes. This allows changing the default fitting parameters. The function fitting is done with the interactive curve fitting tools described under the help topic **icfit**. There are two levels of queries when fitting the spectrum positions interactively; prompts for each image and prompts for each aperture in an image. These prompts may be answered individually with the lower case responses "yes" or "no" or answered for all further prompts with the responses "YES" or "NO". Responding with "yes" or "YES" to image prompt allows interactive fitting of the traced positions for the spectra. Prompts are then given for each aperture in the image. When an spectrum is not fit interactively the last set of fitting parameters are used (initially the default function and order given by the task parameters). Note that answering "YES" or "NO" to a aperture prompt applies to all further aperture in the current image only. Responding with "no" or "NO" to the image prompt fits the spectrum positions for all apertures in all images with the last set of fitting parameters.

The tracing may also be done from the interactive aperture editor with the 't' key. The aperture tracing algorithm may be selected from many of the tasks in the package with the *trace* parameter.

APTRACE DATABASE COEFFICIENTS

The path of an aperture is described by a function that gives an additive offset relative to the aperture center as stored under the database keyword center. The function is saved in the database as a coefficients. The section containing the coefficients starts with the keyword "curve" and the number of coefficients.

The first four coefficients define the type of function, the order or number of spline pieces, and the range of the independent variable (the line or column coordinate along the dispersion). The first coefficient is the function type code with values:

Code	Type
1	Chebyshev polynomial
2	Legendre polynomial
3	Cubic spline
4	Linear spline

The second coefficient is the order (actually the number of terms) of the polynomial or the number of pieces in the spline.

The next two coefficients are the range of the independent variable over which the function is defined. These values are used to normalize the input variable to the range -1 to 1 in the polynomial functions. If independent variable is x and the normalized variable is n, then

$$n = (2 * x - (xmax + xmin)) / (xmax - xmin)$$

where xmin and xmax are the two coefficients.

The spline functions divide the range into the specified number of pieces. A spline coordinate s and the nearest integer below s, denoted as j, are defined by

$$s = (x - xmin) / (xmax - xmin) * npieces$$

$$j = \text{integer part of } s$$

where npieces are the number of pieces.

The remaining coefficients are those for the appropriate function. The number of coefficients is either the same as the function order for the polynomials, npieces+1 for the linear spline, or npieces + 2 for the cubic spline.

1. Chebyshev Polynomial

The polynomial can be expressed as the sum

$$y = \text{sum from } i=1 \text{ to order } \{c_i * z_i\}$$

where the c_i are the coefficients and the z_i are defined interactively as:

$$\begin{aligned} z_1 &= 1 \\ z_2 &= n \\ z_i &= 2 * n * z_{i-1} - z_{i-2} \end{aligned}$$

2. Legendre Polynomial

The polynomial can be expressed as the sum

$$y = \text{sum from } i=1 \text{ to order } \{c_i * z_i\}$$

where the c_i are the coefficients and the z_i are defined interactively as:

$$\begin{aligned} z_1 &= 1 \\ z_2 &= n \\ z_i &= ((2*i-3) * n * z_{i-1} - (i-2) * z_{i-2}) / (i - 1) \end{aligned}$$

3. Linear Spline

The linear spline is evaluated as

$$y = c_j * a + c_{j+1} * b$$

where j is as defined earlier and a and b are fractional difference between s and the nearest integers above and below

$$\begin{aligned} a &= (j + 1) - s \\ b &= s - j \end{aligned}$$

4. Cubic Spline

The cubic spline is evaluated as

$$y = \text{sum from } i=0 \text{ to } 3 \{c_{i+j} * z_i\}$$

where j is as defined earlier. The term z_i are computed from a and b , as defined earlier, as follows

$$\begin{aligned} z_0 &= a^3 \\ z_1 &= 1 + 3 * a * (1 + a * b) \\ z_2 &= 1 + 3 * b * (1 + a * b) \\ z_3 &= b^3 \end{aligned}$$

EXAMPLES

REVISIONS

APTRACEV2.11

The "apertures" parameter can be used to select apertures for resizing, recentering, tracing, and extraction. This parameter name was previously used for selecting apertures in the recentering algorithm. The new parameter name for this is now "aprecenter".

SEE ALSO

apdefault, apfind, aprecenter, apresize, apedit, apall, center1d, icfit, gtools

NAME

calibrate -- Apply extinction corrections and flux calibrations

USAGE

calibrate input output [records]

PARAMETERS**input**

List of input spectra to be calibrated. When using record format extensions the root names are specified, otherwise full image names are used.

output

List of calibrated spectra. If no output list is specified or if output name is the same as the input name then the calibrated spectra replace the input spectra. When using record format extensions the output names consist of root names to which the appropriate record number extension is added. The record number extension will be the same as the input record number extension. The output spectra are coerced to have real datatype pixels regardless of the pixel type.

records(imred.irsandimred.iidsonly)

The set of record number extensions to be applied to each input and output root name when using record number extension format. The syntax consists of comma separated numbers or ranges of numbers. A range consists of two numbers separated by a hyphen. This parameter is not queried when record number formats are not used.

extinct= yes

Apply extinction correction if a spectrum has not been previously corrected? When applying an extinction correction, an extinction file is required.

flux = yes

Apply a flux calibration if a spectrum has not been previously calibrated? When applying a flux calibration, sensitivity spectra are required.

extinction= <no default>

Extinction file for the observation. Standard extinction files are available in the "onedstds\$" directory.

observatory =")_.observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to the package parameter of the same name. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters in the **observatory** task. See **observatory** for additional information.

ignoreaps= no

Ignore aperture numbers and apply a single flux calibration to all apertures? Normally multiaperture instruments have separate sensitivity functions for each aperture while long slit or Fabry-Perot data use a single sensitivity function where the apertures are to be ignored. The sensitivity spectra are obtained by adding the aperture number as an extension to the sensitivity spectrum root name. When apertures are ignored the specified sensitivity spectrum name is used without adding an extension and applied to all input apertures.

sensitivity = "sens"

The root name for the sensitivity spectra produced by **sensfunc**. Normally with multiaperture instruments, **sensfunc** will produce a spectrum appropriate to each aperture with an aperture number extension. If the apertures are ignored (*ignoreaps* = yes) then the sensitivity spectrum specified is used for all apertures and no aperture number is appended automatically.

fnu = no

The default calibration is into units of flux per unit wavelength (F-lambda). If *fnu* = yes then the calibrated spectrum will be in units of flux per unit frequency (F-nu).

airmass,exptime

If the airmass and exposure time are not in the header nor can they be determined from other keywords in the header then these query parameters are used to request the airmass and exposure time. The values are updated in the input and output images.

DESCRIPTION

The input spectra are corrected for extinction and calibrated to a scale using sensitivity spectra produced by the task **sensfunc**. One or both calibrations may be performed by selecting the appropriate parameter flags. It is an error if no calibration is specified. Normally the spectra should be extinction corrected if also flux calibrating. The image header keywords DC-FLAG (or the dispersion type field in the "multispec" world coordinate system), EX-FLAG, and CA-FLAG are checked for dispersion solution (required), previous extinction correction, and previous flux calibration. If previously calibrated the spectrum is skipped and a new output image is not created.

The input spectra are specified by a list of root names (when using record extension format) or full image names. The output calibrated spectra may replace the input spectra if no output spectra list is specified or if output name is the same as the input name. When using record number extensions the output spectra will have the same extensions applied to the root names as those used for the input spectra.

When applying an extinction correction the AIRMASS keyword is sought. If the keyword is not present then the airmass at the time defined by the other header keywords is computed using the latitude of the observatory and observation parameters in the image header. The observatory is first determined from the image under the keyword OBSERVAT. If absent the observatory specified by the task parameter "observatory" is used. See **observatory** for further details of the observatory database. If the air mass cannot be determined an error results. Currently a single airmass is used and no correction for changing extinction during the observation is made and adjustment to the middle of the exposure. The task **setairmass** provides a correction for the exposure time to compute an effective air mass. Running this task before calibration is recommended.

If the airmass is not in the header and cannot be computed then the user is queried for a value. The value entered is then recorded in both the input and output image headers. Also if the exposure time is not found then it is also queried and recorded in the image headers.

The extinction correction is given by the factor

$$10. ** (0.4 * \text{airmass} * \text{extinction})$$

where the extinction is the value interpolated from the specified extinction file for the wavelength of each pixel. After extinction correction the EX-FLAG is set to 0.

When applying a flux calibration the spectra are divided by the aperture sensitivity which is represented by a spectrum produced by the task **sensfunc**. The sensitivity spectrum is in units of:

$$2.5 * \text{Log}_{10} [\text{counts/sec/Ang} /$$

A new spectrum is created in "F-lambda" units - or "F-nu" units - ergs/cm2/sec/Hz. The sensitivity must span the range of wavelengths in the spectrum and interpolation is used if the wavelength coordinates are not identical. If some pixels in the spectrum being calibrated fall outside the wavelength range of the sensitivity function spectrum a warning message giving the number of pixels outside the range. In this case the sensitivity value for the nearest wavelength in the sensitivity function is used.

Multiaperture instruments typically have a separate aperture sensitivity function for each aperture. The appropriate sensitivity function for each input spectrum is selected based on the spectrum's aperture by appending this number to the root sensitivity function spectrum name. If the *ignoreaps* flag is set, however, the aperture number relation is ignored and the single sensitivity spectrum (without extension) is applied.

EXAMPLES

1. To flux calibrates a series of spectra replacing the input spectra by the calibrated spectra:

```
cl> calibrate nite1 ""
```

2. To only extinction correct echelle spectra:

```
cl> calibrate ccd*.ec.imh new//ccd*.ec.imh flux-
```

3. To flux calibrate a long slit spectrum:

```
cl> dispaxis = 2
cl> calibrate obj.imh fcobj.imh
```

REVISIONS

CALIBRATEV2.10.3

This task was revised to operate on 2D and 3D spatial spectra; i.e. long slit and Fabry-Perot data cubes. This task now includes the functionality previously found in **longslit.extinction** and **longslit.fluxcalib**.

A query for the airmass and exposure time is now made if the information is not in the header and cannot be computed from other header keywords.

CALIBRATEV2.10

This task was revised to operate on nonlinear dispersion corrected spectra and 3D images (the **apextract** "extras"). The aperture selection parameter was eliminated (since the header structure does not allow mixing calibrated and uncalibrated spectra) and the latitude parameter was replaced by the observatory parameter. The observatory mechanism insures that if the observatory latitude is needed for computing an airmass and the observatory is specified in the image header the correct calibration will be applied. The record format syntax is available in the **irs/iids** packages. The output spectra are coerced to have real pixel datatype.

SEE ALSO

setairmass, standard, sensfunc, observatory, continuum

NAME

darkcombine -- Combine and process dark count images

USAGE

darkcombine input

PARAMETERS**input**

List of dark count images to combine. The *ccdtype* parameter may be used to select the zero level images from a list containing all types of data.

output = "Dark"

Output dark count root image name.

combine = "average" (average|median)

Type of combining operation performed on the final set of pixels (after rejection). The choices are "average" or "median". The median uses the average of the two central values when the number of pixels is even.

reject = "minmax" (none|minmax|ccdclip|crreject|sigclip|avsigclip|pclip)

Type of rejection operation. See **combine** for details.

ccdtype = "dark"

CCD image type to combine. If no image type is given then all input images are combined.

process = yes

Process the input images before combining?

delete = no

Delete input images after combining? Only those images combined are deleted.

clobber = no

Clobber existing output images?

scale = "exposure" (none|mode|median|mean|exposure)

Multiplicative image scaling to be applied. The choices are none, scale by the mode, median, or mean of the specified statistics section, or scale by the exposure time given in the image header.

statsec = ""

Section of images to use in computing image statistics for scaling. If no section is given then the entire region of the image is sampled (for efficiency the images are sampled if they are big enough).

Algorithm Parameters**nlow = 0, nhigh = 1 (minmax)**

The number of low and high pixels to be rejected by the "minmax" algorithm.

nkeep = 1

The minimum number of pixels to retain or the maximum number to reject when using the clipping algorithms (ccdclip, crreject, sigclip, avsigclip, or pclip). When given as a positive value this is the minimum number to keep. When given as a negative value the absolute value is the maximum

number to reject. This is actually converted to a number to keep by adding it to the number of images.

mclip = yes (ccdclip, crreject, sigclip, avsigclip)

Use the median as the estimate for the true intensity rather than the average with high and low values excluded in the "ccdclip", "crreject", "sigclip", and "avsigclip" algorithms? The median is a better estimator in the presence of data which one wants to reject than the average. However, computing the median is slower than the average.

lsigma = 3., hsigma = 3. (ccdclip, crreject, sigclip, avsigclip, pclip)

Low and high sigma clipping factors for the "ccdclip", "crreject", "sigclip", "avsigclip", and "pclip" algorithms. They multiply a "sigma" factor produced by the algorithm to select a point below and above the average or median value for rejecting pixels. The lower sigma is ignored for the "crreject" algorithm.

rdnoise = "0.", gain = "1.", snoise = "0." (ccdclip, crreject)

CCD readout noise in electrons, gain in electrons/DN, and sensitivity noise as a fraction. These parameters are used with the "ccdclip" and "crreject" algorithms. The values may be either numeric or an image header keyword which contains the value.

pclip = -0.5 (pclip)

Percentile clipping algorithm parameter. If greater than one in absolute value then it specifies a number of pixels above or below the median to use for computing the clipping sigma. If less than one in absolute value then it specifies the fraction of the pixels above or below the median to use. A positive value selects a point above the median and a negative value selects a point below the median. The default of -0.5 selects approximately the quartile point. See **combine** for further details.

blank = 0.

Output value to be used when there are no pixels.

DESCRIPTION

The dark count images in the input image list are combined. The input images may be processed first if desired. The original images may be deleted automatically if desired. The output pixel datatype will be real.

This task is a script which applies **ccdproc** and **combine**. The parameters and combining algorithms are described in detail in the help for **combine**. This script has default parameters specifically set for dark count images and simplifies the combining parameters. There are other combining options not included in this task. For these additional features, such as thresholding, offsetting, masking, and projecting, use **combine**.

EXAMPLES

1. The image data contains four dark count images. To automatically select them and combine them as a background job using the default combining algorithm:

```
cl> darkcombine ccd*.imh&
```

SEE ALSO

ccdproc, combine

NAME

dispcor -- Dispersion correct and resample spectra

USAGE

dispcor input output [records]

PARAMETERS**input**

List of input spectra or root names to be dispersion corrected. These may be echelle or non-echelle spectra, the task will determine which from the database dispersion functions. When using the record number extension format, record number extensions will be appended to each root name in the list.

output

List of dispersion corrected output spectra or root names. When using the record number extension format, record number extensions will be appended to each root name in the list. The output extension will be the same as the input extension. If "no" output list is specified then the output spectrum will replace the input spectrum after dispersion correction.

records(imred.irsand imred.iidsonly)

List of records or ranges of records to be appended to the input and output root names when using record number extension format. The syntax of this list is comma separated record numbers or ranges of record numbers. A range consists of two numbers separated by a used if no record number extensions are desired. This is a positional query parameter only if the record format is specified.

linearize= yes

Interpolate the spectra to a linear dispersion sampling? If yes, the spectra will be interpolated to a linear or log linear sampling using the linear dispersion parameters specified by other parameters. If no, the nonlinear dispersion function(s) from the dispersion function database are assigned to the input image world coordinate system and the spectral data are not interpolated.

database ="database"

Database containing dispersion solutions created by **identify** or **ecidentify**. If the spectra have been previous dispersion corrected this parameter is ignored unless a new reference spectra are defined.

table = ""

Wavelength coordinate table or reference image. Elements in this optional table or reference image override the wavelength coordinates given below for specified apertures. See the DISCUSSION for additional information.

w1 = INDEF, w2 = INDEF, dw = INDEF, nw = INDEF

The starting wavelength, ending wavelength, wavelength interval per pixel, and the number of pixels in the output spectra. Any combination of these parameters may be used to restrict the wavelength coordinates of the output spectra. If two or more have the value INDEF then suitable defaults based on the number of input pixels and the wavelength range of the reference dispersion solutions are used. These defaults may either come from all spectra, all spectra of the same aperture, or individually for each spectrum depending on the values of the *global* and *samedisp* parameters. Note that these parameters are specified in linear units even if a logarithmic wavelength scale is selected. The conversion between linear and logarithmic intervals between pixels is given below. These values may be overridden for specified apertures by a wavelength table or reference image. Otherwise these values apply to all apertures.

log = no

Transform to linear logarithmic wavelength coordinates? Linear logarithmic wavelength coordinates have wavelength intervals which are constant in the logarithm (base 10) of the wavelength. Note that if conserving flux this will change the flux units to flux per log lambda interval. Note that if the input spectra are in log sampling then *log=no* will resample back to linear sampling and *log=yes* will resample keeping the output spectra in log sampling.

flux = yes

Conserve the total flux during interpolation rather than the flux density? If "no", the output spectrum is average of the input spectrum across each output wavelength coordinate. This conserves flux density. If "yes" the input spectrum is integrated over the extent of each output pixel. This conserves the total flux. Note that in this case units of the flux will change; for example rebinning to logarithmic wavelengths will produce flux per log lambda. For flux calibrated data you most likely would not want to conserve flux.

blank= 0.

Output value corresponding to points outside the range of the input data. In other words, the out of bounds value. This only has an effect when linearizing and the output spectral coordinates extend beyond the input spectral range.

samedisp= no

Use the same dispersion parameters for all apertures? If yes then all apertures in a single image will have the same dispersion parameters. If the *global* parameter is all selected then all spectra in all images will have the same dispersion parameters. This parameter would not normally be used with echelle spectra where each order has a different wavelength coverage.

global = no

Apply global wavelength defaults? Defaults for the INDEF wavelength coordinate parameters are determined if two or less of the wavelength parameters are specified. The defaults are based on the number of pixels and the wavelengths of the first and last pixel as given by the dispersion solution. If this parameter is "no" this is done independently for each input spectrum. If this parameter is "yes" then the maximum number of pixels and the minimum and maximum wavelengths of all the input spectra or those of the same aperture are used to provide defaults for the spectra. The parameter *samedisp* determines whether the global coordinates are over all spectra or only those with the same aperture number. The global option is used to have all the dispersion corrected spectra have the same wavelength coordinates without actually specifying the wavelength parameters.

ignoreaps= no

If a reference dispersion solution is not found for an aperture use the first reference dispersion solution and ignore the aperture number? If not ignoring the apertures all spectra must have a aperture for the dispersion solution and the task aborts if this is not the case. Ignoring the apertures avoids this abort and instead the first dispersion solution is used. Note this parameter does not mean ignore matches between reference and spectrum aperture numbers but only ignore the aperture number if no matching reference is found.

Also if a reference table or image is given and *ignoreaps=yes* then the default dispersion parameters for any aperture not defined by the table or image will be that of the first defined aperture. This can still be overridden by giving explicit values for *w1*, *w2*, *dw* and *nw*.

confirm= no

Confirm the wavelength parameters for each spectrum? If yes the wavelength parameters will be printed and the user will be asked whether to accept them. If the parameters are not acceptable the

user will be queried for new values. The confirmation and parameter changes are repeated until an acceptable set of parameters is obtained. When the *global* parameter is *yes* changes to the wavelength parameters will remain in effect until changed again.

listonly = no

List the dispersion coordinates only? If set then the dispersion coordinates are listed but the spectra are not dispersion corrected. This may be used to determine what the default wavelengths would be based on the dispersion solutions.

verbose = yes

Print the dispersion function and coordinate assignments?

logfile = ""

Log file for recording the dispersion correction operations. If no file name is given then no log information is recorded.

DESCRIPTION

The dispersion coordinate systems of the input spectra are set or changed in the output spectra. The output spectra may be the same as the input spectra if no output spectra are specified or the output name is the same as the input name. The input and output spectra are specified by image templates or lists. In the **irs/iids** packages the input and output spectra are specified as root names and the record numbers are specified by the *record* parameter. The records are given as a set of comma separate single numbers or ranges of hyphen separated numbers. If no records are specified then the input and output images are assumed to be full names.

The dispersion coordinate system is defined either in the image header or by dispersion functions in the specified database. To use reference spectra dispersion functions they must first be assigned to the image with **identify** (**reidentify**), **ecidentify** (**ecreidentify**), **refspectra**, or **hedit**. These tasks define the image header keywords REFSPEC1, REFSPEC2, REFSHFT1, and REFSHFT2. The test which determines whether to use the current dispersion coordinate system or reference spectra dispersion solutions is the presence of the REFSPEC1 keyword. Since it is an error to apply a dispersion function to data which have already been dispersion corrected the any dispersion function keywords are deleted after use and a record of them entered in sequential image header keywords beginning with DCLOG.

Dispersion functions are specified by one or both of the reference spectrum image header keywords REFSPEC1 and REFSPEC2 containing the name of calibration spectra with dispersion function solutions (either echelle dispersion functions from **ecidentify** or non-echelle dispersion functions from **identify**) in the database. There must be a dispersion function for each aperture in the input spectrum unless the *ignoreaps* flag is set. If the flag is not set the task will abort if a aperture is not found while if it is set spectra without a matching aperture in the reference dispersion solutions will use the first dispersion solution. Note that aperture number matching is done in both cases and the *ignoreaps* parameter only applies to non-matching spectra. The common situation for using the *ignoreaps* option is there is a single reference dispersion solution which is to be applied to a number of spectra with different aperture numbers; hence effectively ignoring the reference spectrum aperture number.

If two reference spectra are specified the names may be followed by a weighting factor (assumed to be 1 if missing). The wavelength of a pixel is then the weighted average of the wavelengths of the two dispersion functions. The task **refspectra** provides a number of ways to assign reference spectra. Note, however, that these assignments may be made directly using the task **hedit** or with some other task or script if none of the methods are suitable. Also note that **identify** and **reidentify** add the REFSPEC1 keyword referring to the image itself when a database entry is written.

In addition to the one or two reference dispersion functions for each input aperture there may also be image header keywords REFSHFT1 and REFSHFT2 specifying reference spectra whose dispersion function zero

point shifts (the "shift" parameter in the database files) are to be applied to the reference dispersion functions. The shifts from REFSHFT1 will be applied to the dispersion functions from REFSPEC1 and similarly for the second dispersion functions. The reference shifts need not be present for every aperture in a multispectrum image. By default the mean shift from all the reference apertures having a zero point shift is applied to all the reference dispersion functions. If the REFSHFT keyword has the modifier word "nearest" following the spectrum name then the shift from the nearest aperture in spatial position (from the aperture extraction limits in the original 2D spectrum as recorded in the 6th and 7th fields of the APNUM keywords) is used for a particular input aperture. If the modifier word is "interp" then the nearest two apertures are used to interpolate a zero point shift spatially.

The purpose of the reference shift keywords is to apply a point correction to the reference dispersion functions determined from separate arc calibration observations using a same time as object observations. For example, consider multifiber observations in which one or more fibers are assigned to arc lamps at the same time the other fibers are used to observe various objects. The basic dispersion reference, the REFSPEC keywords, will come from arc observations taken through all the fibers. The arc fibers used during an object observation are then calibrated against their corresponding fibers in the arc calibration observations to determine a zero point shift. The REFSHFT keywords will contain the name of the object spectrum itself and the shifts from the simultaneous arc fibers will be interpolated spatially to the nonarc object fibers and applied to the dispersion functions from the arc calibrations for those fibers.

The reference shift keywords are currently added with **hedit** and zero point shifts computed with **identify/reidentify**. The complexities of this have been hidden in the multifiber **imred** instrument reduction packages. The reference shift correction feature was added primarily for use in those reduction packages.

If the *linearize* parameter is no the dispersion functions, weights, and shifts are transferred from the database to the world coordinate system keywords in the image header. Except for printing processing information that is all that is done to the spectra.

If the *linearize* parameter is yes the spectra are interpolated to a linear wavelength scale and the dispersion coordinate system in the header is set appropriately. A linear wavelength coordinate system is defined by a starting wavelength, an ending wavelength, a wavelength interval per pixel, and the number of pixels. These four parameters actually overspecify the coordinate system and only three of these values are needed to define it. The output coordinate system is specified by giving a these parameters using the parameters *w1*, *w2*, *dw*, and *nw*.

When the *log* option is used these parameters are still specified and computed in non-log units but the effective interval per pixel is

$$\begin{aligned} dw_log &= (\log_{10}(w2) - \log_{10}(w1)) / (nw - 1) \\ dw_log &= (\log_{10}(w1+dw*(nw-1)) - \log_{10}(w1)) / (nw - 1) \end{aligned}$$

In other words, the logarithmic interval divides the starting and ending wavelength into the required number of pixels in log step. To avoid confusion in this case it is best to specify the starting and ending wavelengths (in non-log units) and the number of pixels.

Note that if *log*=yes the input spectra in either linear or log sampling will be resampled to produces an output spectrum in log sampling. Similarly, if *log*=no the input spectra will be resampled to linear sampling. This means that log sampled input spectra will be resampled to linear sampling.

Default values for any parameters which are not specified, by using the value INDEF, are supplied based on the wavelengths of the first and last pixel as given by the dispersion function and the number of pixels in the input image. The defaults may either be determined separately for each spectrum (*global* = no), from all spectra with the same aperture (*global* = yes and *samedisp* = no), or from all the spectra (*global* = yes and *samedisp* = yes). As indicated, the parameter *samedisp* determines whether defaults are determined independently for each aperture or set the same for all apertures.

Another way to specify the wavelengths when there are many apertures is use a wavelength table or reference image. If an spectrum image name is specified with the *table* parameter then the dispersion parameters for each apertures are set to be the same as the reference spectrum. Alternatively, a text file table consisting of lines containing an aperture number, the starting wavelength, the ending wavelength, the wavelength interval per pixel, and the number of output pixels may be specified. Any of these values may be specified as INDEF (though usually the aperture number is not). One way to view the wavelength table/reference spectrum is that an entry in the wavelength table/reference spectrum overrides the values of the parameters *w1*, *w2*, *dw*, and *nw*, which normally apply to all apertures, for the specified aperture. The wavelength table is used to specify explicit independent values for apertures. The global mechanism can supply independent values for the INDEF parameters when the *samedisp* parameter is no.

If one wishes to verify and possibly change the defaults assigned, either globally or individually, the *confirm* flag may be set. The user is asked whether to accept these values. By responding with no the user is given the chance to change each parameter value. Then the new parameters are printed and the user is again asked to confirm the parameters. This is repeated until the desired parameters are set. When the defaults are not global the changed parameters will not be used for the next spectrum. When the global option is used any changes made are retained (either for all apertures or independently for each aperture) until changed again.

When adjusting the wavelengths the user should specify which parameter is free to change by entering INDEF. If none of the parameters are specified as INDEF then those values which were not changed, i.e. by accepting the current value, are the first to be changed.

Once the wavelength scale has been defined the input spectrum is interpolated for each output pixel. Output wavelengths outside the range of the input spectrum are set to the value given by the *blank* parameter value. The default interpolation function is a 5th order polynomial. The choice of interpolation type is made with the package parameter "interp". It may be set to "nearest", "linear", "spline3", "poly5", or "sinc". Remember that this applies to all tasks which might need to interpolate spectra in the **onedspec** and associated packages. For a types see **onedspec**.

When it is desired to conserve total flux, particularly when the dispersion is significantly reduced, the parameter *flux* is set to yes and the output pixel value is obtained by integrating the interpolation function across the wavelength limits of the output pixel. If it is set to no then the flux density is conserved by averaging across the output pixel limits.

The input spectrum name, reference spectra, and the wavelength parameters will be printed on the standard output if the *verbose* parameter is set and printed to a log file if one is specified with the *logfile* parameter. If one wishes to only check what wavelengths will be determined for the defaults without actually dispersion correcting the spectra the *listonly* flag may be set.

Other tasks which may be used to change the dispersion coordinate system are **scopy**, **specshift**, and **sapertures**.

EXAMPLES

In the examples when the task is used in the IRS and IIDS packages, shown with the "ir>" prompt the spectra have a record number extension image name format and the records parameter must be specified. In the other case shown with the "on>" prompt the records parameter is not used.

1. Dispersion correct spectra so that they have the same number of pixels and the wavelengths limits are set by the reference spectra.

```
ir> dispcor spec dcspec 9,10,447-448
dcspec.0009: ap = 0, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
dcspec.0010: ap = 1, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
dcspec.0447: ap = 0, w1 = 5082.57, w2 = 6551.45, dw = 1.794, nw = 820
dcspec.0448: ap = 1, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
```

```
on> dispcor allspec.ms dcallspec.ms
dcallspec.ms: ap = 1, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
dcallspec.ms: ap = 2, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
dcallspec.ms: ap = 3, w1 = 5082.57, w2 = 6551.45, dw = 1.794, nw = 820
dcallspec.ms: ap = 4, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
```

2. Confirm and change assignments.

```
on> dispcor spec* %spec%new%* confirm+
new009: ap = 0, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
Change wavelength coordinate assignments? (yes):
Starting wavelength (5078.8421234): 5070
Ending wavelength (6550.535123):
Wavelength interval per pixel (1.79693812):
Number of output pixels (820): INDEF
new009: ap = 0, w1 = 5070., w2 = 6550.53, dw = 1.795, nw = 826
Change wavelength coordinate assignments? (yes): no
new010: ap = 1, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
Change wavelength coordinate assignments? (no): yes
Starting wavelength (5078.7071234): 5100
Ending wavelength (6550.805123): 6500
Wavelength interval per pixel (1.79987512): INDEF
Number of output pixels (820): INDEF
new010: ap = 1, w1 = 5100., w2 = 6500., dw = 1.797, nw = 780
Change wavelength coordinate assignments? (yes): no
new447: ap = 0, w1 = 5082.57, w2 = 6551.45, dw = 1.793, nw = 820
Change wavelength coordinate assignments? (yes): no
new448: ap = 1, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
Change wavelength coordinate assignments? (no):
```

3. Confirm global assignments and do dispersion correction in place. record format.

```
ir> dispcor irs "" 9,10,447,448 confirm+ global+ samedisp+
irs.0009: ap = 0, w1 = 5078.71, w2 = 6553.66, dw = 1.801, nw = 820
Change wavelength coordinate assignments? (yes):
Starting wavelength (5078.7071234): 5100
Ending wavelength (6553.664123): 6500
Wavelength interval per pixel (1.80092412):
Number of output pixels (820):
irs.0009: ap = 0, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (yes): no
irs.0010: ap = 1, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
irs.0447: ap = 0, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
irs.0448: ap = 1, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
```

4. Make a nonlinear dispersion correction in place.

```
on> dispcor spec* "" linearize=no verbose- logfile=logfile
```

5. Apply a single dispersion solution to a set of record number format images.

```
ir> dispcor nite101 dcnite101 "1-10" ignore+ confirm-
```

REVISIONS

DISPCORV2.12.3

Added the blank parameter value.

DISPCORV2.11.3

Long slit and data cubes can be used with this task to either resample using the existing WCS or to use a single dispersion function from IDENTIFY. It uses the first one found.

DISPCORV2.10.3

Provision was added for IDENTIFY dispersion solutions consisting of only a shift (as produced by the 'g' key in IDENTIFY or the refit=no flag in REIDENTIFY) to be applied to previously LINEARIZED spectra. Thus it is possible to use IDENTIFY/REIDENTIFY to automatically compute a zero point shift based on 1 or more lines and then shift all the spectra to that zero point.

DISPCOR will now allow multiple uses of IDENTIFY dispersion solutions in a simple way with but with continuing protection against accidental multiple uses of the same dispersion solutions. When a first dispersion corrected using one or more reference spectra keywords the dispersion flag is set and the reference spectra keywords are moved to DCLOGn keywords. If DISPCOR is called again without setting new reference spectra keywords then the spectra are resampled (rebinned) using the current coordinate system. If new reference spectra are set then DISPCOR will apply these new dispersion functions. Thus the user now explicitly enables multiple dispersion functions by adding reference spectra keywords and DISPCOR eliminates accidental multiple uses of the same dispersion function by renaming the reference spectra. The renamed keywords also provide a

The flux conservation option now computes an average across the output pixel rather than interpolating to the middle of the output pixel when *flux* is no. This preserves the flux density and includes all the data; i.e. a coarse resampling will not eliminate features which don't fall at the output pixel coordinates.

Some additional log and verbose output was added to better inform the user about what is done.

Better error information is now printed if a is not found.

DISPCORV2.10

This is a new version with many differences. It replaces the previous three tasks **dispcor**, **ecdispcor** and **msdispcor**. It applies both one dimensional and echelle dispersion functions. The new parameter *linearize* selects whether to interpolate the spectra to a linear dispersion (the only option available previously) or to assign a nonlinear dispersion function to the image without any interpolation. The interpolation function parameter has been eliminated and the package parameter *interp* is used to select the interpolation function. The new interpolation type "sinc" may be used but care should be exercised. The new task supports applying a secondary zero point shift spectrum to a master dispersion function and a spatial interpolation of the shifts when calibration spectra are taken at the same time on a different region of the same 2D image. The optional wavelength table may now also be an image to match dispersion parameters. The *apertures* and *rebin* parameters have been eliminated. If an input spectrum has been previously dispersion corrected it will be resampled as desired. Verbose and log file parameters have been added to log the dispersion operations as desired. The record format syntax is available in the **irs/iids** packages.

SEE ALSO

package, refspectra, scopy, specshift, sapertures

NAME

help -- print online documentation for the named modules or packages

USAGE

help [template]

PARAMETERS**template**

A string listing the modules or packages for which help is desired. Each list element may be a simple name or a pattern matching template. Abbreviations are permitted. If *template* is omitted a menu will be printed for the current package, listing each task (or subpackage) and describing briefly what it is.

file_template= no

If this switch is set the template is interpreted as a template, and all help blocks found in the named files are output. The help database is not searched, hence manual pages can be printed or documents may be formatted without entering the files into a help database. In other words, "help file.hlp fi+" makes it possible to use *help* as a conventional text formatter.

all = no

Print help for all help modules matching *template*, rather than only the first one found.

parameter = "all"

If the value of this parameter is not "all", only the help text for the given parameter will be printed.

section = "all"

If the value of this parameter is not "all", only the help text for the given section (e.g. "usage", "description", "examples") will be printed.

option = help

The option parameter specifies the type of help desired, chosen from the following:

help Print the full help block for the named module.

source

Print the source code for the module (which often contains additional detailed comments).

sysdoc

Print the technical system documentation for the named module.

directory

Print a directory of all help blocks available for the named package.

alldoc

Print all help blocks in the file containing the help block for the named procedure (i.e., both the user and system documentation).

files

Print the names of all help files associated with the named modules or packages.

summary

Print only the titles and sizes of help blocks in referenced help files. The contents of the blocks are skipped. Titles are printed for *all* help blocks found in the file containing the help block for the named module.

page = yes

Pause after every page of output text. Turning this off for large documents speeds up output

considerably.

nlpp = 59

The number of lines per page if output is redirected, e.g., to *lprint*.

lmargin = 1

Left margin on output.

rmargin = 72

Right margin on output.

search = no

If enabled the template is interpreted as a search string and the task is started with the search panel open with the results of the search. The *file_template* parameter is ignored with search turned on.

home = ""

The home page for the task. If not set and no template is specified the task will start with the online help in the main window, otherwise it may be set to a filename to be displayed when the task starts. This file may contain a text help block which will be formatted before display, or it may be a valid HTML file. See below for a description of the format of a homepage file which provides links to tasks.

printer = "printer"

Default hardcopy printer name. If the *value* of the parameter is reserved string "printer", the actual device is the value of the CL environment variable *printer*.

showtype = no

Add task-type suffix in package menus?

quickref = "uparm\$quick.ref"

Name of the quick-reference file used for searching. This file is the first time the task is run in GUI mode or whenever it doesn't exist, or when any help database file has been updated.

uifname = "lib\$scr/help.gui"

The user interface file. This file is what defines the look and behavior of all the graphical user interface elements. Experts may create variants of this file.

helpdb = "helpdb"

The filename of the help database to be searched. If the *value* of the parameter is the reserved string "helpdb", the actual filename is the value of the CL environment variable *helpdb*.

device = "terminal"

Output device if the standard output is not redirected. Allowable values include:

terminal

If the *value* of the parameter is the reserved string "terminal", the actual device name is the value of the CL environment variable *terminal*.

text Output the formatted help page as plain text.

gui Invoke the GUI for browsing the help system. This option will only work if the *stdgraph* environment variable is set the *xgterm*, and the user is running IRAF from an *XGterm* window.

html Output the formatted help page as HTML text.

ps (or postscript)

Output the formatted help page as postscript.

BASIC USAGE

Despite the complex appearing hidden parameters, **help** is easy to use for simple tasks. **Help** is most commonly used to get help on the current package, and to get help on a program named in a the current package one need only type **help** without any arguments. For example, if the current package is **plot**, the command and its output might appear as follows:

```
pl> help
      contour - Make a contour plot of an image
      graph   - Graph one or more image sections or lists
      pcol    - Plot a column of an image
      pcols   - Plot the average of a range of image columns
      prow    - Plot a line (row) of an image
      pros    - Plot the average of a range of image lines
      surface - Make a surface plot of an image
pl>
```

To get help on a module one supplies the module name as an argument,

```
pl> help graph
```

and the manual page for the **plot.graph** program will be printed on the terminal. To get a hardcopy of the manual page on the printer, the output may be redirected to the line printer, as follows:

```
pl> help graph | lprint
```

DESCRIPTION

The function of the **help** program is to perform a of the help database *helpdb*, printing help for all packages and modules matching the template. By default the standard IRAF help database is but any other help database may be searched if desired. A help database is precompiled with the **mkhelpdb** program to speed up runtime searches for help modules. The standard IRAF help database contains the documentation and source for all CL programs and system and math library procedures installed in IRAF.

A help template is a string type parameter to the CL. The form of a is a list of patterns delimited by commas, i.e.,

```
"pattern1, pattern2, ..., patternN"
```

The form of a pattern is

```
package_pattern.module_pattern
```

If the "." is omitted *module_pattern* is assumed. The standard pattern matching meta-characters, i.e., `"*?["]`, are permitted in patterns. Simple patterns are assumed to be abbreviations.

GUI OPERATION

The GUI component of the task is a front-end to the IRAF **help** task which provides on-the-fly conversion of help documents to HTML for presentation in the GUI or formatted PostScript for hardcopy. The GUI is started by setting the *device* parameter to the special value *gui*, it is only available when using an XGterm window to start IRAF and assuming the *stdgraph* environment variable is set to *xgterm*.

Help pages may be loaded on the command line, through use of a file browser, or by navigating the help databases using a familiar CL package menu scheme. It also features a search capability similar to the **references** task and a complete history mechanism.

When invoked with no command line arguments the task starts as a `task` and the user is presented with a GUI that has the toplevel CL package menu in the upper navigation window. The main display window below will contain any help page specified in the template parameter or loaded on the command line by specifying the template and file_template parameters. If the search parameter is enabled the template is taken to be a search phrase and the database is searched for tasks matching the keyword and the GUI will appear with the search panel mapped so the user can select the task help to view. When no template is given the main display window will start with the page specified by the home parameter, this can be a user-defined HTML file giving links to specific tasks (see below for details) or if home is empty the display will contain the online help for the task.

The first time the task is run, or whenever the help database is updated, a quick reference file (specified by the task quickref parameter) and package menu file will be created in the user's *uparm* directory to speed up help searching and subsequent startups of the task.

NAVIGATING THE HELP SYSTEM

When run as a GUI browser *HELP* works very much like any WWW browser. The top panel is a list widget that will always contain a CL package listing, at startup this will be the toplevel "Home" package menu one would see when first logging into the CL containing the core system packages, NOAO package, and any site-specific external package, or in the case of starting with a specific task it will be the parent package for the task. Additionally, system documents for the **os** HSI routines and the **imfort** and **math** interfaces will be available in the *Home* package although these are programmatic interfaces and not tasks which can be executed.

New packages or task help pages are loaded by selecting an item from the package menu list using the left mouse button. If the requested item is a package, the menu listing will change as though the package were loaded in the CL, and the help display panel will contain a list of tasks with a one-line description for each task such as would be seen with a "help <package>" command using the standard task. New items may then be selected using either the menu list or links in the display panel. If the item is a task, the help page for the task will appear in the display panel. In either case new pages may be selected from the menu listing.

Specific help documents may also be requested by entering the task/package name in the **Topic** text widget above the menu list. As when selecting from the package menu list, items selected this way will cause the menu list to change to the package menu for the parent package if task (displaying the help page in the display panel) or the package menu if the item is a package (displaying the one-liner package listing in the display panel).

Using the **Back** button will revert to the previous page in the history list which will either be the previously loaded package or help page. Similarly, selecting the **Forward** button will move the next page further down in the history list, either button will become insensitive when the end of the list on either end is reached. Selecting the **Up** button will cause the browser to immediately jump up the previous package, skipping over any help pages that were loaded in between. The **Home** button will cause the default homepage (either the user-defined page if specified by the task *home* parameter or the online help) to be displayed. Browsing in this way can also be done using the navigation menu created by hitting the right mouse button while in the main display panel.

Users can also jump to specific pages in the history list using the **History** button on the main menubar. The right column of the menu will indicate whether the item is a task, package, internal link or a file. The history list is truncated at about 40 entries in the menu but the user may work back incrementally by selecting the last item of the menu, after which the History button will display the previous 40 entries. The history list may be cleared except for the current page by selecting the *Clear History* menu item.

BROWSING A HELP DOCUMENT

Once a help page is loaded the middle menubar above the display panel will change to activate widgets based on the position within the history list and options available for a particular page. The left-most group of buttons are the standard navigation buttons described above. The middle group of buttons contains the **Sections** and **Parameters** buttons which are used to browse within a . The *Sections* button is a menu listing all of the sections found within a help page, allowing the user to jump to a rather than scrolling through the entire document. The *Sections* menu is also available using the middle mouse button from the main display area. The *Parameters* button is similarly a menu listing of all task parameter help sections found within the document. Both or either of these buttons will become insensitive when no section or parameter information is found in the document.

The right-most group of buttons represent the various help options available for each page. The default is to get the task help, however help pages may have an associated **source** file or **sysdoc** (e.g. if a CL script there may be a pointer to the script source itself, or a package may have a general overview document listed as the system document). Once a help page is loaded these buttons will change become sensitive if is available, simply select the button to view the option. Selecting the **Files** button will bring up a panel listing all the files associated with a particular help topic. When a help topic is selected and an option is defined but the file does not exist, the options button will display a yellow diamond icon even if the button is insensitive, a green icon indicates the currently selected option. This feature may be disabled by selecting the "Show missing files" item from the main menubar **Options** menu.

SEARCHING

Searching the help database is done by selecting the **Search** button from the main menubar to bring up the search panel. Users may then enter one or more keywords into the **Topic** field at the bottom of the panel and initiate the search with either a carriage return or hitting the *Search* button just beside it. The panel will then show a list of all tasks and packages which match the search phrase along with a description of the task. Help pages may be displayed by selecting either the task or package link with the left mouse button, in both case the package menu list on the main help window will be updated to list the package contents allowing other tasks from that package to be selected in the normal way.

By default the exact phrase entered in the topic window will be used for the search. This can be relaxed by toggling the "Require exact match" button at the top of the panel. For example, to search for all tasks matching *either* the keyword "flat" or "field" turn off the exact match toggle and the search will return not only tasks matching "flat field" but also any task description containing only one of the words such as the VELVECT task which plots velocity *fields*.

Within a help document itself one can search for a the **Find** button from the main menubar to bring up a enter the search string. When the text is entered the main display window will reposition itself and highlight the text found within the document. Searches can be repeated and will wrap around the document automatically, searches can be done either forward or backward through the text and may be case insensitive.

USER_DEFINED HOME PAGES

By default the *help* GUI will start with the online help page displayed in the main help window. The user can change this by setting the task **home** parameter to be a path to any valid file. This file may be plain text, a help document in LROFF format which will be converted to HTML for display, or a native HTML document.

HTML files may contain URLs of the form

```
<a href=[package.]task>url_text</a>
```

where *url_text* is the text to appear in the window and the URL itself consists of an optional package and task name delimited by a period. For example, to create a link to the **splot** task in a document one would

use the URL

```
<a href=onedspec.splot>splot</a>
```

In this way users can create a homepage which serves as a file or index of shortcuts to the most commonly accessed help pages.

LOADING FILES

Text files may be loaded on the command line when starting the task by specifying the filename and setting the `file_template` task parameter. The named file will be searched for a `.help` LROFF directing indicating it a help block that will be converted to HTML for display. If no help block is found the file will be displayed as-is, meaning existing HTML documents can be loaded and will be formatted correctly.

Once the task is running users may load a file by selecting the **Open File...** menu item from the main menubar **File** menu or the right-mouse-button menu from within the main display area. This will open a file browser allowing users to change directories by using the navigation buttons at the top of the panel, or selecting items from the leftmost directory listing. Selecting a file on the rightmost list will cause it to be loaded and automatically formatted if it contains a help block. The file list may be filtered to select only those files matching a particular template by changing the **Filter** box at the top of the panel. Filenames or directories may be entered directly using the **Selection** box at the bottom of the panel.

SAVING FILES

Once a file has been loaded in the browser it may be saved to disk as either *source* (i.e. the original LROFF file if that was converted for the display, or whatever file is currently displayed regardless of format), *text* to save formatted plain text such as that produced by the standard **help** task, *HTML* to save the converted HTML used in the display, or *PostScript* to save formatted PostScript of the document such as that sent to the printer using the **Print** button. Not all options will be available depending on the format of the input text, unavailable options will be insensitive in the GUI.

The **Save** panel is opened by selecting the **Save As...** menu item from the main menubar **File** menu or the right-mouse-button menu from within the main display area. The file browser operates the same as when loading images, the only difference is that file selection simply defines the filename to be used and does not cause the save to occur automatically. Users can overwrite existing files by selecting the *Options* toggle at the bottom of the panel.

HARDCOPY OUTPUT AND SAVING DISK FILES.

Help pages may be output to any configured IRAF printer by selecting the main menubar **Print** button to bring up the print panel. Task help pages will be converted to formatted PostScript and may be sent to either a printer or saved to disk depending on the selection made in the printer panel. If the printer name is set to the special value "printer" then the device named by the CL *printer* environment variable will be used. When saving to disk files the default action is to save to a name is the task name plus a ".ps" extension. Either of these are changeable within the GUI as is the default page size to be used when generating the PostScript.

The main menubar **File** button can also be used to bring up the file browser in order to save the current document to disk. Help pages may be saved as either the origin LROFF source for the file, formatted text as you would get from the standard help task, HTML as is displayed in the GUI, or formatted PostScript. The choice of formats is dictated by the type of file being displayed (e.g. you cannot save PostScript of a

LROFF DIRECTIVE EXTENSIONS FOR HTML

To better support HTML links within documents and to other help pages two new directives have been added to the LROFF text formatter. These are **.hr** to specify a link (an HTML *HREF* directive) and **.hn** to specify a name (an HTML *NAME* directive). The syntax for these are as follows:

```
.hn <name>
.hr <link> <text>
```

where *<name>* is the destination name of an internal link, *<link>* is the URL of the link to be created, and *<text>* is the text to be displayed in the HTML. The URL syntax is either a by a destination name, a simple *task* name or *package* name, or a *package.task* pair giving a more precise task. For internal links the current document is repositioned so the name is at the top of the display, for task help links new help pages will be loaded in the browser.

These directives are ignored when converting the LROFF to either formatted plain text or PostScript.

GUI EXAMPLES

- 1) Start *help* as a GUI browser:

```
cl> help dev=gui
```

- 2) Begin by searching for the phrase 'gauss', tasks and packages may be selected from the search panel which will appear when the task starts:

```
cl> help gauss dev=gui search+
```

- 3) Load an LROFF help page in the browser at startup

```
cl> help mytask.hlp dev=gui file+
```

EXAMPLES

1. Print the help text for the program *delete* in the package *system* (output will be directed to the terminal):

```
cl> help system.delete
or
cl> help delete
or
cl> help del
```

2. Print the help text on the line printer:

```
cl> help delete | lprint
```

3. Print help for the current package:

```
cl> help
```

4. Print the usage section of all modules in the package **images**:

```
cl> help images.* section=usage
```

5. Print a directory of all help blocks in the packages **clpackage** and **clio** (and any others whose names begin with the string "cl"):

```
cl> help cl* op=dir
```

6. Print a directory of each package in the database (useful for getting an overview of the contents of a help database):

```
cl> help * op=dir
```

7. Print the source for all of the string utilities in the system library package **fmtio**:

```
cl> help fmtio.str* op=source
```

8. Find all tasks that delete something:

```
cl> help * | match delete
```

9. Print the manual pages for the *help* and *lprint* tasks on the default printer device:

```
cl> help help, lprint | lpr
```

10. Capture the manual page for task *hedit* in a suitable for printing on any device.

```
cl> help hedit dev=text > hedit.txt
```

11. Print the manual page for task *hedit* as a

```
cl> help hedit dev=ps | lprint
```

BUGS

On some systems, typing the next command keystroke before the end-of-page prompt is printed may result in the character being echoed (messing up the output) and then ignored when raw mode is enabled for the prompt.

SEE ALSO

references , phelp , mkhelpdb , hdbexamine , mkmanpage , lroff , the online task help documents.

NAME

imarith -- binary image arithmetic

USAGE

imarith operand1 op operand2 result

PARAMETERS**operand1,operand2**

Lists of images and constants to be used as operands. Image templates and image sections are allowed.

op Operator to be applied to the operands. The allowed operators are "+", "-", "*", "/", "min", and "max".

result

List of resultant images.

title = ""

Title for the resultant images. If null("") then the title is from operand1 if operand1 is an image or from operand2 otherwise.

divzero = 0.

Replacement value for division by zero. When the denominator is zero or nearly zero the result is replaced by this value.

hparams = ""

List of header parameters to be operated upon. This is primarily used for adding exposure times when adding images.

pixtype = "", calctype = ""

Pixel datatype for the resultant image and the internal calculation datatype. The choices are given below. They may be abbreviated to one character.

""

Calctype defaults to the highest precedence operand datatype. If highest precedence datatype is an integer type and the operation is division then the calculation type will be "real". If the highest precedence operand is type "ushort", *calctype* will default to "long". *Pixtype* defaults to *calctype*. Users who want type "ushort" images on output will need to set *pixtype* to "ushort" explicitly.

"1", "2"

The pixel datatype of the first or second operand.

"short", "ushort", "integer", "long", "real", "double"

Allowed IRAF pixel datatypes.

verbose = no

Print the operator, operands, calculation datatype, and the resultant image name, title, and pixel datatype.

noact = no

Like the verbose option but the operations are not actually performed.

DESCRIPTION

Binary image arithmetic is performed of the form:

operand1 op operand2 = result

where the operators are addition, subtraction, multiplication, division, and minimum and maximum. The division operator checks for nearly zero denominators and replaces the ratio by the value specified by the parameter *divzero*. The operands are lists of images and numerical constants and the result is a list of images. The number of elements in an operand list must either be one or equal the number of elements in the resultant list. If the number of elements is one then it is used for each resultant image. If the number is equal to the number of resultant images then the elements in the operand list are matched with the elements in the resultant list. The only limitation on the combination of images and constants in the operand lists is both operands for a given resultant image may not be constants. The resultant images may have the same name as one of the operand images in which case a temporary image is created and after the operation is successfully completed the image to be replaced is overwritten by the temporary image.

If both operands are images the lengths of each axis for the common dimensions must be the same though the dimensions need not be the same. The resultant image header will be a with the greater dimension. If the dimensions are the same then image header for the resultant image is copied from operand1. The title of the resultant image may be changed using the parameter *title*. The pixel datatype for the resultant image may be set using the parameter *pixtype*. If no pixel datatype is specified then the pixel datatype defaults to the calculation datatype given by the parameter *calctype*. The calculation datatype defaults to the highest precedence datatype of the operand images or constants except that a division operation will default to real for integer images. The precedence of the datatypes, highest first, is double, real, long, integer, and short. The datatype of a constant operand is either short integer or real. A real constant has a

Arithmetic on images of unequal dimensions implies that the operation is repeated for each element of the higher dimensions. For example subtracting a two dimensional image from a three dimensional image consists of subtracting the two dimensional image from each band of the three dimensional image. This works for any combination of image dimensions. As an extreme example dividing a by a one dimension image consists of dividing each line of each plane of each band ... by the one dimensional image.

There are two points to emphasize when using images of unequal dimensions. First, a one dimensional image operates on a line of a two or higher dimension image. To apply a to the columns of a higher dimensional image increase the image dimensionality with **imstack**, transpose the resultant image, and then replicate the columns with **blkrep** (see the EXAMPLE section). The second point of confusion is that an image with a size given by **imheader** of [20,1] is a while an image with size of [20] is a one dimensional image. To reduce the dimensionality of an image use **imcopy**.

In addition to operating on the image pixels the image header parameters specified by the list *hparams* are also operated upon. The operation is the same as performed on the pixels and the values are either the values associated with named header parameters or the operand constant values. The primary purpose of this feature is to add exposure times when adding images.

The verbose option is used to record the image arithmetic. The output consists of the operator, the operand image names, the resultant image name and pixel datatype, and the calculation datatype.

EXAMPLES

1. To add two images and the exposure times:

```
cl> imarith ccd1 + ccd2 sum
>>> hparams="itime,otime,ttime,exposure"
```

2. To subtract a constant from an image and replace input image by the subtracted image:

```
cl> imarith m31 - 223.2 m31
```

Note that the final pixel datatype and the calculation datatype will be at least of type real because the constant operand is real.

3. To scale two exposures, divide one by the other, and extract the central portion:

```
cl> imarith exp1[10:90,10:90] * 1.2 temp1
cl> imarith exp2[10:90,10:90] * 0.9 temp2
cl> imarith temp1 / temp2 final title='Ratio of exp1 and exp 2'
cl> imdelete temp1,temp2
```

Note that in this example the images temp1, temp2, and final will be of real pixel datatype (or double if either exp1 or exp2 are of pixel datatype double) because the numerical constants are real numbers.

4. To divide two images of arbitrary pixel datatype using real arithmetic and create a short pixel datatype resultant image:

```
cl> imarith image1 / image2 image3 pixtype=short \
>>> calctype=real title="Ratio of image1 and image2"
```

5. To divide several images by calibration image using the image pixel type of the numerator images to determine the pixel type of the calibrated images and the calculation arithmetic type:

```
cl> imarith image1,image2,image3 / calibration \
>>> image1a,image2a,image3a pixtype=1 calctype=1
```

The same operation can be done in place with image template expansion by:

```
cl> imarith image* / calibration image* pixtype=1 calctype=1
```

6. To subtract a two dimensional bias from stacked observations (multiple two dimensional observations stacked to form a three dimensional image):

```
cl> imarith obs* - bias obs*//b
```

Note that the output observations obs101b, ..., will be three dimensional.

7. To divide a 50 x 50 image by the average column:

```
cl> blkavg img avcol 50 1
cl> blkrep avcol avcol 50 1
cl> imarith img / avcol flat
```

8. To subtract a one dimensional image from the lines of a image:

```
cl> imarith im2d - im1d diff
```

9. To subtract a one dimensional image from the columns of a image:

```
cl> imstack im1d imcol
cl> imtranspose imcol imcol
cl> blkrep imcol imcol 100 1
cl> imarith im2d - imcol diff
```

Note the need to make a two dimensional image with each column replicated since a one dimensional image will operate on the lines of a two dimensional image.

SEE ALSO

blkrep, imdivide, imfunction, imstack, imtranspose

NAME

imcopy -- copy images

USAGE

imcopy input output

PARAMETERS**input**

Images to be copied.

output

Output images or directory.

verbose = yes

Print each operation as it takes place?

DESCRIPTION

Each of the input images, which may be given as a including sections, is copied to the corresponding output image list, which may also be given as an image template, or the output directory. If the output is a list of images then the number of input images must be equal to the number of output images and the input and output images are paired in order. If the output image name exists and contains a input image (provided it is the same size as the section) will be copied into that section of the input image. If the output image name does not have a section specification and if it is the same as the input image name then the input image is copied to a temporary file which replaces the input image when the copy is successfully concluded. Note that these are the only cases where clobber checking is bypassed; that is, if an output image name is not equal to the input image name or a subsection of an existing image and the file already exists then a clobber error will occur if clobber checking is in effect.

The verbose options prints for each copy lines of the form:

input image -> output image

EXAMPLES

1. For a simple copy of an image:

```
cl> imcopy image imagecopy
```

2. To copy a portion of an image:

```
cl> imcopy image[10:20,*] subimage
```

3. To copy several images:

```
cl> imcopy image1,image2,frame10 a,b,c
```

4. To trim an image:

```
cl> imcopy image[10:20,*] image
```

In the above example the specified section of the input image replaces the original input image. To trim several images using an image template:

```
cl> imcopy frame*[1:512,1:512] frame*
```

In this example all images beginning with "frame" are trimmed to 512 x

5. To copy a set of images to a new directory:

```
cl> imcopy image* directory
```

or

```
cl> imcopy image* directory$
```

or

```
cl> imcopy image* osdirectory
```

where "osdirectory" is an operating system directory name (i.e. /user/me in UNIX).

6. To copy a section of an image in an already existing image of sufficient size to contain the input section.

```
cl> imcopy image[1:512,1:512] outimage[257:768,257:768]
```

BUGS

The distinction between copying to a section of an existing image and overwriting a input image is rather inobvious.

NAME

instruments -- Instrument specific data files

DESCRIPTION

The **ccdred** package has been designed to accommodate many different instruments, detectors, and observatories. This is done by having instrument specific data files. Note that by instrument we mean a combination of detector, instrument, application, and observatory, so there might be several "instruments" associated with a particular CCD detector. Creating and maintaining the instrument files is generally the responsibility of the support staff, though the user may create or copy and modify his/her own instrument/application specific files. The task **setinstrument** makes this information available to the user and package easily.

There are three instrument data files, all of which are optional. The package may be used without the instrument files but much of the convenience of the package, particularly with respect to using the CCD image types, will be lost. The three files are an instrument image header translation file, an initialization task which mainly sets default task parameters, and a bad pixel file identifying the cosmic bad pixels in the detector. These files are generally stored in a system data directory which is a subdirectory of the logical directory "ccddb\$". Each file has a root name which identifies the instrument.

1. Instrument Translation File

The instrument translation file translates the parameter names used by the **ccdred** package into instrument specific parameters and also supplies instrument specific default values. The package parameter *ccdred.instrument* specifies this file to the package. The task **setinstrument** sets this parameter, though it can be set explicitly like any other parameter. For the standard instrument translation file the root name is the instrument identification and the extension is "dat" ("*.dat" files are protected from being removed in a "stripped" system, i.e. when all nonessential files are removed). Private instrument files may be given any name desired.

The instrument translation proceeds as follows. When a parameter for an image, for example "imagetyp", it looks in the instrument translation file. If the file is not found or none is specified then the image header keyword that is requested has the same name. If an instrument translation file is defined then the requested parameter is translated to an image header keyword, provided an entry is given. If no translation is given the package name is used. For example the package parameter "imagetyp" might be translated to "data-typ" (the old NOAO CCD keyword). If the parameter is not found then the default value specified in the translation file, if present, is returned. For recording parameter information in the header, such as processing flags, the translation is also used. The default value has no meaning in this case. For example, if the flag specifying that the image has been corrected by a flat field is to be set then the package parameter name "flatcor" might be translated to "ff-flag". If no translation is given then the new image header parameter is entered as "flatcor".

The format of the translation file are lines consisting of the package parameter name, followed by the image header keyword, followed by the default value. The first two fields are parameter names. The fields are separated by whitespace (blanks and tabs). String default values containing blanks must be quoted. An example is given below.

```
# Sample translation file.
exptime      itime
darktime     itime
imagetyp     data-typ
subset       flpos
biassec      biassec      [411:431,2:573]
datasec      datasec      [14:385,2:573]

fixpix       bp-flag      0
```

```

overscan    bt-flag    0
zerocor     bi-flag    0
darkcor     dk-flag    0
flatcor     ff-flag    0
fringcor    fr-flag    0

```

The first comment line is ignored as are blank lines. The first two lines translate the CCD image type, and the subset parameter without default values (see **ccdtypes** and **subsets** for more information). The next two lines give the overscan bias strip section and the data section with default values for the instrument. Note that these parameters may be overridden in the task **ccdproc**.

The next set of translations requires further discussion. For processing flags the package assumes that the absence of a keyword means that the processing has not been done. If processing is always to be done with the **CCDRED** package and no processing keywords are recorded in the raw data then these parameters should be absent (unless you don't like the names used by the package). However, for compatibility with the original NOAO CCD images, which may be processed outside of IRAF and which use 0 no processing value, the processing flags are translated and the false values are indicated by the default values.

If there is more than one translation for the same CCDRED parameter, for example more than one `exptime`, then the last one is used.

In addition to the parameter name translations the translation file contains translations between the value of the image type parameter and the image types used by the package. These lines consist of the image header type string as the first field (with quotes if there are blanks) and the image type as recognized by the package. The following example will make this clearer.

```

'OBJECT (0)'      object
'DARK (1)'        dark
'PROJECTOR FLAT (2)' flat
'SKY FLAT (3)'    other
'COMPARISON LAMP (4)' other
'BIAS (5)'        zero
'DOME FLAT (6)'   flat

```

The values of the image type strings in the header contain blanks so they are quoted. Also the case of the strings is important. Note that there are two types of flat field images and three types of object images.

The CCD image types recognized by the package are:

```

zero   - zero level image such as a bias or preflash
dark   - dark count image
flat   - flat field image
illum  - illumination image such as a sky image
fringe - fringe correction image
object - object image

```

There may be more than one image type that maps to the same package type. In particular other standard CCD image types, such as comparison spectra, multiple exposure, standard star, etc., should be mapped to object or other. There may also be more than one type of flat field, i.e. dome flat, sky flat, and lamp flat. For more on the CCD image types see **ccdtypes**.

The complete set of package parameters are given below. The package parameter names are generally the same as the standard image header keywords being adopted by NOAO.

General Image Header and Default Parameters			
ccdmean	darktime	exptime	fixfile
imagetyp	ncombine	biassec	subset
title	datasec	nscanrow	

CCDRED Processing Flags

ccdproc	darkcor	fixpix	flatcor
fringcor	illumcor	overscan	trim
zerocor			
CCDRED CCD Image Types			
dark	flat	fringe	illum
none	object	unknown	zero

The translation mechanism described here may become more sophisticated in the future and a general IRAF system facility may be implemented eventually. For the present the translation mechanism is quite simple.

2. Instrument Setup Script

The task **setinstrument** translates an instrument ID into a CL script in the instrument directory. This script is then executed. Generally this script simply sets the task parameters for an instrument/application. However, it could do anything else the support staff desires. Below are the first few lines of a script.

```
ccdred.instrument = "ccddb$kpno/example.dat"
ccdred.pixeltype = "real"
ccdproc.fixpix = yes
ccdproc.overscan = yes
ccdproc.trim = yes
ccdproc.zerocor = no
ccdproc.darkcor = no
ccdproc.flatcor = yes
ccdproc.biassec = "[411:431,2:573]"
ccdproc.datasec = "[14:385,2:573]"
```

The instrument parameter should always be set unless there is no translation file for the instrument. The **ccdproc** parameters illustrate setting the appropriate processing flags for the instrument. The overscan bias and trim data sections show an alternate method of setting these instrument specific parameters. They may be set in the setup script in which case they are given explicitly in the user parameter list for **ccdproc**. If the value is "image" then the parameters may be determined either through the default value in the instrument translation file, as illustrated in the previous section, or from the image header itself.

The instrument setup script for setting default task parameters may be easily created by the support person as follows. Set the package parameters using **eparam** or with CL statements. Setting the parameters might involve testing. When satisfied with the way the package is set then the parameters may be dumped to a using the task **dparam**. The final step is editing this script to delete unimportant and query parameters. For example,

```
cl> dparam ccdred >> file.cl
cl> dparam ccdproc >> file.cl
cl> dparam combine >> file.cl
...
cl> ed file.cl
```

3. Instrument Bad Pixel File

The bad pixel file describes the bad pixels, columns, and lines in the detector which are to be replaced by interpolation when processing the images. This file is clearly detector specific. The file consists of lines describing rectangular regions of the image. The regions are specified by four numbers giving the starting and ending columns followed by the starting and ending lines. The starting and ending points may be the same to specify a single column or line. The example below illustrates a bad pixel file.

```
# RCA1 CCD untrimmed
25 25 1 512
108 108 1 512
302 302 403 512
```

```
1 512 70 70
245 246 312 315
```

If there is a comment line in the file containing the word "untrimmed" then the coordinates of the bad pixel regions apply to the original CCD detector coordinates. If the image has been trimmed and the bad pixels are replaced at a later stage then this word indicates that the trim region be determined from the image header and the necessary coordinate conversion made to the original CCD pixel coordinates. Note that if a subraster readout is used the coordinates must still refer to the original CCD coordinates and not the raw, untrimmed readout image. If the word "untrimmed" does not appear then the coordinates are assumed to apply to the image directly; i.e. the trimmed coordinates if the image has been trimmed or the original coordinates if the image has not been trimmed. The standard bad pixel files should always refer to the original, untrimmed coordinates.

The first two bad pixel regions are complete bad columns (the image is 512 x 512), the next line is a partial bad column, the next line is a bad line, and the last line is a small bad region. These files are easy to create, provided you have a good image to work from and a way to measure the positions with an image or graphics display.

SEE ALSO

ccdtypes, subsets, setinstrument

NAME

lcalib -- List information about the spectral calibration data

USAGE

lcalib option star_name

PARAMETERS**option**

Chooses calibration data to be listed. Option may be: "bands" to list the bandpasses at each wavelength, "ext" to list the extinction at each wavelength, "mags", "fnu", or "flam" to list the magnitude, or flux of the star (selected by the star_name parameter) at each wavelength, or "stars" to list the star names available in the calibration directory.

star_name

Selects which star's magnitude list is chosen if the option parameter is "mags", "fnu", "flam", or "bands". Also if '?' a list of available stars in the specified calibration directory is given.

The following three queried parameters apply if the selected calibration file is for a blackbody. See **standard** for further details.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

extinction

Extinction file. The current standard extinction files:

```
onedstds$kpnoextinct.dat - KPNO standard extinction
onedstds$ctioextinct.dat - CTIO standard extinction
```

caldir

Calibration directory containing standard star data. The directory name must end with /. The current calibration directories available in the onedstds\$ may be listed with the command:

```
cl> page onedstds$README
```

fnuzero= 3.68e-20

The absolute flux per unit frequency at a magnitude of zero. This is to convert the calibration

magnitudes to absolute flux by the formula

$$\text{Flux} = \text{fnuzero} * 10. ** (-0.4 * \text{magnitude})$$

The flux units are also determined by this parameter. However, the frequency to wavelength interval conversion assumes frequency in hertz. The default value is based on a calibration of Vega at 5556 Angstroms of $3.52\text{e-}20$ ergs/cm²/s/hz for a magnitude of 0.048. This default value is that used in earlier versions of this task which did not allow the user to change this calibration.

DESCRIPTION

LCALIB provides a means of checking the flux calibration data. The calibration data consists of extinction, bandpasses, and stellar magnitudes.

The extinction is given in an extinction file consisting of lines with wavelength and extinction. The wavelengths must be order in increasing wavelength and the wavelengths must be in Angstroms. There are two standard extinction files currently available, "onedstds\$kpnoextinct.dat", and "onedstds\$ctioextinct.dat".

The standard star data are in files in a calibration directory specified with the parameter *caldir*. A standard star file is selected by taking the star name given, by the parameter *star_name*, removing blanks, +’s and -’s, appending ".dat", and converting to lower case. This file name is appended to the specified calibration directory. A calibration file consists of lines containing a stellar magnitude, and a bandpass full width. The wavelengths are in Angstroms. Comment lines beginning with # may be included in the file. The star names printed by this task are just the first line of each file in the calibration directory with the first character (#) removed. The calibration files may be typed, copied, and printed. **Lcalib** may also be used to list data from the calibration files.

EXAMPLES

```
# List the extinction table
cl> lcalib ext
# Plot the extinction table
cl> lcalib ext | graph
# Plot the energy distribution
cl> lcalib mags "bd+28 4211" | graph
# List the names of all the stars
cl> lcalib stars caldir=onedstds$irscal/
# As above but for IIDS file
cl> lcalib stars calib_file=onedstds$iidscal/
```

REVISIONS

LCALIBV2.10

This task has a more compact listing for the "stars" option and allows paging a list of stars when the star name query is not recognized.

SEE ALSO

standard, sensfunc, onedstds\$README

NAME

mk1dspec -- Make/add artificial 1D spectra

USAGE

mk1dspec input

PARAMETERS**input**

Spectra to create or modify.

output = ""

Output spectra when modifying input spectra. If no output spectra are given then existing spectra in the input list are modified directly. If an output list is given then it must match in number the input list.

ap = 1

Image line to be created or modified in images of dimension greater than 1.

rv = 0.

Radial velocity (km/s) or redshift, as selected by the parameter *z*, applied to line positions and continuum. Velocities are converted to redshift using the relativistic relation $1+z = \sqrt{(1+rv/c)/(1-rv/c)}$. Note the shift is not a shift in the dispersion parameters but in the underlying artificial spectrum.

z = no

Is the velocity parameter a radial velocity or a

WHEN CREATING NEW SPECTRA**title = ""**

Image title to be given to the spectra. Maximum of 79 characters.

ncols = 512

Number of columns.

naps = 1

Number of lines or apertures.

header = "artdata\$stdheader.dat"

Image or header keyword data file. If an image is given then the image header is copied. If a file is given then the FITS format cards are copied. This only applies to new images. The data file consists of lines in FITS format with leading whitespace ignored. A FITS card must begin with an uppercase/numeric keyword. Lines not beginning with a FITS keyword such as comments or lower case are ignored. The user keyword output of **imheader** is an acceptable data file. See **mkheader** for further information.

wstart= 4000., wend = 8000.

Starting and ending wavelengths in Angstroms. The dispersion is determined by these values and the number of columns.

CONTINUUM PARAMETERS

continuum= 1000., slope = 0.

Continuum of the starting wavelength at rest and the slope of the continuum.

temperature= 5700.

Blackbody continuum temperature in Kelvin. A value of 0 no blackbody continuum is desired. The intensity level is set by scaling to the continuum level of the starting wavelength at rest.

fnu = no

Compute the continuum as flux per unit frequency (F-nu) if yes or flux per unit wavelength (F-lambda) if no.

LINE PARAMETERS

lines = ""

List of spectral line files. Spectral line files contain lines of rest wavelength, peak, profile type, and widths (see the DESCRIPTION section). The latter parameters may be missing or INDEF in which case they default to the task *peak*, *profile*, *gfhwm*, and *lfwhm* parameters (note that the *peak* parameter is not a constant but the random number scaling). If no file or a new (nonexistent) file is specified then a number of random lines given by the parameter *nlines* is generated. If a new file name is specified then the lines generated are recorded in the file. If the list of spectral line files is shorter than the list of input spectra, the last spectral line list file is reused.

nlines = 0

If no spectral line file or a new file is specified then the task will generate this number of random spectral lines. The rest wavelengths are uniformly random within the limits of the spectrum, the peaks are uniformly random between zero and the value of the *peak* parameter, the profile type is given by *profile*, and the widths are fixed at the values of the *gfhwm* and *lfwhm* parameters. If a rest wavelengths are shifted and repeated periodically.

profile = "gaussian" (gaussian|lorentzian|voigt)

The default profile type for random lines or when not specified in the spectral line file. The profile types are:

```
gaussian - Gaussian profile
lorentzian - Lorentzian profile
voigt - Voigt profile
```

peak = -0.5

The maximum spectral line peak value when generating random lines or when the peak is missing from the spectral line file. This value is relative to the continuum unless the continuum is zero. Negative values are absorption lines and positive values are emission lines.

gfhwm = 20., lfwhm = 20.

The default gaussian and lorentzian full widths at half maximum (FWHM), in Angstroms, used when generating random lines or when the widths are missing from the spectral line file.

seed = 1

Random number seed. If a value of "INDEF" is given then the clock time (integer seconds since 1980) is used as the seed yielding different random numbers for each execution.

comments= yes

Include comments recording task parameters in the image header?

PACKAGE PARAMETERS

nxsub = 10

Number of pixel subsamples used in computing the gaussian spectral line profiles.

dynrange= 100000.

The gaussian line profiles extend to infinity so a of the peak intensity to the cutoff intensity, is imposed to cutoff the profiles.

DESCRIPTION

This task creates or modifies one dimensional spectra. with a of blackbody and linear sloped continuum and emission and absorption spectral lines. The spectral lines may be gaussian, lorentzian, or voigt profiles. A velocity shift may be applied to the underlying artificial spectrum which is shifted into the specified observed wavelength region. No noise is included but may be added with the task **mknoise**. New spectra are created with the specified number of pixels, wavelength range, and real datatype. When *nlines* is greater than 1 then an image with the specified number of lines is created though only the line given by the *ap* is will have a spectrum. Existing spectra may be modified in place or new spectra output. Spectra are modified by adding the continuum and lines defined by the parameters.

For new images a set of header keywords may be added by specifying an image or data file with the *header* parameter (see also **mkheader**). If a data file is specified lines beginning with FITS keywords are entered in the image header. Leading whitespace is ignored and any lines beginning with words having lowercase and nonvalid FITS keyword characters are ignored. In addition to this optional header, parameters for the wavelength coordinates are defined. Finally, comments may be added to the image header recording the task parameters and any information from the line file which are not line definitions.

Initially all spectra are created without a pixel coordinates. For multiple spectra in an image this task must be executed for each image line to set the dispersion function and add data. When an image line is selected if it has a defined dispersion function that is used otherwise the task wavelength parameters are used.

A continuum is defined by the value at the starting wavelength at rest, a slope, and a blackbody function of a given temperature. The blackbody function is scaled to have the specified continuum value at the starting wavelength at rest. The blackbody flux units are per unit wavelength (F-lambda). A zero continuum value or a zero temperature will not produce a blackbody continuum.

Spectral lines are modeled by gaussian, lorentzian, or voigt profiles of specified wavelength, peak, and widths. The lines are defined in a spectral line file or generated randomly. A spectral line file consists of text lines giving rest wavelength, peak, profile type, gaussian full width at half maximum and/or lorentzian full width at half maximum. Only the wavelength is required and subsequent fields may be missing or given as INDEF. The following table shows the possible formats where wavelength, peak, gfwhm, and lfwhm are values of wavelength, peak, gaussian FWHM, and lorentzian FWHM. The profile types are as shown though they may be abbreviated to one character.

```
wavelength
wavelength peak
wavelength peak gaussian
wavelength peak gaussian gfwhm
wavelength peak gaussian gfwhm
wavelength peak lorentzian
wavelength peak lorentzian lfwhm
wavelength peak lorentzian lfwhm
wavelength peak voigt
wavelength peak voigt gfwhm
wavelength peak voigt gfwhm lfwhm
wavelength peak voigt gfwhm lfwhm
```

When a field is missing or INDEF the values given by the parameters *peak*, *profile*, *gfwhm*, and *lfwhm* are used. If a peak value is missing, random values between zero and the *peak* value are generated. Note that to get random line intensities with some specified profile type and widths the value INDEF would be used for the peak field.

If no spectral line file is specified or a new (nonexistent) file is then the number of random lines given by the parameter *nlines* is generated. The rest wavelengths are uniformly random within the wavelength range of the spectrum and extend periodically outside this range in the case of an applied velocity shift, the peaks are uniformly random between zero and the *peak* parameter, and the profile type and widths are given by the *profile*, *gfwhm*, and *lfwhm* parameters. If a new file is named then the parameters of the generated lines will be output.

The peak values are taken relative to a positive continuum. In other words the generated line profile is multiplied by the continuum (with a minimum of zero for fully saturated absorption lines). If the continuum is less than or equal to zero, as in the case of an artificial arc spectrum or pure emission line spectrum, then the peak values are absolute intensities. Positive peak values produce emission lines and negative values produce absorption lines. Odd results will occur if the continuum has both positive and zero or negative values.

The underlying rest spectrum may be shifted. This is used primarily for testing radial velocity measuring algorithms and is not intended as a complete model of redshift effects. The starting and ending wavelengths are not changed by redshifting; these are the instrumental observed wavelengths. Input line wavelengths are specified at rest and then shifted into or out of the final spectrum. To be realistic the line list should include wavelengths over a great enough range to cover all desired redshifts. The peaks and widths are also appropriately modified by a redshift. As an example, if the redshift is 1 lines will appear broader by a factor of 2 and the peaks will be down by a factor of 2 in order to maintain the same flux.

The random line generation is difficult in that one wants to have the same set of lines (for a given seed) observed at different redshifts. What is done is that the specified number of random lines is generated within the observed wavelength interval taken at rest. This set is then repeated periodical over all wavelengths. A redshift will then shift these rest lines in to or out of the observed spectrum. If lines are output, they are given at rest. **Note that this periodicity may be important in interpreting cross correlation redshift tests for large shifts between template and object spectra.**

The definitions of the continuum are also affected by a The reference point for the continuum level, slope, and blackbody continuum is the starting wavelength taken at rest. Shifts will then modify the continuum level at the first pixel appropriately. In particular a large redshift will shift the blackbody in such a the flux is still given by the *continuum* parameter at the starting wavelength at rest.

EXAMPLES

1. Create a simple blackbody continuum between the default wavelengths.

```
cl> mk1dspec bb title=Blackbody
```

2. Create a random absorption spectrum on a saving the line list.

```
cl> mk1dspec bbab title=Absorption nlines=100
```

3. Create a random absorption spectrum with noise and cosmic rays.

```
cl> mk1dspec bbab title=Absorption nlines=100
cl> mknoise bbab rdnoise=10 poisson+ ncoss=5 energy=1000
```

4. Create a random emission spectrum on a blackbody continuum and save the line list.

```
cl> mk1dspec bbem title=Emission nl=30 peak=0.6 lines=bbem.dat
```

5. Create an artificial random arc line spectrum.

```
cl> mk1dspec arc title="Arc lines" cont=0 peak=500 nl=30
```

6. Create a test spectrum with a line list.

```
cl> type linelist
4100 -.1 g 20
4200 -.2 g 20
4300 -.3 g 20
5100 -.9 g 2
5200 -.9 g 4
5300 -.9 g 8
6700 .9 g 8
6800 .9 g 2
6900 .9 g 4
7700 .3 g 20
7800 .2 g 20
7900 .1 g 20
cl> mk1dspec testspec title=Test cont=500 temp=0 lines=linelist
```

7. Add absorption lines to a spectrum.

```
cl> mk1dspec bb out=artspec cont=0 lines=STDIN
4300 -60
5000 -200
[EOF]
```

Normally the input spectrum would be a real spectrum.

8. Make two spectra taken from the same set of random lines but differing in redshift.

```
cl> mk1dspec restspect nl=30
cl> mk1dspec redspec rv=3000 nl=30
cl> mk1dspec bluespec rv=-.01 z+ nl=30
```

9. Make a multispec image with 5 apertures and a

```
cl> mk1dspec spec.ms ap=1 nl=30 rv=0 naps=5
cl> mk1dspec spec.ms ap=2 nl=30 rv=1000
cl> mk1dspec spec.ms ap=3 nl=30 rv=2000
cl> mk1dspec spec.ms ap=4 nl=30 rv=3000
cl> mk1dspec spec.ms ap=5 nl=30 rv=4000
```

REVISIONS

MK1DSPECV2.11+

The random number seed can be set from the clock time by using the value "INDEF" to yield different random numbers for each execution.

MK1DSPECV2.11

Lorentzian and Voigt profiles were added and the parameters and input line list format were changed. The widths are now FWHM instead of gaussian sigmas.

MK1DSPECV2.10.3

The format parameter was eliminated and the task updated to produce the current coordinate system format.

SEE ALSO

mknoise, mk2dspec, mkheader, onedspec.sinterp

NAME

nhedit - edit or view an image header interactively or using a

USAGE

```
nhedit images fields value comment
```

PARAMETERS**images**

Template specifying the images to be edited.

fields

Template specifying the fields to be edited in each image. The template is expanded independently for each image against the set of all fields in the image header. Special values for fields includes 'default_pars' that works only with a command file; 'add_blank' to add a blank field value with a value; 'add_textf' to add a text file content to the header. See description for more details.

value

Either a string constant or a general expression (if the first character is a left parenthesis) to be evaluated to compute the new value of each field. With the rename switch the value is the new field name (keyword). A single expression is used for all fields. The special value "." causes the value of each field to be printed rather than edited.

comment

String constant for the comment section of the header card. This value will replace the existing comment of a header or clear it if is empty (""). The special value "." causes the field to be printed rather than edited.

comfile = ""

Alternate command file. If specified, the *fields*, *value*, and *comment* parameters are ignored and commands are taken from the named file. See below for a detailed discussion and examples.

after = ""

Insert the new field after the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

before = ""

Insert the new field before the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

add= no

Change the operation of the editor from update to add new field. If field already exists it is edited. If this option is selected the field list may name only a single field. The add switch takes precedence over the addonly, delete, and rename switches.

addonly= no

Change the operation of the editor from update to add a field already exists it is not changed. If this option is selected the field list may name only a single field. The addonly switch takes precedence over the delete and rename switches.

delete = no

Change the operation of the editor from update to delete field. The listed fields are deleted from each image. This takes precedence over the rename switch.

rename= no

Change the operation of the editor from update field to rename field. The listed fields are renamed in each image if they exist. The value parameter specifies the new keyword name. There is no error if the field does not exist. The comment value is ignored since this operation only affects the field name.

verify = yes

Interactively verify all operations which modify the image database. The editor will describe the operation to be performed, prompting with the new value of the parameter in the case of a return or "yes" to complete the operation, or enter a new value explicitly as a string. Respond with "no" if you do not wish to change the value of the parameter.

show = yes

Print a record of each operation which modifies the database upon the standard output. Old values are given as well as new values, making it possible to undo an edit operation.

update= yes

Enable updating of the image database. If updating is disabled the edit operations are performed in memory but image headers will not be updated on disk.

DESCRIPTION**1. Basic Usage**

The most basic functions of the image header editor are modification and inspection of the fields of an image header. Both the "standard" and "user" fields may be edited in the same fashion, although not all standard fields are writable. For example, to change the value of the standard field "title" of the image "m74" to "sky flat" and enter a comment field we would enter the following command.

```
cl> nhedit m74 title "sky flat" "comment field"
```

If *verify* mode is selected the editor will print the old value of the field and query with the new value, allowing some other value to be entered instead, e.g.:

```
cl> nhedit m74 title "sky flat" "comment field"
m74,i_title ("old title" -> "sky flat"):
```

To accept the new value shown to the right of the arrow, type carriage return or "yes" or "y" followed by carriage return. To continue without changing the value of the field in question enter "no" or "n" followed by carriage return. To enter some other value merely type in the new value. If the new value is one of the reserved strings, e.g., "yes" or "no", enter it preceded by a backslash. If verification is enabled you will also be asked if you want to update the header, once all header fields have been edited. This is your last chance to change your mind before the header is modified on disk. If you respond negatively the image header will not be updated, and editing will continue with the next image. If the response is "q" the editor will exit entirely.

To conveniently print the value of the field "title" without modifying the image header, we repeat the command with the special value "." and "." for the comment portion.

```
cl> nhedit m74 title . .
```


To print (or edit) the values of all header fields a given.

```
cl> nhedit m74 * . .
```

To print (or edit) the values of only a few fields the field template may be given as a list.

```
cl> nhedit m74 w0,wpc . .
```

To print the value of one or more fields in a may be given. Both image templates and field templates may be given if desired.

```
cl> nhedit n1.* exp . .
```

Abbreviations are not permitted for field names, i.e., the given template must match the full field name. Currently, field name matches are case insensitive since image headers are often converted to and from FITS headers, which are case insensitive.

2. Advanced Usage

The header editor is capable of performing global edits on entire image databases wherein the new value of each field is computed automatically at edit time and may depend on the values of other fields in the image header. Editing may be performed in either batch or interactive mode. An audit trail may be maintained (via the *show* switch and i/o redirection), permitting restoration of the database in the event of an error. Trial runs may be made with updating disabled, before committing to an actual edit which modifies the database.

The major editing functions of the *nhedit* task are the following:

update	modify the value of a field or fields
addonly	add a new field
add	add a new field or modify an old one
delete	delete a set of fields
rename	rename a set of fields

In addition, *nhedit* may be used merely to inspect the values of the header fields, without modification of the image database.

2.1 Special header fields

add_blank	Add blank keyword field with optional comment
ex: nhedit add_blank " this is a comment with no kw"	
add_textf	Add the content of a text file into the header
ex: nhedit add_textf "my_text.txt" add+	

All keyword addition can be inserted after or before an existent keyword; use the 'after' and 'before' parameter.

2.2 Input commands from a command file.

All header editing command can be put together in a

```
nhedit file*.fits comfile=command_file.txt
```

2.3 Standard header fields

The header editor may be used to access both the standard image header fields and any user or application defined fields. The standard header fields currently defined are shown below. There is no guarantee that the names and/or usage of these fields will not change in the future.

i_ctime	int	create time
i_history	string	history comments

i_lmtime	int	time when min,max last updated
i_maxpixval	real	maximum pixel value
i_minpixval	real	minimum pixel value
i_mtime	int	time of last modify
i_naxis	int	number of axes (dimensionality)
i_naxis[1-7]	int	length of each axis
i_pixfile	string	pathname of pixel storage file
i_pixtype	int	pixel datatype code
i_title	string	title string

The standard header field names have an "i_" prefix to reduce the possibility of a name collision with a user field name, and to distinguish the two classes of parameters in templates. The prefix may be omitted provided the simple name is unique.

2.4 Field name template

The form of the field name list or template parameter *fields* is equivalent to that of a filename template except that "@listfile" is not supported, and of course the template is expanded upon the field name list of an image, rather than upon a directory. Abbreviations are not permitted in field names and case is not significant. Case is ignored in this context due to the present internal storage format for the user parameters (FITS), which also limits the length of a user field name to 8

2.5 Value expression

The *value* parameter is a string type parameter. If the first character in the string is a left parenthesis the string is interpreted as an algebraic expression wherein the operands may be constants, image header variables (field names), special variables (defined below), or calls to intrinsic functions. The expression syntax is equivalent to that used in the CL and SPP languages. If the value string is not parenthesized it assumed to be a string constant. The *value* string will often contain blanks, quotes, parenthesis, etc., and hence must usually be quoted to avoid interpretation by the CL rather than by the header editor.

For example, the command

```
cl> nhedit m74 title "title // ';'ss'" "."
```

would change the title to the literal string constant "title // " whereas the command

```
cl> nhedit m74 title "(title // ';'ss')"
```

would concatenate the string ";ss" to the old title string. We require parenthesis for expression evaluation to avoid the need to doubly quote simple string constant values, which would be even more confusing for the user than using parenthesis. For example, if expressions did not have to be parenthesized, the first example in the basic usage section would have to be entered as shown below.

```
cl> nhedit m74 title '"sky flat"' # invalid command
```

Expression evaluation for *nhedit*, *hselect*, and similar tasks is carried out internally by the FMTIO library routine **evexpr**. For completeness minimal documentation is given here, but the documentation for *evexpr* itself should be consulted if additional detail is required or if problems occur.

2.5.1 operators

The following operators are recognized in value expressions. With the exception of the operators "?", "=?", and "@", the operator set is equivalent to that available in the CL and SPP languages.

+	-	*	/	arithmetic operators
**				exponentiation
//				string concatenation
!	-			boolean not, unary negation
<	<=	>	>=	order comparison (works for strings)
==	!=	&&		equals, not equals, and, or

<code>?=</code>	string equals pattern
<code>? :</code>	conditional expression
<code>@</code>	reference a variable

The operators "=", "&&", and "||" may be abbreviated as "=", "&", and "|" if desired. The `?=` operator performs pattern matching upon strings. For example, the boolean expression shown below will be true whenever the field "title" contains the substring "sky".

```
(title ?= '*sky*')
```

The conditional expression operator '`??`', which is patterned after a `?:` operator in C, is used to make IF ELSE like decisions within an expression. The syntax is as follows:

```
<bool_expr> '??' <true_expr> ':' <>false_expr>
```

e.g., the expression

```
((a > b) ? 1 : 0)
```

has the value 1 if A is greater than B, and 0 otherwise. The datatypes of the true and false expressions need not be the same, unlike a compiled language. Note that if the parenthesis are omitted ambiguous forms of the expression are possible, e.g.:

```
(a > b) ? 1 : a + 1
```

could be interpreted either as

```
((a > b) ? 1 : a) + 1 or as (a > b) ? 1 : (a + 1)
```

If the parenthesis are omitted the latter interpretation is assumed.

The operator `@` must be used to dereference variables that have names with funny (nonalphanumeric) characters in them, forcing the variable name to be given as a string constant. For example, the value of the expression

```
@"co-flag"
```

is the value of the variable "co-flag". If the variable were referenced directly by name the "-" would be interpreted as the subtraction operator, causing an unknown variable reference (e.g., to "co"). The operand following the `@` may be any string valued expression. The `@` operator is right associative, hence the construct "`@@param`" is the value of the parameter named by the value of the parameter "param".

An expression may contain operands of datatypes bool, int, real, and string. Mixed mode expressions are permitted with automatic type coercion. Most type coercions from boolean or string to other datatypes are illegal. The boolean constants "yes" and "no" are predefined and may be used within expressions.

2.5.2 intrinsic functions

A number of standard intrinsic functions are recognized within expressions. The set of functions currently supported is shown below.

<code>abs</code>	<code>acos</code>	<code>asin</code>	<code>atan</code>	<code>atan2</code>	<code>bool</code>	<code>cos</code>
<code>exp</code>	<code>int</code>	<code>log</code>	<code>log10</code>	<code>max</code>	<code>min</code>	<code>mod</code>
<code>nint</code>	<code>real</code>	<code>sin</code>	<code>sqrt</code>	<code>str</code>	<code>tan</code>	

The trigonometric functions operate in units of degrees rather than radians. The *min* and *max* functions may have any number of arguments up to a maximum of sixteen or so (configurable). The arguments need not all be of the same datatype.

A function call may take either of the following forms:

```

    <identifier> '(' arglist ')'
or
    <string_expr> '(' arglist ')'

```

The first form is the conventional form found in all programming languages. The second permits the generation of function names by string valued expressions and might be useful on rare occasions.

2.5.3 special operands

As noted earlier, expression operands may be constants, variables (header fields), function calls, or references to any of the special variables. The following special variables are recognized within expressions:

.	A string constant, used to flag printing
\$	The value of the "current field"
\$F	The name of the "current field"
\$I	The name of the "current image"
\$T	The current clock time (an integer value)

These builtin variables are especially useful for constructing context dependent expressions. For example, the value of a field may be incremented by 100 by assigning it the value "\$ + 100".

EXAMPLES

1. Globally edit the database "n1", setting the value of the string parameter "obs" to "sky" if "s-flag" is 1, to "obj" otherwise.

```
cl> nhedit n1.* obs '(@"s-flag" == 1 ? "sky" : "obj")' "Observation value"
```

2. Globally edit the same database, replacing the value of the parameter "variance" by the square root of the original value.

```
cl> nhedit n1.* var '(sqrt(var))' "Variance value"
```

3. Replace the values of the fields A and B by the absolute value of the original value:

```
cl> nhedit n1.* a,b '(abs($))' 'Absolute value'
```

4. Add a blank field with a comment after a

```
cl> nhedit file.fits add_blank "INSTRUMENT DESCRIPTION " after=k5dx add+
```

Notice the use of the special field value 'add_blank' which will be replaced by a blank keyword in the header.

5. Add HISTORY card before a given keyword

```
cl> nhedit file.fits history \
    "History text from column 9 to 80, no quotes" before=wcsdim add+
```

6. Run a command file through the first 50 extensions

```
cl> for(i=1;i<51;i=i+1) {
    nhedit("mymef["//i//"]",comfile="home$hh.in")
}
```

7. Add a text file to the header. This will be put as HISTORY lines with text appropriately split when long lines are encountered. Start putting the text after the keyword KEYWN.

```
cl> nhedit add_textf "mytext_file.tx" after=KEYWN add+
```

8. Run nhedit through all the extensions in a

```
cl> for(i=1;i<7;i=i+1)
    nhedit("mymef.fits["//i//"]",comfi="home$myheader.txt")
```

9. Run several fits files with the same set of header commands from the file "hdrc.txt".

```
cl> nhedit file*.fits commfile=hdrc.txt
```

As an example the 'hdrc.txt' content can be: (Notice the 'default_pars' command)

```
#
# Sample command file for nhedit task.
#
# Establish the default parameters for the rest of the commands.

default_pars upda+ add+ show- veri-

# Notice the use of commas if you desire.
"DETECTOR" 'Newfirm', "comment string"
ONELE 'A' "comment to A"
#
# Now delete a keyword
ONELE1 del+ show+
add_blank " /blank keyw"

# add a boolean value T
ONELE1 '(1==1)', "comment to A"

"DETSIZE", '[1:2048,1:2048]'
"ENVTEM", 1.56000000000000E+01

# Add a field with string value 'T'
ONELEi2 'T'

bafkeyw1 123.456 "comment to key1" before="WCSDIM" addonly+ show-
add_blank "COMMENT FOR A BLANK" after="FR-SCALE" add+ show-
history "this is a hist to append" add+ show-
history "this is a hist 22 after trim pkey" after="TRIM" add+ show-
comment "this is a comment" after="FR-SCALE" add+ show-
# END OF HDRC.TXT FILE
```

SEE ALSO

hselect, hedit, mkheader, imgets, imheader

NAME

observatory -- Examine and set observatory parameters

USAGE

observatory command obsid [images]

PARAMETERS**command = "list" (set|list|images)**

Command option which is one of "set", "list", or "images". The set command sets the default observatory task parameters for the specified observatory. The list command lists the observatory parameters for the specified observatory but does not modify the task parameters. The images command lists the observatory parameters for a list of images. The list and images commands examine and verify the observatory parameters applied by other tasks using the observatory database facility.

obsid = "?"

Observatory identification to be set, listed, or used as the default for images without the OBSERVAT keyword. The observatory ID is one of those in the database (case ignored), the special string "observatory" to default to the environment variable "observatory" or the *observatory.observatory* parameter, "obspars" to select the parameters in the **observatory** task, or "?" to list the observatories defined in the database.

images

List of images to be examined with the "images" command. The images are checked for the OBSERVAT keyword to determine the observatory parameters to be listed, otherwise the observatory given by *obsid* is used.

verbose = no

Verbose output? Because there are a number of different ways in which observatory information is determined this option prints detailed information on how the observatory database and parameters are ultimately selected.

observatory

The default observatory used by tasks which use the special observatory identification "observatory". The value is one of the observatory names in the observatory database (case ignored) or the special value "obspars" to select the parameters defined in this task. There is no default to force users to set it at least once.

name Observatory name.

longitude

Observatory longitude given in degrees west.

latitude

Observatory latitude in degrees. Positive latitudes are north and negative latitudes are south.

altitude

Observatory altitude in meters above sea level.

timezone

Observatory time zone. The time zone is the number of hours west of Greenwich or the number of hours to be added to local time to obtain Greenwich time.

ENVIRONMENT VARIABLES**obsdb**

This variable selects the observatory database. If not defined it to noao\$lib/obsdb.dat.

observatory

This variable selects the observatory entry whenever a observatory name "observatory". If not defined the value of the task parameter *observatory.observatory* is used.

IMAGE HEADER KEYWORDS

The observatory identification for images is first sought under the image header keyword OBSERVAT. This always takes precedence over any other means of defining the observatory.

DESCRIPTION**OBSERVATORY PARAMETERS IN THE NOAO PACKAGE**

Some astronomical data reduction and analysis tasks perform computations requiring information about where the data was observed. For example a number of **noao** tasks make corrections for the airmass. Generally they look for an airmass in the image header and if it is not present they attempt to compute it from other image header parameters. The information about time and telescope coordinates of the observation are often in the image header but the observatory latitude is not. The task must get this information somehow.

Prior to IRAF V2.10 tasks generally had explicit parameters, such as latitude, with default values pointing (using parameter redirection) to the parameter of the same name in the **observatory** task. The user was required to know the values of the observatory parameters and manually change them for data from different observatories. In V2.10 an observatory database has been implemented. Observatory parameters are stored in a simple text file and tasks obtain observatory related parameters by specifying an observatory identification.

In general the information about the observatory should be directly associated with the image data. Unless stated otherwise in the description of a task, tasks which require observatory information will first look for the image header keyword OBSERVAT. The value of this keyword is the observatory identification used to index the observatory database. The task will then look up any observatory parameters it needs in the observatory database. Data from observatories that support this keyword will, therefore, always use the correct observatory parameters without user intervention. All observatories which export FITS image data are urged to adopt the OBSERVAT keyword (a keyword recommended by the FITS standard).

For image data which do not identify the observatory in this way and in tasks which do not operate on images (such as astronomical calculator tools), the observatory must be specified by the user. Most tasks provide an "observatory" parameter which either directly selects the observatory or use special values for defining the observatory with an environment variable or the parameters from the **observatory** task.

An observatory is specified by the identification name used in the observatory database. The names in the database may be listed using the **observatory** task as described below. If the desired observatory is not in the database a user may copy/create their own database and select it with the environment variable "obsdb", modify the standard database if allowed (any changes to the distributed version should be forwarded to iraf\$noao.edu), or use the special observatory name "obspars". The last option directly uses

the parameters in the **observatory** task which can be set to any values using the normal parameter editing mechanism.

The default value for the observatory parameter in a "observatory". This special name directs the task to look first for the environment variable of the same name and then at the *observatory* parameter of the **observatory** task. The environment variable allows users or sites to set the default observatory in their login files and site defaults. Also it is simple to change the default observatory either with a **reset** command or the **observatory** command.

The observatory database is selected by the environment variable "obsdb". The default when the variable is not defined is the **noao** package library database file "noao\$lib/obsdb.dat". The use of an environment variable allows users to permanently change the default database in the OS environment (when IRAF has access to it such as in UNIX systems) or in the startup IRAF environment as set in the "login.cl" or "loginuser.cl" files. One can, of course, change it during a session with the set or reset commands. For sites which want to customize the observatory mechanism the environment variables can also be set and changed in the files "hlib\$zzsetenv.def", "noao\$lib/zzsetenv.def", and the template login file "hlib\$login.cl".

An observatory database file consist of a simple list of keyword=value pairs with arbitrary whitespace allowed. An observatory entry begins with the observatory keyword and extends to the next observatory keyword or the end of the file. The observatory identification should be the same as the string used in the OBSERVAT image header parameter for data from that observatory. The default file noao\$lib/obsdb.dat begins as follows:

```
# Observatory Parameters. Taken from the Almanac.
#
# Observatories wishing to be added or make changes in the default
# distributed database should send information to iraf@noao.edu.

observatory = "kpno"
    name = "Kitt Peak National Observatory"
    longitude = 111:36.0
    latitude = 31:58.8
    altitude = 2120.
    timezone = 7

observatory = "ctio"
    <etc>
```

In summary, access to observatory parameters is now done by referencing the image header keyword OBSERVAT and, if not defined, determine the observatory name from a task parameter. The environment variables "observatory" and "obsdb" can be set by the user to select alternate observatories and observatory database files. For data without an observatory entry the observatory can be set to "obspars" or the user may make their own observatory database.

THE OBSERVATORY TASK

The **observatory** task serves a number of functions. It examine the observatory database, verify the observatory parameters which will be used by other tasks, particularly those operating on images, set the default observatory if not defined by other means, set observatory parameters explicitly, especially when there is no observatory database entry, and as a parameter set for tasks which explicitly reference observatory parameters. The **verbose** parameter also provides a detailed check of the steps used to determine the observatory database, observatory identification, and observatory parameters.

The *command* parameter takes the values "set", "list", or "images". The *obsid* parameter supplies the observatory identification and the *images* parameter is used to specify a command. The parameters are query parameters and so may be either queried or simply typed on the command line.

The "set" command prints the observatory parameters for the specified observatory and sets many of these in the **observatory** task parameters. This command is used to set the default observatory parameters for tasks where images are not used, the images do not contain the observatory id, or direct references to specific parameters with parameter redirection (for example ")observatory.latitude") are used.

The "list" command is similar to the "set" command except the task parameters are not modified. It is used to list observatory parameters. It is also use with the special observatory identifications to list the entries in an observatory database and verify the observatory to be used by tasks which do not operate on images. The special value "?" lists the entries in the database. The special value "observatory" lists the observatory defined by the "observatory" environment variable or that given by the *observatory.observatory* parameter. The special value "obspars" simply lists the observatory task parameters.

The "images" command lists the observatory information applicable to one or more images. In particular, the observatory identification is first sought in OBSERVAT image header keyword and, if not found, the *obsid* parameter is used. Often the default observatory is "observatory" to follow the same search path used by other tasks.

The *verbose* parameter prints additional detailed information. It prints the database used and whether it is selected by default (noao\$lib/obsdb.dat) or by the "obsdb" environment variable. When the observatory is defined as "observatory" it indicates whether the observatory is defined by the environment variable "observatory" or by the observatory task. When listing images it prints the OBSERVAT keyword or the default observatory assigned.

For observatories not in a database the name, latitude, longitude, altitude, and time zone parameters may be set using **eparam**. The observatory id must be set to "obspars" in this case. These parameters will then be referenced by other tasks in which the observatory is specified as "obspars". This allows arbitrary observatory parameters to be set without creating or modifying an observatory database. However, it is advisable to create a local database and also send the observatory information to the IRAF group at NOAO for inclusion in the default database.

EXAMPLES

1. List the observatory entries in the database:

```
cl> observatory list ? v+
Using default observatory database: noao$lib/obsdb.dat
```

```
default: Kitt Peak National Observatory
kпно: Kitt Peak National Observatory
ctio: Cerro Tololo Interamerican Observatory
eso: European Southern Observatory
lick: Lick Observatory
mmt: Whipple Observatory
cfht: Canada-France-Hawaii Telescope
lapalma: Roque de los Mucachos, La Palma
```

2. Set the observatory parameters for Cerro Tololo:

```
cl> observatory set ctio
Observatory parameters for Cerro Tololo...
    observatory = ctio
    timezone = 5
    altitude = 2215.
    latitude = -30:09.9
    longitude = 70:48.9
    name = 'Cerro Tololo Interamerican Observatory'
cl> lpar observatory
```

command = "set"	Command (set list images)
argument = ctio	Observatory or images
(observatory = "ctio")	Observatory identification
(name = "Cerro Tololo...")	Observatory name
(longitude = 70.815)	Observatory longitude (degrees)
(latitude = -30.165)	Observatory latitude (degrees)
(altitude = 2215.)	Observatory altitude (meters)
(timezone = 4)	Observatory time zone
(verbose = no)	Verbose output?
(mode = "q")	

3. Set the observatory parameters to use the environment variable "observatory" and verify it.

```
cl> set observatory=cfht
cl> observatory list observatory
Observatory parameters for Canada-France-Hawaii Telescope
  observatory = cfht
  timezone = 10
  altitude = 4215
  latitude = 19:49.6
  longitude = 155:28.3
  name = 'Canada-France-Hawaii Telescope'
```

4. Change the default observatory database and verify verbosely:

```
cl> set observatory="sco"
cl> set obsdb="/local/iraf/obsdb.dat"
cl> type obsdb$
# Local Observatory Parameters.

observatory = "sco"
  name = "Small College Observatory"
  longitude = 100:20.0
  latitude = 35:58.8
  altitude = 212.
  timezone = 6
cl> observ set observatory v+
Using database defined by 'obsdb' environment variable:
/tmp/test/obsdb.dat
Using obs... defined by 'obs...' environment variable: sco
Using observatory parameters for database entry: sco
Observatory parameters for Small College Observatory
  observatory = sco
  timezone = 6
  altitude = 212.
  latitude = 35:58.8
  longitude = 100:20.0
  name = 'Small College Observatory'
```

5. List the observatory assigned to some images with a determined either by the "observatory" environment variable or that set in the observatory task.

```
cl> observ images observatory dev$pix,demoobj1
Observatory parameters for Small College Observatory
  observatory = sco
  timezone = 6
  altitude = 212.
  latitude = 35:58.8
  longitude = 100:20.0
  name = 'Small College Observatory'
Images: dev$pix (default observatory)
Observatory parameters for Kitt Peak National Observatory
  observatory = kpno
```

```
timezone = 7
altitude = 2120.
latitude = 31:58.8
longitude = 111:36.0
name = 'Kitt Peak National Observatory'
Images: demoobj1 (OBSERVAT keyword)
```

6. Set explicit observatory parameters:

```
cl> epar observatory
<set observatory parameters>
cl> observ list obspars
Observatory parameters for North Pole
  observatory = obspars
  timezone = 0
  altitude = 0.
  latitude = 90.
  longitude = 0.
  name = 'North Pole'
```

7. Use observatory parameters in expressions:

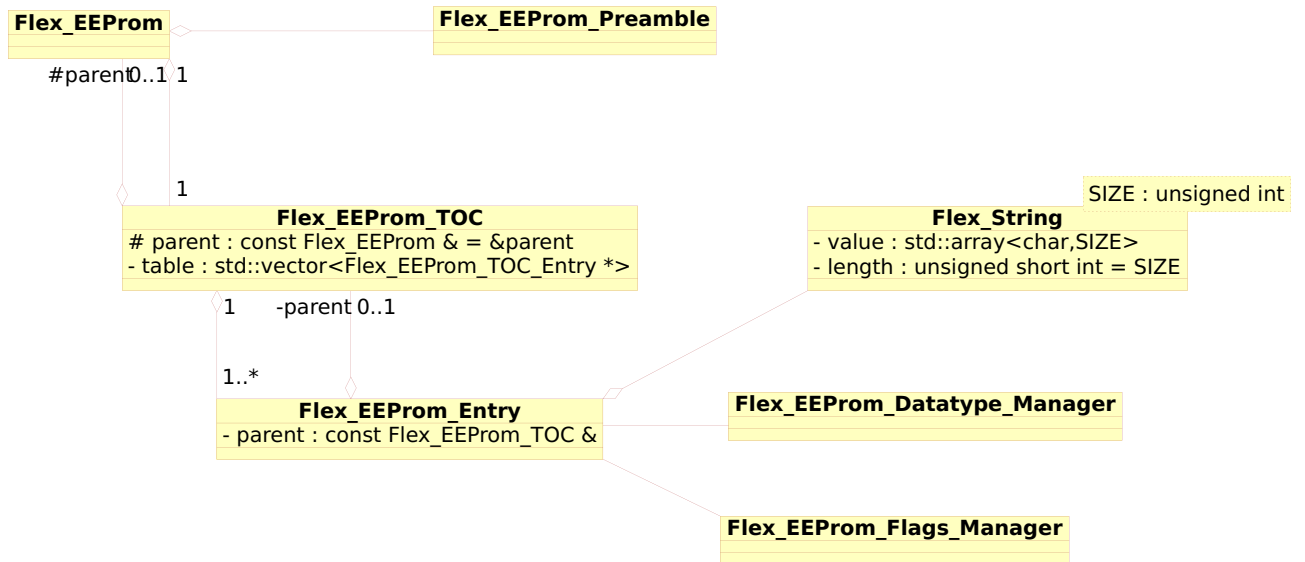
```
cl> observ set kpno
Observatory parameters for Kitt Peak National Observatory
  observatory = kpno
  timezone = 7
  altitude = 2120.
  latitude = 31:58.8
  longitude = 111:36.0
  name = 'Kitt Peak National Observatory'
cl> = observ.lat
31.98
cl> = sin (3.14159/180 * observ.lat)
0.52962280742153
```

SEE ALSO

Tasks in astutil, imred, onedspec, and twodspec.

Flex_CRC

- crc_table : std::vector<uint16_t>
- polynomial : uint16_t

Flex_Messages

NAME

response -- Determine response calibrations

USAGE

response calibration normalization response

PARAMETERS**calibration**

Images to use in determining response calibrations. These are generally quartz continuum spectra. An image section may be used to select only a portion of the image.

normalization

Images to use determining the normalization spectrum. In almost all cases the normalization images are the same as the calibration images or a subsection of the calibration images.

responses

Response calibration images to be created. Each response image is with a calibration image. If the image exists then it will be modified otherwise it is created.

interactive= yes

Graph the average calibration spectrum and fit the normalization spectrum interactively?

threshold= INDEF

Set the response to 1 when the normalization spectrum or input image data fall below this value. If INDEF then no threshold is applied.

sample = "*"

Sample of points to use in fitting the average calibration spectrum. The sample is selected with a range string.

naverage= 1

Number of sample points to average or median before fitting the function. If the number is positive the average of each set of naverage sample points is formed while if the number is negative then the median of each set of points (in absolute value) is formed. This subsample of points is used in fitting the normalization spectrum.

function = "spline3"

Function to fit to the average image spectrum to form the normalization spectrum. The options are "spline1", "spline3", "legendre", and "chebyshev".

order= 1

Order of the fitting function or the number of spline pieces.

low_reject= 0., high_reject= 0.

Rejection limits below and above the fit in units of the residual sigma.

niterate= 1

Number of rejection iterations.

grow = 0

Reject additional points within this distance of points exceeding the rejection threshold.

CURSOR KEYS

The interactive curve fitting package **icfit** is used to fit a to the average calibration spectrum. Help for this package is found under the name "icfit".

DESCRIPTION

A response calibration, in the form of an image, is created for each input image, normally a quartz spectrum. The response calibration is formed by dividing the calibration image by a normalization spectrum which is the same at all points along the spatial axis. The normalization spectrum is obtained by averaging the normalization image across the dispersion to form a one dimensional spectrum and smoothing the spectrum by fitting a function. The threshold value does not apply to creating or fitting of the normalization spectrum but only the final creation of the response values. When normalizing (that is dividing the data values by the fit to the normalization spectrum) only pixels in which both the fitted normalization value and the data value are above the threshold are computed. If either the normalization value or the data value is below the threshold the output response value is one.

The image header keyword DISPAXIS must be present with a dispersion parallel to the lines (varying with the column coordinate) or 2 for dispersion parallel to the columns (varying with line coordinate). This parameter may be added using **hedit**. Note that if the image has been transposed (**imtranspose**) the dispersion axis should still refer to the original dispersion axis unless the physical world coordinate system is first reset (see **wcsreset**). This is done in order to allow images which have DISPAXIS defined prior to transposing to still work correctly without requiring this keyword to be changed.

If the output image does not exist it is first created with unit response everywhere. Subsequently the response is only modified in those regions occupied by the input calibration image. Thus, image sections may be used to select regions in which the response is desired. This ability is particularly useful when dealing with multiple slits within an image or to exclude regions outside the slit.

Normally the normalization images are the same as the calibration images. In other words the calibration image is normalized by the average spectrum of the calibration image itself. Sometimes, however, the normalization image may be a smaller image section of the calibration image to avoid contaminating the normalization spectrum by effects at the edge of the slit. Again, this may be quite useful in multi-slit images.

The normalization spectrum is smoothed by fitting a using the interactive curve fitting package (**icfit**). The parameters determining the fitted normalization spectrum are the sample points, the averaging bin size, the fitting function, the order of the function, the rejection sigmas, the number of rejection iterations, and the rejection width. The sample points for the average spectrum are selected by a range string. Points in the normalization spectrum not in the sample are not used in determining the fitted function. The selected sample points may be binned into a set of averages or medians which are used in the function fit instead of the sample points with the averaging bin size parameter *naverage*. This parameter selects the number of sample points to be averaged if its value is positive or the number of points to be medianed if its value is negative (naturally, the absolute value is used for the number of points). A value of one uses all sample points without binning. The fitted function may be used to reject points from the fit using the parameters *low_reject*, *high_reject*, *niterate* and *grow*. If one or both of the rejection limits are greater than zero then the sigma of the residuals is computed and points with residuals less than *-low_reject* times the sigma and greater than *high_reject* times the sigma are removed and the function fitted again. In addition points within a distance given by the parameter *grow* of the a rejected point are also rejected. A value of zero for this parameter rejects only the points exceeding the rejection threshold. Finally, the rejection procedure may be iterated the number of times given by the parameter *niterate*.

The fitted function may be examined and modified interactively when the parameter *interactive* is set. In this case the normalization spectrum and the fitted function or the residuals of the fit are graphed. Deleted points are marked with an x and rejected points by a diamond. The sample regions are indicated along the bottom of the graph. The cursor keys and colon commands are used to change the values of the fitting

parameters, delete points, and window and expand the graph. When the fitted function is satisfactory exit with a carriage return or 'q' and the calibration image will be created. Changes in the fitted parameters are remembered from image to image within the task but not outside the task.

When the task finishes creating a response image the fitting parameters are updated in the parameter file.

EXAMPLES

1. To create a response image non-interactively:

```
cl> response quartz quartz response order=20 interactive=no
```

2. To determine independent responses for a image sections defining each slit. Then the responses are computed as follows:

```
cl> response quartz[10:20,*],quartz[35:45,*] \  
>>> quartz[12:18,*],quartz[12:18,*] resp,resp
```

Generally the slit image sections are prepared in a used to define the lists of input images and response.

```
cl> response @slits @slits @responses
```

3. If the DISPAXIS keyword is missing and the dispersion is running vertically (varying with the image lines):

```
cl> hedit *.imh dispaxis 2 add+
```

SEE ALSO

icfit, iillumination

NAME

sarith -- Spectrum arithmetic

USAGE

sarith input1 op input2 output

PARAMETERS**input1**

List of input images to be used as operands.

op Operator to be applied to the first operand or to both operands. The unary or single operand operators are:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu
- inv - inverse
- ln - natural logarithm
- log - base 10 logarithm
- lum - convert magnitude to luminosity
- mag - convert luminosity to magnitude
- sqrt - square root

The binary or two operand operators are:

- replace - replace first operand values by second operand values
- + - addition
- - subtraction
- * - multiplication
- / - division
- ^ - exponentiation

input2

Lists of input spectra or constants to be used as second operands for binary operations. If a single value is specified it applies to all the first operand input images otherwise the list must match the first operand list in number.

output

List of resultant output images or root names. Image sections are ignored and if the output format is "onedspec" then any record extensions are stripped to form the root name. If no output list is specified then the input list is used and the input images are replaced by the resultant spectra. If a single output name is specified then all resultant spectra are written to the same output image or image root name. This allows packing or merging multiple spectra and requires properly setting the *clobber*, *merge*, *renumber* and *offset* parameters to achieve the desired output. If more than one output image is specified then it must match the input image list in number.

w1 = INDEF, w2 = INDEF

Starting and ending wavelengths to be copied. If w1 is not specified then the wavelength of the starting edge of the first pixel is used (wavelength at pixel coordinate 0.5) and if w2 is not specified

then the wavelength of the ending edge of the last pixel is used (wavelength of the last pixel plus 0.5). If both are not specified, that is set to INDEF, then the whole spectrum is copied and the *rebin* parameter is ignored. Note that by specifying both endpoints the copied region can be set to have increasing or decreasing wavelength per pixel. If the spectrum only partially covers the specified range only that portion of the spectrum within the range is copied. It is an error if the range is entirely outside that of a spectrum.

apertures = "", beams = ""

List of apertures and beams to be selected from the input spectra. The logical intersection of the two lists is selected. The null list selects all apertures or beams. A list consists of comma separated numbers and ranges of numbers. A range is specified by a hyphen. An optional step size may be given by 'x' followed by a number. See **xtools.ranges** for more information. If the first character is "!" then the apertures/beams not in the list are selected. Note that a "!" in either of the lists complements the intersection of the two lists. For longslit input spectra the aperture numbers selects the lines or columns to be extracted. For 3D Fabry-Perot spectra the aperture numbers select the first spatial axis.

bands = ""

List of bands in 3D multispec. For 3D spatial spectra the band parameter applies to the second spatial axis. The null list selects all bands. The syntax is as described above.

apmodulus= 0

Modulus to be applied to the input aperture numbers before matching against the aperture list. If zero then no modulus is used. This is used to select apertures which are related by the same modulus, typically a factor of 10; for example, 10, 1010, and 2010 with a related.

reverse= no

Reverse the order of the operands in a binary operation? Because the first operand is used as the image header template, dispersion coordinate template, and output image in the case of a image and not a constant. To allow certain operations, for example subtracting a spectra from a constant or using the subtractand as the dispersion coordinate template, the reverse option is used to reverse the order of the operands in a binary operation.

ignoreaps= no

Ignore aperture numbers in the second operand? Normally, spectra in binary operations must have matching aperture numbers, otherwise an error is printed. If this parameter is yes then the spectra are matched by line number with the last line being used if the second operand spectrum has fewer lines than the first operand spectrum. This is generally used to allow using a single spectrum with multiple aperture spectra.

format="multispec" (multispec|onedspec)

Output image format and name syntax. The "multispec" format consists of one or more spectra in the same image file. The "onedspec" format consists of a single spectrum per image with names having a root name and a four digit aperture number extension. Note that converting to "onedspec" format from three dimensional images where the third dimension contains associated spectra will not include data from the extra dimension. Image sections may be used in this case.

renumber= no

Renumber the output aperture numbers? If set the output aperture numbers, including any preexisting spectra when merging, are renumbered beginning with 1. The *offset* parameter may be used to change the starting number.

offset = 0

Offset to be added to the input or renumbered aperture number to form the final output aperture number.

clobber = no

Modify an existing output image either by overwriting or merging?

merge = no

Merge apertures into existing spectra? This requires that the *clobber* parameter be set. If not merging then the selected spectra entirely replace those in existing output images. If merging then the input spectra replace those in the output image with the same aperture number and new apertures are added if not present.

rebin = yes

Rebin the spectrum to the exact wavelength range specified by the *w1* and *w2* parameters? If the range is given as INDEF for both endpoints this parameter does not apply. If a range is given and this parameter is not set then the pixels in the specified range (using the nearest pixels to the endpoint wavelengths) are copied without rebinning. In this case the wavelength of the first pixel may not be exactly that specified by *w1* and the dispersion, including non-linear dispersions, is unchanged. If this parameter is set the spectra are interpolated to have the first and last pixels at exactly the specified endpoint wavelengths while preserving the same number of pixels in the interval. Linear and log-linear dispersion types are maintained while non-linear dispersions are linearized.

errval = 0.

Value for resultant pixel if an arithmetic error occurs such as dividing by zero or the square root of a negative number.

verbose = no

Print a record of each operation?

DESCRIPTION

Sarith performs arithmetic operations on spectra. It is distinguished from **imarith** in that it includes unary operators, like **imfunction** but with some specific to astronomical spectra, and binary operations between two spectra are performed in dispersion coordinate space (typically wavelength) rather than logical pixel space. In the latter case the spectra are checked for matching dispersion functions (which are not necessarily linear) and, if they don't match, the second operand is interpolated without flux conservation. (If flux conservation is desired then the task **dispcor** should be used first.) Thus, the spectra may have different dispersion functions but the arithmetic is done at matching wavelengths. The default interpolation function is a 5th order polynomial. The choice of interpolation type is made with the package parameter "interp". It may be set to "nearest", "linear", "spline3", "poly5", or "sinc". Remember that this applies to all tasks which might need to interpolate spectra in the **onedspec** and associated packages. For a discussion of interpolation types see **onedspec**.

The unary operators operate on the spectra in the first operand list to produce the specified output spectra, which may be the same as the input spectra. The operators include:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu

inv - inverse
 ln - natural logarithm
 log - base 10 logarithm
 lum - convert magnitude to luminosity
 mag - convert luminosity to magnitude
 sqrt - square root

The luminosity to magnitude and magnitude to luminosity operators are based on the standard relation:

$$\text{mag} = -2.5 * \log(\text{lum})$$

where the log is base 10. The F-nu to F-lambda and F-lambda to F-nu operators are based on the relation:

$$\text{F-nu} = \text{F-lambda} * \text{lambda} / \text{nu}$$

where lambda is wavelength and nu is frequency (currently the wavelength is assumed to be Angstroms and so F-lambda is in units of per Angstrom and F-nu is in units of per Hertz). In all the operators it responsibility of user as to the appropriateness of the operator to the input.

The binary operators operate on the spectra in the first operand list and the spectra or numerical constants in the second operand. Numeric constants are equivalent to spectra having the specified value at all pixels. The binary operators are the standard arithmetic ones plus exponentiation and replacement:

replace - replace first operand values by second operand values
 + - addition
 - - subtraction
 * - multiplication
 / - division
 ^ - exponentiation

If the second operand is a spectrum, as mentioned previously, it is interpolated, without flux conservation, to the dispersion function of the first operand spectrum if necessary.

There is a distinction between the first operand and the second operand. The first operand must always be a spectrum. It supplies the dispersion function to be matched by the second operand spectrum. It also supplies a copy of its image header when a new output spectrum is created. In cases where it is desired to have the second operand be the dispersion/header reference and/or the first operand be a the *reverse* parameter is used. For example to subtract a spectrum from the constant 1:

```
cl> sarith 1 - spec invspec reverse+
```

or to subtract two spectra using the subtractand as the dispersion reference:

```
cl> sarith spec1 - spec2 diff reverse+
```

When a binary operation on a pair of spectra is performed the aperture numbers may be required to be the same if *ignoreaps* is no. For images containing multiple spectra the apertures need not be in the same order but only that matching apertures exist. If this parameter is set to yes then aperture numbers are ignored when the operation is performed. For multiple spectra images the second operand spectra are matched by image line number rather than by aperture. If the second operand image has fewer lines, often just one line, then the last line is used repeatedly. This feature allows multiple spectra in the primary operand list to be operated upon by a single spectrum; for example to subtract one spectrum from all spectra in the in a multiple spectrum image.

If it is an error to perform an operation on certain data values, for example division by zero or the square root of a negative number, then the output value is given the value specified by the parameter *errval*.

A log of the operations performed may be printed to the standard output, which may then be redirected if desired, if the *verbose* parameter is set. In the output the last bracketed number is the aperture number of the spectrum.

INPUT/OUTPUT

The arithmetic part of **sarith** is fairly straightforward and intuitive. The selection of input spectra from input images and the placing of output spectra in output images can be more confusing because there are many possibilities. This section concentrates on the topics of the input and output. Since the concepts apply to all of the operators it simplifies things to think in terms of copying input spectra to output spectra; the "copy" operator. Note that the task **scopy** is actually just this case of **sarith** with parameters set for copying. While the discussion here is similar to that in the help for **scopy**, the examples for that task are more focused for illustrating this topic than the **sarith** examples which concentrate more on the arithmetic aspects of the task.

Input spectra are specified by an image list which may include explicit image names, wildcard templates and @files containing image names. The image names may also include image sections such as to select portions of the wavelength coverage. The input images may be either one or two dimensional spectra. One dimensional spectra may be stored in individual one dimensional images or as lines in two (or three) dimensional images. The one dimensional spectra are identified by an aperture number, which must be unique within an image, and a beam number. Two dimensional long slit and three dimensional Fabry-Perot spectra are treated, for the purpose of this task, as a collection of spectra with dispersion either along any axis specified by the DISPAXIS image header parameter or the *dispaxis* package parameter. The aperture and band parameters specify a spatial position. A number of adjacent lines, columns, and bands, specified by the *nsum* package parameter, will be summed to form an aperture spectrum. If number is odd then the aperture/band number refers to the middle and if it is even it refers to the lower of the two middle lines or columns.

In the case of many spectra each stored in separate one dimensional images, the image names may be such that they have a common root name and a four digit aperture number extension. This name syntax is called "onedspec" format. Including such spectra in an input list may be accomplished either with wildcard templates such as

```
name*
name.????.imh
```

where the image type extension ".imh" must be given to complete the template but the actual extension could also be that for an STF type image, or using an @file prepared with the task **names**. To generate this syntax for output images the *format* parameter is set to "onedspec" (this will be discussed further later).

From the input images one may select a range of wavelengths with the *w1* and *w2* parameters and a subset of spectra based on aperture and beam numbers using the *aperture* and *beam* parameters. If the wavelength range is specified as INDEF the full spectra are used without any resampling. If the aperture and beam lists are not specified, an empty list, then all apertures and beams are selected. The lists may be those spectra desired or the complement obtained by prefixing the list with '!'. Only the selected wavelength range and spectra will be operated upon and passed on to the output images.

Specifying a wavelength range is fairly obvious except for the question of pixel sampling. Either the pixels in the specified range are used without resampling or the pixels are resampled to correspond exactly to the requested range. The choice is made with the *rebin* parameter. In the first case the nearest pixels to the specified wavelength endpoints are determined and those pixels and all those in between are used. The dispersion relation is unchanged. In the second case the spectra are reinterpolated to have the specified starting and ending wavelengths with the same number of pixels between those points as in the original spectrum. The reinterpolation is done in either linear or log-linear dispersion. The non-linear dispersion functions are interpolated to a linear dispersion.

Using **sarith** with long slit and Fabry-Perot images provides a and simple type of extraction as opposed to using the **apextract** package. When summing it is often desired to start each aperture after the number of lines summed. To do this specify a step size in the aperture/band list. For example to extract columns 3 to

23 summing every 5 columns you would use an aperture list of "3-23x5" and an *nsum* of 5. If not use the step in the aperture list you would extract the sum of columns 1 to 5, then columns 2 to 6, and so on.

In the special case of subapertures extracted by **apextract**, related apertures are numbered using a modulus; for example apertures 5, 1005, 2005. To allow selecting all related apertures using a aperture number the *apmodulus* parameter is used to specify the modulus factor; 1000 in the above example. This is a feature which should be ignored by most users.

The output list of images may consist of an empty list, a or a list of images matching the input list in number. Note that it is the number of image names that matters and not the number of spectra since there may be any number of spectra in an image. The empty list converts to the same list as the input and is shorthand for replacing the input image with the output image upon completion; therefore it is equivalent to the case of a matching list. If the input consists of just one image then the distinction between a output and a matching list is moot. The interesting distinction is when there is an input list of two or more images. The two cases are then a mapping of many-to-many or many-to-one. Note that it possible to have more complex mappings by repeating the same output name in a matching list provided clobbering, merging, and possibly renumbering is enabled.

In the case of a matching list, spectra from different input images will go to different output images. In the case of a single output image all spectra will go to the same output image. Note that in this discussion an output image when "onedspec" format is specified is actually a root name for possibly many images. However, it should be thought of as a single image from the point of view of image lists.

When mapping many spectra to a single output image, which may have existing spectra if merging, there may be a conflict with repeated aperture numbers. One option is to consecutively renumber the aperture numbers, including any previous spectra in the output image when merging and then continuing with the input spectra in the order in which they are selected. This is specified with the *renumber* parameter which renumbers beginning with 1.

Another options which may be used independently of renumbering or in conjunction with it is to add an offset as specified by the *offset* parameter. This is last step in determining the output aperture numbers so that if used with the renumber option the final aperture numbers begin with one plus the offset.

It has been mentioned that it is possible to write and add to existing images. If an output image exists an error will be printed unless the *clobber* parameter is set. If clobbering is allowed then the existing output image will be replaced by the new output. Rather than replacing an output image sometimes one wants to replace certain spectra or add new spectra. This is done by selecting the *merge* option. In this case if the output has a spectrum with the same aperture number as the input spectrum it is replaced by the input spectrum. If the input spectrum aperture number is not in the output then the spectrum is added to the output image. To add spectra with the same aperture number and not replace the one in the output use the *renumber* or *offset* options.

EXAMPLES

In addition to the examples in this section there are many examples in the help for **scopy** which illustrate aspects of selecting input spectra and producing various types of output. Those examples are equivalent to using the "copy" operator. The same examples will also apply with other operators where the input spectra are modified arithmetically before being copied to the output images.

I. SIMPLE EXAMPLES

The simple examples use only a single input image and create a output image.

1. Examples of unary operations:

```
cl> sarith example1 mag "" magexample
cl> sarith magexample lum "" example2
cl> sarith example1 log "" logexample
```

Note that a place holder for the second operand is required on the command line which will be ignored.

2. Examples of binary operations using constants:

```
cl> sarith example1 + 1000 example2
cl> sarith example1 - 1000 example2 reverse+
cl> sarith example1 / 1000 example2
cl> sarith example1 ** 2 example2
```

3. Examples of binary operations between spectra with matching apertures:

```
cl> sarith example1 + example2 example3
cl> sarith example1 - example2 example3
```

4. Example of binary operations between spectra with the second image consisting of a single spectrum:

```
cl> sarith example1 / flatspec flatexample1 ignore+ errval=1
```

II. MORE COMPLEX EXAMPLES

5. Unary and constant operations on a list of images:

```
cl> sarith example* fnu "" %example%fnu%
cl> sarith example* + 1000 %example%fnu%
```

6. Binary operations on a list of images using a with matching apertures:

```
cl> sarith example* - skyspec %example%skysub%*
```

7. Selecting apertures to operate upon:

```
cl> sarith example* - skyspec %example%skysub%* aper=1,5,9
```

8. Extract the sum of each 10 columns in a by the central spectrum:

```
cl> nsum = "10"
cl> sarith longslit copy "" longslit.ms aper=5-500x10
longslit[5] --> longslit.ms[5]
longslit[15] --> longslit.ms[15]
longslit[25] --> longslit.ms[25]
...
cl> sarith longslit.ms / longslit.ms[*,25] norm ignore+
longslit.ms[5] / longslit.ms[*,25][245] --> norm[5]
longslit.ms[15] / longslit.ms[*,25][245] --> norm[15]
longslit.ms[25] / longslit.ms[*,25][245] --> norm[25]
...
```

9. In place operations:

```
cl> sarith example* + 1000 example* clobber+
example1[1] + 1000. --> example1[1]
example1[2] + 1000. --> example1[2]
...
example2[1] + 1000. --> example2[1]
example2[2] + 1000. --> example2[2]
...
cl> sarith example* flam "" example* clobber+
example1[1] -- flam --> example1[1]
example1[2] -- flam --> example1[2]
...
example2[1] -- flam --> example2[1]
example2[2] -- flam --> example2[2]
...
cl> sarith example* - skyspec "" clobber+ ignore+
```

```
example1[1] + skyspec[1] --> example1[1]
example1[2] + skyspec[1] --> example1[2]
...
example2[1] + skyspec[1] --> example2[1]
example2[2] + skyspec[1] --> example2[2]
...
```

10. Merging existing spectra with the results of operations:

```
cl> sarith example* / flat "" clobber+ merge+ renum+ ignor+
```

REVISIONS

SARITHV2.11

Previously both w1 and w2 had to be specified to select a band to be used. Now if only one is specified the second endpoint defaults to the first or last pixel.

The noise band in multispec data is only copied from the primary spectrum and not modified. This is a kludge until the noise is handled properly.

SARITHV2.10.3

Additional support for 3D multispec/equispec or spatial spectra has been added. The "bands" parameter allows selecting specific bands and the onedspec output format creates separate images for each selected aperture and band.

SARITHV2.10

This task is new.

SEE ALSO

scopy, splot, imarith, imfunction

NAME

sensfunc -- Determine sensitivity and extinction functions

USAGE

sensfunc standards sensitivity

PARAMETERS

standards="std"

Input standard star data file created by the task **standard**.

sensitivity = "sens"

Output sensitivity function image name or rootname. Generally each aperture results in an independent sensitivity function with the aperture number appended to the rootname. If the parameter *ignoreaps* is set, however, the aperture numbers are ignored and a function is determined with the output image having the specified name with no extension.

apertures=""

List of apertures to be selected from the input file. All other apertures are ignored. If no list is specified then all apertures are selected. See **ranges** for the syntax.

ignoreaps= no

Ignore aperture numbers and create a single sensitivity function? Normally each aperture produces an independent sensitivity function. If the apertures are ignored then all the observations are combined into a single sensitivity function.

logfile = "logfile"

Output log filename for statistical information about the stars used and the sensitivity function and extinction function. If no filename is given then no file is written.

extinction= <no default>

Input extinction file. Any extinction determination made will be relative to this extinction. If no file is given then no extinction correction is applied and any extinction determination from the standard star data will be an absolute determination of the extinction. The default value is redirected to the package parameter of the same name. The extinction file is generally one of the standard extinctions in the calibration directory "onedstds\$".

If extinction corrected spectra were used as input to **standard** then it is important that the same extinction file be used here. This includes using no extinction file in both tasks.

newextinction ="extinct.dat"

Output revised extinction file. If the extinction is revised and an output filename is given then a revised extinction file is written. It has the same format as the standard extinction files.

observatory ="_)_observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a when graphing flux calibrated data of spectra which do not include the airmass in the image header. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information.

function = "spline3"

Function used to fit the sensitivity data. The function types are "chebyshev" polynomial, "legendre" polynomial, "spline3" cubic spline, and "spline1" linear spline. The default value may be changed interactively.

order= 6

Order of the sensitivity fitting function. The value corresponds to the number of polynomial terms or the number of spline pieces. The default value may be changed interactively.

interactive= yes

Determine the sensitivity function interactively? If yes the user graphically interacts with the data, modifies data and parameters affecting the sensitivity function, and determines a

graphs = "sr"

Graphs to be displayed per frame. From one to four graphs may be displayed per frame. The graph types are selected by single characters and are:

- a - residual sensitivity vs airmass
- c - composite residual sensitivity and error bars vs wavelength
- e - input extinction and revised extinction vs wavelength
- i - Flux calibrated spectrum vs wavelength
- r - residual sensitivity vs wavelength
- s - sensitivity vs wavelength

All other characters including whitespace and commas are ignored. The order and number of graphs determines the positions of the graphs.

marks = "plus cross box"

Symbols used to mark included, deleted, and added data respectively. The available mark types are point, box, plus, cross, diamond, hline (horizontal line), vline (vertical line), hebar (horizontal error bar), vebar (vertical error bar), and circle.

colors = "2 1 3 4"

Colors to use for "lines", "marks", "deleted" data, and "added" data. The colors associated with the numbers is graphics device dependent. For example in XGTERM they are defined by resources while on other devices that don't support colors only one color will appear.

cursor = ""

Graphics cursor input list. If not specified as a graphics cursor is read.

device = "stdgraph"

Graphics output device.

answer

Query parameter for selecting whether to fit apertures interactively.

CURSOR COMMANDS

- ? Print help
- a Add a point at the cursor position
- c Toggle use of composite points
- d Delete point, star, or wavelength nearest the cursor
- e Toggle residual extinction correction
- f Fit data with a sensitivity function and overplot

```

g      Fit data with a sensitivity function and redraw the graph(s)
i      Print information about point nearest the cursor
m      Move point, star, wavelength nearest the cursor to new sensitivity
o      Reset to original data
q      Quit and write sensitivity function for current aperture
r      Redraw graph(s)
s      Toggle shift of standard stars to eliminate mean deviations
u      Undelete point, star, or wavelength nearest the cursor
w      Change weights of point, star, or wavelength nearest the cursor

:flux [min] [max]  Limits for flux calibrated graphs (INDEF for autoscale)
:function [type]   Function to be fit to sensitivity data:
                   chebyshev - Chebyshev polynomial
                   legendre  - Legendre polynomial
                   spline1   - Linear spline
                   spline3   - Cubic spline
:graphs [types]    Graphs to be displayed (up to four):
                   a - Residual sensitivity vs airmass
                   c - Composite residuals and error bars vs wavelength
                   e - Extinction (and revised extinction) vs wavelength
                   i - Flux calibrated image vs wavelength
                   l - Log of flux calibrated image vs wavelength
                   r - Residual sensitivity vs wavelength
                   s - Sensitivity vs wavelength
:images [images]   Images to flux calibrate and plot (up to four)
:marks marks       Mark types to use for included, delete, and added points:
                   point, box, plus, cross, diamond, hline,
                   vline, hebar, vebar, circle
:order [order]     Order of function
:skys [images]     Sky images for flux calibration (up to four)
:stats [file]      Statistics about stars and sensitivity fit
:vstats [file]     Verbose statistics about sensitivity fit

```

DESCRIPTION

Standard star calibration measurements are used to determine the system sensitivity as a function of wavelength for each independent aperture. If the parameter *ignoreaps* is set then the aperture numbers are ignored and a single sensitivity function is determined from all the observations. Using measurements spanning a range of airmass it is also possible to derive an adjustment to the standard extinction curve or even an absolute determination. Extinction determination requires that the observations span a good range of airmass during photometric conditions. When conditions are poor and standard star observations are obtained during periods of variable transparency, the entire sensitivity curve may vary by a constant factor, assuming that the cause of the variations has no color effect. This is often the case during periods of thin clouds. In this case the mean sensitivity of each observation may be shifted to match the observation of greatest sensitivity. This allows for the possibility of deriving correct absolute fluxes if one observation of a standard was obtained during a clear period.

The input data is a file of calibration information produced by the task **standard**. The data consists of a line containing the spectrum image name, the sky image name if beam switching, the aperture number, the length of the spectrum, the exposure time, airmass, wavelength range, and title. Following the identification line are calibration lines consisting of the central bandpass wavelengths, the tabulated fluxes in the bandpasses, the bandpass widths, and the observed counts in the bandpasses. The spectrum identification and calibration lines repeat for each standard star observation. The parameter *apertures* may be used to select only specific apertures from the input data. This parameter is in the form of a range list (see help for **ranges**) and if no list is given (specified by the null string "") then all apertures are selected.

An input extinction file may also be specified. Any extinction determinations are then residuals to this input extinction table. The format of this table is described in **lcalib**.

The calibration factor at each point is computed as

$$(1) C = 2.5 \log (O / (T B F)) + A E$$

where O is the observed counts in a bandpass of an observation, T is the exposure time of the observation, B is the bandpass width, F is the flux per Angstrom at the bandpass for the standard star, A is the airmass of the observation, and E is the extinction at the bandpass. Thus, C is the ratio of the observed count rate per Angstrom corrected to some extinction curve to the expected flux expressed in magnitudes. The goal of the task is to fit the observations to the relation

$$(2) C = S(W) + AE(W)$$

where W is wavelength, S(W) is the sensitivity function, and E(W) is a residual extinction function relative to the extinction used in (1). In later discussion we will also refer to the residual sensitivity which is defined by

$$(3) R = C - S(W) - AE(W)$$

The sensitivity function S(W) is output as an one dimensional image much like the spectra. The sensitivities are in magnitude units to better judge the variations and because the interpolation is smoother in the logarithmic space ($\text{mags} = 2.5 \log_{10}[\text{sensitivity}]$). There is one sensitivity function for each aperture unless the parameter *ignoreaps* is set. In the first case the image names are formed from the specified rootname with the aperture number as a four digit numerical extension. In the latter case a single sensitivity function is determined from all data, ignoring the aperture numbers, and the specified output image is created without an extension. These images are used by **calibrate** to correct observations to a absolute flux scale. If no sensitivity function image rootname is specified then the sensitivity curves are not output.

If a revised extinction function E(W) has been determined for one or more of the apertures then the functions are averaged over all apertures, added to the original extinction, and written to the specified extinction table. The format of this table is the same as the standard extinction tables and are, thus, interchangeable. If no new extinction filename is specified then no extinction table is recorded.

If a log filename is given then statistical information about the sensitivity function determinations are recorded. This includes the names of the input standard star observations and the tabulated sensitivity, extinction, and error information.

Some points to note are that if no input extinction is given then the E in (1) are zero and the E determined in (2) is the absolute extinction. If the data are not good enough to determine extinction then using one of the standard extinction curves the problem reduces to fitting

$$(4) C = S(W)$$

The sensitivity and extinction functions are determined as fitted curves. The curves are defined by a function type and order. There are four function types and the order specifies either the number of terms in the polynomial or the number of pieces in the spline. The order is automatically reduced to the largest value which produces a nonsingular result. In this case the function will attempt to pass through every calibration point. Lower orders provide for a smoother representation of the function. The latter is generally more appropriate for a detector. The initial function type and order for the sensitivity function is specified by the parameters *function* and *order*.

If the *interactive* flag is no then the default function and order is fit to equation (4) (i.e. there is no residual extinction determination or manipulation of the data). The sensitivity functions are output if an image rootname is given and the log information is output if a log filename is given.

When the sensitivity is determined interactively a each aperture. The responses "no" and "yes" select fitting the sensitivity interactively or not for the specified aperture. The responses "NO" and "YES" apply to all apertures and no further queries will be given. When interactive fitting is selected the data are graphed on the specified graphics device and input is through the specified cursor list. The graphics output consists

of from one to four graphs. The user selects how many and which types of graphs to display. The graph types and their single character code used to select them are:

```
a - residual sensitivity vs airmass
c - composite residual sensitivity and error bars vs wavelength
e - input extinction and revised extinction vs wavelength
i - Flux calibrated spectrum vs wavelength
r - residual sensitivity vs wavelength
s - sensitivity vs wavelength
```

The initial graphs are selected with the parameter **graphs** and changed interactively with the colon command `:graphs types`. The ability to view a variety of graphs allows evaluating the effects of the sensitivity curve and extinction in various ways. The flux calibrated spectrum graph uses the current sensitivity function and checks for possible wiggles in the sensitivity curve which affect the shape of the continuum. The choice of graphs also allows the user to trade off plotting speed and resolution against the amount of information available simultaneously. Thus, with some graphics devices or over a slow line one can reduce the number of graphs for greater speed while on very fast devices with large screens one can look at more data. The parameter *marks* and the associated colon command `:marks types` also let the user define the symbols used to mark included, deleted, and added data points.

The list of interactive commands is given in the section on CURSOR COMMANDS. The commands include deleting, undeleting, adding, moving, and identifying individual data points, whole stars, or all points at the same wavelength. Some other commands include `'c'` to create composite points by averaging all points at the same wavelength (this requires exact overlap in the bandpasses) which then replace the individual data points in the fit. This is different than the composite point graph which displays the residual in the mean sensitivity and error in the mean but uses the input data in the fitting. The `'s'` command shifts the data so that the mean sensitivity of each star is the same as the star with the greatest mean sensitivity. This compensates for variable grey extinction due to clouds. Note that delete points are excluded from the shift calculation and a deleted star will not be used as the star of greatest sensitivity. Another useful command is `'o'` to recover the original data. This cancels all changes made due to shifting, extinction corrections, deleting points, creating composite points, etc.

The `'e'` command attempts to compute a correlations between the sensitivity points at different airmass. Note that this is not iterative so that repeating this after having added an extinction correction simply redetermines the correction. At each wavelength or wavelength regions having multiple observations at different airmass a slope with airmass is determined. This slope is the residual extinction at that wavelength. A plot of the residual extinctions at each wavelength is made using the ICFIT procedure. The user may then examine and fit a curve through the residual extinction estimates as a function of wavelength (see **icfit** for a description of the commands). The user must decide how much wavelength dependence is derivable from the data. In many cases only a constant fit to a "gray extinction" or possibly a linear fit is realistic. The fitting is exited by the key `'q'`.

To help evaluate how important the residual extinction determination is a t-statistic significance is computed. This statistic is defined by

$$(5) t = \text{sqrt} (r^{**2} * (N - 2) /$$

where the correlation coefficient

$$(6) r = \text{RMS with correction} /$$

is the fractional improvement in the RMS due to the added extinction correction and N is the number of wavelength points. For large N this approaches a gaussian sigma but a more precise significance requires the t-distribution for N-2 degrees of freedom. Basically this asks, was the improvement in the RMS significantly more than would occur with random errors? A value greater than 3 a value less than 1 is not significant. The user may then accept the revised extinction and apply it to the data.

Note that when there are multiple apertures used each aperture has an independent system sensitivity but the residual extinction is the same. Therefore, the residual extinctions from each aperture are averaged at the end. If one determines a new extinction then one may replace the original input extinction by the new extinction and rederive the sensitivity.

EXAMPLES

1. The following command generates sensitivity spectra

```
cl> sensfunc std sens
```

This command uses the data from the **standard** output file "std" to create sensitivity functions with rootname "sens". If not interactive the task will produce the output with some progress messages being printed. If it is interactive the graphics device will be used to display the data and the fit and user can change the function and order of the fit, delete bad points, shift data to correct for clouds or bandpass errors, and possibly determine a revised extinction function. The statistics of the sensitivity determination are written to the logfile ("logfile" by default).

2. The following examples illustrate the colon command syntax. Generally if no argument is given the current value is displayed. For the statistics commands an optional output file may be given to record the information.

```
:flux 1e-12 INDEF      Set lower limit for flux plots
:flux INDEF INDEF      Restore autoscaling in flux plots
:func spline3          Select cubic spline function
:g srae                Graph sensitivity, residuals, airmass,
                        and extinction
:g sii                 Graph sensitivity and two images
:i n1.0004 n1.0008     Set first two images to graph (the defaults
                        are taken from the standard star list)
:skys n1.0005          Subtract this sky image from first image
                        for flux calibrated spectrum
:m plus                Change the mark type for included points and
                        don't change the deleted or added point mark type
:stats                 Print statistics to terminal
:vstats stdstats       Print verbose statistics to file
```

REVISIONS

SENSFUNC V2.10.3+

Deleted points and stars are now ignored from the grey shift calculation.

SENSFUNC V2.10.3

A color parameter was added for graphics terminals supporting color.

SENSFUNC V2.10

The latitude parameter has been replaced by the observatory parameter. The 'i' flux calibrated graph type now shows flux in linear scaling while the new graph type 'l' shows flux in log scaling. A new colon command allows fixing the flux limits for the flux calibrated graphs.

SENSFUNC V2.8

This task has been completely rewritten from that of versions 2.5 and earlier.

1. The input standard data format is different.
2. Extinction corrections beyond a grey term are now supported.
3. Weighting by the counts is not supported.
4. Tabular input is not supported.
5. The data which can be displayed is greatly improved.
6. The fitting options have been greatly enhanced.

7. The fitting function types available have been extended.
8. One or more flux calibrated images may be displayed using the current sensitivity function.
9. Additional flexibility is provided for treating apertures.

BUGS

If the flux points do not span the wavelength range, set by the standard star observations, then the fitting may fail at some maximum order. When it fails there is no message but the highest order which can be successfully fit is used. To work around this one can either add fake points, truncate the wavelength range in the first line of each tabulated object in the file produced by **standard**, or exclude the part of the image data which cannot be uncalibrated (using **scopy** or **dispcor**).

SEE ALSO

standard, lcalib, calibrate, observatory, icfit, ranges, scopy, dispcor

NAME

setairmass -- update image headers with the effective airmass

USAGE

setairmass images

PARAMETERS**images**

The list of images for which to calculate the airmass. The image headers may optionally be updated with the airmass and the mid-UT of the exposure.

observatory = ")_.observatory"

Observatory for which airmasses are to be computed if the observatory is specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a value. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information. If the input consists of images then the observatory is defined by the OBSERVAT keyword if present.

intype = "beginning"

The time stamp of the observation as recorded at the telescope for the time dependent header keywords. The choices are the "beginning", "middle" or "end" of the observation.

outtype = "effective"

The output time stamp desired for the airmass. The choices are the "effective" airmass, or the airmass at the "beginning", "middle" or "end" of the observation.

ra = "ra"

The name of the keyword that contains the right ascension. The right ascension is assumed to be in hours unless ra is one of the standard CRVALn keywords in which case it is assumed to be in degrees.

dec = "dec"

The name of the keyword that contains the declination in degrees.

equinox = "epoch"

The name of the keyword that contains the equinox of the right ascension and declination coordinates in years.

st = "st"

The name of the keyword containing the sidereal time in hours.

ut = "ut"

The name of the keyword containing the ut time. This keyword can either be in date plus time format or in hours. Note that this allows setting both the "date-obs" and "ut". If no time is found then a time of 0hrs is used.

date = "date-obs"

The name of the keyword that contains the UT date of the observation. The format should be 'DD/MM/YY' (old FITS format), YYYY-MM-DD (new FITS format), or

YYYY-MM-DDTHH:MM:SS (new FITS format with time). If there is a time and no time is found in the ut keyword then it is used for the ut.

exposure = "exptime"

The name of the keyword that contains the exposure time (in seconds) of the image.

airmass = "airmass"

The name of the output keyword that will receive the computed airmass.

utmiddle = "utmiddle"

The name of the output keyword that will receive the universal time for the middle of the observation. The format of the keyword will be the same as that specifying the universal time.

scale = 750.0

The atmospheric scale height.

show = yes

Print the airmasses and mid-UT's for each image?

update = yes

Update the image headers with the airmasses and the mid-UT's?

override = yes

If updating the image headers, override values that were previously recorded ?

DESCRIPTION

SETAIRMASS will calculate the effective airmass of an astronomical image, as described below under "ALGORITHMS". The task requires the instantaneous zenith distance at the beginning, middle and end of the exposure. These are calculated using the right ascension, declination, and equinox as well as the sidereal time, exposure time, UT date, and observatory from the image header. If the observatory is not available in the image header under the keyword OBSERVAT, the observatory is defined by the *observatory* parameter. See help for *observatory* for further information.

The right ascension and declination will be precessed from the given equinox to the date of observation. The name of the right ascension, declination, equinox, sidereal time, ut time, exposure time, and date keywords can be specified as parameters. These keywords should express the right ascension in hours, the declination in degrees, the equinox in years, the sidereal time in hours, the universal time in hours, the exposure time in seconds, and the date in FITS format. If any of the required keywords are missing from the image headers, they can be added using the hedit or asthedit tasks. Note that the universal time keyword may be in either a date plus time format or in hours and any output middle universal time will be in the same format.

Before using this task, you will need to know the "time stamp" of the time varying header quantities (e.g. sidereal time). Do the recorded values represent the beginning, the middle or the end of the exposure ? be set in the **intype** parameter.

If for some reason the effective airmass is not desired, the value of the airmass at the beginning, middle or end of the exposure can be recorded in the header keyword specified by the *airmass* parameter. The **show** parameter can be used to control the output to the terminal. The **update** and **override** parameters control the header keyword output.

SETAIRMASS will also calculate the universal time of the middle of the exposure and place the value in

the header keyword specified by the *utmiddle* parameter. This assumes that the value for the UT is in the date keyword or ut keyword, with the same time stamp as the sidereal time. The mid-observation UT is useful for interpolating calibration arc dispersion solutions using REFSPECTRA, especially when the exposure time is long.

ALGORITHMS

The mean airmass is calculated using the formula described in "Some Factors Affecting the Accuracy of Stellar Photometry with CCDs" by P. Stetson, DAO preprint, September 1988. This simple formula is:

$$AM (eff) = [AM (beginning) + 4*AM (middle) + AM (end)] / 6$$

and is derived by using Simpson's 1/3 rule to approximate the integral that represents the mean airmass.

The beginning, middle and end airmasses are calculated using the relation between airmass and elevation (or zenith distance) in John Ball's book on Algorithms for the HP-45:

$$AM = \sqrt{x^2 + 2*scale + 1} - x, \text{ where}$$

$$x = scale * \sin(elevation) = scale * \cos(ZD)$$

The atmospheric scaling parameter is *scale* (see "Astrophysical Quantities" by Allen, 1973 p.125,133).

KEYWORDS

The input keywords are:

OBSERVAT

Observatory at which the data was taken. If absent the observatory is determined using the *observatory* parameter.

ra

Right ascension in hours at the beginning, middle, or end of the observation. If *ra* is one of the CRVALn keywords it is assumed to be in degrees.

dec

Declination in degrees at the beginning, middle, or end of the observation.

equinox

The equinox of the coordinates. The right ascension and declination will be precessed from this epoch to the date of the observation before being used.

st

Sidereal time in hours at the beginning, middle, or end of the observation.

ut

Universal time in hours at the beginning, middle, or end of the observation. This may be in either date plus time format or just in hours.

date

The value of the date parameter is the keyword name to be used for the date of the observation. The date must be in either the old or new FITS format.

exposure

The value of the exposure parameter is the keyword name to be used for the exposure time in seconds.

The output keywords are:

airmass

The value of the airmass parameter is the keyword name to be used for the computed airmass at either the beginning, middle, or end of the exposure, or for the weighted effective value over the exposure.

utmiddle

The value of the utmiddle parameter is the keyword name to be used for the universal time at the middle of the exposure.

EXAMPLES

1. Calculate the effective airmass of the IRAF test picture, dev\$pix.

```
cl> setairmass dev$pix exposure=itime update-
```

Note that the test picture does not have the correct coordinate epoch listed in its header, so no precession will be performed.

2. Calculate the effective airmass of the IRAF test picture dev\$ypix in two ways.

```
cl> setairmass dev$ypix exposure=itime update-
```

```
cl> setairmass dev$ypix ra=crval1 dec=crval2 equinox=equinox \
    exposure=itime update-
```

Note the first way gives the same results as example 1. The second way uses the J2000 equatorial system rather than the ra and dec at the time of observation.

REVISIONS

SETAIRMASV2.11.4

The ut keyword now has precedence over any time in the date keyword and it can be either date plus time or hours.

SETAIRMASV2.11.3

The right ascension, declination, equinox, st, and ut keywords were made parameters rather than being hard wired.

SETAIRMASV2.11.2

Y2K update: This task was updated to use the new FITS date format.

SEE ALSO

airmass, hedit, refspectra, observatory

NAME

specfocus -- Determine spectral focus and alignment variations

USAGE

specfocus images

PARAMETERS**images**

List of 1D or 2D focus images. Typically the input is a 2D CCD images of arc slit spectra. The 1D image input is provided to allow use of extraction techniques beyond those provided by this task.

focus = ""

List of focus identification values to be associated with each input image or an image header keyword containing the values. The list may be an explicit list of values, a range specification, an @ file containing the values, or an image header keyword. If none of these is given the identification values are simple index values in the order of the input images. A range specification has the forms A, Ax C, A-BxC where A is a starting value, B is an ending value, and C is an increment.

corwidth= 20.

Correlation width in pixels.

level = 0.5

Percent or fraction of the correlation peak at which to measure focus widths. The default is 50% or full width at half maximum.

shifts = yes

Compute dispersion shifts across the dispersion when there are multiple samples? If yes and there are multiple samples across the dispersion (*ndisp* > 1), pixel shifts relative to the central sample are determined by crosscorrelation.

dispaxis = 2

Dispersion axis for 2D images. The image header keyword DISPAXIS has precedence over this value.

nspectra= 1, ndisp = 1

The number of spectral samples across the dispersion and the number of subpieces along the dispersion to divide the spectra into. If *nspectra* is greater than one then information about variations across the dispersion will be determined and if *ndisp* is greater than 1 information about variations along the dispersion will be determined. *Nspectra* applies only to 2D images. For 1D spectra in multispec format each line is used as a separate sample.

slit1 = INDEF, slit2 = INDEF

The lower and upper edges of the slit (or data region) in pixel coordinates (lines or columns) across the dispersion axis. A value of INDEF specifies the image edges.

logfile = "logfile"

File in which to record the results. If no file is specified no log output is produced.

CURSOR COMMANDS

All keys select an image and a sample (one of the *ndisp* samples along the dispersion and one of the *nspectra* samples across the dispersion) which is then generally highlighted.

```
? Help summary
b Best focus at each sample summary graphs
d Delete image, sample, or point
p Profiles at one sample for all images and all samples for one image
q Quit
r Redraw
s Spectra at one sample for all images and all samples for one image
u Undelete spectrum, sample, or point
w Profile widths verses focus and distribution of widths
z Zoom on a single sample showing correlation profile and spectrum
<space> Status line output for selected image and sample
```

DESCRIPTION

This task estimates the dispersion width of spectral lines in sequences of arc spectra taken at different focus settings (or with some other parameter varied). The widths can be measured at different spatial and dispersion positions, called "samples", on the detector. The width estimates are recorded and displayed graphically to investigate dependencies and determine appropriate settings for the spectrograph setup. The task may also measure dispersion shifts when multiple spectral samples are specified. This task does not measure the focus point-spread-function width across the dispersion.

The input images are specified with an image template list. The list may consist of explicit image names, wildcard templates, and @ files. A "focus" value is associated with each image. This may be any numeric quantity (integer or floating point). The focus values may be specified in several ways. If no value is given then index numbers are assigned to the images in the order in which they appear in the image list. A range list may be specified as described in the help topic **ranges**. This consists of individual values, ranges of values, a starting value and a step, and a range with a step. The elements of the list are separated by commas, ranges are separated by hyphens, and a character 'x'. Long range lists, such as a values, may be placed in a file and specified with the @<filename> convention. Finally, a parameter in the image header may be used for the focus values by simply specifying the parameter name.

Two dimensional long slit images are summed into one or more one dimensional spectra across the dispersion. The dispersion axis is defined either by the image header parameter DISPAXIS or the *dispaxis* task parameter with the image header parameter having precedence. The range of lines or columns across the dispersion to be used is specified by the parameters *slit1* and *slit2*. If specified as INDEF then the image limits are used. This range is then divided into the number of spectra given by the parameter *nspectra*. Use of more than one spectrum across the dispersion allows investigation of variations along the slit. In addition, if the parameter *shifts* is set the spectrum nearest the center is used as a reference against which shifts in the dispersion positions of the features in the other spectra are determined by crosscorrelation.

The conversion of two dimensional spectra to one dimensional spectra may also be performed separately using the tasks in the **apextract** package. This would be done typically for multifiber or echelle format spectra. If the two dimensional spectra have been extracted to one dimensional spectra in this way the task ignores the dispersion axis and number of spectra parameters. The data limits (*slit1* and *slit2*) are still used to select a range of lines in "multispec" format images. The *shifts* parameter also applies when there are multiple spectra per image. However, it does not make sense in the case of echelle spectra and so it should be set to no in that case.

In addition to dividing the spatial axis into a dispersion axis may also be divided into a set of subspectra. The number of divisions is specified by the *ndisp* parameter which applies to both long slit and 1D extracted spectra. When the dispersion axis is divided into more than one sample, the dependence of the dispersion widths and shifts along the dispersion may be investigated.

Each spectral sample has a low order continuum subtracted using a noninteractive iterative rejection algorithm to exclude the spectral lines. This technique is described further under the topic *continuum*. The continuum subtracted spectrum is then tapered with a cosine bell function and autocorrelated. The length of the taper and the range of shifts for the correlation is set by the *corwidth* parameter. This parameter should be only slightly bigger than the expected feature widths to prevent correlations between different spectral lines. The correlation profile is offset to zero at the edges of the profile and normalized to unity at the profile center. The profiles may be viewed as described below.

If there is more than one spatial sample the central spectrum is also crosscorrelated against the other spectra at the same dispersion sample. The crosscorrelation is computed in exactly the same way as the autocorrelation. The crosscorrelation profiles are only used for determining shifts between the two samples and are not used in the width determinations.

A cubic spline interpolator is fit to the profiles and this interpolation function is used to determine the profile width and center. The width is measured at a point given by the *level* parameter relative to the profile peak. It may be specified as a fraction of the peak if it is less than one or a percentage of the peak if it is greater than one. The default value of 0.5 selects the full width at half maximum. The autocorrelation width is divided by the square root of two to yield an estimate of the width of the spectral features in the spectrum in units of pixels.

Having computed the width and shift for each input image at each sample, the "best focus" values (focus, width, and shift) are estimated for each sample. As discussed later, it is possible to exclude some samples from this calculation by deleting them graphically. First the images with the smallest measured width at each distinct focus are selected since it is possible to input more than one image at same focus. The selected images are sorted by focus value and the image with the smallest width is found. If that image has the lowest or highest focus (which will always be the case if there are only one or two images) then the best focus, width, and shift are those measured for that image. If there are three or more focus values and the minimum width focus image is not an endpoint then parabolic interpolation is used to find the minimum width. The focus at this minimum width is the "best focus". The dispersion shift is the parabolic interpolation of the shifts at the best focus. The "average best focus" values are then the average of the "best focus" values over all samples.

After computing the correlation profiles, the profile widths and shifts, and the best focus values, an interactive graphics mode is entered. This is described in detail below. The graphics mode is exited with the 'q' key. At this point the results are written to the standard output (usually the terminal) and to a logfile if one is specified. The output begins with a banner identifying the task, version of IRAF, the user, and the date and time. The next line gives the best average focus and width. This banner also appears in all plots. Then each image is listed with the focus value and average width (over all samples). Finally the image with the smallest average width is identified and tables showing the width and shifts (if computed) at each sample position are printed. If there is only one sample then the tables are not output.

INTERACTIVE GRAPHICS MODE

There are five types of plot formats which are selected with the 'b', 'p', 's', 'w', and 'z' keys. The available formats and their content are modified depending on the number of images and the number of samples. If there is only one image or one sample per image some of the plot formats are not available. If there are a large number of images or a large number of samples the content of the plot formats may be abbreviated for legibility.

In all plots there is a concept of the current image and the current sample. In general there is an indication, usually a box, of which image and sample is the current one. The current image and sample are changed by pointing at a particular point, box, circle, or symbol for that image and sample and typing a key.

The 'b' key produces summary graphs of the best focus values (as described above) at each sample position. There must be more than one image and more than one sample (either along or across the

dispersion or both). This is the initial plot shown when this condition is satisfied. The central graph, which is always drawn, represents the best focus (smallest) width at each sample by circles of size proportional to the width. The position of the circle indicates the central line and column of the sample. If there are multiple samples across the dispersion and the *shifts* parameter is set then little vectors are also drawn from the center of the circle in the direction of the shift and with length proportional to the shift. If there are 5 or fewer samples in each dimension the values of the best focus and the width and shift (if computed and nonzero) at that focus, are printed on the graph next to the circles. If there are more samples this information may be obtained by pointing at the sample and typing the space key.

In addition to the spatial graph there may be graphs along the line or column axes. These graphs again show the widths as circles but one axis is either the line or column and the other axis is either the best focus value or the shift. The focus graph marks the best average focus (over all samples) by a dashed line and a solid line connects the mean focus at each column or line. The focus graphs will only appear if there is more than one sample along a particular image axis. The shift graphs will only appear if shifts are computed (*shifts* parameter is yes) and there is more than one sample along a particular dimension. Lines are drawn at zero shift and connecting the mean shift at each point along the spatial axis. Note that there is always a point at zero shift which is the reference sample.

The best focus graphs are the exception in showing a sample. When changing to one of the other plots based on a current image and sample the circle from the central spatial graph nearest the cursor is used (note that the other focus and shift graphs are ignored). The sample is defined by its spatial position and the image is the one with focus closest to the best focus value of that sample.

The 'w' key produces a graph showing the sample widths as a focus value. There must be more than one image and more than one sample for this type of graph. The top graph is a focus. The symbols are crosses except for the current image which is shown with pluses. The current sample is highlighted with a long dashed line connecting the widths for the current sample at each focus value and short dashed lines showing the best average focus and width.

The lower portion of the 'w' key are graphs showing the widths as circles with size proportional to the width and position corresponding to the spatial position of the sample in the image. If there are more than 5 samples in either dimension the graph is for the current image. Otherwise there is a box for each image with the focus value (provided there are not too many images) indicated. The circles are arranged as they would be spatially in columns and rows. The samples closest to the best focus are indicated by pluses. This allows seeing where the best focus values cluster. The current image and sample are indicated by highlighting boxes.

The 'p' key produces graphs of the autocorrelation profiles. This also requires more than one image and more than one sample. The top graph shows the profiles of all images at a particular sample and the bottom graph shows the profiles of all samples at a particular image. The bottom sample boxes are arranged in columns and rows in the same way the samples are distributed in the image. The current image and current sample are highlighted by a box.

The profiles are drawn with a solid line using the interpolator function and the actual pixel lags are indicated with pluses. The profiles are drawn shifted by the amount computed from the crosscorrelation. Note that the shift is added to the autocorrelation profile and the crosscorrelation profile is not what is plotted. The zero shift position is indicated by a vertical line. If there are less than 25 boxes the boxes are labeled by the width, shift (if nonzero), and focus.

The 's' key plot is similar to the 'p' key plot but shows the spectra rather than the profiles. The top graphs are the spectra of each image at a particular sample and the bottom graphs are the spectra of each sample for a particular image. The current image and sample are highlighted by a box.

The 'z' key graphs the autocorrelation profile and the spectrum of a single sample. This graph provides scales which are not provided with the 'p' and 's' graphs. If there is only one image and one sample then this is the only plot available.

It is possible to exclude some of the samples from the calculation of the best focus and best average focus values. This is done by deleting them using the 'd' key. When using the 'd' key you must specify the sample to be deleted in one of the graphs. You are then asked if only that sample (point) is to be deleted, if all samples from that image are to be deleted, or if the same sample from all images is to be deleted. The deleted data is no longer shown explicitly but the space occupied by the data is still present so that the data may be included again by typing the 'u' undelete key. When the task is exited with the 'q' key the printed and logged results will have the deleted data excluded.

The remaining cursor keys do the following. The '?' key gives a summary of the cursor keys. The 'r' key redraws the current plot. The space key prints information about the current sample. This is mostly used when there are too many images or samples to annotate the graphs with the focus, width, and shift. Finally the 'q' key quits the task.

EXAMPLES

1. A series of 2D focus images is obtained with focus values starting at 400 in steps of -50. The slit is between columns 50 and 130. There are 3 samples across the dispersion and 3 the dispersion.

```
cl> lpar specfocus
      images = "@imlist"      List of images
      (focus = "400x-50")    Focus values
      (corwidth = 20)         Correlation width
      (level = 0.5)           Percent or fraction of peak
      (shifts = yes)          Compute shifts across the disp?\n
      (dispaxis = 2)          Dispersion axis (long slit only)
      (nspectra = 3)          Number of spec samples (ls only)
      (ndisp = 3)             Number of dispersion samples
      (slit1 = 50)            Lower slit edge
      (slit2 = 130)          Upper slit edge\n
      (logfile = "logfile")   Logfile
      (mode = "ql")
cl> specfocus @imlist
<Interactive graphics which is exited with the 'q' key>
SPECFOCUS: NOAO/IRAF V2.10EXPORT valdes Thu 19:41:41 17-Sep-92
Best avg focus at 206.6584 with avg width of 2.91 at 50% of peak
```

-- Average Over All Samples

Image	Focus	Width
jdv011.imh	100.	3.78
jdv010.imh	150.	3.28
jdv009.imh	200.	2.95
jdv008.imh	250.	3.17
jdv007.imh	300.	3.41
jdv006.imh	350.	3.74
jdv005.imh	400.	4.16

-- Image jdv009.imh at Focus 200. --

Width at 50% of Peak:

	Columns		
	50-76	77-103	104-130
Lines	+	-----	-----
2-267	2.93	2.58	2.74
268-533	3.17	2.76	2.89
534-799	3.77	2.23	3.50

Position Shifts Relative To Central Sample:

Lines	Columns		
	50-76	77-103	104-130
2-267	0.68	0.00	0.18
268-533	0.64	0.00	0.13
534-799	0.92	0.00	0.16

SEE ALSO

imexamine, implot, ranges, splot

The IRAF/NOAO Spectral World Coordinate Systems

1. Types of Spectral Data

Spectra are stored as one, two, or three dimensional images with one axis being the dispersion axis. A pixel value is the flux over some interval of wavelength and position. The simplest example of a spectrum is a one dimensional image which has pixel values as a function of wavelength.

There are two types of higher dimensional spectral image formats. One type has spatial axes for the other dimensions and the dispersion axis may be along any of the image axes. Typically this type of format is used for long slit (two dimensional) and Fabry-Perot (three dimensional) spectra. This type of spectra is referred to as *spatial* spectra and the world coordinate system (WCS) format is called *ndspec*. The details of the world coordinate systems are discussed later.

The second type of higher dimensional spectral image consists of multiple, independent, one dimensional spectra stored in the higher dimensions with the first image axis being the dispersion axis; i.e. each line is a spectrum. This format allows associating many spectra and related parameters in a single data object. This type of spectra is referred to as *multispec* and there are two coordinate system formats, *equispec* and *multispec*. The *equispec* format applies to the common case where all spectra have the same linear dispersion relation. The *multispec* format applies to the general case of spectra with differing dispersion relations or non-linear dispersion functions. These multi-spectrum formats are important since maintaining large numbers of spectra as individual one dimensional images is very unwieldy for the user and inefficient for the software.

Examples of multispec spectral images are spectra extracted from a multi-fiber or multi-aperture spectrograph or orders from an echelle spectrum. The second axis is some arbitrary indexing of the spectra, called *apertures*, and the third dimension is used for associated quantities. The IRAF **apextract** package may produce multiple spectra from a CCD image in successive image lines with an optimally weighted spectrum, a simple aperture sum spectrum, a background spectrum, and sigma spectrum as the associated quantities along the third dimension of the image.

Many **onedspec** package tasks which are designed to operate on individual one dimensional spectra may operate on spatial spectra by summing a number of neighboring spectra across the dispersion axis. This eliminates the need to "extract" one dimensional spectra from the natural format of this type of data in order to use tasks oriented towards the display and analysis of one dimensional spectra. The dispersion axis is either given in the image header by the keyword DISPAXIS or the package *dispaxis* parameter. The summing factors across the dispersion are specified by the *nsum* package parameter. See "help onedspec .package" for information on these parameters.

One dimensional spectra, whether from multispec spatial spectra, have several associated quantities which may appear in the image header as part of the coordinate system description. The primary identification of a spectrum is an integer aperture number. This number must be unique within a single image. There is also an integer beam number used for various purposes such as discriminating object, sky, and arc spectra in multi-fiber/multi-aperture data or to identifying the order number in echelle data. For spectra summed from spatial spectra the aperture number is the central line, column, or band. In 3D images the aperture index wraps around the lowest non-dispersion axis. Since most one dimensional spectra are derived from an integration over one or more spatial axes, two additional aperture parameters record the aperture limits. These limits refer to the original pixel limits along the spatial axis. This information is primarily for record keeping but in some cases it is used for spatial interpolation during dispersion calibration. These values are set either by the **apextract** tasks or when summing neighboring vectors in spatial spectra.

An important task to be aware of for manipulating spectra between image formats is **scopy**. This task allows selecting spectra from multispec images and grouping them in various ways and also "extracts" apertures from long slit and 3D spectra simply and without resort to the more general **apextract** package.

2. World Coordinate Systems

IRAF images have three types of coordinate systems. The pixel array coordinates of an image or image section, i.e. the lines and columns, are called the *logical* coordinates. The logical coordinates of individual pixels change as sections of the image are used or extracted. Pixel coordinates are tied to the data, i.e. are fixed to features in the image, are called *physical* coordinates. Initially the logical and physical coordinates are the equivalent but differ when image sections or other tasks which modify the sampling of the pixels are applied.

The last type of coordinate system is called the *world* coordinate system. Like the physical coordinates, the world coordinates are tied to the features in the image and remain unchanged when sections of the image are used or extracted. If a world coordinate system is not defined for an image, the physical coordinate system is considered to be the world coordinate system. In spectral images the world coordinate system includes dispersion coordinates such as wavelengths. In many tasks outside the spectroscopy packages, for example the **plot**, **tv** and **images** packages, one may select the type of coordinate system to be used. To make plots and get coordinates in dispersion units for spectra with these tasks one selects the "world" system. The spectral tasks always use world coordinates.

The coordinate systems are defined in the image headers using a reserved keywords which are set, changed, and updated by various tasks. Some of the keywords consist of simple single values following the FITS convention. Others, the WAT keywords, encode long strings of information, one for each coordinate axis and one applying to all axes, into a set of sequential keywords. The values of these keywords must then be pasted together to recover the string. The long strings contain multiple pieces called WCS *attributes*. In general the WCS keywords should be left to IRAF tasks to modify. However, if one wants modify them directly some tasks which may be used are **hedit**, **hfix**, **wcsedit**, **wcsreset**, **specshift**, **dopcor**, and **sapertures**. The first two are useful for the simple keywords, the two "wcs" tasks are useful for the linear ndspec and equispec formats, the next two are for the common cases of shifting the coordinate zero point or applying a doppler correction, and the last one is the one to use for the more complex multispec format attributes.

3. Physical Coordinate System

The physical coordinate system is used by the spectral tasks when there is no dispersion coordinate information (such as before dispersion calibration), to map the physical dispersion axis to the logical dispersion axis, and in the multispec world coordinate system dispersion functions which are defined in terms of physical coordinates.

The transformation between logical and physical coordinates is defined by the header keywords LTVi, LTMi_j (where i and j are axis numbers) through the vector equation

$$l = |m| * p + v$$

where l is a logical coordinate vector, p is a coordinate vector, v is the origin translation vector specified by the LTV keywords and |m| is the scale/rotation matrix specified by the LTM keywords. For spectra rotation terms (nondiagonal matrix elements) generally do not make sense (in fact many tasks will not work if there is a rotation) so the transformations along each axis are given by the linear equation

where l is a logical coordinate vector, p is a v is the origin translation vector specified by the LTV keywords and |m| is the scale/rotation matrix specified by the LTM keywords. For spectra a rotation term (nondiagonal matrix elements) generally does not make sense (in fact many tasks will not work if there is a transformations along each axis are given by the linear equation

$$l_i = LTM_{i_i} * p_i + LTV_i.$$

If all the LTM/LTV keywords are missing they are assumed to have zero values except that the diagonal matrix terms, LTMi_i, are assumed to be 1. Note that if some of the keywords are present then a take the value zero which generally causes an arithmetic or matrix inversion error in the IRAF tasks.

The dimensional mapping between logical and physical axes is given by the keywords WCSDIM and WAXMAP01. The WCSDIM keyword gives the dimensionality of the physical and world coordinate system. There must be coordinate information for that many axes in the header (though some may be missing and take their default values). If the WCSDIM keyword is missing it assumed to be the same as the logical image dimensionality.

The syntax of the WAXMAP keyword are pairs of integer values, one for each physical axis. The first number of each pair indicates which current *logical* axis corresponds to the original *physical* axis (in order) or zero if that axis is missing. When the first number is zero the second number gives the offset to the element of the original axis which is missing. As an example consider a which the second plane is extracted (an IRAF image section of [*,2,*]). The keyword would then appear as WAXMAP01 = '1 0 0' is missing the mapping is 1:1; i.e. the dimensionality and order of the axes are the same.

The dimensional mapping is important because the dispersion axis for the nspec spatial spectra as specified by the DISPAXIS keyword or task parameter, or the axis definitions for the equispec and or multispec formats are always in terms of the original physical axes.

4. Linear Spectral World Coordinate Systems

When there is a linear or logarithmic relation between pixels and dispersion coordinates which is the same for all spectra the WCS header format is simple and uses the FITS convention (with the CD matrix keywords proposed by Hanisch and Wells 1992) for the logical pixel to world coordinate transformation. This format applies to one, two, and three dimensional data. The higher dimensional data may have either linear spatial axes or the equispec format where each one dimensional spectrum stored along the lines of the image has the same dispersion.

The FITS image header keywords describing the spectral world coordinates are CTYPEi, CRPIXi, CRVALi, and CDi_j where i and j are axis numbers. As with the physical coordinate transformation the nondiagonal or rotation terms are not expected in the spectral WCS and may cause problems if are not zero. The CTYPEi keywords will have the value LINEAR to identify the type of coordinate system. The transformation between dispersion coordinate, wi, and logical pixel coordinate, li, along axis i

$$w_i = CRVAL_i + CD_{i,i} * (l_i - CRPIX_i)$$

If the keywords are missing then the values are assumed to be zero except for the diagonal elements of the scale/rotation matrix, the CDi_i, which are assumed to be 1. If only some of the keywords are present then any missing CDi_i keywords will take the value 0 which will cause IRAF tasks to fail with arithmetic or matrix inversion errors. If the CTYPEi keyword is missing it is assumed to be "LINEAR".

If the pixel sampling is logarithmic in the dispersion coordinate, as required for radial velocity cross-correlations, the WCS coordinate values are logarithmic and wi (above) is the logarithm of the dispersion coordinate. The spectral tasks (though not other tasks) will recognize this case and automatically apply the anti-log. The two types of pixel sampling are identified by the value of the keyword DC-FLAG. A value of 0 defines a linear sampling of the dispersion and a logarithmic sampling of the dispersion. Thus, in all cases the spectral tasks will display and analyze the spectra in the same dispersion units regardless of the pixel sampling.

Other keywords which may be present are DISPAXIS for 2 spatial spectra, and the WCS attributes "system", "wtype", "label", and "units". The system attribute will usually have the value "world" for spatial spectra and "equispec" for equispec spectra. The wtype attribute will have the value

"linear". Currently the label will be either "Pixel" or "Wavelength" and the units will be "Angstroms" for dispersion corrected spectra. In the future there will be more generality in the units for dispersion calibrated spectra.

Figure 1 shows the WCS keywords for a two dimensional long slit spectrum. The coordinate system is defined to be a generic "world" system and the wtype attributes and CTYPE keywords define the axes to be linear. The other attributes define a label and unit for the second axis, which is dispersion axis as indicated by the DISPAXIS keyword. The LTM/LTV keywords in this example show that a subsection of the original image has been extracted with a factor of 2 block averaging along the dispersion axis. The dispersion coordinates are given in terms of the *logical* pixel coordinates by the FITS keywords as defined previously.

Figure 1: Long Slit Spectrum

```

WAT0_001= 'system=world'
WAT1_001= 'wtype=linear'
WAT2_001= 'wtype=linear label=Wavelength units=Angstroms'
WCSDIM   =                2
DISPAXIS =                2
DC-FLAG  =                0

CTYPE1   = 'LINEAR  '
LTV1     =                -10.
LTM1_1   =                1.
CRPIX1   =                -9.
CRVAL1   =    19.5743865966797
CD1_1    =    1.01503419876099

CTYPE2   = 'LINEAR  '
LTV2     =                -49.5
LTM2_2   =                0.5
CRPIX2   =                -49.
CRVAL2   =    4204.462890625
CD2_2    =    12.3337936401367

```

Figure 2 shows the WCS keywords for a three dimensional image where each line is an independent spectrum or associated data but where all spectra have the same linear dispersion. This type of coordinate system has the system name "equispec". The ancillary information about each aperture is found in the APNUM keywords. These give the aperture number, beam number, and extraction limits. In this example the LTM/LTV keywords have their default values; i.e. the logical and physical coordinates are the same.

Figure 2: Equispec Spectrum

```

WAT0_001= 'system=equispec'
WAT1_001= 'wtype=linear label=Wavelength units=Angstroms'
WAT2_001= 'wtype=linear'
WAT3_001= 'wtype=linear'
WCSDIM   =                3
DC-FLAG  =                0
APNUM1   = '41 3 7.37 13.48'
APNUM2   = '15 1 28.04 34.15'
APNUM3   = '33 2 43.20 49.32'

CTYPE1   = 'LINEAR  '
LTM1_1   =                1.
CRPIX1   =                1.
CRVAL1   =    4204.463
CD1_1    =    6.166897000000001

```

```

CTYPE2 = 'LINEAR'
LTM2_2 = 1.
CD2_2 = 1.

CTYPE3 = 'LINEAR'
LTM3_3 = 1.
CD3_3 = 1.

```

5. Multispec Spectral World Coordinate System

The *multispec* spectral world coordinate system applies only to one dimensional spectra; i.e. there is no analog for the spatial type spectra. It is used either when there are multiple 1D spectra with differing dispersion functions in a single image or when the dispersion functions are nonlinear.

The multispec coordinate system is always two dimensional though there may be an independent third axis. The two axes are coupled and they both have axis type "multispec". When the image is one dimensional the physical line is given by the dimensional reduction keyword WAXMAP. The second, line axis, has world coordinates of aperture number. The aperture numbers are integer values and need not be in any particular order but do need to be unique. This aspect of the WCS is not of particular user interest but applications use the inverse world to physical transformation to select a spectrum line given a specified aperture.

The dispersion functions are specified by attribute strings with the identifier *specN* where N is the *physical* image line. The attribute strings contain a series of numeric fields. The fields are indicated symbolically as follows.

```
specN = ap beam dtype w1 dw nw z aplo waphigh [functions_i]
```

where there are zero or more functions having the following fields,

```
function_i = wt_i w0_i ftype_i [parameters] [coefficients]
```

The first nine fields in the attribute are common to all the dispersion functions. The first field of the WCS attribute is the aperture number, the second field is the beam number, and the third field is the dispersion type with the same function as DC-FLAG in the *nspec* and *equispec* formats. A value of -1 indicates the coordinates are not dispersion coordinates (the spectrum is not dispersion calibrated), a value of 0 indicates linear dispersion sampling, a value of 1 indicates log-linear dispersion sampling, and a value of 2 indicates a dispersion.

The next two fields are the dispersion coordinate of the first *physical* pixel and the average dispersion interval per *physical* pixel. For linear and log-linear dispersion types the dispersion parameters are exact while for the nonlinear dispersion functions they are approximate. The next field is the number of valid pixels, hence it is possible to have spectra with varying lengths in the same image. In that case the image is as big as the biggest spectrum and the number of pixels selects the actual data in each image line. The next (seventh) field is a doppler factor. This doppler factor is applied to all dispersion coordinates by multiplying by $1/(1+z)$ (assuming wavelength dispersion units). Thus a value of 0 is no doppler correction. The last two fields are extraction aperture limits as discussed previously.

Following these fields are zero or more function descriptions. For linear or log-linear dispersion coordinate systems there are no function fields. For the nonlinear dispersion systems the function fields specify a zero point offset, the type of dispersion function, and the parameters and coefficients describing it. The function type codes, *ftype_i*, are 1 for a chebyshev polynomial, 2 for a legendre polynomial, 3 cubic spline, 4 for a linear spline, 5 for a pixel coordinate array, and 6 for a sampled coordinate array. The number of

fields before the next function and the number of functions are determined from the parameters of the preceding function until the end of the attribute is reached.

The equation below shows how the final wavelength is computed based on the nfunc individual dispersion functions $W_i(p)$. Note that this is completely general in that different function types may be combined. However, in practice when multiple functions are used they are generally of the same type and represent a calibration before and after the actual object observation with the weights based on the relative time difference between the calibration dispersion functions and the object observation.

$$w = \text{sum from } i=1 \text{ to } n\text{func} \{wt_i * (w0_i + W_i(p)) / (1 + z)\}$$

The multispec coordinate systems define a transformation between physical pixel, p , and world coordinates, w . Generally there is an intermediate coordinate system used. The following equations define these coordinates. The first one shows the transformation between logical, l , p , coordinates based on the LTM/LTV keywords. The polynomial functions are defined in terms of a normalized coordinate, n , as shown in the second equation. The normalized coordinates run between -1 and 1 range of physical coordinates, $pmin$ and $pmax$ which are parameters of the function, upon which the coefficients were defined. The spline functions map the physical range into an index over the number of evenly divided spline pieces, $npieces$, which is a parameter of the function. This mapping is shown in the third and fourth equations where s is the continuous spline coordinate and j than or equal to s .

$$\begin{aligned} p &= (l - LTV1) / LTM1_1 \\ n &= (p - pmiddle) / (prange / 2) \\ &= (p - (pmax+pmin)/2) / ((pmax-pmin) / 2) \\ s &= (p - pmin) / (pmax - pmin) * npieces \\ j &= \text{int}(s) \end{aligned}$$

5.1 Linear and Log Linear Dispersion Function

The linear and log-linear dispersion functions are described by a wavelength at the first *physical* pixel and a *physical* pixel. A doppler correction may also be applied. The equations below show the two forms. Note that the coordinates returned are always wavelength even though the pixel sampling and the dispersion parameters may be log-linear.

$$\begin{aligned} w &= (w1 + dw * (p - 1)) / (1 + z) \\ w &= 10 ** \{(w1 + dw * (p - 1)) / (1 + z)\} \end{aligned}$$

Figure 3 shows an example from a multispec image with independent linear dispersion coordinates. This is a linearized echelle spectrum where each order (identified by the beam number) is stored as a separate image line.

Figure 3: Echelle Spectrum with Linear Dispersion Function

```

WAT0_001= 'system=multispec'
WAT1_001= 'wtype=multispec label=Wavelength units=Angstroms'
WAT2_001= 'wtype=multispec spec1 = "1 113 0 4955.44287109375 0.05...'
WAT2_002= '5 256 0. 23.22 31.27" spec2 = "2 112 0 4999.0810546875...'
WAT2_003= '58854293 256 0. 46.09 58.44" spec3 = "3 111 0 5043.505...'
WAT2_004= '928358078002 256 0. 69.28 77.89"
WCSDIM   =                2

CTYPE1   = 'MULTISPE'
LTM1_1   =                1.
CD1_1    =                1.

```

```

CTYPE2 = 'MULTISPE'
LTM2_2 = 1.
CD2_2 = 1.

```

5.2 Chebyshev Polynomial Dispersion Function

The parameters for the chebyshev polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, ci. The equation below shows how to evaluate the function using an iterative definition where $x_1 = x_2 = n$, and $x_i = 2 * n * x_{i-1} - x_{i-2}$.

The parameters for the chebyshev polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, c_i. The equation below shows how to evaluate the function using an iterative definition where $x_1 = 1$, $x_2 = n$, and $x_i = 2 * n * x_{i-1} - x_{i-2}$.

$$W = \text{sum from } i=1 \text{ to order } \{c_i * x_i\}$$

5.3 Legendre Polynomial Dispersion Function

The parameters for the legendre polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, c_i. The equation below shows how to evaluate the function using an iterative definition where $x_1 = 1$, $x_2 = x_i = ((2i-3)*n*x_{i-1} - (i-2)*x_{i-2})/(i-1)$.

$$W = \text{sum from } i=1 \text{ to order } \{c_i * x_i\}$$

Figure 4 shows an example from a multispec image with independent nonlinear dispersion coordinates. This is again from an echelle spectrum. Note that the IRAF **echelle** package determines a function, in this case a bidimensional legendre polynomial, with the independent variables being the order number and the extracted pixel coordinate. To assign and store this function in the image is simply a matter of collapsing the two dimensional dispersion function by fixing the order number and combining all the terms with the same order.

```

Figure 4: Echelle Spectrum with Legendre Polynomial Function
WAT0_001= 'system=multispec'
WAT1_001= 'wtype=multispec label=Wavelength units=Angstroms'
WAT2_001= 'wtype=multispec spec1 = "1 113 2 4955.442888635351 0.05...'
WAT2_002= '83 256 0. 23.22 31.27 1. 0. 2 4 1. 256. 4963.0163112090...'
WAT2_003= '976664 -0.3191636898579552 -0.8169352858733255" spec2 =...'
WAT2_004= '9.081188912082 0.06387049476832223 256 0. 46.09 58.44 1...'
WAT2_005= '56. 5007.401409453303 8.555959076467951 -0.176732458267...'
WAT2_006= '09935064388" spec3 = "3 111 2 5043.505764869474 0.07097...'
WAT2_007= '256 0. 69.28 77.89 1. 0. 2 4 1. 256. 5052.586239197408 ...'
WAT2_008= '271 -0.03173489817897474 -7.190562320405975E-4"
WCSDIM = 2

CTYPE1 = 'MULTISPE'
LTM1_1 = 1.

```

```

CD1_1    =          1.

CTYPE2   = 'MULTISPE'
LTM2_2   =          1.
CD2_2    =          1.

```

5.4 Linear Spline Dispersion Function

The parameters for the linear spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+1 coefficients, c_i. The two coefficients used in a linear combination are selected based on the spline coordinate, where a are the fractions of the interval in the spline piece between the spline knots, a=(j+1)-s, b=s-j, and x_0=a, and x_1=b.

$$W = \text{sum from } i=0 \text{ to } 1 \{c_{(i+j)} * x_i\}$$

5.5 Cubic Spline Dispersion Function

The parameters for the cubic spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+3 coefficients, c_i. The four coefficients used are selected based on the spline coordinate. The fractions of the interval between the integer spline knots are given by a and b, a=(j+1)-s, b=s-j, and x_0=a sup 3, x_1=(1+3*a*(1+a*b)), x_2=(1+3*b*(1+a*b)), and x_3=b**3.

The parameters for the cubic spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+3 coefficients, c_i. The four coefficients used are selected based on the spline coordinate. The fractions of the interval between the integer spline knots are given by a and b, a=(j+1)-s, b=s-j, and x_0=a**3, x_1=(1+3*a*(1+a*b)), x_2=(1+3*b*(1+a*b)), and x_3=b**3.

$$W = \text{sum from } i=0 \text{ to } 3 \{c_{(i+j)} * x_i\}$$

5.6 Pixel Array Dispersion Function

The parameters for the pixel array dispersion function consists of just the number of coordinates ncoords. Following this are the wavelengths at integer physical pixel coordinates starting with 1. To evaluate a wavelength at some physical coordinate, not necessarily an integer, a linear interpolation is used between the nearest integer physical coordinates and the desired physical coordinate where a fractional intervals k=int(p), a=(k+1)-p, b=p-k, and x_0=a, and x_1=b.

$$W = \text{sum from } i=0 \text{ to } 1 \{c_{(i+j)} * x_i\}$$

5.7 Sampled Array Dispersion Function

The parameters for the sampled array dispersion function consists of the number of coordinate pairs, ncoords, and a dummy field. Following these are the physical coordinate and wavelength pairs which are in increasing order. The nearest physical coordinates to the desired physical coordinate are located and a is computed between the two sample points.

NAME

splot -- plot and analyze spectra

USAGE

splot images [line [band]]

PARAMETERS**images**

List of images (spectra) to plot. If the image is 2D or 3D the line and band parameters are used. Successive images are plotted following each 'q' cursor command. One may use an image section to select a desired column, line, or band but the full image will be in memory and any updates to the spectrum will be part of the full image.

line, band

The image line/aperture and band to plot in two or three dimensional images. For multiaperture spectra the aperture specified by the line parameter is first sought and if not found the specified image line is selected. For other two dimensional images, such as long slit spectra, the line parameter specifies a line or column. Note that if the line and band parameters are specified on the command line it will not be possible to change them interactively.

units = ""

Dispersion coordinate units for the plot. If the spectra have known units, currently this is generally Angstroms, the units may be converted to other units for plotting as specified by this task parameter. If this parameter is the null string and the world coordinate system attribute "units_display" is defined then that will be used. If both this task parameters and "units_display" are not given then the spectrum dispersion units will be used. The units may also be changed interactively. See the units section of the **package** help for a further description and available units.

options = "auto" [auto,zero,xydraw,histogram,nosysid,wreset,flip,overplot]

A list of zero or more, possibly abbreviated, options. The options can also be toggled with colon commands. The currently defined options are "auto", "zero", "xydraw", "histogram", "nosysid", "wreset", "flip", and "overplot". Option "auto" automatically replots the graph whenever changes are made. Otherwise the graph is replotted with keystrokes 'c' or 'r'. Option "zero" makes the initial minimum y of the graphs occur at zero. Otherwise the limits are set automatically from the range of the data or the ymin parameter. Option "xydraw" changes the 'x' draw key to use both x and y cursor values for drawing rather than the nearest pixel value for the y value. Option "histogram" plots the spectra in a histogram style rather than connecting the pixel centers. Option "nosysid" excludes the system banner from the graph title. Option "wreset" resets the graph limits to those specified by the xmin, xmax, ymin, ymax parameters whenever a new spectrum is plotted. The "flip" option selects that initially the spectra be plotted with decreasing wavelengths. The options may be queried and changed interactively. The "overplot" options overplots all graphs and a screen erase only occurs with the redraw key.

xmin = INDEF, xmax = INDEF, ymin = INDEF, ymax = INDEF

The default limits for the initial graph. If INDEF then the limit is determined from the range of the data (autoscaling). These values can be changed interactively with 'w' window key options or the cursor commands ":xwindow" and ":ywindow" (see **gtools**).

save_file = "splot.log"

The file to contain any results generated by the equivalent width or deblending functions. Results are added to this file until the file is deleted. If the filename is null (""), then no results are saved.

graphics="stdgraph"

Output graphics device: one of "stdgraph", "stdplot", "stdvdm", or name.

cursor=""

Graphics cursor input. When null the standard cursor is used otherwise the specified file is used.

The following parameters are used for error estimates in the 'd', 'k', and 'e' key measurements. See the ERROR ESTIMATES section for a discussion of the error estimates.

nerrsamples= 0

Number of samples for the error computation. A value less than 10 turns off the error computation. A value of ~10 does a rough error analysis, a value of ~50 does a reasonable error analysis, and a detailed error analysis. The larger this value the longer the analysis takes.

sigma0 = INDEF, **invgain** = INDEF

The pixel sigmas are modeled by the formula:

$$\text{sigma}^2 = \text{sigma0}^2 + \text{invgain} * I$$

where I is the pixel value and "**2" means the square of the quantity. If either parameter is specified as INDEF or with a value less than zero then no sigma estimates are made and so no error estimates for the measured parameters are made.

The following parameters are for the interactive curve fitting function entered with the 't' key. This function is usually used for continuum fitting. The values of these parameters are updated during the fitting. See **icfit** for additional details on interactive curve fitting.

function="spline3"

Function to be fit to the spectra. The functions are "legendre" (legendre polynomial), "chebyshev" (chebyshev polynomial), "spline1" (linear spline), and "spline3" (cubic spline). The functions may be abbreviated.

order= 1

The order of the polynomials or the number of spline pieces.

low_reject= 2., **high_reject**= 4.

Rejection limits below and above the fit in units of the residual sigma. Unequal limits are used to reject spectral lines on one side of the continuum during continuum fitting.

niterate= 10

Number of rejection iterations.

grow = 1.

When a pixel is rejected, pixels within this distance of the rejected pixel are also rejected.

markrej= yes

Mark rejected points? If there are many rejected points it desired to not mark rejected points.

The following parameters are used to overplot standard star fluxes with the 'y' key. See **standard** for more information about these parameters.

star_name

Query parameter for the standard star fluxes to be overplotted. Unrecognized names or a "?" will print a list of the available stars in the specified calibration directory.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

caldir = "_caldir"

The standard star calibration directory. The default value redirects the value to the parameter of the same name in the package parameters.

fnuzero = 3.68e-20

The absolute flux per unit frequency at a magnitude of zero used to convert the calibration magnitudes to absolute flux.

The following parameters are used for queries in response to particular keystrokes.

next_image

In response to 'g' (get next image) this parameter specifies the image.

new_image

In response to 'i' (write current spectrum) this parameter specifies the name of a new image to create or existing image to overwrite.

overwrite = no

Overwrite an existing output image? If set to yes it is possible to write back into the input spectrum or to some other existing image. Otherwise the user is queried again for a new image name.

spec2

When adding, subtracting, multiplying, or dividing by a ('+', '-', '*', '/') keys in the 'f' mode) this parameter is used to get the name of the second spectrum.

constant

When adding or multiplying by a constant ('p' or 'm' keys in the 'f' mode) the parameter is used to get the constant.

wavelength

This parameter is used to get a dispersion coordinate value during deblending or when changing the dispersion coordinates with 'u'.

linelist

During deblending this parameter is used to get a peak values, profile types, and widths.

wstart,wend, dw

In response to 'p' (convert to a linear wavelength scale) these parameters specify the starting wavelength, ending wavelength, and wavelength per pixel.

boxsize

In response to 's' (smooth) this parameter specifies the box size in pixels to be used for the boxcar smooth. The value must be odd. If an even value is specified the next larger odd value is actually used.

DESCRIPTION

Splot provides an interactive facility to display and analyze spectra. See also **bplot** for a version of this task useful for making many plots noninteractively. Each spectrum in the image list is displayed successively. To quit the current image and go on to the next the 'q' cursor command is used. If an image is two-dimensional, such as with multiple aperture or long slit spectra, the aperture or image column/line to be displayed is needed. If the image is three-dimensional, such as with the extra information produced by **apextract**, the band is needed. These parameters are queried unless specified on the command line. If given on the command line it will not be possible to change them interactively.

The plots are made on the specified graphics device which is usually to the graphics terminal. The initial plot limits are set with the parameters *xmin*, *xmax*, *ymin*, and *ymax*. If a is determined from the range of the data. The "zero" option may also be set in the *options* parameter to set the lower intensity limit to zero. Other options that may be set to control the initial plot are to exclude the system identification banner, and to select a histogram line type instead of connecting the pixel centers. The dispersion units used in the plot are set by the *units* parameter. This allows converting to units other than those in which the dispersion coordinates are defined in the spectra.

The *option* parameter, mentioned in the previous paragraph, is a list of zero or more options. As previously noted, some of the options control the initial appearance of the plots. The "auto" option determines how frequently plots are redrawn. For slow terminals or via modems one might wish to minimize the redrawing. The default, however, is to redraw when changes are made. The "xydraw" parameter is specific to the 'x' key.

After the initial graph is made an interactive cursor loop is entered. The *cursor* parameter may be reset to read from a file but generally the graphics device cursor is read. The cursor loop takes single keystroke commands and typed in commands begun with a colon, called colon commands. These commands are described below and a summary of the commands may be produced interactively with the '?' key or a scrolling help on the status line with the '/' key.

Modifications to the spectra being analyzed may be saved using the 'i' key in a new, the current, or other existing spectra. A new image is created as a new copy of the current spectrum and so if the current spectrum is part of a multiple spectrum image (including a other spectra are copied. If other spectra in the same image are then modified and saved use the overwrite option to replace then in the new output image. If the output spectrum already exists then the *overwrite* flag must be set to allow modifying the data. This includes the case when the output spectrum is the same as the input spectrum. The only odd case here is when the input spectrum is one dimensional and the output spectrum is two dimensional. In this case the user is queried for the line to be written.

The other form of output, apart from that produced on the terminal, are measurements of equivalent widths, and other analysis functions. This information will be recorded in the *save_file* if specified.

The following keystrokes are active in addition to the normal IRAF cursor facilities (available with ":.help"):

? Page help information.

/ Cycle through short status line help.

<space>

The space bar prints the cursor position and value of the nearest pixel.

- a** Expand and autoscale to the data range between two cursor positions. See also 'w', and 'z'. Selecting no range, that is the two cursor positions the same, produces an autoscale of the whole spectrum.
- b** Set the plot base level to zero rather than autoscaling.
- c** Clear all windowing and redraw the full current spectrum. This redraws the spectrum and cancels any effects of the 'a', 'z', and 'w' keys. The 'r' key is used to redraw the spectrum with the current windowing.
- d** Mark two continuum points and fit (deblend) multiple line profiles. The center, continuum at the center, core intensity, integrated flux, equivalent width, FWHMs for each profile are printed and saved in the log file. See 'k' for fitting a '-' to subtract the fitted profiles.
- e** Measure equivalent width by marking two continuum points around the line to be measured. The linear continuum is subtracted and the flux is determined by simply summing the pixels with partial pixels at the ends. Returned values are the line center, continuum at the region center, flux above or below the continuum, and the equivalent width.
- f** Enter arithmetic function mode. This mode allows arithmetic functions to be applied to the spectrum. The pixel values are modified according to the function request and may be saved as a new spectrum with the 'i' command. Operations with a second spectrum are done in wavelength space and the second spectrum is automatically resampled if necessary. If one spectrum is longer than the other, only the smaller number of pixels are affected. To exit this mode type 'q'.

The following keystrokes are available in the function mode. Binary operations with a constant or a second spectrum produce a query for the constant value or spectrum name.

- a** Absolute value
 - d** Power of base 10 (inverse log base 10)
 - e** Power of base e (inverse log base e)
 - i** Inverse/reciprocal (values equal to zero are set to 0.0 in the inverse)
 - l** Log base 10 (values less than or equal to 0.0 are set to -0.5)
 - m** Multiply by a constant (constant is queried)
 - n** Log base e (values less than or equal to 0.0 are set to -0.5)
 - p** Add by a constant (constant is queried)
 - q** Quit Function mode
 - s** Square root (values less than 0.0 are set to 0.0)
 - +** Add another spectrum
 - Subtract another spectrum
 - *** Multiply by another spectrum
 - /** Divide by another spectrum
- g** Get another spectrum. The current spectrum is replaced by the new spectrum. The aperture/line and band are queried is necessary.

- h** Measure equivalent widths assuming a gaussian profile with the width measured at a specified point. Note that this is not a gaussian fit (see 'k' to fit a gaussian)! The gaussian profile determined here may be subtracted with the '-' key. A second cursor key is requested with one of the following values:
 - a** Mark the continuum level at the line center and use the LEFT half width at the half flux point.
 - b** Mark the continuum level at the line center and use the RIGHT half width at the half flux point.
 - c** Mark the continuum level at the line center and use the FULL width at the half flux point.
 - l** Mark a flux level at the line center relative to a and use the LEFT width at that flux point.
 - r** Mark a flux level at the line center relative to a and use the RIGHT width at that flux point.
 - k** Mark a flux level at the line center relative to a and use the FULL width at that flux point.
- i** Write the current spectrum out to a new or existing image. The image name is queried and overwriting must be confirmed.
- j** Set the value of the nearest pixel to the x cursor to the y
- k + (g, l or v)** Mark two continuum points and fit a single line profile. The second key selects the type of profile: g for gaussian, l for lorentzian, and v for voigt. Any other second key defaults to gaussian. The center, continuum at the center, core intensity, integrated flux, equivalent width, and FWHMs are printed and saved in the log file. See 'd' for fitting multiple profiles and '-' to subtract the fit.
- l** Convert to flux per unit wavelength (f-lambda). The spectrum is assumed to be flux calibrated in flux per unit frequency (f-nu). See also 'n'.
- m** Compute the mean, RMS, and signal-to-noise over a x cursor positions.
- n** Convert to flux per unit frequency (f-nu). The spectrum is assumed to be flux calibrated in flux per unit wavelength (f-lambda). See also 'l'.
- o** Set overplot flag. The next plot will overplot the current plot. Normally this key is immediately followed by one of 'g', '#', '%', '(', or ')'. The ":overplot" colon command and overplot parameter option may be used to set overplotting to be permanently on.
- p** Define a linear wavelength scale. The user is queried for a wavelength and an ending wavelength. If either (though not both) are specified as INDEF a dispersion is queried for and used to compute an endpoint. A wavelength scale set this way will be used for other spectra which are not dispersion corrected.
- q** Quit and go on to next input spectrum. After the last spectrum exit.
- r** Redraw the spectrum with the current windowing. To redraw the full spectrum and cancel any windowing use the 'c' key.
- s** Smooth via a boxcar. The user is prompted for the box size.
- t** Fit a function to the spectrum using the ICFIT mode. Typically interactive rejection is used to exclude spectra lines from the fit in order to fit a smooth continuum. A second keystroke selects what to do with the fit.

- / Normalize by the fit. When fitting the continuum this continuum normalizes the spectrum.
- Subtract the fit. When fitting the continuum this continuum subtracts the spectrum.
- f** Replace the spectrum by the fit.
- c** Clean the spectrum by replacing any rejected points by the fit.
- n** Do the fitting but leave the spectrum unchanged (a NOP on the spectrum). This is useful to play with the spectrum using the capabilities of ICFIT.
- q** Quit and don't do any fitting. The spectrum is not modified.

- u** Adjust the user coordinate scale. There are three options, 'd' mark a position with the cursor and doppler shift it to a specified value, 'z' mark a position with the cursor and zeropoint shift it value, or 'l' mark two positions and enter two values to define a linear (in wavelength) dispersion scale. The units used for input are those currently displayed. A wavelength scale set this way will be used for other spectra which are not dispersion corrected.

- v** Toggle to a velocity scale using the position of the cursor as the velocity origin and back.

- w** Window the graph. For further help type '?' to the "window:" prompt or see help under **gtools**. To cancel the windowing use 'a'.

- x** "Etch-a-sketch" mode. Straight lines are drawn between successive positions of the cursor. Requires 2 cursor settings in x. The nearest pixels are used as the endpoints. To draw a line between arbitrary y values first use 'j' to adjust the endpoints or set the "xydraw" option.

- y** Overplot standard star values from a calibration file.

- z** Zoom the graph by a factor of 2 in x.

- (** In multiaperture spectra go to the spectrum in the preceding image line. If there is only one line go to the spectrum in the preceding band.

-)** In multiaperture spectra go to the spectrum in the following image line. If there is only one line go to the spectrum in the following band.

- #** Get a different line in multiaperture spectra or two dimensional images. The aperture/line/column is queried.

- %** Get a different band in a three dimensional image.

- \$** Switch between physical pixel coordinates and world (dispersion) coordinates.

- Subtract the fits generated by the 'd' (deblend), 'k' (single profile fit), and 'h' (gaussian of specified width). The region to be subtracted is marked with two cursor positions.

- ' , '** Shift the graph window to the left.

- .** Shift the graph window to the right.

- I** Force a fatal error interrupt to leave the graph. This is used because the normal interrupt character is ignored in graphics mode.

:show

Page the full output of the previous deblend and equivalent width measurements.

:log Enable logging of measurements to the file specified by the parameter *save_file*. When the program is first entered logging is enabled (provided a log file is specified). There is no way to change the file name from within the program.

:nolog

Disable logging of measurements.

:dispaxis<val>

Show or change dispersion axis for 2D images.

:nsum<val>

Show or change summing for 2D images.

:units<value>

Change the coordinate units in the plot. See below for more information.

:# <comment>

Add comment to logfile.

Labels:**:label<label> <format>**

Add a label at the cursor position.

:mabove<label> <format>

Add a tick mark and label above the spectrum at the cursor position.

:mbelow<label> <format>

Add a tick mark and label below the spectrum at the cursor position.

The label must be quoted if it contains blanks. A label beginning with % (i.e. %.2f) is treated as a format for the x cursor position. The optional format is a gtext string (see help on "cursors"). The labels are not remembered between redraws.

:auto[yes|no]

Enable/disable autodraw option

:zero[yes|no]

Enable/disable zero baseline option

:xydraw[yes|no]

Enable/disable xydraw option

:hist[yes|no]

Enable/disable histogram line type option

:nosysid[yes|no]

Enable/disable system ID option

:wreset[yes|no]

Enable/disable window reset for new spectra option

:flip [yes|no]

Enable/disable the flipped coordinates option

:overplot[yes|no]

Enable/disable the permanent overplot option

:/help

Get help on GTOOLS options.

:.help

Get help on standard cursor mode options

PROFILE FITTING AND DEBLENDING

The single profile ('k') and multiple profile deblending ('d') commands fit gaussian, lorentzian, and voigt line profiles with a linear background. The single profile fit, 'k' key, is a fitting designed to be simple to use. Two cursor positions define the region to be fit and a fixed linear continuum. The second key is used to select the type of profile to fit with 'g' for gaussian, 'l' for lorentzian, and 'v' for voigt. Any other second key will default to a gaussian profile. The profile center, peak strength, and width(s) are then determined and the results are printed on the status line and in the log file. The meaning of these quantities is described later. The fit is also overplotted and may be subtracted from the spectrum subsequently with the '-' key.

The more complex deblending function, 'd' key, defines the fitting region and initial linear continuum in the same way with two cursor positions. The continuum may be included in the fitting as an option. The lines to be fit are entered with the cursor near the line center ('g' for gaussian, 'l' for lorentzian, 'v' for voigt), by typing the wavelengths ('t'), or read from a file ('f'). The latter two methods are useful if the lines are known accurately and if fits restricting the absolute or relative positions of the lines will be used. The 't' key is restricted to gaussian fits only.

The 'f' key asks for a line list file. The format of this file has one or more columns. The columns are the wavelength, the peak value (relative to the continuum with negative values being absorption), the profile type (gaussian, lorentzian, or voigt), and the gaussian and/or lorentzian FWHM. End columns may be missing or INDEF values may be used to have values be approximated. Below are examples of the file line formats

```
wavelength
wavelength peak
wavelength peak (gaussian|lorentzian|voigt)
wavelength peak gaussian gfwhm
wavelength peak lorentzian lfwhm
wavelength peak voigt gfwhm
wavelength peak voigt gfwhm lfwhm

1234.5                <- Wavelength only
1234.5 -100           <- Wavelength and peak
1234.5 INDEF v        <- Wavelength and profile type
1234.5 INDEF g 12      <- Wavelength and gaussian FWHM
```

where peak is the peak value, gfwhm is the gaussian FWHM, and lfwhm is the lorentzian FWHM. This format is the same as used by **fitprofs** and also by **artdata.mk1dspec** (except in the latter case the peak is normalized to a continuum of 1).

There are four queries made to define the set of parameters to be fit or constrained. The positions may be held "fixed" at their input values, allowed to shift by a "single" offset from the input values, or "all"

positions may be fit independently. The widths may be constrained to a "single" value or "all" fit independently. The linear background may be included in the fit or kept fixed at that input using the cursor.

As noted above, sometimes the absolute or relative wavelengths of the lines are known a priori and this information may be entered by typing the wavelengths explicitly using the 't' option or read from a 'f' option during marking. In this case one should fix or fit a single shift for the position. The latter may be useful if the lines are known but there is a measurable doppler shift.

After the fit, the modeled lines are overplotted. The line center, flux, equivalent width, and full width half maxima are printed on the status line for the first line. The values for the other lines and the RMS of the fit may be examined by scrolling the status line using the '+', '-', and 'r' keys. To continue enter 'q'.

The fitting may be repeated with different options until exited with 'q'. For each line in the blend the line center, continuum intensity at the line center, the core intensity above or below the continuum, the FWHM for the gaussian and lorentzian parts, the flux above or below the continuum, and the equivalent width are recorded in the log file. All these parameters except the continuum are based on the fitted analytic profiles. Thus, even though the fitted region may not extend into the wings of a line the equivalent width measurements include the wings in the fitted profile. For direct integration of the flux use the 'e' key.

The fitted model may be subtracted from the data (after exiting the deblending function) using the '-' (minus) keystroke to delimit the region for which the subtraction is to be performed. This allows you to fit a portion of a line which may be contaminated by a away the entire line to examine the remaining components.

The fitting uses an interactive algorithm based on the Levenberg-Marquardt method. The iterations attempt to improve the fit by varying the parameters along the gradient of improvement in the chi square. This method requires that the initial values for the parameters be close enough that the gradient leads to the correct solution rather than an incorrect local minimum in the chi square. The initial values are determined as follows:

1. If the lines are input from a data file then those values in the file are used. Missing information is determined as below.
2. The line centers are those specified by the user either by marking with the cursor, entering the wavelengths, for read from a file.
3. The initial widths are obtained by dividing the width of the marked fitting region by the number of lines and then dividing this width by a factor depending on the profile type.
4. The initial peak intensities are the data values at the given line centers with the marked continuum subtracted.

Note that each time a new fitting option is specified the initial parameters are those from the previous fits. Thus the results do depend on the history of previous fits until the fitting is exited. Within each fit an iteration of parameters is performed as described next.

The iteration is more likely to fail if one initially attempts to fit too many parameters simultaneously. A constrained approach to the solution is obtained by iterating starting with a few parameters and then adding more parameters as the solution approaches the true chi square minimum. This is done by using the solutions from the more constrained options as the starting point for the less constrained options. In particular, the positions and a single width are fit first with fixed background. Then multiple widths and the background are added.

To conclude, here are some general comments. The most restrictive (fixed positions and single width(s)) will give odd results if the initial positions are not close to the true centers. The most general (simultaneous positions, widths, and background) can also lead to incorrect results by using unphysically different widths

to make one line very narrow and another very broad in an attempt to fit very blended lines. The algorithm works well when the lines are not severely blended and the shapes of the lines are close to the profile type.

CENTROID, FLUX, AND EQUIVALENT WIDTH DETERMINATIONS

There are currently five techniques in SPLOT to measure equivalent widths and other line profile parameters. The simplest (conceptually) is by integration of the pixel values between two marked pixels. This is invoked with the 'e' keystroke. The user marks the two edges of the line at the continuum. The measured line center, continuum value, line flux, and equivalent width are given by:

```
center = sum (w(i) * (I(i)-C(i))**3/2) / sum ((I(i)-C(i))**3/2)
continuum = C(midpoint)
flux = sum ((I(i)-C(i)) * (w(i2) - w(i1)) / (i2 - i1))
eq. width = sum (1 - I(i)/C(i))
```

where $w(i)$ is the wavelength of pixel i , $i1$ and $i2$ are the nearest integer pixel limits of the integrated wavelength range, $I(i)$ is the data value of pixel i , $C(i)$ is the continuum at pixel (i) , and the sum is over the marked range of pixels. The continuum is a linear function between the two points marked. The factor multiplying the continuum subtracted pixel values in the flux calculation is the wavelength interval per pixel so that the flux integration is done in wavelength units. (See the discussion at the end of this section concerning flux units).

The most complex method for computing line profile parameters is performed by the profile fitting and deblending commands which compute a non-linear least-squares fit to the line(s). These are invoked with the 'd' or 'k' keystroke. These were described in detail previously.

The fourth and fifth methods, selected with the 'h' key, determine the equivalent width from a gaussian profile defined by a constant continuum level "cont", a core depth "core", and the width of the line "dw" at some intermediate level "Iw".

```
I(w) = cont + core * exp (-0.5*((w-center)/sigma)**2)
sigma = dw / 2 / sqrt (2 * ln (core/Iw))
fwhm = 2.355 * sigma
flux = core * sigma * sqrt (2*pi)
eq. width = abs (flux) / cont
```

where w is wavelength.

For ease of use with a large number of lines only one cursor position is used to mark the center of the line and one flux level. Note that both the x and y cursor positions are read simultaneously. From the x position the line center and core intensity are determined. The region around the specified line position is searched for a minimum or maximum and a parabola is fit to better define the extremum.

The two methods based on the simple gaussian profile model differ in how they use the y cursor position and what part of the line is used. After typing 'h' one selects the method and whether to use the left, right, or both sides of the line by a second keystroke. The 'l', 'r', and 'k' keys require a continuum level of one. The y cursor position defines where the width of the line is determined. The 'a', 'b', and 'c' keys use the y cursor position to define the continuum and the line width is determined at the point half way between the line core and the continuum. In both cases the width at the appropriate level is determined by the interception of the y level with the data using linear interpolation between pixels. The one-sided measurements use the half-width on the appropriate side and the two-sided measurements use the full-width.

The adopted gaussian line profile is drawn over the spectrum and the horizontal and vertical lines show the measured line width and the depth of the line center from the continuum. This model may also be subtracted from the spectrum using the '-' key.

The major advantages of these methods are that only a (both the x and y positions are used) is required and they are fast. The 'l', 'r', and 'k' keys give more flexibility in adjusting the width of gaussian line at

the expense of requiring that the spectrum be normalized to a unit continuum. The 'a', 'b', and 'c' keys allow measurements at any continuum level at the expense of only using the half flux level to determine the gaussian line width.

All these methods print and record in the log file the line center, continuum intensity at the line center, the flux, and the equivalent width. For the 'e' key the flux is directly integrated while for the other methods the fitted gaussian is integrated. In addition, for the profile fitting methods the core intensity above or below the continuum, and the FWHMs are also printed. A zero value is record for the gaussian or lorentzian width if the value is not determined by profile fit. A brief line of data for each measurement is printed on the graphics status line. To get the full output and the output from previous measurements use the command ": show". This pages the output on the text output which may involve erasing the graphics.

The integrated fluxes for all the methods are in the same units as the intensities and the integration is done in the same units as the plotted scale. It is the user's responsibility to keep track of the flux units. As a caution, if the data is in flux per unit frequency, say ergs/cm2/sec/hz, and the dispersion in Angstroms then the integrated flux will not be in the usual units but will be A-ergs/cm2/sec/hz. For flux in wavelength units, ergs/cm2/sec/A and the dispersion scale in Angstroms the integrated flux will be correct; i.e. ergs/cm2/sec.

Note that one can compute integrated flux in pixel units by using the '\$' to plot in pixels. This is appropriate if the pixel values are in data numbers or photon counts to get total data number or photons.

ERROR ESTIMATES

The deblending ('d'), single profile fitting ('k'), and profile integration and equivalent width ('e') functions provide error estimates for the measured parameters. This requires a model for the pixel sigmas. Currently this model is based on a Poisson statistics model of the data. The model parameters are a constant gaussian sigma and an "inverse gain" as specified by the parameters *sigma0* and *invgain*. These parameters are used to compute the pixel value sigma from the following formula:

$$\text{sigma}^2 = \text{sigma0}^2 + \text{invgain} * I$$

where I is the pixel value and "I**2" means the square of the quantity.

If either the constant sigma or the inverse gain are specified as INDEF or with values less than zero then no noise model is applied and no error estimates are computed. Also if the number of error samples is less than 10 then no error estimates are computed. Note that for processed spectra this noise model will not generally be the same as the detector readout noise and gain. These parameters would need to be estimated in some way using the statistics of the spectrum. The use of an inverse gain rather than a direct gain was choosed to allow a value of zero for this parameters. This provides a model with constant uncertainties.

The direct profile integration error estimates are computed by error propagation assuming independent pixel sigmas. Also it is assumed that the marked linear background has no errors. The error estimates are one sigma estimates. They are given in the log output (which may also be view without exiting the program using the :show command) below the value to which they apply and in parenthesis.

The deblending and profile fit error estimates are computed by Monte-Carlo simulation. The model is fit to the data (using the sigmas) and this model is used to describe the noise-free spectrum. A number of simulations, given by the *nerrs* parameter, are created in which random gaussian noise is added to the noise-free spectrum using the pixel sigmas from the noise model. The model fitting is done for each simulation and the absolute deviation of each fitted parameter to model parameter is recorded. The error estimate for the each parameter is then the absolute deviation containing 68.3% of the parameter estimates. This corresponds to one sigma if the distribution of parameter estimates is gaussian though this method does not assume this.

The Monte-Carlo technique automatically includes all effects of parameter correlations and does not depend on any approximations. However the computation of the errors does take a amount of time. The amount of

time and the accuracy of the error estimates depend on how many simulations are done. A small number of samples (of order 10) is fast but gives crude estimates. A large number (greater than 100) is slow but gives good estimates. A compromise value of 50 is recommended for many applications.

UNITS

The dispersion units capability of **splot** allows specifying the units with the *units* parameter and interactively changing the units with the ":units" command. In addition the 'v' key allows plotting in velocity units with the zero point velocity defined by the cursor position.

The units are specified by strings having a along with the possible preceding modifiers, "inverse", to select the inverse of the unit and "log" to select logarithmic units. For example "log angstroms" to plot the logarithm of wavelength in Angstroms and "inv microns" to plot inverse microns. The various identifiers may be abbreviated as words but the syntax is not sophisticated enough to recognize standard scientific abbreviations except as noted below.

```

    angstroms - Wavelength in Angstroms
    nanometers - Wavelength in nanometers
    millimicrons - Wavelength in millimicrons
    microns - Wavelength in microns
    millimeters - Wavelength in millimeters
    centimeter - Wavelength in centimeters
    meters - Wavelength in meters
    hertz - Frequency in hertz (cycles per second)
    kilohertz - Frequency in kilohertz
    megahertz - Frequency in megahertz
    gigahertz - Frequency in gigahertz
    m/s - Velocity in meters per second
    km/s - Velocity in kilometers per second
    ev - Energy in electron volts
    kev - Energy in kilo electron volts
    mev - Energy in mega electron volts

    nm - Wavelength in nanometers
    mm - Wavelength in millimeters
    cm - Wavelength in centimeters
    m - Wavelength in meters
    Hz - Frequency in hertz (cycles per second)
    KHz - Frequency in kilohertz
    MHz - Frequency in megahertz
    GHz - Frequency in gigahertz
    wn - Wave number (inverse centimeters)

```

The velocity units require a trailing value and unit defining the velocity zero point. For example to plot velocity relative to a wavelength of 1 micron the unit string would be:

```
km/s 1 micron
```

Some additional examples of units strings are:

```

milliang
megahertz
inv mic
log hertz
m/s 3 inv mic

```

EXAMPLES

This task has a very large number of commands and capabilities which are interactive and graphical. Therefore these examples are fairly superficial. The user is encouraged to simply experiment with the task. To get some help use the '?' or '/' keys.

1. To plot a single spectrum and record any measurements in the file 'ngc7662':

```
cl> splot spectrum save_file=ngc7662
```

2. To force all plots to display zero as the minimum y

```
cl> splot spectrum options="auto, zero"
```

Note that the options auto and zero can be abbreviated to one character.

3. To successively display graphs for a set of spectra with the wavelength limits set to 3000 to 6000 angstroms:

```
cl> splot spec* xmin=3000 xmax=6000
```

4. To make batch plots create a file containing the simple cursor command

```
0 0 0 q
```

or an empty file and then execute one of the following:

```
cl> splot spec* graphics=stdplot cursor=curfile
cl> set stdvdm=splot.mc
cl> splot spec* graphics=stdvdm cursor=curfile
cl> splot spec* cursor=curfile >G splot.mc
```

The first example sends the plots to the standard plot device specified by the environment variable "stdplot". The next example sends the plots to the standard virtual display metacode file specified by the environment variable "stdvdm". The last example redirects the standard graphics to the metacode file splot.mc. To spool the metacode file the tasks **stdplot** and **gkimosaic** may be used. For a large number of plots **gkimosaic** is preferred since it places many plots on one page instead of one plot per page. The other GKI tasks in the **plot** package may be used to examine the contents of a metacode file. A simple script call **bplot** is provided which has the default cursor file given above and default device of "stdplot".

5. More complex plots may be produced both interactively using the '=' key or the ":.snap" or ":.write" commands or by preparing a script of cursor commands.

REVISIONS

SPL0TV2.11

The profile fitting and deblending was expanded to include lorentzian and voigt profiles. A new parameter controls the number of Monte-Carlo samples used in the error estimates.

Added colon commands for labeling.

SPL0TV2.10.3

The 'u' key now allows three ways to adjust the dispersion scale. The old method of setting a linear dispersion scale is retained as well as adding a doppler and zeropoint adjustment. The coordinates are input in the currently displayed units.

If a wavelength scale is set with either 'p' or 'u' then any other spectra which are not dispersion corrected will adopt this wavelength scale.

The '(' and ') ' keys cycle through bands if there is only one spectrum.

A new option, "flip", has been added to the options parameter to select that the spectra are plotted in decreasing wavelength.

A new options "overplot" has been added to the options parameters and colon commands to

permanently set overplotting. This allows quickly overplotting many spectra.

This task will now write out the current display units in the "units_display" WCS attribute. The default task units have been changed to "" to allow picking up the "units_display" units if defined.

The deblending and gaussian fitting code now subsamples the profile by a factor of 3 and fits the data pixels to the sum of the three subsamples. This accounts for finite sampling of the data.

Error estimates are provided for the deblending ('d'), gaussian fitting ('k'), and profile integration ('e') results.

SPLITV2.10

This is a new version with a significant number of changes. In addition to the task changes the other general changes to the spectroscopy packages also apply. In particular, long slit spectra and spectra with nonlinear dispersion functions may be used with this task. The image header or package dispaxis and nsum parameters allow automatically extracting spectra from 2D image. The task parameters have been modified primarily to obtain the desired initial graph without needing to do it interactively. In particular, the new band parameter selects the band in 3D images, the units parameter selects the dispersion units, and the new histogram, nosysid, and xydraw options select histogram line type, whether to include a system ID banner, and allow editing a spectrum using different endpoint criteria.

Because nearly every key is used there has been some shuffling, consolidating, or elimination of keys. One needs to check the run time '?' help or the help to determine the key changes.

Deblending may now use any number of components and simultaneous fitting of a linear background. A new simplified version of Gaussian fitting for a single line has been added in the 'k' key. The old 'k', 'h', and 'v' equivalent width commands are all part of the single 'h' command using a second key to select a specific option. The Gaussian line model from these modes may now be subtracted from the spectrum in the same way as the Gaussian fitting. The one-sided options, in particular, are interesting in this regard as a new capability.

The arithmetic functions between two spectra are now done in wavelength with resampling to a common dispersion done automatically. The 't' key now provides for the full power of the ICFIT package to be used on a spectrum for continuum normalization, subtraction, or line and cosmic ray removal. The 'x' editing key may now use the nearest pixel values rather than only the y cursor position to replace regions by straight line segments. The mode is selected by the task option parameter "xydraw".

Control over the graph window (plotting limits) is better integrated so that redrawing, zooming, shifting, and the GTOOLS window commands all work well together. The new 'c' key resets the window to the full spectrum allowing the 'r' redraw key to redraw the current window to clean up overplots from the Gaussian fits or spectrum editing.

The dispersion units may now be selected and changed to be from hertz to Mev and the log or inverse (for wave numbers) of units taken. As part of the units package the 'v' key or colon commands may be used to plot in velocity relative to some origin. The \$ key now easily toggles between the dispersion units (whatever they may be) and pixels coordinates.

Selection of spectra has become more complex with multiaperture and long slit spectra. New keys allow selecting apertures, lines, columns, and bands as well as quickly scrolling through the lines in multiaperture spectra. Overplotting is also more general and consistent with other tasks by using the 'o' key to toggle the next plot to be overplotted. Overplots, including those of the Gaussian line models, are now done in a different line type.

There are new colon commands to change the dispersion axis and summing parameters for 2D image, to toggle logging, and also to put comments into the log file. All the options may also be set with colon commands.

SEE ALSO

bplot, gtools, icfit, standard, package, specplot, graph, implot, fitprofs

NAME

standard -- Add standard stars to sensitivity file

USAGE

standard input [records] output

PARAMETERS**input**

List of input standard star spectra or root names if using the record number extension format. All spectra of the same aperture must be of the same standard star. In beam switch mode or when the same star parameter is set all spectra must be of the same standard star regardless of aperture number. Normally the spectra will not be extinction corrected but if then the extinction file should also be given and the same extinction file should be used with **sensfunc**.

records(imred.irsandimred.iidsonly)

List of records or ranges of records to be appended to the input spectra names when using record number extension format. The syntax of this list is comma separated record numbers or ranges of record numbers. A range consists of two numbers separated by a . A null list may be used if no record number extensions are desired. This is a positional query parameter only if the record format is specified.

output

The name of a text file which will contain the output from **standard**. Each execution of **standard** appends to this file information about the standard stars, the calibration bandpasses, and observed counts (see the DESCRIPTION section for more details). The output must be explicitly deleted by the user if the filename is to be reused.

samestar= yes

Is the same star in all apertures? If set to no then each aperture may contain a different standard star. The standard star name is queried each time a new aperture is encountered. Note that this occurs only once per aperture and multiple spectra with the same aperture number must be of the same star. If set to yes the standard star name is only queried once. When in beam switch mode this parameter is ignored since all apertures must contain the same star.

beam_switch= no

Beam switch the spectra? If yes then a beam switch mode is used for the spectra in which successive pairs of object and sky observations from the same aperture are sky subtracted. This requires that the object type flag OFLAG be present and that the spectra are appropriately ordered. All object observations must be of the same standard star and the *samestar* parameter is ignored.

apertures =""

List of apertures to be selected from the input list of spectra. If is specified then all apertures are selected. The syntax is the same as the record number extensions.

bandwidth= INDEF,bandsep= INDEF

Bandpass widths and separations in wavelength units. If INDEF then the default bandpasses are those given in the standard star calibration file. If values for these parameters are specified then a of bandpasses of equal width and separation are defined over the range of the input spectrum. In both cases the default bandpasses can be changed interactively if desired.

fnuzero= 3.68e-20

The absolute flux per unit frequency at an AB magnitude of zero. This is to convert the calibration AB magnitudes to absolute flux by the formula

$$f_{\nu} = \text{fnuzero} * 10. ** (-0.4 * m_{\text{AB}})$$

The flux units are also determined by this parameter. However, the frequency to wavelength interval conversion assumes frequency in hertz. The default value is based on a calibration of Vega at 5556 Angstroms of $3.52\text{e-}20$ ergs/cm²/s/Hz for an AB magnitude of 0.0336. This default value is that used in earlier versions of this task which did not allow the user to change this calibration.

extinction= <no default>

Extinction file used to make second order extinction corrections across the bandpasses. The default value is redirected to the package parameter of the same name. See **lcalib** for a extinctionfiles. Normally the input spectra will not be extinction corrected. But if they are this file will be used to remove the extinction and then the same file should be specified in **sensfunc**. Note that one can choose to use a null extinction file in both.

caldir =")_.caldir"

Calibration directory containing standard star data. The default value of ")_.caldir" means to use the package parameter "caldir". A list of standard calibration directories may be obtained by listing the file "onedstds\$README"; for example:

```
cl> page onedstds$README
```

The user may copy or create their own calibration files and specify the directory. The directory "" refers to the current working directory. The standard calibration directory for blackbody curves is "onedstds\$blackbody/".

observatory =")_.observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information.

interact= no

If set to no, then the default wavelength set (either that from the star calibration file or the set given by the *bandwidth* and *bandsep* parameters) is used to select wavelength points along the spectrum where the sensitivity is measured. If set to yes, the spectra may be plotted and the bandpasses adjusted.

graphics ="stdgraph"

Graphics output device for use with the interactive mode. Normally this is the user's graphics terminal.

cursor =""

Graphics cursor input for use with the interactive mode. When null the standard graphics cursor is used otherwise the specified file is used.

star_name

The name of the star observed in the current series of spectra. Calibration data for the star must be in the specified calibration directory "caldir". This is normally a interactive query parameter and

should not be specified on the command line unless all spectra are of the same standard star.

The following three queried parameters apply if the selected calibration file is for a blackbody.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

The following two parameters are queried if the image does not contain the information.

airmass,exptime

If the airmass and exposure time are not in the header nor can they be determined from other keywords in the header then these query parameters are used to request the airmass and exposure time. The values are updated in the image.

The following parameter is for the task to make queries.

answer

Interactive query parameter.

CURSOR KEYS

```
? Display help page
a Add a new band by marking the endpoints
d Delete band nearest the cursor in wavelength
r Redraw current plot
q Quit with current bandpass definitions
w Window plot (follow with '?' for help)
I Interrupt task immediately
```

```
:show Show current bandpass data
```

DESCRIPTION

Observations of standard stars are integrated over calibration bandpasses and written to an output file along with the associated calibration fluxes. The fluxes are obtained from tabulated standard star calibration files or a model flux distribution (currently just a blackbody) based on the magnitude and spectral type of the star. The output data is used by the task **sensfunc** to determine the detector sensitivity function and possibly the extinction. The spectra are required to be dispersion corrected. The input spectra may be in either "onedspec" or "echelle" format and may have many different observation apertures. The spectra may also be beam switched and use the a record number extension format.

The input spectra are specified by a list of names or root names if the record number extension format. In the latter case each name in the list has each of the specified record numbers appended. A subset of the

input spectra may be selected by their aperture numbers using the parameter *apertures*. The spectrum name, aperture number, and title are printed to the standard output. The airmass is required but if absent from the image header it may be computed from the observation header parameters and the latitude task parameter (normally obtained from the **observatory** task). If the airmass cannot be computed, due to missing keywords, then a query is made for the airmass. The airmass is then updated in the header.

The name of the standard star or blackbody curve is obtained by querying the user. If the parameter *samestar* is yes or beam switch mode is selected then all spectra are assumed to be of the same standard star and the query is made once. If the parameter is no then a each aperture. This allows each aperture to contain a different standard star. Note however that multiple observations with the same aperture number must be of the same standard star.

The standard star name is either the name of an actual standard star or of a blackbody calibration. The latter generally have a star name consisting of just the standard bandpass identifier. If the standard star name is not recognized a menu of the available standard stars in the calibration directory, the file "standards.men", is printed and then the query is repeated. Thus, to get a list you can type ?

The standard star names must map to a file containing tabulated calibration data. The calibration filename is formed from the star name with blanks, "+", and "-" removed, converted to lower case, and the extension ".dat" added. This name is appended to a calibration directory, so the directory name must have an appropriate directory delimiter such as "\$" or "/". Generally one of the system calibration directories is used but one may copy and modify or create new calibration files in a personal directory. For the current working directory the calibration directory is either null or "./".

The calibration files may include comment parameter information consisting of the comment character '#', a parameter name, and the parameter value. These elements are separated by whitespace. Any other comment where the first word does not match one of the allowed parameter names is ignored by the program. The parameter names are "type" identifying the type of calibration file, "units" identifying wavelength units, "band" identifying the band for magnitudes, and "weff" identifying the effective wavelength of the band.

There are two types of standard star calibration files as described below.

STANDARD STAR CALIBRATION FILES

This type of file is any file that does not contain the parameter "type" with a value of "blackbody". The only other parameter used by this type of calibration file is the "units" parameter for the wavelength units. If the units are not specified then the wavelengths default to Angstroms. All older calibration files will have no parameter information so they are interpreted as standard star calibration files with wavelengths in Angstroms.

The calibration files consist of lines with wavelengths, calibration magnitudes, and bandpass widths. The magnitudes are *m_AB* defined as

$$m_{AB}(\text{star}) = -2.5 * \log_{10} (f_{\nu}) - 48.60$$

where *f_nu* is in erg/cm²/s/Hz. The *m_AB* calibration magnitudes are converted to absolute flux per unit frequency using the parameter *fnuzero* defined by

$$f_{\nu} = fnuzero * 10. ** (-0.4 * m_{AB})$$

Thus, *fnuzero* is the flux at *m_AB* of zero. The flux units are determined by this number. The default value was chosen such that Vega at 5556 Angstroms has an AB magnitude of 0.0336 and a ergs/cm²/s/Hz. This is the same value that was used by all previous versions of this task.

BLACKBODY CALIBRATION FILES

This type of file has the comment parameter "type" with a "blackbody". It must also include the "band" and "weff" comment parameters. If no "units" comment parameter is given then the default units are Angstroms.

The rest of the file consists of lines with wavelengths, m_{AB} of a magnitude star (in that band magnitude system), and the bandpass widths. The m_{AB} are defined as described previously. Normally all the m_{AB} values will be the same though it is possible to adjust them to produce a departure from a pure blackbody flux distribution.

The actual m_{AB} calibration magnitudes for the star are obtained by the relation

$$m_{AB}(\text{star}) = \text{mag} + m_{AB}(m=0) - 2.5 * \log_{10} (B(\text{weff}, \text{teff})/B(w, \text{teff}))$$

where m is the magnitude of the star in the calibration band, $m_{AB}(m=0)$ is the calibration value in the calibration file representing the magnitude of a $m=0$ star (basically the m_{AB} of Vega), weff is the effective wavelength for the calibration file, and teff is the effective temperature of the star. The function $B(w,T)$ is the blackbody function in f_{ν} that provides the shape of the calibration. Note how the normalization is such that at weff the last term is zero and $m_{AB}(\text{star}) = m + m_{AB}(m=0)$.

The $m_{AB}(\text{star})$ computed using the calibration values and the blackbody function are then in the same units and form as for the standard star files. The conversion to f_{ν} and the remaining processing proceeds in the same way as for standard star calibration data.

The parameters $\backslash \text{mag}$ and teff are specified by the user for each star as described in the section BLACKBODY PARAMETERS. These parameters are queried by the task for each star (unless forced to a value on the command line).

The beam switch mode is selected with the *beam_switch* parameter. This mode requires that all apertures are of the same star, the header keyword OFLAG be present to identify object and sky spectra, and that the sequence of spectra specified are paired such that if an object spectrum is encountered first the next spectrum for that aperture (spectra from other apertures may appear in between) is a or the reverse. These restrictions are not fundamental but are made so that this mode behaves the same as with the previous version of this task. The sky spectrum is subtracted from the object spectrum and the result is then used in generating the observed intensities in the calibration bandpasses.

If the spectra have been extinction corrected (EX-FLAG = extinction correction is removed. The specified extinction file is used for this operation and so must be the same as that used when the extinction correction was made. The airmass is also required in this step and, if needed to compute the airmass, the observatory specified in the image or observatory parameter is used. The treatment of extinction in this task is subtle. The aim of this task is to produce observed integrated instrumental intensities without extinction correction. Thus, the extinction correction is removed from extinction corrected spectra. However, a correction is made for an extinction gradient across the bandpasses. This is done by applying an extinction correction, integrating across the bandpass, and then correcting the integrated intensity for the extinction at the center of the bandpass. An alternative way to look at this is that the integral is weighted by the ratio of the extinction correction at each pixel to the extinction correction at the center of the bandpass. This correction or weighting is why the extinction file and latitude are parameters in this task even though for nonextinction corrected spectra they appear not to be needed.

The observed instrumental intensities are integrated within a bandpasses by summing the pixels using partial pixels at the bandpass edges. Initial bandpasses are defined in one of two ways. A set of evenly spaced bandpasses of constant width covering the range of the input spectrum may be specified using the parameters *bandwidth* and *bandsep* in the same units as the spectrum dispersion. If these parameters have the value INDEF then the bandpasses from the calibration file which are entirely within the spectrum are

selected. Generally these bandpasses are the actual measured bandpasses though one is free to make calibration files using estimated points. The calibration bandpasses are preferable because they have been directly measured and they have been placed to avoid troubles with spectral lines. However, when the coverage or resolution is such that these bandpasses do not allow a good determination of the instrumental response the evenly spaced bandpasses may be needed. The calibration fluxes are linearly interpolated (or extrapolated) from the calibration data points to the defined bandpasses.

Each spectrum adds a line to the output file containing the spectrum image name, the sky spectrum image name if beam switching, the aperture or beam number, the number of points in the spectrum, the exposure time, airmass, wavelength range, and title. If the airmass is not found in the image header it is computed using the latitude parameter and observation information from the header. If the airmass cannot be computed, due to missing keywords, then a query is made for the airmass.

Following the spectrum information, calibration data is added for each bandpass. The bandpass wavelength, absolute flux (per Angstrom), bandpass width, and observed instrumental intensity in the bandpass are added to the output file. As discussed above, the observed intensity does not include an extinction term but does apply a small correction or weighting for the variation of the extinction across the bandpass.

The setting and editing of the bandpasses may be performed interactively if the *interact* flag is set. In this case the user is queried for each spectrum. The answers to this query may be "no" or "yes" to skip editing or edit the bandpasses for this spectrum, "NO" or "YES" to skip or not skip editing all spectra of the same aperture with no further queries for this aperture, and "NO!" or "YES!" to skip editing or edit all spectra with no further queries.

When editing the bandpasses a graph of the spectrum is made with the bandpasses plotted at the computed intensity per pixel. The cursor and colon commands available are summarized in the section CURSOR KEYS. Basically bandpasses may be added or deleted and the current bandpass data may be examined. Additional keys allow the usual windowing and cursor mode operations. When satisfied with the bandpasses exit with 'q'. The edited bandpasses for that aperture remain in effect until changed again by the user. Thus if there are many spectra from the same aperture one may reply with "NO" to queries for the next spectra to accept the current bandpasses for all other spectra of the same aperture.

BLACKBODY PARAMETERS

When a blackbody calibration is selected (the calibration file selected by the *star_name* parameter has "# type blackbody") there are two quantities needed to scale the blackbody ~~the~~ observation. These are the magnitude of the star in the same band as the observation and the effective temperature. The magnitude is used for the flux scaling and the effective temperature for the shape of the flux distribution. The values are obtained or derived from the user specified parameters *mag*, *magband*, and *teff*. This section describes how the values are derived from other parameters using the data file "params.dat" in the calibration directory.

The effective temperature in degrees Kelvin may be specified directly or it may be derived from a spectral type for the star. In the latter case the file "params.dat" is searched for the effective temperature. The file consists of lines with the first value being the spectral type and the second the effective temperature. Other columns are described later. The spectral type can be any string without whitespace that matches what is in the file. However, the program finds the last spectral type that matches the first two characters when there is no complete match. This scheme is intended for the case where the spectral types are of the form A0I, A0III, or A0V, where A can be any spectral type letter OBAFGKM, the single digit subtype is between 0 and 9, and the luminosity class is one of I, III, or V. The two character match selects the last spectral type independent of the luminosity class. The standard file "onedstds\$blackbody/params.dat" uses these spectral type identifiers with the dwarf class acting as the default.

The magnitude band is specified along with the input magnitude. If band is the same as the calibration band given in the calibration file then no further transformation is required. However if the magnitude is

specified in a different band, a conversion is performed using information from the "params.dat" file based on the spectral type of the star.

When an effective temperature is specified rather than the nearest tabulated temperature for the spectral types that have "V" as the third character is used. For the standard spectral type designations this means that when an effective temperature is specified the dwarf spectral type is used for the magnitude transformation.

As mentioned previously, the "params.dat" data file has additional columns following the spectral type and effective temperature. These columns are relative magnitudes in various bands. The standard file has V magnitudes of zero so in this case the columns are also the X-V colors (where X is appropriate magnitude). Given the spectral type the relative magnitudes for the calibration band, m_1, and the input magnitude band, m_2, are found and the calibration magnitude for the star is given by

$$m_{\text{calibration}} = m_{\text{input}} + m_1 - m_2$$

If one of the magnitudes is missing, given as "INDEF" because the transformation is not available for the spectral type, the last spectral type matching the first two characters which does specify the two magnitudes will be used. For example if there is no information for a B3III star for a M-J color then the spectral type B3V might be used.

In order for the program to determine the bands for each column in the data file there must be a comment before the data with the column names. It must begin with "# Type Teff" and then be followed by the same band identifiers used in the blackbody calibration files and as specified by the *magband* parameter. Any amount whitespace (space or tab) is used to separate the various fields in the comment and in the fields of the table. For example the file might have the comment

```
# Type      Teff      V      J      H      K      L      Lprime      M
```

identifying the third column of the file as the V magnitude and the ninth file as the M magnitude.

EXAMPLES

1. To compile observations of three standard stars using a switched instrument like the IIDS:

```
cl> standard.recformat=yes
cl> standard nite1 1001-1008 std beam_switch+ interact-
[nite1.1001][0]: HZ 44 - Night 1
[nite1.1004][0]: HZ 44 - Night 1
[nite1.1005][0]: HZ 44 - Night 1
[nite1.1008][0]: HZ 44 - Night 1
Star name in calibration list: hz 44
cl> standard nite1 1009-1016 std beam_switch+ interact-
...
cl> standard nite1 1017-1024 std beam_switch+ interact-
...
```

This will create a file "std" which will contain sensitivity measurements from the beam-switched observations of the three standard stars given. Note that **standard** is run separately for each standard star.

The spectra will be from the images: nite1.1001, nite.1002 ... nite1.1024, and the default calibration file, "onedstds\$irscal.dat" will be used.

2. For echelle spectra all apertures, the orders, are of the same star and so the *samestar* parameter is set. Usually the resolution is much higher than the calibration data so in order to get sufficient coverage bandpasses must be interpolated from the calibration data. Therefore the evenly spaced bandpasses are used.

```
cl> standard.recformat=no
cl> standard.samestar=yes
```



```

cl> standard ech001.ec std bandwidth=10 bandsep=15
[ech001.ec][0]: Feige 110
Star name in calibration list: feige 110
[ech001.ec][0]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): yes
[ech001.ec][1]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): yes
[ech001.ec][2]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): NO!

```

3. To use a blackbody infrared calibration where the V magnitude of the star is known.

```

cl> standard std1.ms std caldir=onedstds$blackbody/
std1.ms(1): Standard Star
Star name in calibration list: J
Magnitude of star: 10.3
Magnitude type (|V|J|H|K|L|Lprime|M|): V
Effective temperature or spectral type: B3III
WARNING: Effective temperature for B3III not found - using B3V
Blackbody: V = 10.30, J = 10.32, Teff = 19000
std1[1]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!) (yes):

```

Note the warning message and the confirmation information.

REVISIONS

STANDARD V2.10.4

The calibration files can be defined to compute blackbody values.

STANDARD V2.10.3

A query for the airmass and exposure time is now made if the information is not in the header and cannot be computed from other header keywords.

STANDARD V2.10

Giving an unrecognized standard star name will page a stars available in the calibration directory and then repeat the query.

SEE ALSO

observatory, lcalib, sensfunc

NAME

subsets -- Description of CCD subsets

DESCRIPTION

The **ccdred** package groups observation into subsets. The image header parameter used to identify the subsets is defined in the instrument translation file (see help for **instruments**). For example to select subsets by the header parameter "filters" the instrument translation file would contain the line:

```
subset filters
```

Observations are generally grouped into subsets based on a instrument configuration such as a filter, aperture mask, grating setting, etc. This allows combining images from several different subsets automatically and applying the appropriate flat field image when processing the observations. For example if the subsets are by filter then **flatcombine** will search through all the images, find the flat field images (based on the CCD type parameter), and combine the flat field images from each filter separately. Then when processing the images the flat field with the same filter as the observation is used.

Each subset is assigned a short identifier. This is listed when using **ccdlist** and is appended to a root name when combining images. Because the subset parameter in the image header may be any string there must be a mapping applied to generate unique identifiers. This mapping is defined in the file given by the package parameter *ccdred.ssfile*. The file consists of lines with two fields (except that comment lines may be included as a line by itself or following the second field):

```
'subset string' subset_id
```

where the subset string is the image header string and the subset_id is the identifier. A field must be quoted if it contains blanks. The user may create this file but generally it is created by the tasks. The tasks use the first word of the subset string as the default identifier and a number is appended if the first word is not unique. The following steps define the subset identifier:

- (1) Search the subset file, if present, for a matching subset string and use the defined subset identifier.
- (2) If there is no matching subset string use the first word of the image header subset string and, if it is not unique, add successive integers until it is unique.
- (3) If the identifier is not in the subset file create the file and add an entry if necessary.

EXAMPLES

1. The subset file is "subsets" (the default). The subset parameter is translated to "f1pos" in the image header (the old NOAO CCD parameter) which is an integer filter position. After running a "ccdlist *.imh" to cause all filters to be checked, the subset file contains:

```
'2'      2
'5'      5
'3'      3
```

The order reflects the order in which the filters were encountered. Suppose the user wants to have more descriptive names then the subset file can be created or edited to the form:

```
# Sample translation file.
'2'      U
'3'      B
'4'      V
```

(This is only an example and does not mean these are standard filters.)

2. As another example suppose the image header parameter is "filter" and contains more descriptive strings. The subset file might become:

```
'GG 385 Filter' GG  
'GG 495 Filter' GG1  
'RG 610 Filter' RG  
'H-ALPHA'      H_ALPHA
```

In this case use of the first word was not very good but it is better if the filters are encoded with the thought that the first word will be used by **ccdred**; it should be short and unique.

SEE ALSO

instruments

NAME

zerocombine -- Combine and process zero level images

USAGE

zerocombine input

PARAMETERS**input**

List of zero level images to combine. The *ccdtype* parameter may be used to select the zero level images from a list containing all types of data.

output = "Zero"

Output zero level root image name.

combine = "average" (average|median)

Type of combining operation performed on the final set of pixels (after rejection). The choices are "average" or "median". The median uses the average of the two central values when the number of pixels is even.

reject = "minmax" (none|minmax|ccdclip|crreject|sigclip|avsigclip|pclip)

Type of rejection operation. See **combine** for details.

ccdtype = "zero"

CCD image type to combine. If no image type is given then all input images are combined.

process = no

Process the input images before combining?

delete = no

Delete input images after combining? Only those images combined are deleted.

clobber = no

Clobber existing output images?

scale = "none" (none|mode|median|mean|exposure)

Multiplicative image scaling to be applied. The choices are none, scale by the mode, median, or mean of the specified statistics section, or scale by the exposure time given in the image header.

statsec = ""

Section of images to use in computing image statistics for scaling. If no section is given then the entire region of the image is sampled (for efficiency the images are sampled if they are big enough).

Algorithm Parameters**nlow = 0, nhigh = 1 (minmax)**

The number of low and high pixels to be rejected by the "minmax" algorithm.

nkeep = 1

The minimum number of pixels to retain or the maximum number to reject when using the clipping algorithms (ccdclip, crreject, sigclip, avsigclip, or pclip). When given as a positive value this is the minimum number to keep. When given as a negative value the absolute value is the maximum

number to reject. This is actually converted to a number to keep by adding it to the number of images.

mclip = yes (ccdclip, crreject, sigclip, avsigclip)

Use the median as the estimate for the true intensity rather than the average with high and low values excluded in the "ccdclip", "crreject", "sigclip", and "avsigclip" algorithms? The median is a better estimator in the presence of data which one wants to reject than the average. However, computing the median is slower than the average.

lsigma = 3., hsigma = 3. (ccdclip, crreject, sigclip, avsigclip, pclip)

Low and high sigma clipping factors for the "ccdclip", "crreject", "sigclip", "avsigclip", and "pclip" algorithms. They multiply a "sigma" factor produced by the algorithm to select a point below and above the average or median value for rejecting pixels. The lower sigma is ignored for the "crreject" algorithm.

rdnoise = "0.", gain = "1.", snoise = "0." (ccdclip, crreject)

CCD readout noise in electrons, gain in electrons/DN, and sensitivity noise as a fraction. These parameters are used with the "ccdclip" and "crreject" algorithms. The values may be either numeric or an image header keyword which contains the value.

pclip = -0.5 (pclip)

Percentile clipping algorithm parameter. If greater than one in absolute value then it specifies a number of pixels above or below the median to use for computing the clipping sigma. If less than one in absolute value then it specifies the fraction of the pixels above or below the median to use. A positive value selects a point above the median and a negative value selects a point below the median. The default of -0.5 selects approximately the quartile point. See **combine** for further details.

blank = 0.

Output value to be used when there are no pixels.

DESCRIPTION

The zero level images in the input image list are combined. The input images may be processed first if desired. The original images may be deleted automatically if desired. The output pixel datatype will be real.

This task is a script which applies **ccdproc** and **combine**. The parameters and combining algorithms are described in detail in the help for **combine**. This script has default parameters specifically set for zero level images and simplifies the combining parameters. There are other combining options not included in this task. For these additional features, such as thresholding, offsetting, masking, and projecting, use **combine**.

EXAMPLES

1. The image data contains four zero level images. To automatically select them and combine them as a using the default combining algorithm:

```
cl> zerocombine ccd*.imh&
```

SEE ALSO

ccdproc, combine

NAME

darkcombine -- Combine and process dark count images

USAGE

darkcombine input

PARAMETERS**input**

List of dark count images to combine. The *ccdtype* parameter may be used to select the zero level images from a list containing all types of data.

output = "Dark"

Output dark count root image name.

combine = "average" (average|median)

Type of combining operation performed on the final set of pixels (after rejection). The choices are "average" or "median". The median uses the average of the two central values when the number of pixels is even.

reject = "minmax" (none|minmax|ccdclip|crreject|sigclip|avsigclip|pclip)

Type of rejection operation. See **combine** for details.

ccdtype = "dark"

CCD image type to combine. If no image type is given then all input images are combined.

process = yes

Process the input images before combining?

delete = no

Delete input images after combining? Only those images combined are deleted.

clobber = no

Clobber existing output images?

scale = "exposure" (none|mode|median|mean|exposure)

Multiplicative image scaling to be applied. The choices are none, scale by the mode, median, or mean of the specified statistics section, or scale by the exposure time given in the image header.

statsec = ""

Section of images to use in computing image statistics for scaling. If no section is given then the entire region of the image is sampled (for efficiency the images are sampled if they are big enough).

Algorithm Parameters**nlow = 0, nhigh = 1 (minmax)**

The number of low and high pixels to be rejected by the "minmax" algorithm.

nkeep = 1

The minimum number of pixels to retain or the maximum number to reject when using the clipping algorithms (ccdclip, crreject, sigclip, avsigclip, or pclip). When given as a positive value this is the minimum number to keep. When given as a negative value the absolute value is the maximum

number to reject. This is actually converted to a number to keep by adding it to the number of images.

mclip = yes (ccdclip, crreject, sigclip, avsigclip)

Use the median as the estimate for the true intensity rather than the average with high and low values excluded in the "ccdclip", "crreject", "sigclip", and "avsigclip" algorithms? The median is a better estimator in the presence of data which one wants to reject than the average. However, computing the median is slower than the average.

lsigma = 3., hsigma = 3. (ccdclip, crreject, sigclip, avsigclip, pclip)

Low and high sigma clipping factors for the "ccdclip", "crreject", "sigclip", "avsigclip", and "pclip" algorithms. They multiply a "sigma" factor produced by the algorithm to select a point below and above the average or median value for rejecting pixels. The lower sigma is ignored for the "crreject" algorithm.

rdnoise = "0.", gain = "1.", snoise = "0." (ccdclip, crreject)

CCD readout noise in electrons, gain in electrons/DN, and sensitivity noise as a fraction. These parameters are used with the "ccdclip" and "crreject" algorithms. The values may be either numeric or an image header keyword which contains the value.

pclip = -0.5 (pclip)

Percentile clipping algorithm parameter. If greater than one in absolute value then it specifies a number of pixels above or below the median to use for computing the clipping sigma. If less than one in absolute value then it specifies the fraction of the pixels above or below the median to use. A positive value selects a point above the median and a negative value selects a point below the median. The default of -0.5 selects approximately the quartile point. See **combine** for further details.

blank = 0.

Output value to be used when there are no pixels.

DESCRIPTION

The dark count images in the input image list are combined. The input images may be processed first if desired. The original images may be deleted automatically if desired. The output pixel datatype will be real.

This task is a script which applies **ccdproc** and **combine**. The parameters and combining algorithms are described in detail in the help for **combine**. This script has default parameters specifically set for dark count images and simplifies the combining parameters. There are other combining options not included in this task. For these additional features, such as thresholding, offsetting, masking, and projecting, use **combine**.

EXAMPLES

1. The image data contains four dark count images. To automatically select them and combine them as a background job using the default combining algorithm:

```
cl> darkcombine ccd*.imh&
```

SEE ALSO

ccdproc, combine

NAME

dispcor -- Dispersion correct and resample spectra

USAGE

dispcor input output [records]

PARAMETERS**input**

List of input spectra or root names to be dispersion corrected. These may be echelle or non-echelle spectra, the task will determine which from the database dispersion functions. When using the record number extension format, record number extensions will be appended to each root name in the list.

output

List of dispersion corrected output spectra or root names. When using the record number extension format, record number extensions will be appended to each root name in the list. The output extension will be the same as the input extension. If "no" output list is specified then the output spectrum will replace the input spectrum after dispersion correction.

records(imred.irsand imred.iidsonly)

List of records or ranges of records to be appended to the input and output root names when using record number extension format. The syntax of this list is comma separated record numbers or ranges of record numbers. A range consists of two numbers separated by a used if no record number extensions are desired. This is a positional query parameter only if the record format is specified.

linearize= yes

Interpolate the spectra to a linear dispersion sampling? If yes, the spectra will be interpolated to a linear or log linear sampling using the linear dispersion parameters specified by other parameters. If no, the nonlinear dispersion function(s) from the dispersion function database are assigned to the input image world coordinate system and the spectral data are not interpolated.

database ="database"

Database containing dispersion solutions created by **identify** or **ecidentify**. If the spectra have been previous dispersion corrected this parameter is ignored unless a new reference spectra are defined.

table = ""

Wavelength coordinate table or reference image. Elements in this optional table or reference image override the wavelength coordinates given below for specified apertures. See the DISCUSSION for additional information.

w1 = INDEF, w2 = INDEF, dw = INDEF, nw = INDEF

The starting wavelength, ending wavelength, wavelength interval per pixel, and the number of pixels in the output spectra. Any combination of these parameters may be used to restrict the wavelength coordinates of the output spectra. If two or more have the value INDEF then suitable defaults based on the number of input pixels and the wavelength range of the reference dispersion solutions are used. These defaults may either come from all spectra, all spectra of the same aperture, or individually for each spectrum depending on the values of the *global* and *samedisp* parameters. Note that these parameters are specified in linear units even if a logarithmic wavelength scale is selected. The conversion between linear and logarithmic intervals between pixels is given below. These values may be overridden for specified apertures by a wavelength table or reference image. Otherwise these values apply to all apertures.

log = no

Transform to linear logarithmic wavelength coordinates? Linear logarithmic wavelength coordinates have wavelength intervals which are constant in the logarithm (base 10) of the wavelength. Note that if conserving flux this will change the flux units to flux per log lambda interval. Note that if the input spectra are in log sampling then *log=no* will resample back to linear sampling and *log=yes* will resample keeping the output spectra in log sampling.

flux = yes

Conserve the total flux during interpolation rather than the flux density? If "no", the output spectrum is average of the input spectrum across each output wavelength coordinate. This conserves flux density. If "yes" the input spectrum is integrated over the extent of each output pixel. This conserves the total flux. Note that in this case units of the flux will change; for example rebinning to logarithmic wavelengths will produce flux per log lambda. For flux calibrated data you most likely would not want to conserve flux.

blank= 0.

Output value corresponding to points outside the range of the input data. In other words, the out of bounds value. This only has an effect when linearizing and the output spectral coordinates extend beyond the input spectral range.

samedisp= no

Use the same dispersion parameters for all apertures? If yes then all apertures in a single image will have the same dispersion parameters. If the *global* parameter is all selected then all spectra in all images will have the same dispersion parameters. This parameter would not normally be used with echelle spectra where each order has a different wavelength coverage.

global = no

Apply global wavelength defaults? Defaults for the INDEF wavelength coordinate parameters are determined if two or less of the wavelength parameters are specified. The defaults are based on the number of pixels and the wavelengths of the first and last pixel as given by the dispersion solution. If this parameter is "no" this is done independently for each input spectrum. If this parameter is "yes" then the maximum number of pixels and the minimum and maximum wavelengths of all the input spectra or those of the same aperture are used to provide defaults for the spectra. The parameter *samedisp* determines whether the global coordinates are over all spectra or only those with the same aperture number. The global option is used to have all the dispersion corrected spectra have the same wavelength coordinates without actually specifying the wavelength parameters.

ignoreaps= no

If a reference dispersion solution is not found for an aperture use the first reference dispersion solution and ignore the aperture number? If not ignoring the apertures all spectra must have a aperture for the dispersion solution and the task aborts if this is not the case. Ignoring the apertures avoids this abort and instead the first dispersion solution is used. Note this parameter does not mean ignore matches between reference and spectrum aperture numbers but only ignore the aperture number if no matching reference is found.

Also if a reference table or image is given and *ignoreaps=yes* then the default dispersion parameters for any aperture not defined by the table or image will be that of the first defined aperture. This can still be overridden by giving explicit values for *w1*, *w2*, *dw* and *nw*.

confirm= no

Confirm the wavelength parameters for each spectrum? If yes the wavelength parameters will be printed and the user will be asked whether to accept them. If the parameters are not acceptable the

user will be queried for new values. The confirmation and parameter changes are repeated until an acceptable set of parameters is obtained. When the *global* parameter is *yes* changes to the wavelength parameters will remain in effect until changed again.

listonly = no

List the dispersion coordinates only? If set then the dispersion coordinates are listed but the spectra are not dispersion corrected. This may be used to determine what the default wavelengths would be based on the dispersion solutions.

verbose = yes

Print the dispersion function and coordinate assignments?

logfile = ""

Log file for recording the dispersion correction operations. If no file name is given then no log information is recorded.

DESCRIPTION

The dispersion coordinate systems of the input spectra are set or changed in the output spectra. The output spectra may be the same as the input spectra if no output spectra are specified or the output name is the same as the input name. The input and output spectra are specified by image templates or lists. In the **irs/iids** packages the input and output spectra are specified as root names and the record numbers are specified by the *record* parameter. The records are given as a set of comma separate single numbers or ranges of hyphen separated numbers. If no records are specified then the input and output images are assumed to be full names.

The dispersion coordinate system is defined either in the image header or by dispersion functions in the specified database. To use reference spectra dispersion functions they must first be assigned to the image with **identify** (**reidentify**), **ecidentify** (**ecreidentify**), **refspectra**, or **hedit**. These tasks define the image header keywords REFSPEC1, REFSPEC2, REFSHFT1, and REFSHFT2. The test which determines whether to use the current dispersion coordinate system or reference spectra dispersion solutions is the presence of the REFSPEC1 keyword. Since it is an error to apply a dispersion function to data which have already been dispersion corrected the any dispersion function keywords are deleted after use and a record of them entered in sequential image header keywords beginning with DCLOG.

Dispersion functions are specified by one or both of the reference spectrum image header keywords REFSPEC1 and REFSPEC2 containing the name of calibration spectra with dispersion function solutions (either echelle dispersion functions from **ecidentify** or non-echelle dispersion functions from **identify**) in the database. There must be a dispersion function for each aperture in the input spectrum unless the *ignoreaps* flag is set. If the flag is not set the task will abort if a aperture is not found while if it is set spectra without a matching aperture in the reference dispersion solutions will use the first dispersion solution. Note that aperture number matching is done in both cases and the *ignoreaps* parameter only applies to non-matching spectra. The common situation for using the *ignoreaps* option is there is a single reference dispersion solution which is to be applied to a number of spectra with different aperture numbers; hence effectively ignoring the reference spectrum aperture number.

If two reference spectra are specified the names may be followed by a weighting factor (assumed to be 1 if missing). The wavelength of a pixel is then the weighted average of the wavelengths of the two dispersion functions. The task **refspectra** provides a number of ways to assign reference spectra. Note, however, that these assignments may be made directly using the task **hedit** or with some other task or script if none of the methods are suitable. Also note that **identify** and **reidentify** add the REFSPEC1 keyword referring to the image itself when a database entry is written.

In addition to the one or two reference dispersion functions for each input aperture there may also be image header keywords REFSHFT1 and REFSHFT2 specifying reference spectra whose dispersion function zero

point shifts (the "shift" parameter in the database files) are to be applied to the reference dispersion functions. The shifts from REFSHFT1 will be applied to the dispersion functions from REFSPEC1 and similarly for the second dispersion functions. The reference shifts need not be present for every aperture in a multispectrum image. By default the mean shift from all the reference apertures having a zero point shift is applied to all the reference dispersion functions. If the REFSHFT keyword has the modifier word "nearest" following the spectrum name then the shift from the nearest aperture in spatial position (from the aperture extraction limits in the original 2D spectrum as recorded in the 6th and 7th fields of the APNUM keywords) is used for a particular input aperture. If the modifier word is "interp" then the nearest two apertures are used to interpolate a zero point shift spatially.

The purpose of the reference shift keywords is to apply a point correction to the reference dispersion functions determined from separate arc calibration observations using a same time as object observations. For example, consider multifiber observations in which one or more fibers are assigned to arc lamps at the same time the other fibers are used to observe various objects. The basic dispersion reference, the REFSPEC keywords, will come from arc observations taken through all the fibers. The arc fibers used during an object observation are then calibrated against their corresponding fibers in the arc calibration observations to determine a zero point shift. The REFSHFT keywords will contain the name of the object spectrum itself and the shifts from the simultaneous arc fibers will be interpolated spatially to the nonarc object fibers and applied to the dispersion functions from the arc calibrations for those fibers.

The reference shift keywords are currently added with **hedit** and zero point shifts computed with **identify/reidentify**. The complexities of this have been hidden in the multifiber **imred** instrument reduction packages. The reference shift correction feature was added primarily for use in those reduction packages.

If the *linearize* parameter is no the dispersion functions, weights, and shifts are transferred from the database to the world coordinate system keywords in the image header. Except for printing processing information that is all that is done to the spectra.

If the *linearize* parameter is yes the spectra are interpolated to a linear wavelength scale and the dispersion coordinate system in the header is set appropriately. A linear wavelength coordinate system is defined by a starting wavelength, an ending wavelength, a wavelength interval per pixel, and the number of pixels. These four parameters actually overspecify the coordinate system and only three of these values are needed to define it. The output coordinate system is specified by giving a these parameters using the parameters *w1*, *w2*, *dw*, and *nw*.

When the *log* option is used these parameters are still specified and computed in non-log units but the effective interval per pixel is

$$\begin{aligned} dw_log &= (\log_{10}(w2) - \log_{10}(w1)) / (nw - 1) \\ dw_log &= (\log_{10}(w1+dw*(nw-1)) - \log_{10}(w1)) / (nw - 1) \end{aligned}$$

In other words, the logarithmic interval divides the starting and ending wavelength into the required number of pixels in log step. To avoid confusion in this case it is best to specify the starting and ending wavelengths (in non-log units) and the number of pixels.

Note that if *log*=yes the input spectra in either linear or log sampling will be resampled to produces an output spectrum in log sampling. Similarly, if *log*=no the input spectra will be resampled to linear sampling. This means that log sampled input spectra will be resampled to linear sampling.

Default values for any parameters which are not specified, by using the value INDEF, are supplied based on the wavelengths of the first and last pixel as given by the dispersion function and the number of pixels in the input image. The defaults may either be determined separately for each spectrum (*global* = no), from all spectra with the same aperture (*global* = yes and *samedisp* = no), or from all the spectra (*global* = yes and *samedisp* = yes). As indicated, the parameter *samedisp* determines whether defaults are determined independently for each aperture or set the same for all apertures.

Another way to specify the wavelengths when there are many apertures is use a wavelength table or reference image. If an spectrum image name is specified with the *table* parameter then the dispersion parameters for each apertures are set to be the same as the reference spectrum. Alternatively, a text file table consisting of lines containing an aperture number, the starting wavelength, the ending wavelength, the wavelength interval per pixel, and the number of output pixels may be specified. Any of these values may be specified as INDEF (though usually the aperture number is not). One way to view the wavelength table/reference spectrum is that an entry in the wavelength table/reference spectrum overrides the values of the parameters *w1*, *w2*, *dw*, and *nw*, which normally apply to all apertures, for the specified aperture. The wavelength table is used to specify explicit independent values for apertures. The global mechanism can supply independent values for the INDEF parameters when the *samedisp* parameter is no.

If one wishes to verify and possibly change the defaults assigned, either globally or individually, the *confirm* flag may be set. The user is asked whether to accept these values. By responding with no the user is given the chance to change each parameter value. Then the new parameters are printed and the user is again asked to confirm the parameters. This is repeated until the desired parameters are set. When the defaults are not global the changed parameters will not be used for the next spectrum. When the global option is used any changes made are retained (either for all apertures or independently for each aperture) until changed again.

When adjusting the wavelengths the user should specify which parameter is free to change by entering INDEF. If none of the parameters are specified as INDEF then those values which were not changed, i.e. by accepting the current value, are the first to be changed.

Once the wavelength scale has been defined the input spectrum is interpolated for each output pixel. Output wavelengths outside the range of the input spectrum are set to the value given by the *blank* parameter value. The default interpolation function is a 5th order polynomial. The choice of interpolation type is made with the package parameter "interp". It may be set to "nearest", "linear", "spline3", "poly5", or "sinc". Remember that this applies to all tasks which might need to interpolate spectra in the **onedspec** and associated packages. For a types see **onedspec**.

When it is desired to conserve total flux, particularly when the dispersion is significantly reduced, the parameter *flux* is set to yes and the output pixel value is obtained by integrating the interpolation function across the wavelength limits of the output pixel. If it is set to no then the flux density is conserved by averaging across the output pixel limits.

The input spectrum name, reference spectra, and the wavelength parameters will be printed on the standard output if the *verbose* parameter is set and printed to a log file if one is specified with the *logfile* parameter. If one wishes to only check what wavelengths will be determined for the defaults without actually dispersion correcting the spectra the *listonly* flag may be set.

Other tasks which may be used to change the dispersion coordinate system are **scopy**, **specshift**, and **sapertures**.

EXAMPLES

In the examples when the task is used in the IRS and IIDS packages, shown with the "ir>" prompt the spectra have a record number extension image name format and the records parameter must be specified. In the other case shown with the "on>" prompt the records parameter is not used.

1. Dispersion correct spectra so that they have the same number of pixels and the wavelengths limits are set by the reference spectra.

```
ir> dispcor spec dcspec 9,10,447-448
dcspec.0009: ap = 0, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
dcspec.0010: ap = 1, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
dcspec.0447: ap = 0, w1 = 5082.57, w2 = 6551.45, dw = 1.794, nw = 820
dcspec.0448: ap = 1, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
```

```
on> dispcor allspec.ms dcallspec.ms
dcallspec.ms: ap = 1, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
dcallspec.ms: ap = 2, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
dcallspec.ms: ap = 3, w1 = 5082.57, w2 = 6551.45, dw = 1.794, nw = 820
dcallspec.ms: ap = 4, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
```

2. Confirm and change assignments.

```
on> dispcor spec* %spec%new%* confirm+
new009: ap = 0, w1 = 5078.84, w2 = 6550.54, dw = 1.797, nw = 820
Change wavelength coordinate assignments? (yes):
Starting wavelength (5078.8421234): 5070
Ending wavelength (6550.535123):
Wavelength interval per pixel (1.79693812):
Number of output pixels (820): INDEF
new009: ap = 0, w1 = 5070., w2 = 6550.53, dw = 1.795, nw = 826
Change wavelength coordinate assignments? (yes): no
new010: ap = 1, w1 = 5078.71, w2 = 6552.81, dw = 1.800, nw = 820
Change wavelength coordinate assignments? (no): yes
Starting wavelength (5078.7071234): 5100
Ending wavelength (6550.805123): 6500
Wavelength interval per pixel (1.79987512): INDEF
Number of output pixels (820): INDEF
new010: ap = 1, w1 = 5100., w2 = 6500., dw = 1.797, nw = 780
Change wavelength coordinate assignments? (yes): no
new447: ap = 0, w1 = 5082.57, w2 = 6551.45, dw = 1.793, nw = 820
Change wavelength coordinate assignments? (yes): no
new448: ap = 1, w1 = 5082.03, w2 = 6553.66, dw = 1.797, nw = 820
Change wavelength coordinate assignments? (no):
```

3. Confirm global assignments and do dispersion correction in place. record format.

```
ir> dispcor irs "" 9,10,447,448 confirm+ global+ samedisp+
irs.0009: ap = 0, w1 = 5078.71, w2 = 6553.66, dw = 1.801, nw = 820
Change wavelength coordinate assignments? (yes):
Starting wavelength (5078.7071234): 5100
Ending wavelength (6553.664123): 6500
Wavelength interval per pixel (1.80092412):
Number of output pixels (820):
irs.0009: ap = 0, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (yes): no
irs.0010: ap = 1, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
irs.0447: ap = 0, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
irs.0448: ap = 1, w1 = 5100., w2 = 6500., dw = 1.799, nw = 779
Change wavelength coordinate assignments? (no):
```

4. Make a nonlinear dispersion correction in place.

```
on> dispcor spec* "" linearize=no verbose- logfile=logfile
```

5. Apply a single dispersion solution to a set of record number format images.

```
ir> dispcor nite101 dcnite101 "1-10" ignore+ confirm-
```

REVISIONS

DISPCORV2.12.3

Added the blank parameter value.

DISPCORV2.11.3

Long slit and data cubes can be used with this task to either resample using the existing WCS or to use a single dispersion function from IDENTIFY. It uses the first one found.

DISPCORV2.10.3

Provision was added for IDENTIFY dispersion solutions consisting of only a shift (as produced by the 'g' key in IDENTIFY or the refit=no flag in REIDENTIFY) to be applied to previously LINEARIZED spectra. Thus it is possible to use IDENTIFY/REIDENTIFY to automatically compute a zero point shift based on 1 or more lines and then shift all the spectra to that zero point.

DISPCOR will now allow multiple uses of IDENTIFY dispersion solutions in a simple way with but with continuing protection against accidental multiple uses of the same dispersion solutions. When a first dispersion corrected using one or more reference spectra keywords the dispersion flag is set and the reference spectra keywords are moved to DCLOGn keywords. If DISPCOR is called again without setting new reference spectra keywords then the spectra are resampled (rebinned) using the current coordinate system. If new reference spectra are set then DISPCOR will apply these new dispersion functions. Thus the user now explicitly enables multiple dispersion functions by adding reference spectra keywords and DISPCOR eliminates accidental multiple uses of the same dispersion function by renaming the reference spectra. The renamed keywords also provide a

The flux conservation option now computes an average across the output pixel rather than interpolating to the middle of the output pixel when *flux* is no. This preserves the flux density and includes all the data; i.e. a coarse resampling will not eliminate features which don't fall at the output pixel coordinates.

Some additional log and verbose output was added to better inform the user about what is done.

Better error information is now printed if a is not found.

DISPCORV2.10

This is a new version with many differences. It replaces the previous three tasks **dispcor**, **ecdispcor** and **msdispcor**. It applies both one dimensional and echelle dispersion functions. The new parameter *linearize* selects whether to interpolate the spectra to a linear dispersion (the only option available previously) or to assign a nonlinear dispersion function to the image without any interpolation. The interpolation function parameter has been eliminated and the package parameter *interp* is used to select the interpolation function. The new interpolation type "sinc" may be used but care should be exercised. The new task supports applying a secondary zero point shift spectrum to a master dispersion function and a spatial interpolation of the shifts when calibration spectra are taken at the same time on a different region of the same 2D image. The optional wavelength table may now also be an image to match dispersion parameters. The *apertures* and *rebin* parameters have been eliminated. If an input spectrum has been previously dispersion corrected it will be resampled as desired. Verbose and log file parameters have been added to log the dispersion operations as desired. The record format syntax is available in the **irs/iids** packages.

SEE ALSO

package, refspectra, scopy, specshift, sapertures

NAME

help -- print online documentation for the named modules or packages

USAGE

help [template]

PARAMETERS**template**

A string listing the modules or packages for which help is desired. Each list element may be a simple name or a pattern matching template. Abbreviations are permitted. If *template* is omitted a menu will be printed for the current package, listing each task (or subpackage) and describing briefly what it is.

file_template= no

If this switch is set the template is interpreted as a template, and all help blocks found in the named files are output. The help database is not searched, hence manual pages can be printed or documents may be formatted without entering the files into a help database. In other words, "help file.hlp fi+" makes it possible to use *help* as a conventional text formatter.

all = no

Print help for all help modules matching *template*, rather than only the first one found.

parameter = "all"

If the value of this parameter is not "all", only the help text for the given parameter will be printed.

section = "all"

If the value of this parameter is not "all", only the help text for the given section (e.g. "usage", "description", "examples") will be printed.

option = help

The option parameter specifies the type of help desired, chosen from the following:

help Print the full help block for the named module.

source

Print the source code for the module (which often contains additional detailed comments).

sysdoc

Print the technical system documentation for the named module.

directory

Print a directory of all help blocks available for the named package.

alldoc

Print all help blocks in the file containing the help block for the named procedure (i.e., both the user and system documentation).

files

Print the names of all help files associated with the named modules or packages.

summary

Print only the titles and sizes of help blocks in referenced help files. The contents of the blocks are skipped. Titles are printed for *all* help blocks found in the file containing the help block for the named module.

page = yes

Pause after every page of output text. Turning this off for large documents speeds up output

considerably.

nlpp = 59

The number of lines per page if output is redirected, e.g., to *lprint*.

lmargin = 1

Left margin on output.

rmargin = 72

Right margin on output.

search = no

If enabled the template is interpreted as a search string and the task is started with the search panel open with the results of the search. The *file_template* parameter is ignored with search turned on.

home = ""

The home page for the task. If not set and no template is specified the task will start with the online help in the main window, otherwise it may be set to a filename to be displayed when the task starts. This file may contain a text help block which will be formatted before display, or it may be a valid HTML file. See below for a description of the format of a homepage file which provides links to tasks.

printer = "printer"

Default hardcopy printer name. If the *value* of the parameter is reserved string "printer", the actual device is the value of the CL environment variable *printer*.

showtype = no

Add task-type suffix in package menus?

quickref = "uparm\$quick.ref"

Name of the quick-reference file used for searching. This file is the first time the task is run in GUI mode or whenever it doesn't exist, or when any help database file has been updated.

uifname = "lib\$scr/help.gui"

The user interface file. This file is what defines the look and behavior of all the graphical user interface elements. Experts may create variants of this file.

helpdb = "helpdb"

The filename of the help database to be searched. If the *value* of the parameter is the reserved string "helpdb", the actual filename is the value of the CL environment variable *helpdb*.

device = "terminal"

Output device if the standard output is not redirected. Allowable values include:

terminal

If the *value* of the parameter is the reserved string "terminal", the actual device name is the value of the CL environment variable *terminal*.

text Output the formatted help page as plain text.

gui Invoke the GUI for browsing the help system. This option will only work if the *stdgraph* environment variable is set the *xgterm*, and the user is running IRAF from an *XGterm* window.

html Output the formatted help page as HTML text.

ps (or postscript)

Output the formatted help page as postscript.

BASIC USAGE

Despite the complex appearing hidden parameters, **help** is easy to use for simple tasks. **Help** is most commonly used to get help on the current package, and to get help on a program named in a the current package one need only type **help** without any arguments. For example, if the current package is **plot**, the command and its output might appear as follows:

```
pl> help
      contour - Make a contour plot of an image
      graph   - Graph one or more image sections or lists
      pcol    - Plot a column of an image
      pcols   - Plot the average of a range of image columns
      prow    - Plot a line (row) of an image
      pros    - Plot the average of a range of image lines
      surface - Make a surface plot of an image
pl>
```

To get help on a module one supplies the module name as an argument,

```
pl> help graph
```

and the manual page for the **plot.graph** program will be printed on the terminal. To get a hardcopy of the manual page on the printer, the output may be redirected to the line printer, as follows:

```
pl> help graph | lprint
```

DESCRIPTION

The function of the **help** program is to perform a of the help database *helpdb*, printing help for all packages and modules matching the template. By default the standard IRAF help database is but any other help database may be searched if desired. A help database is precompiled with the **mkhelpdb** program to speed up runtime searches for help modules. The standard IRAF help database contains the documentation and source for all CL programs and system and math library procedures installed in IRAF.

A help template is a string type parameter to the CL. The form of a is a list of patterns delimited by commas, i.e.,

```
"pattern1, pattern2, ..., patternN"
```

The form of a pattern is

```
package_pattern.module_pattern
```

If the "." is omitted *module_pattern* is assumed. The standard pattern matching meta-characters, i.e., "*" "[]", are permitted in patterns. Simple patterns are assumed to be abbreviations.

GUI OPERATION

The GUI component of the task is a front-end to the IRAF **help** task which provides on-the-fly conversion of help documents to HTML for presentation in the GUI or formatted PostScript for hardcopy. The GUI is started by setting the *device* parameter to the special value *gui*, it is only available when using an XGterm window to start IRAF and assuming the *stdgraph* environment variable is set to *xgterm*.

Help pages may be loaded on the command line, through use of a file browser, or by navigating the help databases using a familiar CL package menu scheme. It also features a search capability similar to the **references** task and a complete history mechanism.

When invoked with no command line arguments the task starts as a `task` and the user is presented with a GUI that has the toplevel CL package menu in the upper navigation window. The main display window below will contain any help page specified in the template parameter or loaded on the command line by specifying the template and file_template parameters. If the search parameter is enabled the template is taken to be a search phrase and the database is searched for tasks matching the keyword and the GUI will appear with the search panel mapped so the user can select the task help to view. When no template is given the main display window will start with the page specified by the home parameter, this can be a user-defined HTML file giving links to specific tasks (see below for details) or if home is empty the display will contain the online help for the task.

The first time the task is run, or whenever the help database is updated, a quick reference file (specified by the task quickref parameter) and package menu file will be created in the user's *uparm* directory to speed up help searching and subsequent startups of the task.

NAVIGATING THE HELP SYSTEM

When run as a GUI browser *HELP* works very much like any WWW browser. The top panel is a list widget that will always contain a CL package listing, at startup this will be the toplevel "Home" package menu one would see when first logging into the CL containing the core system packages, NOAO package, and any site-specific external package, or in the case of starting with a specific task it will be the parent package for the task. Additionally, system documents for the **os** HSI routines and the **imfort** and **math** interfaces will be available in the *Home* package although these are programmatic interfaces and not tasks which can be executed.

New packages or task help pages are loaded by selecting an item from the package menu list using the left mouse button. If the requested item is a package, the menu listing will change as though the package were loaded in the CL, and the help display panel will contain a list of tasks with a one-line description for each task such as would be seen with a "help <package>" command using the standard task. New items may then be selected using either the menu list or links in the display panel. If the item is a task, the help page for the task will appear in the display panel. In either case new pages may be selected from the menu listing.

Specific help documents may also be requested by entering the task/package name in the **Topic** text widget above the menu list. As when selecting from the package menu list, items selected this way will cause the menu list to change to the package menu for the parent package if task (displaying the help page in the display panel) or the package menu if the item is a package (displaying the one-liner package listing in the display panel).

Using the **Back** button will revert to the previous page in the history list which will either be the previously loaded package or help page. Similarly, selecting the **Forward** button will move the next page further down in the history list, either button will become insensitive when the end of the list on either end is reached. Selecting the **Up** button will cause the browser to immediately jump up the previous package, skipping over any help pages that were loaded in between. The **Home** button will cause the default homepage (either the user-defined page if specified by the task *home* parameter or the online help) to be displayed. Browsing in this way can also be done using the navigation menu created by hitting the right mouse button while in the main display panel.

Users can also jump to specific pages in the history list using the **History** button on the main menubar. The right column of the menu will indicate whether the item is a task, package, internal link or a file. The history list is truncated at about 40 entries in the menu but the user may work back incrementally by selecting the last item of the menu, after which the History button will display the previous 40 entries. The history list may be cleared except for the current page by selecting the *Clear History* menu item.

BROWSING A HELP DOCUMENT

Once a help page is loaded the middle menubar above the display panel will change to activate widgets based on the position within the history list and options available for a particular page. The left-most group of buttons are the standard navigation buttons described above. The middle group of buttons contains the **Sections** and **Parameters** buttons which are used to browse within a . The *Sections* button is a menu listing all of the sections found within a help page, allowing the user to jump to a rather than scrolling through the entire document. The *Sections* menu is also available using the middle mouse button from the main display area. The *Parameters* button is similarly a menu listing of all task parameter help sections found within the document. Both or either of these buttons will become insensitive when no section or parameter information is found in the document.

The right-most group of buttons represent the various help options available for each page. The default is to get the task help, however help pages may have an associated **source** file or **sysdoc** (e.g. if a CL script there may be a pointer to the script source itself, or a package may have a general overview document listed as the system document). Once a help page is loaded these buttons will change become sensitive if is available, simply select the button to view the option. Selecting the **Files** button will bring up a panel listing all the files associated with a particular help topic. When a help topic is selected and an option is defined but the file does not exist, the options button will display a yellow diamond icon even if the button is insensitive, a green icon indicates the currently selected option. This feature may be disabled by selecting the "Show missing files" item from the main menubar **Options** menu.

SEARCHING

Searching the help database is done by selecting the **Search** button from the main menubar to bring up the search panel. Users may then enter one or more keywords into the **Topic** field at the bottom of the panel and initiate the search with either a carriage return or hitting the *Search* button just beside it. The panel will then show a list of all tasks and packages which match the search phrase along with a description of the task. Help pages may be displayed by selecting either the task or package link with the left mouse button, in both case the package menu list on the main help window will be updated to list the package contents allowing other tasks from that package to be selected in the normal way.

By default the exact phrase entered in the topic window will be used for the search. This can be relaxed by toggling the "Require exact match" button at the top of the panel. For example, to search for all tasks matching *either* the keyword "flat" or "field" turn off the exact match toggle and the search will return not only tasks matching "flat field" but also any task description containing only one of the words such as the VELVECT task which plots velocity *fields*.

Within a help document itself one can search for a the **Find** button from the main menubar to bring up a enter the search string. When the text is entered the main display window will reposition itself and highlight the text found within the document. Searches can be repeated and will wrap around the document automatically, searches can be done either forward or backward through the text and may be case insensitive.

USER_DEFINED HOME PAGES

By default the *help* GUI will start with the online help page displayed in the main help window. The user can change this by setting the task **home** parameter to be a path to any valid file. This file may be plain text, a help document in LROFF format which will be converted to HTML for display, or a native HTML document.

HTML files may contain URLs of the form

```
<a href=[package.]task>url_text</a>
```

where *url_text* is the text to appear in the window and the URL itself consists of an optional package and task name delimited by a period. For example, to create a link to the **splot** task in a document one would

use the URL

```
<a href=onedspec.splot>splot</a>
```

In this way users can create a homepage which serves as a file or index of shortcuts to the most commonly accessed help pages.

LOADING FILES

Text files may be loaded on the command line when starting the task by specifying the filename and setting the `file_template` task parameter. The named file will be searched for a `.help` LROFF directing indicating it a help block that will be converted to HTML for display. If no help block is found the file will be displayed as-is, meaning existing HTML documents can be loaded and will be formatted correctly.

Once the task is running users may load a file by selecting the **Open File...** menu item from the main menubar **File** menu or the right-mouse-button menu from within the main display area. This will open a file browser allowing users to change directories by using the navigation buttons at the top of the panel, or selecting items from the leftmost directory listing. Selecting a file on the rightmost list will cause it to be loaded and automatically formatted if it contains a help block. The file list may be filtered to select only those files matching a particular template by changing the **Filter** box at the top of the panel. Filenames or directories may be entered directly using the **Selection** box at the bottom of the panel.

SAVING FILES

Once a file has been loaded in the browser it may be saved to disk as either *source* (i.e. the original LROFF file if that was converted for the display, or whatever file is currently displayed regardless of format), *text* to save formatted plain text such as that produced by the standard **help** task, *HTML* to save the converted HTML used in the display, or *PostScript* to save formatted PostScript of the document such as that sent to the printer using the **Print** button. Not all options will be available depending on the format of the input text, unavailable options will be insensitive in the GUI.

The **Save** panel is opened by selecting the **Save As...** menu item from the main menubar **File** menu or the right-mouse-button menu from within the main display area. The file browser operates the same as when loading images, the only difference is that file selection simply defines the filename to be used and does not cause the save to occur automatically. Users can overwrite existing files by selecting the *Options* toggle at the bottom of the panel.

HARDCOPY OUTPUT AND SAVING DISK FILES.

Help pages may be output to any configured IRAF printer by selecting the main menubar **Print** button to bring up the print panel. Task help pages will be converted to formatted PostScript and may be sent to either a printer or saved to disk depending on the selection made in the printer panel. If the printer name is set to the special value "printer" then the device named by the CL *printer* environment variable will be used. When saving to disk files the default action is to save to a name is the task name plus a ".ps" extension. Either of these are changeable within the GUI as is the default page size to be used when generating the PostScript.

The main menubar **File** button can also be used to bring up the file browser in order to save the current document to disk. Help pages may be saved as either the origin LROFF source for the file, formatted text as you would get from the standard help task, HTML as is displayed in the GUI, or formatted PostScript. The choice of formats is dictated by the type of file being displayed (e.g. you cannot save PostScript of a

LROFF DIRECTIVE EXTENSIONS FOR HTML

To better support HTML links within documents and to other help pages two new directives have been added to the LROFF text formatter. These are **.hr** to specify a link (an HTML *HREF* directive) and **.hn** to specify a name (an HTML *NAME* directive). The syntax for these are as follows:

```
.hn <name>
.hr <link> <text>
```

where <name> is the destination name of an internal link, <link> is the URL of the link to be created, and <text> is the text to be displayed in the HTML. The URL syntax is either a by a destination name, a simple *task* name or *package* name, or a *package.task* pair giving a more precise task. For internal links the current document is repositioned so the name is at the top of the display, for task help links new help pages will be loaded in the browser.

These directives are ignored when converting the LROFF to either formatted plain text or PostScript.

GUI EXAMPLES

- 1) Start *help* as a GUI browser:

```
cl> help dev=gui
```

- 2) Begin by searching for the phrase 'gauss', tasks and packages may be selected from the search panel which will appear when the task starts:

```
cl> help gauss dev=gui search+
```

- 3) Load an LROFF help page in the browser at startup

```
cl> help mytask.hlp dev=gui file+
```

EXAMPLES

1. Print the help text for the program *delete* in the package *system* (output will be directed to the terminal):

```
cl> help system.delete
or
cl> help delete
or
cl> help del
```

2. Print the help text on the line printer:

```
cl> help delete | lprint
```

3. Print help for the current package:

```
cl> help
```

4. Print the usage section of all modules in the package **images**:

```
cl> help images.* section=usage
```

5. Print a directory of all help blocks in the packages **clpackage** and **clio** (and any others whose names begin with the string "cl"):

```
cl> help cl* op=dir
```

6. Print a directory of each package in the database (useful for getting an overview of the contents of a help database):

```
cl> help * op=dir
```

7. Print the source for all of the string utilities in the system library package **fmtio**:

```
cl> help fmtio.str* op=source
```

8. Find all tasks that delete something:

```
cl> help * | match delete
```

9. Print the manual pages for the *help* and *lprint* tasks on the default printer device:

```
cl> help help, lprint | lpr
```

10. Capture the manual page for task *hedit* in a suitable for printing on any device.

```
cl> help hedit dev=text > hedit.txt
```

11. Print the manual page for task *hedit* as a

```
cl> help hedit dev=ps | lprint
```

BUGS

On some systems, typing the next command keystroke before the end-of-page prompt is printed may result in the character being echoed (messing up the output) and then ignored when raw mode is enabled for the prompt.

SEE ALSO

references , phelp , mkhelpdb , hdbexamine , mkmanpage , lroff , the online task help documents.

NAME

imarith -- binary image arithmetic

USAGE

imarith operand1 op operand2 result

PARAMETERS**operand1,operand2**

Lists of images and constants to be used as operands. Image templates and image sections are allowed.

op Operator to be applied to the operands. The allowed operators are "+", "-", "*", "/", "min", and "max".

result

List of resultant images.

title = ""

Title for the resultant images. If null("") then the title is from operand1 if operand1 is an image or from operand2 otherwise.

divzero = 0.

Replacement value for division by zero. When the denominator is zero or nearly zero the result is replaced by this value.

hparams = ""

List of header parameters to be operated upon. This is primarily used for adding exposure times when adding images.

pixtype = "", calctype = ""

Pixel datatype for the resultant image and the internal calculation datatype. The choices are given below. They may be abbreviated to one character.

""

Calctype defaults to the highest precedence operand datatype. If highest precedence datatype is an integer type and the operation is division then the calculation type will be "real". If the highest precedence operand is type "ushort", *calctype* will default to "long". *Pixtype* defaults to *calctype*. Users who want type "ushort" images on output will need to set *pixtype* to "ushort" explicitly.

"1", "2"

The pixel datatype of the first or second operand.

"short", "ushort", "integer", "long", "real", "double"

Allowed IRAF pixel datatypes.

verbose = no

Print the operator, operands, calculation datatype, and the resultant image name, title, and pixel datatype.

noact = no

Like the verbose option but the operations are not actually performed.

DESCRIPTION

Binary image arithmetic is performed of the form:

operand1 op operand2 = result

where the operators are addition, subtraction, multiplication, division, and minimum and maximum. The division operator checks for nearly zero denominators and replaces the ratio by the value specified by the parameter *divzero*. The operands are lists of images and numerical constants and the result is a list of images. The number of elements in an operand list must either be one or equal the number of elements in the resultant list. If the number of elements is one then it is used for each resultant image. If the number is equal to the number of resultant images then the elements in the operand list are matched with the elements in the resultant list. The only limitation on the combination of images and constants in the operand lists is both operands for a given resultant image may not be constants. The resultant images may have the same name as one of the operand images in which case a temporary image is created and after the operation is successfully completed the image to be replaced is overwritten by the temporary image.

If both operands are images the lengths of each axis for the common dimensions must be the same though the dimensions need not be the same. The resultant image header will be a with the greater dimension. If the dimensions are the same then image header for the resultant image is copied from operand1. The title of the resultant image may be changed using the parameter *title*. The pixel datatype for the resultant image may be set using the parameter *pixtype*. If no pixel datatype is specified then the pixel datatype defaults to the calculation datatype given by the parameter *calctype*. The calculation datatype defaults to the highest precedence datatype of the operand images or constants except that a division operation will default to real for integer images. The precedence of the datatypes, highest first, is double, real, long, integer, and short. The datatype of a constant operand is either short integer or real. A real constant has a

Arithmetic on images of unequal dimensions implies that the operation is repeated for each element of the higher dimensions. For example subtracting a two dimensional image from a three dimensional image consists of subtracting the two dimensional image from each band of the three dimensional image. This works for any combination of image dimensions. As an extreme example dividing a by a one dimension image consists of dividing each line of each plane of each band ... by the one dimensional image.

There are two points to emphasize when using images of unequal dimensions. First, a one dimensional image operates on a line of a two or higher dimension image. To apply a to the columns of a higher dimensional image increase the image dimensionality with **imstack**, transpose the resultant image, and then replicate the columns with **blkrep** (see the EXAMPLE section). The second point of confusion is that an image with a size given by **imheader** of [20,1] is a while an image with size of [20] is a one dimensional image. To reduce the dimensionality of an image use **imcopy**.

In addition to operating on the image pixels the image header parameters specified by the list *hparams* are also operated upon. The operation is the same as performed on the pixels and the values are either the values associated with named header parameters or the operand constant values. The primary purpose of this feature is to add exposure times when adding images.

The verbose option is used to record the image arithmetic. The output consists of the operator, the operand image names, the resultant image name and pixel datatype, and the calculation datatype.

EXAMPLES

1. To add two images and the exposure times:

```
cl> imarith ccd1 + ccd2 sum
>>> hparams="itime,otime,ttime,exposure"
```

2. To subtract a constant from an image and replace input image by the subtracted image:

```
cl> imarith m31 - 223.2 m31
```

Note that the final pixel datatype and the calculation datatype will be at least of type real because the constant operand is real.

3. To scale two exposures, divide one by the other, and extract the central portion:

```
cl> imarith exp1[10:90,10:90] * 1.2 temp1
cl> imarith exp2[10:90,10:90] * 0.9 temp2
cl> imarith temp1 / temp2 final title='Ratio of exp1 and exp 2'
cl> imdelete temp1,temp2
```

Note that in this example the images temp1, temp2, and final will be of real pixel datatype (or double if either exp1 or exp2 are of pixel datatype double) because the numerical constants are real numbers.

4. To divide two images of arbitrary pixel datatype using real arithmetic and create a short pixel datatype resultant image:

```
cl> imarith image1 / image2 image3 pixtype=short \
>>> calctype=real title="Ratio of image1 and image2"
```

5. To divide several images by calibration image using the image pixel type of the numerator images to determine the pixel type of the calibrated images and the calculation arithmetic type:

```
cl> imarith image1,image2,image3 / calibration \
>>> image1a,image2a,image3a pixtype=1 calctype=1
```

The same operation can be done in place with image template expansion by:

```
cl> imarith image* / calibration image* pixtype=1 calctype=1
```

6. To subtract a two dimensional bias from stacked observations (multiple two dimensional observations stacked to form a three dimensional image):

```
cl> imarith obs* - bias obs*//b
```

Note that the output observations obs101b, ..., will be three dimensional.

7. To divide a 50 x 50 image by the average column:

```
cl> blkavg img avcol 50 1
cl> blkrep avcol avcol 50 1
cl> imarith img / avcol flat
```

8. To subtract a one dimensional image from the lines of a image:

```
cl> imarith im2d - im1d diff
```

9. To subtract a one dimensional image from the columns of a image:

```
cl> imstack im1d imcol
cl> imtranspose imcol imcol
cl> blkrep imcol imcol 100 1
cl> imarith im2d - imcol diff
```

Note the need to make a two dimensional image with each column replicated since a one dimensional image will operate on the lines of a two dimensional image.

SEE ALSO

blkrep, imdivide, imfunction, imstack, imtranspose

NAME

imcopy -- copy images

USAGE

imcopy input output

PARAMETERS**input**

Images to be copied.

output

Output images or directory.

verbose = yes

Print each operation as it takes place?

DESCRIPTION

Each of the input images, which may be given as a including sections, is copied to the corresponding output image list, which may also be given as an image template, or the output directory. If the output is a list of images then the number of input images must be equal to the number of output images and the input and output images are paired in order. If the output image name exists and contains a input image (provided it is the same size as the section) will be copied into that section of the input image. If the output image name does not have a section specification and if it is the same as the input image name then the input image is copied to a temporary file which replaces the input image when the copy is successfully concluded. Note that these are the only cases where clobber checking is bypassed; that is, if an output image name is not equal to the input image name or a subsection of an existing image and the file already exists then a clobber error will occur if clobber checking is in effect.

The verbose options prints for each copy lines of the form:

input image -> output image

EXAMPLES

1. For a simple copy of an image:

```
cl> imcopy image imagecopy
```

2. To copy a portion of an image:

```
cl> imcopy image[10:20,*] subimage
```

3. To copy several images:

```
cl> imcopy image1,image2,frame10 a,b,c
```

4. To trim an image:

```
cl> imcopy image[10:20,*] image
```

In the above example the specified section of the input image replaces the original input image. To trim several images using an image template:

```
cl> imcopy frame*[1:512,1:512] frame*
```

In this example all images beginning with "frame" are trimmed to 512 x

5. To copy a set of images to a new directory:

```
cl> imcopy image* directory
```

or

```
cl> imcopy image* directory$
```

or

```
cl> imcopy image* osdirectory
```

where "osdirectory" is an operating system directory name (i.e. /user/me in UNIX).

6. To copy a section of an image in an already existing image of sufficient size to contain the input section.

```
cl> imcopy image[1:512,1:512] outimage[257:768,257:768]
```

BUGS

The distinction between copying to a section of an existing image and overwriting a input image is rather inobvious.

NAME

instruments -- Instrument specific data files

DESCRIPTION

The **ccdred** package has been designed to accommodate many different instruments, detectors, and observatories. This is done by having instrument specific data files. Note that by instrument we mean a combination of detector, instrument, application, and observatory, so there might be several "instruments" associated with a particular CCD detector. Creating and maintaining the instrument files is generally the responsibility of the support staff, though the user may create or copy and modify his/her own instrument/application specific files. The task **setinstrument** makes this information available to the user and package easily.

There are three instrument data files, all of which are optional. The package may be used without the instrument files but much of the convenience of the package, particularly with respect to using the CCD image types, will be lost. The three files are an instrument image header translation file, an initialization task which mainly sets default task parameters, and a bad pixel file identifying the cosmic bad pixels in the detector. These files are generally stored in a system data directory which is a subdirectory of the logical directory "ccddb\$". Each file has a root name which identifies the instrument.

1. Instrument Translation File

The instrument translation file translates the parameter names used by the **ccdred** package into instrument specific parameters and also supplies instrument specific default values. The package parameter *ccdred.instrument* specifies this file to the package. The task **setinstrument** sets this parameter, though it can be set explicitly like any other parameter. For the standard instrument translation file the root name is the instrument identification and the extension is "dat" ("*.dat" files are protected from being removed in a "stripped" system, i.e. when all nonessential files are removed). Private instrument files may be given any name desired.

The instrument translation proceeds as follows. When a parameter for an image, for example "imagetyp", it looks in the instrument translation file. If the file is not found or none is specified then the image header keyword that is requested has the same name. If an instrument translation file is defined then the requested parameter is translated to an image header keyword, provided an entry is given. If no translation is given the package name is used. For example the package parameter "imagetyp" might be translated to "data-typ" (the old NOAO CCD keyword). If the parameter is not found then the default value specified in the translation file, if present, is returned. For recording parameter information in the header, such as processing flags, the translation is also used. The default value has no meaning in this case. For example, if the flag specifying that the image has been corrected by a flat field is to be set then the package parameter name "flatcor" might be translated to "ff-flag". If no translation is given then the new image header parameter is entered as "flatcor".

The format of the translation file are lines consisting of the package parameter name, followed by the image header keyword, followed by the default value. The first two fields are parameter names. The fields are separated by whitespace (blanks and tabs). String default values containing blanks must be quoted. An example is given below.

```
# Sample translation file.
exptime      itime
darktime     itime
imagetyp     data-typ
subset       flpos
biassec      biassec      [411:431,2:573]
datasec      datasec      [14:385,2:573]

fixpix       bp-flag      0
```

```

overscan    bt-flag    0
zerocor     bi-flag    0
darkcor     dk-flag    0
flatcor     ff-flag    0
fringcor    fr-flag    0

```

The first comment line is ignored as are blank lines. The first two lines translate the CCD image type, and the subset parameter without default values (see **ccdtypes** and **subsets** for more information). The next two lines give the overscan bias strip section and the data section with default values for the instrument. Note that these parameters may be overridden in the task **ccdproc**.

The next set of translations requires further discussion. For processing flags the package assumes that the absence of a keyword means that the processing has not been done. If processing is always to be done with the **CCDRED** package and no processing keywords are recorded in the raw data then these parameters should be absent (unless you don't like the names used by the package). However, for compatibility with the original NOAO CCD images, which may be processed outside of IRAF and which use 0 no processing value, the processing flags are translated and the false values are indicated by the default values.

If there is more than one translation for the same CCDRED parameter, for example more than one `exptime`, then the last one is used.

In addition to the parameter name translations the translation file contains translations between the value of the image type parameter and the image types used by the package. These lines consist of the image header type string as the first field (with quotes if there are blanks) and the image type as recognized by the package. The following example will make this clearer.

```

'OBJECT (0)'      object
'DARK (1)'        dark
'PROJECTOR FLAT (2)' flat
'SKY FLAT (3)'    other
'COMPARISON LAMP (4)' other
'BIAS (5)'        zero
'DOME FLAT (6)'   flat

```

The values of the image type strings in the header contain blanks so they are quoted. Also the case of the strings is important. Note that there are two types of flat field images and three types of object images.

The CCD image types recognized by the package are:

```

zero   - zero level image such as a bias or preflash
dark   - dark count image
flat   - flat field image
illum  - illumination image such as a sky image
fringe - fringe correction image
object - object image

```

There may be more than one image type that maps to the same package type. In particular other standard CCD image types, such as comparison spectra, multiple exposure, standard star, etc., should be mapped to object or other. There may also be more than one type of flat field, i.e. dome flat, sky flat, and lamp flat. For more on the CCD image types see **ccdtypes**.

The complete set of package parameters are given below. The package parameter names are generally the same as the standard image header keywords being adopted by NOAO.

General Image Header and Default Parameters			
ccdmean	darktime	exptime	fixfile
imagetyp	ncombine	biassec	subset
title	datasec	nscanrow	

CCDRED Processing Flags

ccdproc	darkcor	fixpix	flatcor
fringcor	illumcor	overscan	trim
zerocor			
CCDRED CCD Image Types			
dark	flat	fringe	illum
none	object	unknown	zero

The translation mechanism described here may become more sophisticated in the future and a general IRAF system facility may be implemented eventually. For the present the translation mechanism is quite simple.

2. Instrument Setup Script

The task **setinstrument** translates an instrument ID into a CL script in the instrument directory. This script is then executed. Generally this script simply sets the task parameters for an instrument/application. However, it could do anything else the support staff desires. Below are the first few lines of a script.

```
ccdred.instrument = "ccddb$kpno/example.dat"
ccdred.pixeltype = "real"
ccdproc.fixpix = yes
ccdproc.overscan = yes
ccdproc.trim = yes
ccdproc.zerocor = no
ccdproc.darkcor = no
ccdproc.flatcor = yes
ccdproc.biassec = "[411:431,2:573]"
ccdproc.datasec = "[14:385,2:573]"
```

The instrument parameter should always be set unless there is no translation file for the instrument. The **ccdproc** parameters illustrate setting the appropriate processing flags for the instrument. The overscan bias and trim data sections show an alternate method of setting these instrument specific parameters. They may be set in the setup script in which case they are given explicitly in the user parameter list for **ccdproc**. If the value is "image" then the parameters may be determined either through the default value in the instrument translation file, as illustrated in the previous section, or from the image header itself.

The instrument setup script for setting default task parameters may be easily created by the support person as follows. Set the package parameters using **eparam** or with CL statements. Setting the parameters might involve testing. When satisfied with the way the package is set then the parameters may be dumped to a using the task **dparam**. The final step is editing this script to delete unimportant and query parameters. For example,

```
cl> dparam ccdred >> file.cl
cl> dparam ccdproc >> file.cl
cl> dparam combine >> file.cl
...
cl> ed file.cl
```

3. Instrument Bad Pixel File

The bad pixel file describes the bad pixels, columns, and lines in the detector which are to be replaced by interpolation when processing the images. This file is clearly detector specific. The file consists of lines describing rectangular regions of the image. The regions are specified by four numbers giving the starting and ending columns followed by the starting and ending lines. The starting and ending points may be the same to specify a single column or line. The example below illustrates a bad pixel file.

```
# RCA1 CCD untrimmed
25 25 1 512
108 108 1 512
302 302 403 512
```

```
1 512 70 70
245 246 312 315
```

If there is a comment line in the file containing the word "untrimmed" then the coordinates of the bad pixel regions apply to the original CCD detector coordinates. If the image has been trimmed and the bad pixels are replaced at a later stage then this word indicates that the trim region be determined from the image header and the necessary coordinate conversion made to the original CCD pixel coordinates. Note that if a subraster readout is used the coordinates must still refer to the original CCD coordinates and not the raw, untrimmed readout image. If the word "untrimmed" does not appear then the coordinates are assumed to apply to the image directly; i.e. the trimmed coordinates if the image has been trimmed or the original coordinates if the image has not been trimmed. The standard bad pixel files should always refer to the original, untrimmed coordinates.

The first two bad pixel regions are complete bad columns (the image is 512 x 512), the next line is a partial bad column, the next line is a bad line, and the last line is a small bad region. These files are easy to create, provided you have a good image to work from and a way to measure the positions with an image or graphics display.

SEE ALSO

ccdtypes, subsets, setinstrument

NAME

lcalib -- List information about the spectral calibration data

USAGE

lcalib option star_name

PARAMETERS**option**

Chooses calibration data to be listed. Option may be: "bands" to list the bandpasses at each wavelength, "ext" to list the extinction at each wavelength, "mags", "fnu", or "flam" to list the magnitude, or flux of the star (selected by the star_name parameter) at each wavelength, or "stars" to list the star names available in the calibration directory.

star_name

Selects which star's magnitude list is chosen if the option parameter is "mags", "fnu", "flam", or "bands". Also if '?' a list of available stars in the specified calibration directory is given.

The following three queried parameters apply if the selected calibration file is for a blackbody. See **standard** for further details.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

extinction

Extinction file. The current standard extinction files:

```
onedstds$kpnoextinct.dat - KPNO standard extinction
onedstds$ctioextinct.dat - CTIO standard extinction
```

caldir

Calibration directory containing standard star data. The directory name must end with /. The current calibration directories available in the onedstds\$ may be listed with the command:

```
cl> page onedstds$README
```

fnuzero= 3.68e-20

The absolute flux per unit frequency at a magnitude of zero. This is to convert the calibration

magnitudes to absolute flux by the formula

$$\text{Flux} = \text{fnuzero} * 10. ** (-0.4 * \text{magnitude})$$

The flux units are also determined by this parameter. However, the frequency to wavelength interval conversion assumes frequency in hertz. The default value is based on a calibration of Vega at 5556 Angstroms of $3.52\text{e-}20$ ergs/cm²/s/hz for a magnitude of 0.048. This default value is that used in earlier versions of this task which did not allow the user to change this calibration.

DESCRIPTION

LCALIB provides a means of checking the flux calibration data. The calibration data consists of extinction, bandpasses, and stellar magnitudes.

The extinction is given in an extinction file consisting of lines with wavelength and extinction. The wavelengths must be order in increasing wavelength and the wavelengths must be in Angstroms. There are two standard extinction files currently available, "onedstds\$kpnoextinct.dat", and "onedstds\$ctioextinct.dat".

The standard star data are in files in a calibration directory specified with the parameter *caldir*. A standard star file is selected by taking the star name given, by the parameter *star_name*, removing blanks, +’s and -’s, appending ".dat", and converting to lower case. This file name is appended to the specified calibration directory. A calibration file consists of lines containing a stellar magnitude, and a bandpass full width. The wavelengths are in Angstroms. Comment lines beginning with # may be included in the file. The star names printed by this task are just the first line of each file in the calibration directory with the first character (#) removed. The calibration files may be typed, copied, and printed. **Lcalib** may also be used to list data from the calibration files.

EXAMPLES

```
# List the extinction table
cl> lcalib ext
# Plot the extinction table
cl> lcalib ext | graph
# Plot the energy distribution
cl> lcalib mags "bd+28 4211" | graph
# List the names of all the stars
cl> lcalib stars caldir=onedstds$irscal/
# As above but for IIDS file
cl> lcalib stars calib_file=onedstds$iidscal/
```

REVISIONS

LCALIBV2.10

This task has a more compact listing for the "stars" option and allows paging a list of stars when the star name query is not recognized.

SEE ALSO

standard, sensfunc, onedstds\$README

NAME

mk1dspec -- Make/add artificial 1D spectra

USAGE

mk1dspec input

PARAMETERS**input**

Spectra to create or modify.

output = ""

Output spectra when modifying input spectra. If no output spectra are given then existing spectra in the input list are modified directly. If an output list is given then it must match in number the input list.

ap = 1

Image line to be created or modified in images of dimension greater than 1.

rv = 0.

Radial velocity (km/s) or redshift, as selected by the parameter *z*, applied to line positions and continuum. Velocities are converted to redshift using the relativistic relation $1+z = \sqrt{(1+rv/c)/(1-rv/c)}$. Note the shift is not a shift in the dispersion parameters but in the underlying artificial spectrum.

z = no

Is the velocity parameter a radial velocity or a

WHEN CREATING NEW SPECTRA**title = ""**

Image title to be given to the spectra. Maximum of 79 characters.

ncols = 512

Number of columns.

naps = 1

Number of lines or apertures.

header = "artdata\$stdheader.dat"

Image or header keyword data file. If an image is given then the image header is copied. If a file is given then the FITS format cards are copied. This only applies to new images. The data file consists of lines in FITS format with leading whitespace ignored. A FITS card must begin with an uppercase/numeric keyword. Lines not beginning with a FITS keyword such as comments or lower case are ignored. The user keyword output of **imheader** is an acceptable data file. See **mkheader** for further information.

wstart= 4000., wend = 8000.

Starting and ending wavelengths in Angstroms. The dispersion is determined by these values and the number of columns.

CONTINUUM PARAMETERS

continuum= 1000., slope = 0.

Continuum of the starting wavelength at rest and the slope of the continuum.

temperature= 5700.

Blackbody continuum temperature in Kelvin. A value of 0 no blackbody continuum is desired. The intensity level is set by scaling to the continuum level of the starting wavelength at rest.

fnu = no

Compute the continuum as flux per unit frequency (F-nu) if yes or flux per unit wavelength (F-lambda) if no.

LINE PARAMETERS

lines = ""

List of spectral line files. Spectral line files contain lines of rest wavelength, peak, profile type, and widths (see the DESCRIPTION section). The latter parameters may be missing or INDEF in which case they default to the task *peak*, *profile*, *gfhwm*, and *lfwhm* parameters (note that the *peak* parameter is not a constant but the random number scaling). If no file or a new (nonexistent) file is specified then a number of random lines given by the parameter *nlines* is generated. If a new file name is specified then the lines generated are recorded in the file. If the list of spectral line files is shorter than the list of input spectra, the last spectral line list file is reused.

nlines = 0

If no spectral line file or a new file is specified then the task will generate this number of random spectral lines. The rest wavelengths are uniformly random within the limits of the spectrum, the peaks are uniformly random between zero and the value of the *peak* parameter, the profile type is given by *profile*, and the widths are fixed at the values of the *gfhwm* and *lfwhm* parameters. If a rest wavelengths are shifted and repeated periodically.

profile = "gaussian" (gaussian|lorentzian|voigt)

The default profile type for random lines or when not specified in the spectral line file. The profile types are:

```
gaussian - Gaussian profile
lorentzian - Lorentzian profile
voigt - Voigt profile
```

peak = -0.5

The maximum spectral line peak value when generating random lines or when the peak is missing from the spectral line file. This value is relative to the continuum unless the continuum is zero. Negative values are absorption lines and positive values are emission lines.

gfhwm = 20., lfwhm = 20.

The default gaussian and lorentzian full widths at half maximum (FWHM), in Angstroms, used when generating random lines or when the widths are missing from the spectral line file.

seed = 1

Random number seed. If a value of "INDEF" is given then the clock time (integer seconds since 1980) is used as the seed yielding different random numbers for each execution.

comments= yes

Include comments recording task parameters in the image header?

PACKAGE PARAMETERS

nxsub = 10

Number of pixel subsamples used in computing the gaussian spectral line profiles.

dynrange= 100000.

The gaussian line profiles extend to infinity so a of the peak intensity to the cutoff intensity, is imposed to cutoff the profiles.

DESCRIPTION

This task creates or modifies one dimensional spectra. with a of blackbody and linear sloped continuum and emission and absorption spectral lines. The spectral lines may be gaussian, lorentzian, or voigt profiles. A velocity shift may be applied to the underlying artificial spectrum which is shifted into the specified observed wavelength region. No noise is included but may be added with the task **mknoise**. New spectra are created with the specified number of pixels, wavelength range, and real datatype. When *nlines* is greater than 1 then an image with the specified number of lines is created though only the line given by the *ap* is will have a spectrum. Existing spectra may be modified in place or new spectra output. Spectra are modified by adding the continuum and lines defined by the parameters.

For new images a set of header keywords may be added by specifying an image or data file with the *header* parameter (see also **mkheader**). If a data file is specified lines beginning with FITS keywords are entered in the image header. Leading whitespace is ignored and any lines beginning with words having lowercase and nonvalid FITS keyword characters are ignored. In addition to this optional header, parameters for the wavelength coordinates are defined. Finally, comments may be added to the image header recording the task parameters and any information from the line file which are not line definitions.

Initially all spectra are created without a pixel coordinates. For multiple spectra in an image this task must be executed for each image line to set the dispersion function and add data. When an image line is selected if it has a defined dispersion function that is used otherwise the task wavelength parameters are used.

A continuum is defined by the value at the starting wavelength at rest, a slope, and a blackbody function of a given temperature. The blackbody function is scaled to have the specified continuum value at the starting wavelength at rest. The blackbody flux units are per unit wavelength (F-lambda). A zero continuum value or a zero temperature will not produce a blackbody continuum.

Spectral lines are modeled by gaussian, lorentzian, or voigt profiles of specified wavelength, peak, and widths. The lines are defined in a spectral line file or generated randomly. A spectral line file consists of text lines giving rest wavelength, peak, profile type, gaussian full width at half maximum and/or lorentzian full width at half maximum. Only the wavelength is required and subsequent fields may be missing or given as INDEF. The following table shows the possible formats where wavelength, peak, gfw hm, and lfwhm are values of wavelength, peak, gaussian FWHM, and lorentzian FWHM. The profile types are as shown though they may be abbreviated to one character.

```
wavelength
wavelength peak
wavelength peak gaussian
wavelength peak gaussian gfw hm
wavelength peak gaussian gfw hm
wavelength peak lorentzian
wavelength peak lorentzian lfwhm
wavelength peak lorentzian lfwhm
wavelength peak voigt
wavelength peak voigt gfw hm
wavelength peak voigt gfw hm lfwhm
wavelength peak voigt gfw hm lfwhm
```

When a field is missing or INDEF the values given by the parameters *peak*, *profile*, *gfwhm*, and *lfwhm* are used. If a peak value is missing, random values between zero and the *peak* value are generated. Note that to get random line intensities with some specified profile type and widths the value INDEF would be used for the peak field.

If no spectral line file is specified or a new (nonexistent) file is then the number of random lines given by the parameter *nlines* is generated. The rest wavelengths are uniformly random within the wavelength range of the spectrum and extend periodically outside this range in the case of an applied velocity shift, the peaks are uniformly random between zero and the *peak* parameter, and the profile type and widths are given by the *profile*, *gfwhm*, and *lfwhm* parameters. If a new file is named then the parameters of the generated lines will be output.

The peak values are taken relative to a positive continuum. In other words the generated line profile is multiplied by the continuum (with a minimum of zero for fully saturated absorption lines). If the continuum is less than or equal to zero, as in the case of an artificial arc spectrum or pure emission line spectrum, then the peak values are absolute intensities. Positive peak values produce emission lines and negative values produce absorption lines. Odd results will occur if the continuum has both positive and zero or negative values.

The underlying rest spectrum may be shifted. This is used primarily for testing radial velocity measuring algorithms and is not intended as a complete model of redshift effects. The starting and ending wavelengths are not changed by redshifting; these are the instrumental observed wavelengths. Input line wavelengths are specified at rest and then shifted into or out of the final spectrum. To be realistic the line list should include wavelengths over a great enough range to cover all desired redshifts. The peaks and widths are also appropriately modified by a redshift. As an example, if the redshift is 1 lines will appear broader by a factor of 2 and the peaks will be down by a factor of 2 in order to maintain the same flux.

The random line generation is difficult in that one wants to have the same set of lines (for a given seed) observed at different redshifts. What is done is that the specified number of random lines is generated within the observed wavelength interval taken at rest. This set is then repeated periodical over all wavelengths. A redshift will then shift these rest lines in to or out of the observed spectrum. If lines are output, they are given at rest. **Note that this periodicity may be important in interpreting cross correlation redshift tests for large shifts between template and object spectra.**

The definitions of the continuum are also affected by a The reference point for the continuum level, slope, and blackbody continuum is the starting wavelength taken at rest. Shifts will then modify the continuum level at the first pixel appropriately. In particular a large redshift will shift the blackbody in such a the flux is still given by the *continuum* parameter at the starting wavelength at rest.

EXAMPLES

1. Create a simple blackbody continuum between the default wavelengths.

```
cl> mk1dspec bb title=Blackbody
```

2. Create a random absorption spectrum on a saving the line list.

```
cl> mk1dspec bbab title=Absorption nlines=100
```

3. Create a random absorption spectrum with noise and cosmic rays.

```
cl> mk1dspec bbab title=Absorption nlines=100
cl> mknoise bbab rdnoise=10 poisson+ ncoss=5 energy=1000
```

4. Create a random emission spectrum on a blackbody continuum and save the line list.

```
cl> mk1dspec bbem title=Emission nl=30 peak=0.6 lines=bbem.dat
```

5. Create an artificial random arc line spectrum.

```
cl> mk1dspec arc title="Arc lines" cont=0 peak=500 nl=30
```

6. Create a test spectrum with a line list.

```
cl> type linelist
4100 -.1 g 20
4200 -.2 g 20
4300 -.3 g 20
5100 -.9 g 2
5200 -.9 g 4
5300 -.9 g 8
6700 .9 g 8
6800 .9 g 2
6900 .9 g 4
7700 .3 g 20
7800 .2 g 20
7900 .1 g 20
cl> mk1dspec testspec title=Test cont=500 temp=0 lines=linelist
```

7. Add absorption lines to a spectrum.

```
cl> mk1dspec bb out=artspec cont=0 lines=STDIN
4300 -60
5000 -200
[EOF]
```

Normally the input spectrum would be a real spectrum.

8. Make two spectra taken from the same set of random lines but differing in redshift.

```
cl> mk1dspec restspect nl=30
cl> mk1dspec redspec rv=3000 nl=30
cl> mk1dspec bluespec rv=-.01 z+ nl=30
```

9. Make a multispec image with 5 apertures and a

```
cl> mk1dspec spec.ms ap=1 nl=30 rv=0 naps=5
cl> mk1dspec spec.ms ap=2 nl=30 rv=1000
cl> mk1dspec spec.ms ap=3 nl=30 rv=2000
cl> mk1dspec spec.ms ap=4 nl=30 rv=3000
cl> mk1dspec spec.ms ap=5 nl=30 rv=4000
```

REVISIONS

MK1DSPECV2.11+

The random number seed can be set from the clock time by using the value "INDEF" to yield different random numbers for each execution.

MK1DSPECV2.11

Lorentzian and Voigt profiles were added and the parameters and input line list format were changed. The widths are now FWHM instead of gaussian sigmas.

MK1DSPECV2.10.3

The format parameter was eliminated and the task updated to produce the current coordinate system format.

SEE ALSO

mknoise, mk2dspec, mkheader, onedspec.sinterp

NAME

nhedit - edit or view an image header interactively or using a

USAGE

```
nhedit images fields value comment
```

PARAMETERS**images**

Template specifying the images to be edited.

fields

Template specifying the fields to be edited in each image. The template is expanded independently for each image against the set of all fields in the image header. Special values for fields includes 'default_pars' that works only with a command file; 'add_blank' to add a blank field value with a value; 'add_textf' to add a text file content to the header. See description for more details.

value

Either a string constant or a general expression (if the first character is a left parenthesis) to be evaluated to compute the new value of each field. With the rename switch the value is the new field name (keyword). A single expression is used for all fields. The special value "." causes the value of each field to be printed rather than edited.

comment

String constant for the comment section of the header card. This value will replace the existing comment of a header or clear it if is empty (""). The special value "." causes the field to be printed rather than edited.

comfile = ""

Alternate command file. If specified, the *fields*, *value*, and *comment* parameters are ignored and commands are taken from the named file. See below for a detailed discussion and examples.

after = ""

Insert the new field after the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

before = ""

Insert the new field before the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

add= no

Change the operation of the editor from update to add new field. If field already exists it is edited. If this option is selected the field list may name only a single field. The add switch takes precedence over the addonly, delete, and rename switches.

addonly= no

Change the operation of the editor from update to add a field already exists it is not changed. If this option is selected the field list may name only a single field. The addonly switch takes precedence over the delete and rename switches.

delete = no

Change the operation of the editor from update to delete field. The listed fields are deleted from each image. This takes precedence over the rename switch.

rename= no

Change the operation of the editor from update field to rename field. The listed fields are renamed in each image if they exist. The value parameter specifies the new keyword name. There is no error if the field does not exist. The comment value is ignored since this operation only affects the field name.

verify = yes

Interactively verify all operations which modify the image database. The editor will describe the operation to be performed, prompting with the new value of the parameter in the case of a return or "yes" to complete the operation, or enter a new value explicitly as a string. Respond with "no" if you do not wish to change the value of the parameter.

show = yes

Print a record of each operation which modifies the database upon the standard output. Old values are given as well as new values, making it possible to undo an edit operation.

update= yes

Enable updating of the image database. If updating is disabled the edit operations are performed in memory but image headers will not be updated on disk.

DESCRIPTION**1. Basic Usage**

The most basic functions of the image header editor are modification and inspection of the fields of an image header. Both the "standard" and "user" fields may be edited in the same fashion, although not all standard fields are writable. For example, to change the value of the standard field "title" of the image "m74" to "sky flat" and enter a comment field we would enter the following command.

```
cl> nhedit m74 title "sky flat" "comment field"
```

If *verify* mode is selected the editor will print the old value of the field and query with the new value, allowing some other value to be entered instead, e.g.:

```
cl> nhedit m74 title "sky flat" "comment field"
m74,i_title ("old title" -> "sky flat"):
```

To accept the new value shown to the right of the arrow, type carriage return or "yes" or "y" followed by carriage return. To continue without changing the value of the field in question enter "no" or "n" followed by carriage return. To enter some other value merely type in the new value. If the new value is one of the reserved strings, e.g., "yes" or "no", enter it preceded by a backslash. If verification is enabled you will also be asked if you want to update the header, once all header fields have been edited. This is your last chance to change your mind before the header is modified on disk. If you respond negatively the image header will not be updated, and editing will continue with the next image. If the response is "q" the editor will exit entirely.

To conveniently print the value of the field "title" without modifying the image header, we repeat the command with the special value "." and "." for the comment portion.

```
cl> nhedit m74 title . .
```

To print (or edit) the values of all header fields a given.

```
cl> nhedit m74 * . .
```

To print (or edit) the values of only a few fields the field template may be given as a list.

```
cl> nhedit m74 w0,wpc . .
```

To print the value of one or more fields in a may be given. Both image templates and field templates may be given if desired.

```
cl> nhedit n1.* exp . .
```

Abbreviations are not permitted for field names, i.e., the given template must match the full field name. Currently, field name matches are case insensitive since image headers are often converted to and from FITS headers, which are case insensitive.

2. Advanced Usage

The header editor is capable of performing global edits on entire image databases wherein the new value of each field is computed automatically at edit time and may depend on the values of other fields in the image header. Editing may be performed in either batch or interactive mode. An audit trail may be maintained (via the *show* switch and i/o redirection), permitting restoration of the database in the event of an error. Trial runs may be made with updating disabled, before committing to an actual edit which modifies the database.

The major editing functions of the *nhedit* task are the following:

update	modify the value of a field or fields
addonly	add a new field
add	add a new field or modify an old one
delete	delete a set of fields
rename	rename a set of fields

In addition, *nhedit* may be used merely to inspect the values of the header fields, without modification of the image database.

2.1 Special header fields

add_blank	Add blank keyword field with optional comment
ex: nhedit add_blank " this is a comment with no kw"	
add_textf	Add the content of a text file into the header
ex: nhedit add_textf "my_text.txt" add+	

All keyword addition can be inserted after or before an existent keyword; use the 'after' and 'before' parameter.

2.2 Input commands from a command file.

All header editing command can be put together in a

```
nhedit file*.fits comfile=command_file.txt
```

2.3 Standard header fields

The header editor may be used to access both the standard image header fields and any user or application defined fields. The standard header fields currently defined are shown below. There is no guarantee that the names and/or usage of these fields will not change in the future.

i_ctime	int	create time
i_history	string	history comments

i_litime	int	time when min,max last updated
i_maxpixval	real	maximum pixel value
i_minpixval	real	minimum pixel value
i_mtime	int	time of last modify
i_naxis	int	number of axes (dimensionality)
i_naxis[1-7]	int	length of each axis
i_pixfile	string	pathname of pixel storage file
i_pixtype	int	pixel datatype code
i_title	string	title string

The standard header field names have an "i_" prefix to reduce the possibility of a name collision with a user field name, and to distinguish the two classes of parameters in templates. The prefix may be omitted provided the simple name is unique.

2.4 Field name template

The form of the field name list or template parameter *fields* is equivalent to that of a filename template except that "@listfile" is not supported, and of course the template is expanded upon the field name list of an image, rather than upon a directory. Abbreviations are not permitted in field names and case is not significant. Case is ignored in this context due to the present internal storage format for the user parameters (FITS), which also limits the length of a user field name to 8

2.5 Value expression

The *value* parameter is a string type parameter. If the first character in the string is a left parenthesis the string is interpreted as an algebraic expression wherein the operands may be constants, image header variables (field names), special variables (defined below), or calls to intrinsic functions. The expression syntax is equivalent to that used in the CL and SPP languages. If the value string is not parenthesized it assumed to be a string constant. The *value* string will often contain blanks, quotes, parenthesis, etc., and hence must usually be quoted to avoid interpretation by the CL rather than by the header editor.

For example, the command

```
cl> nhedit m74 title "title // ';'ss'" "."
```

would change the title to the literal string constant "title // " whereas the command

```
cl> nhedit m74 title "(title // ';'ss')"
```

would concatenate the string ";ss" to the old title string. We require parenthesis for expression evaluation to avoid the need to doubly quote simple string constant values, which would be even more confusing for the user than using parenthesis. For example, if expressions did not have to be parenthesized, the first example in the basic usage section would have to be entered as shown below.

```
cl> nhedit m74 title '"sky flat"' # invalid command
```

Expression evaluation for *nhedit*, *hselect*, and similar tasks is carried out internally by the FMTIO library routine **evexpr**. For completeness minimal documentation is given here, but the documentation for *evexpr* itself should be consulted if additional detail is required or if problems occur.

2.5.1 operators

The following operators are recognized in value expressions. With the exception of the operators "?", "=?", and "@", the operator set is equivalent to that available in the CL and SPP languages.

+	-	*	/	arithmetic operators
**				exponentiation
//				string concatenation
!	-			boolean not, unary negation
<	<=	>	>=	order comparison (works for strings)
==	!=	&&		equals, not equals, and, or

<code>?=</code>	string equals pattern
<code>? :</code>	conditional expression
<code>@</code>	reference a variable

The operators "=", "&&", and "||" may be abbreviated as "=", "&", and "|" if desired. The `?=` operator performs pattern matching upon strings. For example, the boolean expression shown below will be true whenever the field "title" contains the substring "sky".

```
(title ?= '*sky*')
```

The conditional expression operator '`??`', which is patterned after a operator in C, is used to make IF ELSE like decisions within an expression. The syntax is as follows:

```
<bool_expr> '??' <true_expr> ':' <false_expr>
```

e.g., the expression

```
((a > b) ? 1 : 0)
```

has the value 1 if A is greater than B, and 0 otherwise. The datatypes of the true and false expressions need not be the same, unlike a compiled language. Note that if the parenthesis are omitted ambiguous forms of the expression are possible, e.g.:

```
(a > b) ? 1 : a + 1
```

could be interpreted either as

```
((a > b) ? 1 : a) + 1 or as (a > b) ? 1 : (a + 1)
```

If the parenthesis are omitted the latter interpretation is assumed.

The operator `@` must be used to dereference variables that have names with funny (nonalphanumeric) characters in them, forcing the variable name to be given as a string constant. For example, the value of the expression

```
@ "co-flag"
```

is the value of the variable "co-flag". If the variable were referenced directly by name the "-" would be interpreted as the subtraction operator, causing an unknown variable reference (e.g., to "co"). The operand following the `@` may be any string valued expression. The `@` operator is right associative, hence the construct "`@@param`" is the value of the parameter named by the value of the parameter "param".

An expression may contain operands of datatypes bool, int, real, and string. Mixed mode expressions are permitted with automatic type coercion. Most type coercions from boolean or string to other datatypes are illegal. The boolean constants "yes" and "no" are predefined and may be used within expressions.

2.5.2 intrinsic functions

A number of standard intrinsic functions are recognized within expressions. The set of functions currently supported is shown below.

abs	acos	asin	atan	atan2	bool	cos
exp	int	log	log10	max	min	mod
nint	real	sin	sqrt	str	tan	

The trigonometric functions operate in units of degrees rather than radians. The *min* and *max* functions may have any number of arguments up to a maximum of sixteen or so (configurable). The arguments need not all be of the same datatype.

A function call may take either of the following forms:

```

    <identifier> '(' arglist ')'
or
    <string_expr> '(' arglist ')'

```

The first form is the conventional form found in all programming languages. The second permits the generation of function names by string valued expressions and might be useful on rare occasions.

2.5.3 special operands

As noted earlier, expression operands may be constants, variables (header fields), function calls, or references to any of the special variables. The following special variables are recognized within expressions:

.	A string constant, used to flag printing
\$	The value of the "current field"
\$F	The name of the "current field"
\$I	The name of the "current image"
\$T	The current clock time (an integer value)

These builtin variables are especially useful for constructing context dependent expressions. For example, the value of a field may be incremented by 100 by assigning it the value "\$ + 100".

EXAMPLES

1. Globally edit the database "n1", setting the value of the string parameter "obs" to "sky" if "s-flag" is 1, to "obj" otherwise.

```
cl> nhedit n1.* obs '(@s-flag == 1 ? "sky" : "obj")' "Observation value"
```

2. Globally edit the same database, replacing the value of the parameter "variance" by the square root of the original value.

```
cl> nhedit n1.* var '(sqrt(var))' "Variance value"
```

3. Replace the values of the fields A and B by the absolute value of the original value:

```
cl> nhedit n1.* a,b '(abs($))' 'Absolute value'
```

4. Add a blank field with a comment after a

```
cl> nhedit file.fits add_blank "INSTRUMENT DESCRIPTION " after=k5dx add+
```

Notice the use of the special field value 'add_blank' which will be replaced by a blank keyword in the header.

5. Add HISTORY card before a given keyword

```
cl> nhedit file.fits history \
    "History text from column 9 to 80, no quotes" before=wcsdim add+
```

6. Run a command file through the first 50 extensions

```
cl> for(i=1;i<51;i=i+1) {
    nhedit("mymef["//i//"]",comfile="home$hh.in")
}
```

7. Add a text file to the header. This will be put as HISTORY lines with text appropriately split when long lines are encountered. Start putting the text after the keyword KEYWN.

```
cl> nhedit add_textf "mytext_file.tx" after=KEYWN add+
```

8. Run nhedit through all the extensions in a

```
cl> for(i=1;i<7;i=i+1)
    nhedit("mymef.fits["//i//"]",comfi="home$myheader.txt")
```

9. Run several fits files with the same set of header commands from the file "hdrc.txt".

```
cl> nhedit file*.fits commfile=hdrc.txt
```

As an example the 'hdrc.txt' content can be: (Notice the 'default_pars' command)

```
#
# Sample command file for nhedit task.
#
# Establish the default parameters for the rest of the commands.

default_pars upda+ add+ show- veri-

# Notice the use of commas if you desire.
"DETECTOR" 'Newfirm', "comment string"
ONELE 'A' "comment to A"
#
# Now delete a keyword
ONELE1 del+ show+
add_blank " /blank keyw"

# add a boolean value T
ONELE1 '(1==1)', "comment to A"

"DETSIZE", '[1:2048,1:2048]'
"ENVTEM", 1.56000000000000E+01

# Add a field with string value 'T'
ONELEi2 'T'

bafkeyw1 123.456 "comment to key1" before="WCSDIM" addonly+ show-
add_blank "COMMENT FOR A BLANK" after="FR-SCALE" add+ show-
history "this is a hist to append" add+ show-
history "this is a hist 22 after trim pkey" after="TRIM" add+ show-
comment "this is a comment" after="FR-SCALE" add+ show-
# END OF HDRC.TXT FILE
```

SEE ALSO

hselect, hedit, mkheader, imgets, imheader

NAME

observatory -- Examine and set observatory parameters

USAGE

observatory command obsid [images]

PARAMETERS**command = "list" (set|list|images)**

Command option which is one of "set", "list", or "images". The set command sets the default observatory task parameters for the specified observatory. The list command lists the observatory parameters for the specified observatory but does not modify the task parameters. The images command lists the observatory parameters for a list of images. The list and images commands examine and verify the observatory parameters applied by other tasks using the observatory database facility.

obsid = "?"

Observatory identification to be set, listed, or used as the default for images without the OBSERVAT keyword. The observatory ID is one of those in the database (case ignored), the special string "observatory" to default to the environment variable "observatory" or the *observatory.observatory* parameter, "obspars" to select the parameters in the **observatory** task, or "?" to list the observatories defined in the database.

images

List of images to be examined with the "images" command. The images are checked for the OBSERVAT keyword to determine the observatory parameters to be listed, otherwise the observatory given by *obsid* is used.

verbose = no

Verbose output? Because there are a number of different ways in which observatory information is determine this option prints detailed information on how the observatory database and parameters are ultimately selected.

observatory

The default observatory used by tasks which use the special observatory identification "observatory". The value is one of the observatory names in the observatory database (case ignored) or the special value "obspars" to select the parameters defined in this task. There is no default to force users to set it at least once.

name Observatory name.

longitude

Observatory longitude given in degrees west.

latitude

Observatory latitude in degrees. Positive latitudes are north and negative latitudes are south.

altitude

Observatory altitude in meters above sea level.

timezone

Observatory time zone. The time zone is the number of hours west of Greenwich or the number of hours to be added to local time to obtain Greenwich time.

ENVIRONMENT VARIABLES**obsdb**

This variable selects the observatory database. If not defined it to noao\$lib/obsdb.dat.

observatory

This variable selects the observatory entry whenever a observatory name "observatory". If not defined the value of the task parameter *observatory.observatory* is used.

IMAGE HEADER KEYWORDS

The observatory identification for images is first sought under the image header keyword OBSERVAT. This always takes precedence over any other means of defining the observatory.

DESCRIPTION**OBSERVATORY PARAMETERS IN THE NOAO PACKAGE**

Some astronomical data reduction and analysis tasks perform computations requiring information about where the data was observed. For example a number of **noao** tasks make corrections for the airmass. Generally they look for an airmass in the image header and if it is not present they attempt to compute it from other image header parameters. The information about time and telescope coordinates of the observation are often in the image header but the observatory latitude is not. The task must get this information somehow.

Prior to IRAF V2.10 tasks generally had explicit parameters, such as latitude, with default values pointing (using parameter redirection) to the parameter of the same name in the **observatory** task. The user was required to know the values of the observatory parameters and manually change them for data from different observatories. In V2.10 an observatory database has been implemented. Observatory parameters are stored in a simple text file and tasks obtain observatory related parameters by specifying an observatory identification.

In general the information about the observatory should be directly associated with the image data. Unless stated otherwise in the description of a task, tasks which require observatory information will first look for the image header keyword OBSERVAT. The value of this keyword is the observatory identification used to index the observatory database. The task will then look up any observatory parameters it needs in the observatory database. Data from observatories that support this keyword will, therefore, always use the correct observatory parameters without user intervention. All observatories which export FITS image data are urged to adopt the OBSERVAT keyword (a keyword recommended by the FITS standard).

For image data which do not identify the observatory in this way and in tasks which do not operate on images (such as astronomical calculator tools), the observatory must be specified by the user. Most tasks provide an "observatory" parameter which either directly selects the observatory or use special values for defining the observatory with an environment variable or the parameters from the **observatory** task.

An observatory is specified by the identification name used in the observatory database. The names in the database may be listed using the **observatory** task as described below. If the desired observatory is not in the database a user may copy/create their own database and select it with the environment variable "obsdb", modify the standard database if allowed (any changes to the distributed version should be forwarded to iraf\$noao.edu), or use the special observatory name "obspars". The last option directly uses

the parameters in the **observatory** task which can be set to any values using the normal parameter editing mechanism.

The default value for the observatory parameter in a "observatory". This special name directs the task to look first for the environment variable of the same name and then at the *observatory* parameter of the **observatory** task. The environment variable allows users or sites to set the default observatory in their login files and site defaults. Also it is simple to change the default observatory either with a **reset** command or the **observatory** command.

The observatory database is selected by the environment variable "obsdb". The default when the variable is not defined is the **noao** package library database file "noao\$lib/obsdb.dat". The use of an environment variable allows users to permanently change the default database in the OS environment (when IRAF has access to it such as in UNIX systems) or in the startup IRAF environment as set in the "login.cl" or "loginuser.cl" files. One can, of course, change it during a session with the set or reset commands. For sites which want to customize the observatory mechanism the environment variables can also be set and changed in the files "hlib\$zzsetenv.def", "noao\$lib/zzsetenv.def", and the template login file "hlib\$login.cl".

An observatory database file consist of a simple list of keyword=value pairs with arbitrary whitespace allowed. An observatory entry begins with the observatory keyword and extends to the next observatory keyword or the end of the file. The observatory identification should be the same as the string used in the OBSERVAT image header parameter for data from that observatory. The default file noao\$lib/obsdb.dat begins as follows:

```
# Observatory Parameters. Taken from the Almanac.
#
# Observatories wishing to be added or make changes in the default
# distributed database should send information to iraf@noao.edu.

observatory = "kpno"
    name = "Kitt Peak National Observatory"
    longitude = 111:36.0
    latitude = 31:58.8
    altitude = 2120.
    timezone = 7

observatory = "ctio"
    <etc>
```

In summary, access to observatory parameters is now done by referencing the image header keyword OBSERVAT and, if not defined, determine the observatory name from a task parameter. The environment variables "observatory" and "obsdb" can be set by the user to select alternate observatories and observatory database files. For data without an observatory entry the observatory can be set to "obspars" or the user may make their own observatory database.

THE OBSERVATORY TASK

The **observatory** task serves a number of functions. It examine the observatory database, verify the observatory parameters which will be used by other tasks, particularly those operating on images, set the default observatory if not defined by other means, set observatory parameters explicitly, especially when there is no observatory database entry, and as a parameter set for tasks which explicitly reference observatory parameters. The **verbose** parameter also provides a detailed check of the steps used to determine the observatory database, observatory identification, and observatory parameters.

The *command* parameter takes the values "set", "list", or "images". The *obsid* parameter supplies the observatory identification and the *images* parameter is used to specify a command. The parameters are query parameters and so may be either queried or simply typed on the command line.

The "set" command prints the observatory parameters for the specified observatory and sets many of these in the **observatory** task parameters. This command is used to set the default observatory parameters for tasks where images are not used, the images do not contain the observatory id, or direct references to specific parameters with parameter redirection (for example ")observatory.latitude") are used.

The "list" command is similar to the "set" command except the task parameters are not modified. It is used to list observatory parameters. It is also use with the special observatory identifications to list the entries in an observatory database and verify the observatory to be used by tasks which do not operate on images. The special value "?" lists the entries in the database. The special value "observatory" lists the observatory defined by the "observatory" environment variable or that given by the *observatory.observatory* parameter. The special value "obspars" simply lists the observatory task parameters.

The "images" command lists the observatory information applicable to one or more images. In particular, the observatory identification is first sought in OBSERVAT image header keyword and, if not found, the *obsid* parameter is used. Often the default observatory is "observatory" to follow the same search path used by other tasks.

The *verbose* parameter prints additional detailed information. It prints the database used and whether it is selected by default (noao\$lib/obsdb.dat) or by the "obsdb" environment variable. When the observatory is defined as "observatory" it indicates whether the observatory is defined by the environment variable "observatory" or by the observatory task. When listing images it prints the OBSERVAT keyword or the default observatory assigned.

For observatories not in a database the name, latitude, longitude, altitude, and time zone parameters may be set using **eparam**. The observatory id must be set to "obspars" in this case. These parameters will then be referenced by other tasks in which the observatory is specified as "obspars". This allows arbitrary observatory parameters to be set without creating or modifying an observatory database. However, it is advisable to create a local database and also send the observatory information to the IRAF group at NOAO for inclusion in the default database.

EXAMPLES

1. List the observatory entries in the database:

```
cl> observatory list ? v+
Using default observatory database: noao$lib/obsdb.dat
```

```
default: Kitt Peak National Observatory
kпно: Kitt Peak National Observatory
ctio: Cerro Tololo Interamerican Observatory
eso: European Southern Observatory
lick: Lick Observatory
mmt: Whipple Observatory
cfht: Canada-France-Hawaii Telescope
lapalma: Roque de los Mucachos, La Palma
```

2. Set the observatory parameters for Cerro Tololo:

```
cl> observatory set ctio
Observatory parameters for Cerro Tololo...
    observatory = ctio
    timezone = 5
    altitude = 2215.
    latitude = -30:09.9
    longitude = 70:48.9
    name = 'Cerro Tololo Interamerican Observatory'
cl> lpar observatory
```

command = "set"	Command (set list images)
argument = ctio	Observatory or images
(observatory = "ctio")	Observatory identification
(name = "Cerro Tololo...")	Observatory name
(longitude = 70.815)	Observatory longitude (degrees)
(latitude = -30.165)	Observatory latitude (degrees)
(altitude = 2215.)	Observatory altitude (meters)
(timezone = 4)	Observatory time zone
(verbose = no)	Verbose output?
(mode = "q")	

3. Set the observatory parameters to use the environment variable "observatory" and verify it.

```
cl> set observatory=cfht
cl> observatory list observatory
Observatory parameters for Canada-France-Hawaii Telescope
  observatory = cfht
  timezone = 10
  altitude = 4215
  latitude = 19:49.6
  longitude = 155:28.3
  name = 'Canada-France-Hawaii Telescope'
```

4. Change the default observatory database and verify verbosely:

```
cl> set observatory="sco"
cl> set obsdb="/local/iraf/obsdb.dat"
cl> type obsdb$
# Local Observatory Parameters.

observatory = "sco"
  name = "Small College Observatory"
  longitude = 100:20.0
  latitude = 35:58.8
  altitude = 212.
  timezone = 6
cl> observ set observatory v+
Using database defined by 'obsdb' environment variable:
/tmp/test/obsdb.dat
Using obs... defined by 'obs...' environment variable: sco
Using observatory parameters for database entry: sco
Observatory parameters for Small College Observatory
  observatory = sco
  timezone = 6
  altitude = 212.
  latitude = 35:58.8
  longitude = 100:20.0
  name = 'Small College Observatory'
```

5. List the observatory assigned to some images with a determined either by the "observatory" environment variable or that set in the observatory task.

```
cl> observ images observatory dev$pix,demoobj1
Observatory parameters for Small College Observatory
  observatory = sco
  timezone = 6
  altitude = 212.
  latitude = 35:58.8
  longitude = 100:20.0
  name = 'Small College Observatory'
Images: dev$pix (default observatory)
Observatory parameters for Kitt Peak National Observatory
  observatory = kpno
```

```
timezone = 7
altitude = 2120.
latitude = 31:58.8
longitude = 111:36.0
name = 'Kitt Peak National Observatory'
Images: demoobj1 (OBSERVAT keyword)
```

6. Set explicit observatory parameters:

```
cl> epar observatory
<set observatory parameters>
cl> observ list obspars
Observatory parameters for North Pole
  observatory = obspars
  timezone = 0
  altitude = 0.
  latitude = 90.
  longitude = 0.
  name = 'North Pole'
```

7. Use observatory parameters in expressions:

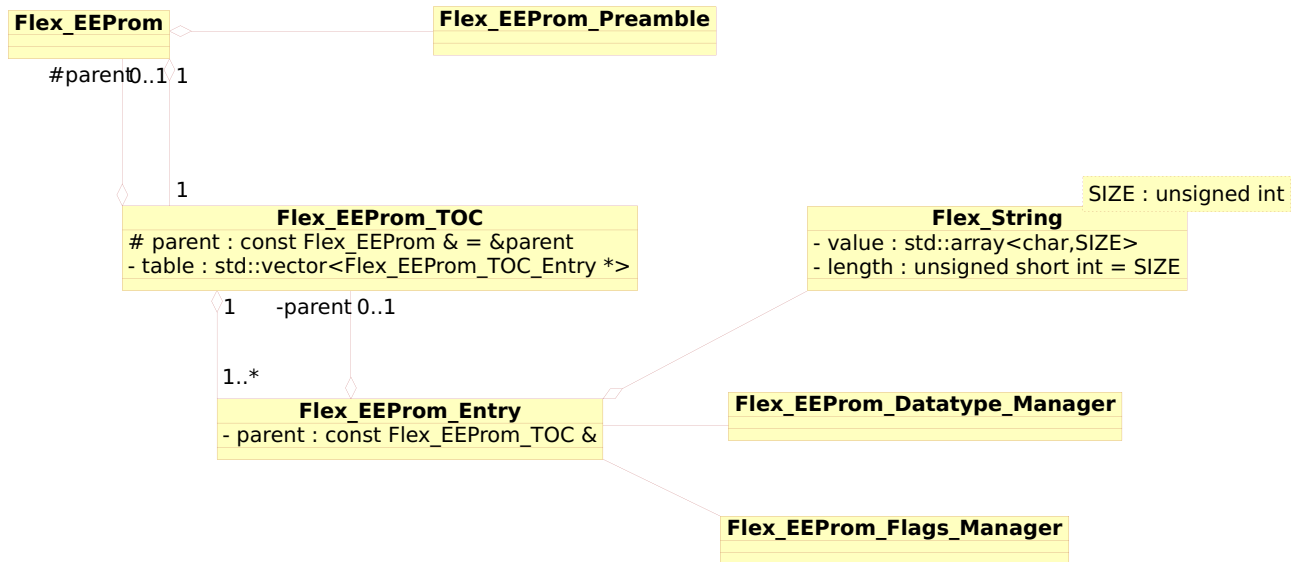
```
cl> observ set kpno
Observatory parameters for Kitt Peak National Observatory
  observatory = kpno
  timezone = 7
  altitude = 2120.
  latitude = 31:58.8
  longitude = 111:36.0
  name = 'Kitt Peak National Observatory'
cl> = observ.lat
31.98
cl> = sin (3.14159/180 * observ.lat)
0.52962280742153
```

SEE ALSO

Tasks in astutil, imred, onedspec, and twodspec.

Flex_CRC

- crc_table : std::vector<uint16_t>
- polynomial : uint16_t

Flex_Messages

NAME

response -- Determine response calibrations

USAGE

response calibration normalization response

PARAMETERS**calibration**

Images to use in determining response calibrations. These are generally quartz continuum spectra. An image section may be used to select only a portion of the image.

normalization

Images to use determining the normalization spectrum. In almost all cases the normalization images are the same as the calibration images or a subsection of the calibration images.

responses

Response calibration images to be created. Each response image is with a calibration image. If the image exists then it will be modified otherwise it is created.

interactive= yes

Graph the average calibration spectrum and fit the normalization spectrum interactively?

threshold= INDEF

Set the response to 1 when the normalization spectrum or input image data fall below this value. If INDEF then no threshold is applied.

sample = "*"

Sample of points to use in fitting the average calibration spectrum. The sample is selected with a range string.

naverage= 1

Number of sample points to average or median before fitting the function. If the number is positive the average of each set of naverage sample points is formed while if the number is negative then the median of each set of points (in absolute value) is formed. This subsample of points is used in fitting the normalization spectrum.

function = "spline3"

Function to fit to the average image spectrum to form the normalization spectrum. The options are "spline1", "spline3", "legendre", and "chebyshev".

order= 1

Order of the fitting function or the number of spline pieces.

low_reject= 0., high_reject= 0.

Rejection limits below and above the fit in units of the residual sigma.

niterate= 1

Number of rejection iterations.

grow = 0

Reject additional points within this distance of points exceeding the rejection threshold.

CURSOR KEYS

The interactive curve fitting package **icfit** is used to fit a to the average calibration spectrum. Help for this package is found under the name "icfit".

DESCRIPTION

A response calibration, in the form of an image, is created for each input image, normally a quartz spectrum. The response calibration is formed by dividing the calibration image by a normalization spectrum which is the same at all points along the spatial axis. The normalization spectrum is obtained by averaging the normalization image across the dispersion to form a one dimensional spectrum and smoothing the spectrum by fitting a function. The threshold value does not apply to creating or fitting of the normalization spectrum but only the final creation of the response values. When normalizing (that is dividing the data values by the fit to the normalization spectrum) only pixels in which both the fitted normalization value and the data value are above the threshold are computed. If either the normalization value or the data value is below the threshold the output response value is one.

The image header keyword DISPAXIS must be present with a dispersion parallel to the lines (varying with the column coordinate) or 2 for dispersion parallel to the columns (varying with line coordinate). This parameter may be added using **hedit**. Note that if the image has been transposed (**imtranspose**) the dispersion axis should still refer to the original dispersion axis unless the physical world coordinate system is first reset (see **wcsreset**). This is done in order to allow images which have DISPAXIS defined prior to transposing to still work correctly without requiring this keyword to be changed.

If the output image does not exist it is first created with unit response everywhere. Subsequently the response is only modified in those regions occupied by the input calibration image. Thus, image sections may be used to select regions in which the response is desired. This ability is particularly useful when dealing with multiple slits within an image or to exclude regions outside the slit.

Normally the normalization images are the same as the calibration images. In other words the calibration image is normalized by the average spectrum of the calibration image itself. Sometimes, however, the normalization image may be a smaller image section of the calibration image to avoid contaminating the normalization spectrum by effects at the edge of the slit. Again, this may be quite useful in multi-slit images.

The normalization spectrum is smoothed by fitting a using the interactive curve fitting package (**icfit**). The parameters determining the fitted normalization spectrum are the sample points, the averaging bin size, the fitting function, the order of the function, the rejection sigmas, the number of rejection iterations, and the rejection width. The sample points for the average spectrum are selected by a range string. Points in the normalization spectrum not in the sample are not used in determining the fitted function. The selected sample points may be binned into a set of averages or medians which are used in the function fit instead of the sample points with the averaging bin size parameter *naverage*. This parameter selects the number of sample points to be averaged if its value is positive or the number of points to be medianed if its value is negative (naturally, the absolute value is used for the number of points). A value of one uses all sample points without binning. The fitted function may be used to reject points from the fit using the parameters *low_reject*, *high_reject*, *niterate* and *grow*. If one or both of the rejection limits are greater than zero then the sigma of the residuals is computed and points with residuals less than *-low_reject* times the sigma and greater than *high_reject* times the sigma are removed and the function fitted again. In addition points within a distance given by the parameter *grow* of the a rejected point are also rejected. A value of zero for this parameter rejects only the points exceeding the rejection threshold. Finally, the rejection procedure may be iterated the number of times given by the parameter *niterate*.

The fitted function may be examined and modified interactively when the parameter *interactive* is set. In this case the normalization spectrum and the fitted function or the residuals of the fit are graphed. Deleted points are marked with an x and rejected points by a diamond. The sample regions are indicated along the bottom of the graph. The cursor keys and colon commands are used to change the values of the fitting

parameters, delete points, and window and expand the graph. When the fitted function is satisfactory exit with a carriage return or 'q' and the calibration image will be created. Changes in the fitted parameters are remembered from image to image within the task but not outside the task.

When the task finishes creating a response image the fitting parameters are updated in the parameter file.

EXAMPLES

1. To create a response image non-interactively:

```
cl> response quartz quartz response order=20 interactive=no
```

2. To determine independent responses for a image sections defining each slit. Then the responses are computed as follows:

```
cl> response quartz[10:20,*],quartz[35:45,*] \  
>>> quartz[12:18,*],quartz[12:18,*] resp,resp
```

Generally the slit image sections are prepared in a used to define the lists of input images and response.

```
cl> response @slits @slits @responses
```

3. If the DISPAXIS keyword is missing and the dispersion is running vertically (varying with the image lines):

```
cl> hedit *.imh dispaxis 2 add+
```

SEE ALSO

icfit, iillumination

NAME

sarith -- Spectrum arithmetic

USAGE

sarith input1 op input2 output

PARAMETERS**input1**

List of input images to be used as operands.

op Operator to be applied to the first operand or to both operands. The unary or single operand operators are:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu
- inv - inverse
- ln - natural logarithm
- log - base 10 logarithm
- lum - convert magnitude to luminosity
- mag - convert luminosity to magnitude
- sqrt - square root

The binary or two operand operators are:

- replace - replace first operand values by second operand values
- + - addition
- - subtraction
- * - multiplication
- / - division
- ^ - exponentiation

input2

Lists of input spectra or constants to be used as second operands for binary operations. If a single value is specified it applies to all the first operand input images otherwise the list must match the first operand list in number.

output

List of resultant output images or root names. Image sections are ignored and if the output format is "onedspec" then any record extensions are stripped to form the root name. If no output list is specified then the input list is used and the input images are replaced by the resultant spectra. If a single output name is specified then all resultant spectra are written to the same output image or image root name. This allows packing or merging multiple spectra and requires properly setting the *clobber*, *merge*, *renumber* and *offset* parameters to achieve the desired output. If more than one output image is specified then it must match the input image list in number.

w1 = INDEF, w2 = INDEF

Starting and ending wavelengths to be copied. If w1 is not specified then the wavelength of the starting edge of the first pixel is used (wavelength at pixel coordinate 0.5) and if w2 is not specified

then the wavelength of the ending edge of the last pixel is used (wavelength of the last pixel plus 0.5). If both are not specified, that is set to INDEF, then the whole spectrum is copied and the *rebin* parameter is ignored. Note that by specifying both endpoints the copied region can be set to have increasing or decreasing wavelength per pixel. If the spectrum only partially covers the specified range only that portion of the spectrum within the range is copied. It is an error if the range is entirely outside that of a spectrum.

apertures = "", beams = ""

List of apertures and beams to be selected from the input spectra. The logical intersection of the two lists is selected. The null list selects all apertures or beams. A list consists of comma separated numbers and ranges of numbers. A range is specified by a hyphen. An optional step size may be given by 'x' followed by a number. See **xtools.ranges** for more information. If the first character is "!" then the apertures/beams not in the list are selected. Note that a "!" in either of the lists complements the intersection of the two lists. For longslit input spectra the aperture numbers selects the lines or columns to be extracted. For 3D Fabry-Perot spectra the aperture numbers select the first spatial axis.

bands = ""

List of bands in 3D multispec. For 3D spatial spectra the band parameter applies to the second spatial axis. The null list selects all bands. The syntax is as described above.

apmodulus= 0

Modulus to be applied to the input aperture numbers before matching against the aperture list. If zero then no modulus is used. This is used to select apertures which are related by the same modulus, typically a factor of 10; for example, 10, 1010, and 2010 with a related.

reverse= no

Reverse the order of the operands in a binary operation? Because the first operand is used as the image header template, dispersion coordinate template, and output image in the case of a image and not a constant. To allow certain operations, for example subtracting a spectra from a constant or using the subtractand as the dispersion coordinate template, the reverse option is used to reverse the order of the operands in a binary operation.

ignoreaps= no

Ignore aperture numbers in the second operand? Normally, spectra in binary operations must have matching aperture numbers, otherwise an error is printed. If this parameter is yes then the spectra are matched by line number with the last line being used if the second operand spectrum has fewer lines than the first operand spectrum. This is generally used to allow using a single spectrum with multiple aperture spectra.

format="multispec" (multispec|onedspec)

Output image format and name syntax. The "multispec" format consists of one or more spectra in the same image file. The "onedspec" format consists of a single spectrum per image with names having a root name and a four digit aperture number extension. Note that converting to "onedspec" format from three dimensional images where the third dimension contains associated spectra will not include data from the extra dimension. Image sections may be used in this case.

renumber= no

Renumber the output aperture numbers? If set the output aperture numbers, including any preexisting spectra when merging, are renumbered beginning with 1. The *offset* parameter may be used to change the starting number.

offset = 0

Offset to be added to the input or renumbered aperture number to form the final output aperture number.

clobber = no

Modify an existing output image either by overwriting or merging?

merge = no

Merge apertures into existing spectra? This requires that the *clobber* parameter be set. If not merging then the selected spectra entirely replace those in existing output images. If merging then the input spectra replace those in the output image with the same aperture number and new apertures are added if not present.

rebin = yes

Rebin the spectrum to the exact wavelength range specified by the *w1* and *w2* parameters? If the range is given as INDEF for both endpoints this parameter does not apply. If a range is given and this parameter is not set then the pixels in the specified range (using the nearest pixels to the endpoint wavelengths) are copied without rebinning. In this case the wavelength of the first pixel may not be exactly that specified by *w1* and the dispersion, including non-linear dispersions, is unchanged. If this parameter is set the spectra are interpolated to have the first and last pixels at exactly the specified endpoint wavelengths while preserving the same number of pixels in the interval. Linear and log-linear dispersion types are maintained while non-linear dispersions are linearized.

errval = 0.

Value for resultant pixel if an arithmetic error occurs such as dividing by zero or the square root of a negative number.

verbose = no

Print a record of each operation?

DESCRIPTION

Sarith performs arithmetic operations on spectra. It is distinguished from **imarith** in that it includes unary operators, like **imfunction** but with some specific to astronomical spectra, and binary operations between two spectra are performed in dispersion coordinate space (typically wavelength) rather than logical pixel space. In the latter case the spectra are checked for matching dispersion functions (which are not necessarily linear) and, if they don't match, the second operand is interpolated without flux conservation. (If flux conservation is desired then the task **dispcor** should be used first.) Thus, the spectra may have different dispersion functions but the arithmetic is done at matching wavelengths. The default interpolation function is a 5th order polynomial. The choice of interpolation type is made with the package parameter "interp". It may be set to "nearest", "linear", "spline3", "poly5", or "sinc". Remember that this applies to all tasks which might need to interpolate spectra in the **onedspec** and associated packages. For a discussion of interpolation types see **onedspec**.

The unary operators operate on the spectra in the first operand list to produce the specified output spectra, which may be the same as the input spectra. The operators include:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu

inv - inverse
 ln - natural logarithm
 log - base 10 logarithm
 lum - convert magnitude to luminosity
 mag - convert luminosity to magnitude
 sqrt - square root

The luminosity to magnitude and magnitude to luminosity operators are based on the standard relation:

$$\text{mag} = -2.5 * \log(\text{lum})$$

where the log is base 10. The F-nu to F-lambda and F-lambda to F-nu operators are based on the relation:

$$\text{F-nu} = \text{F-lambda} * \text{lambda} / \text{nu}$$

where lambda is wavelength and nu is frequency (currently the wavelength is assumed to be Angstroms and so F-lambda is in units of per Angstrom and F-nu is in units of per Hertz). In all the operators it responsibility of user as to the appropriateness of the operator to the input.

The binary operators operate on the spectra in the first operand list and the spectra or numerical constants in the second operand. Numeric constants are equivalent to spectra having the specified value at all pixels. The binary operators are the standard arithmetic ones plus exponentiation and replacement:

replace - replace first operand values by second operand values
 + - addition
 - - subtraction
 * - multiplication
 / - division
 ^ - exponentiation

If the second operand is a spectrum, as mentioned previously, it is interpolated, without flux conservation, to the dispersion function of the first operand spectrum if necessary.

There is a distinction between the first operand and the second operand. The first operand must always be a spectrum. It supplies the dispersion function to be matched by the second operand spectrum. It also supplies a copy of its image header when a new output spectrum is created. In cases where it is desired to have the second operand be the dispersion/header reference and/or the first operand be a the *reverse* parameter is used. For example to subtract a spectrum from the constant 1:

```
cl> sarith 1 - spec invspec reverse+
```

or to subtract two spectra using the subtractand as the dispersion reference:

```
cl> sarith spec1 - spec2 diff reverse+
```

When a binary operation on a pair of spectra is performed the aperture numbers may be required to be the same if *ignoreaps* is no. For images containing multiple spectra the apertures need not be in the same order but only that matching apertures exist. If this parameter is set to yes then aperture numbers are ignored when the operation is performed. For multiple spectra images the second operand spectra are matched by image line number rather than by aperture. If the second operand image has fewer lines, often just one line, then the last line is used repeatedly. This feature allows multiple spectra in the primary operand list to be operated upon by a single spectrum; for example to subtract one spectrum from all spectra in the in a multiple spectrum image.

If it is an error to perform an operation on certain data values, for example division by zero or the square root of a negative number, then the output value is given the value specified by the parameter *errval*.

A log of the operations performed may be printed to the standard output, which may then be redirected if desired, if the *verbose* parameter is set. In the output the last bracketed number is the aperture number of the spectrum.

INPUT/OUTPUT

The arithmetic part of **sarith** is fairly straightforward and intuitive. The selection of input spectra from input images and the placing of output spectra in output images can be more confusing because there are many possibilities. This section concentrates on the topics of the input and output. Since the concepts apply to all of the operators it simplifies things to think in terms of copying input spectra to output spectra; the "copy" operator. Note that the task **scopy** is actually just this case of **sarith** with parameters set for copying. While the discussion here is similar to that in the help for **scopy**, the examples for that task are more focused for illustrating this topic than the **sarith** examples which concentrate more on the arithmetic aspects of the task.

Input spectra are specified by an image list which may include explicit image names, wildcard templates and @files containing image names. The image names may also include image sections such as to select portions of the wavelength coverage. The input images may be either one or two dimensional spectra. One dimensional spectra may be stored in individual one dimensional images or as lines in two (or three) dimensional images. The one dimensional spectra are identified by an aperture number, which must be unique within an image, and a beam number. Two dimensional long slit and three dimensional Fabry-Perot spectra are treated, for the purpose of this task, as a collection of spectra with dispersion either along any axis specified by the DISPAXIS image header parameter or the *dispaxis* package parameter. The aperture and band parameters specify a spatial position. A number of adjacent lines, columns, and bands, specified by the *nsum* package parameter, will be summed to form an aperture spectrum. If number is odd then the aperture/band number refers to the middle and if it is even it refers to the lower of the two middle lines or columns.

In the case of many spectra each stored in separate one dimensional images, the image names may be such that they have a common root name and a four digit aperture number extension. This name syntax is called "onedspec" format. Including such spectra in an input list may be accomplished either with wildcard templates such as

```
name*
name.????.imh
```

where the image type extension ".imh" must be given to complete the template but the actual extension could also be that for an STF type image, or using an @file prepared with the task **names**. To generate this syntax for output images the *format* parameter is set to "onedspec" (this will be discussed further later).

From the input images one may select a range of wavelengths with the *w1* and *w2* parameters and a subset of spectra based on aperture and beam numbers using the *aperture* and *beam* parameters. If the wavelength range is specified as INDEF the full spectra are used without any resampling. If the aperture and beam lists are not specified, an empty list, then all apertures and beams are selected. The lists may be those spectra desired or the complement obtained by prefixing the list with '!'. Only the selected wavelength range and spectra will be operated upon and passed on to the output images.

Specifying a wavelength range is fairly obvious except for the question of pixel sampling. Either the pixels in the specified range are used without resampling or the pixels are resampled to correspond exactly to the requested range. The choice is made with the *rebin* parameter. In the first case the nearest pixels to the specified wavelength endpoints are determined and those pixels and all those in between are used. The dispersion relation is unchanged. In the second case the spectra are reinterpolated to have the specified starting and ending wavelengths with the same number of pixels between those points as in the original spectrum. The reinterpolation is done in either linear or log-linear dispersion. The non-linear dispersion functions are interpolated to a linear dispersion.

Using **sarith** with long slit and Fabry-Perot images provides a and simple type of extraction as opposed to using the **apextract** package. When summing it is often desired to start each aperture after the number of lines summed. To do this specify a step size in the aperture/band list. For example to extract columns 3 to

23 summing every 5 columns you would use an aperture list of "3-23x5" and an *nsum* of 5. If not use the step in the aperture list you would extract the sum of columns 1 to 5, then columns 2 to 6, and so on.

In the special case of subapertures extracted by **apextract**, related apertures are numbered using a modulus; for example apertures 5, 1005, 2005. To allow selecting all related apertures using a aperture number the *apmodulus* parameter is used to specify the modulus factor; 1000 in the above example. This is a feature which should be ignored by most users.

The output list of images may consist of an empty list, a or a list of images matching the input list in number. Note that it is the number of image names that matters and not the number of spectra since there may be any number of spectra in an image. The empty list converts to the same list as the input and is shorthand for replacing the input image with the output image upon completion; therefore it is equivalent to the case of a matching list. If the input consists of just one image then the distinction between a output and a matching list is moot. The interesting distinction is when there is an input list of two or more images. The two cases are then a mapping of many-to-many or many-to-one. Note that it possible to have more complex mappings by repeating the same output name in a matching list provided clobbering, merging, and possibly renumbering is enabled.

In the case of a matching list, spectra from different input images will go to different output images. In the case of a single output image all spectra will go to the same output image. Note that in this discussion an output image when "onedspec" format is specified is actually a root name for possibly many images. However, it should be thought of as a single image from the point of view of image lists.

When mapping many spectra to a single output image, which may have existing spectra if merging, there may be a conflict with repeated aperture numbers. One option is to consecutively renumber the aperture numbers, including any previous spectra in the output image when merging and then continuing with the input spectra in the order in which they are selected. This is specified with the *renumber* parameter which renumbers beginning with 1.

Another options which may be used independently of renumbering or in conjunction with it is to add an offset as specified by the *offset* parameter. This is last step in determining the output aperture numbers so that if used with the renumber option the final aperture numbers begin with one plus the offset.

It has been mentioned that it is possible to write and add to existing images. If an output image exists an error will be printed unless the *clobber* parameter is set. If clobbering is allowed then the existing output image will be replaced by the new output. Rather than replacing an output image sometimes one wants to replace certain spectra or add new spectra. This is done by selecting the *merge* option. In this case if the output has a spectrum with the same aperture number as the input spectrum it is replaced by the input spectrum. If the input spectrum aperture number is not in the output then the spectrum is added to the output image. To add spectra with the same aperture number and not replace the one in the output use the *renumber* or *offset* options.

EXAMPLES

In addition to the examples in this section there are many examples in the help for **scopy** which illustrate aspects of selecting input spectra and producing various types of output. Those examples are equivalent to using the "copy" operator. The same examples will also apply with other operators where the input spectra are modified arithmetically before being copied to the output images.

I. SIMPLE EXAMPLES

The simple examples use only a single input image and create a output image.

1. Examples of unary operations:

```
cl> sarith example1 mag "" magexample
cl> sarith magexample lum "" example2
cl> sarith example1 log "" logexample
```

Note that a place holder for the second operand is required on the command line which will be ignored.

2. Examples of binary operations using constants:

```
cl> sarith example1 + 1000 example2
cl> sarith example1 - 1000 example2 reverse+
cl> sarith example1 / 1000 example2
cl> sarith example1 ** 2 example2
```

3. Examples of binary operations between spectra with matching apertures:

```
cl> sarith example1 + example2 example3
cl> sarith example1 - example2 example3
```

4. Example of binary operations between spectra with the second image consisting of a single spectrum:

```
cl> sarith example1 / flatspec flatexample1 ignore+ errval=1
```

II. MORE COMPLEX EXAMPLES

5. Unary and constant operations on a list of images:

```
cl> sarith example* fnu "" %example%fnu%
cl> sarith example* + 1000 %example%fnu%
```

6. Binary operations on a list of images using a with matching apertures:

```
cl> sarith example* - skyspec %example%skysub%*
```

7. Selecting apertures to operate upon:

```
cl> sarith example* - skyspec %example%skysub%* aper=1,5,9
```

8. Extract the sum of each 10 columns in a by the central spectrum:

```
cl> nsum = "10"
cl> sarith longslit copy "" longslit.ms aper=5-500x10
longslit[5] --> longslit.ms[5]
longslit[15] --> longslit.ms[15]
longslit[25] --> longslit.ms[25]
...
cl> sarith longslit.ms / longslit.ms[*,25] norm ignore+
longslit.ms[5] / longslit.ms[*,25][245] --> norm[5]
longslit.ms[15] / longslit.ms[*,25][245] --> norm[15]
longslit.ms[25] / longslit.ms[*,25][245] --> norm[25]
...
```

9. In place operations:

```
cl> sarith example* + 1000 example* clobber+
example1[1] + 1000. --> example1[1]
example1[2] + 1000. --> example1[2]
...
example2[1] + 1000. --> example2[1]
example2[2] + 1000. --> example2[2]
...
cl> sarith example* flam "" example* clobber+
example1[1] -- flam --> example1[1]
example1[2] -- flam --> example1[2]
...
example2[1] -- flam --> example2[1]
example2[2] -- flam --> example2[2]
...
cl> sarith example* - skyspec "" clobber+ ignore+
```



```
example1[1] + skyspec[1] --> example1[1]
example1[2] + skyspec[1] --> example1[2]
...
example2[1] + skyspec[1] --> example2[1]
example2[2] + skyspec[1] --> example2[2]
...
```

10. Merging existing spectra with the results of operations:

```
cl> sarith example* / flat "" clobber+ merge+ renum+ ignor+
```

REVISIONS

SARITHV2.11

Previously both w1 and w2 had to be specified to select a band to be used. Now if only one is specified the second endpoint defaults to the first or last pixel.

The noise band in multispec data is only copied from the primary spectrum and not modified. This is a kludge until the noise is handled properly.

SARITHV2.10.3

Additional support for 3D multispec/equispec or spatial spectra has been added. The "bands" parameter allows selecting specific bands and the onedspec output format creates separate images for each selected aperture and band.

SARITHV2.10

This task is new.

SEE ALSO

scopy, splot, imarith, imfunction

NAME

sensfunc -- Determine sensitivity and extinction functions

USAGE

sensfunc standards sensitivity

PARAMETERS

standards="std"

Input standard star data file created by the task **standard**.

sensitivity = "sens"

Output sensitivity function image name or rootname. Generally each aperture results in an independent sensitivity function with the aperture number appended to the rootname. If the parameter *ignoreaps* is set, however, the aperture numbers are ignored and a function is determined with the output image having the specified name with no extension.

apertures=""

List of apertures to be selected from the input file. All other apertures are ignored. If no list is specified then all apertures are selected. See **ranges** for the syntax.

ignoreaps= no

Ignore aperture numbers and create a single sensitivity function? Normally each aperture produces an independent sensitivity function. If the apertures are ignored then all the observations are combined into a single sensitivity function.

logfile = "logfile"

Output log filename for statistical information about the stars used and the sensitivity function and extinction function. If no filename is given then no file is written.

extinction= <no default>

Input extinction file. Any extinction determination made will be relative to this extinction. If no file is given then no extinction correction is applied and any extinction determination from the standard star data will be an absolute determination of the extinction. The default value is redirected to the package parameter of the same name. The extinction file is generally one of the standard extinctions in the calibration directory "onedstds\$".

If extinction corrected spectra were used as input to **standard** then it is important that the same extinction file be used here. This includes using no extinction file in both tasks.

newextinction ="extinct.dat"

Output revised extinction file. If the extinction is revised and an output filename is given then a revised extinction file is written. It has the same format as the standard extinction files.

observatory =")_observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a when graphing flux calibrated data of spectra which do not include the airmass in the image header. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information.

function = "spline3"

Function used to fit the sensitivity data. The function types are "chebyshev" polynomial, "legendre" polynomial, "spline3" cubic spline, and "spline1" linear spline. The default value may be changed interactively.

order= 6

Order of the sensitivity fitting function. The value corresponds to the number of polynomial terms or the number of spline pieces. The default value may be changed interactively.

interactive= yes

Determine the sensitivity function interactively? If yes the user graphically interacts with the data, modifies data and parameters affecting the sensitivity function, and determines a

graphs = "sr"

Graphs to be displayed per frame. From one to four graphs may be displayed per frame. The graph types are selected by single characters and are:

- a - residual sensitivity vs airmass
- c - composite residual sensitivity and error bars vs wavelength
- e - input extinction and revised extinction vs wavelength
- i - Flux calibrated spectrum vs wavelength
- r - residual sensitivity vs wavelength
- s - sensitivity vs wavelength

All other characters including whitespace and commas are ignored. The order and number of graphs determines the positions of the graphs.

marks = "plus cross box"

Symbols used to mark included, deleted, and added data respectively. The available mark types are point, box, plus, cross, diamond, hline (horizontal line), vline (vertical line), hebar (horizontal error bar), vebar (vertical error bar), and circle.

colors = "2 1 3 4"

Colors to use for "lines", "marks", "deleted" data, and "added" data. The colors associated with the numbers is graphics device dependent. For example in XGTERM they are defined by resources while on other devices that don't support colors only one color will appear.

cursor = ""

Graphics cursor input list. If not specified as a graphics cursor is read.

device = "stdgraph"

Graphics output device.

answer

Query parameter for selecting whether to fit apertures interactively.

CURSOR COMMANDS

- ? Print help
- a Add a point at the cursor position
- c Toggle use of composite points
- d Delete point, star, or wavelength nearest the cursor
- e Toggle residual extinction correction
- f Fit data with a sensitivity function and overplot

```

g      Fit data with a sensitivity function and redraw the graph(s)
i      Print information about point nearest the cursor
m      Move point, star, wavelength nearest the cursor to new sensitivity
o      Reset to original data
q      Quit and write sensitivity function for current aperture
r      Redraw graph(s)
s      Toggle shift of standard stars to eliminate mean deviations
u      Undelete point, star, or wavelength nearest the cursor
w      Change weights of point, star, or wavelength nearest the cursor

:flux [min] [max]  Limits for flux calibrated graphs (INDEF for autoscale)
:function [type]   Function to be fit to sensitivity data:
                   chebyshev - Chebyshev polynomial
                   legendre  - Legendre polynomial
                   spline1   - Linear spline
                   spline3   - Cubic spline
:graphs [types]    Graphs to be displayed (up to four):
                   a - Residual sensitivity vs airmass
                   c - Composite residuals and error bars vs wavelength
                   e - Extinction (and revised extinction) vs wavelength
                   i - Flux calibrated image vs wavelength
                   l - Log of flux calibrated image vs wavelength
                   r - Residual sensitivity vs wavelength
                   s - Sensitivity vs wavelength
:images [images]   Images to flux calibrate and plot (up to four)
:marks marks       Mark types to use for included, delete, and added points:
                   point, box, plus, cross, diamond, hline,
                   vline, hebar, vebar, circle
:order [order]     Order of function
:skys [images]     Sky images for flux calibration (up to four)
:stats [file]      Statistics about stars and sensitivity fit
:vstats [file]     Verbose statistics about sensitivity fit

```

DESCRIPTION

Standard star calibration measurements are used to determine the system sensitivity as a function of wavelength for each independent aperture. If the parameter *ignoreaps* is set then the aperture numbers are ignored and a single sensitivity function is determined from all the observations. Using measurements spanning a range of airmass it is also possible to derive an adjustment to the standard extinction curve or even an absolute determination. Extinction determination requires that the observations span a good range of airmass during photometric conditions. When conditions are poor and standard star observations are obtained during periods of variable transparency, the entire sensitivity curve may vary by a constant factor, assuming that the cause of the variations has no color effect. This is often the case during periods of thin clouds. In this case the mean sensitivity of each observation may be shifted to match the observation of greatest sensitivity. This allows for the possibility of deriving correct absolute fluxes if one observation of a standard was obtained during a clear period.

The input data is a file of calibration information produced by the task **standard**. The data consists of a line containing the spectrum image name, the sky image name if beam switching, the aperture number, the length of the spectrum, the exposure time, airmass, wavelength range, and title. Following the identification line are calibration lines consisting of the central bandpass wavelengths, the tabulated fluxes in the bandpasses, the bandpass widths, and the observed counts in the bandpasses. The spectrum identification and calibration lines repeat for each standard star observation. The parameter *apertures* may be used to select only specific apertures from the input data. This parameter is in the form of a range list (see help for **ranges**) and if no list is given (specified by the null string "") then all apertures are selected.

An input extinction file may also be specified. Any extinction determinations are then residuals to this input extinction table. The format of this table is described in **lcalib**.

The calibration factor at each point is computed as

$$(1) C = 2.5 \log (O / (T B F)) + A E$$

where O is the observed counts in a bandpass of an observation, T is the exposure time of the observation, B is the bandpass width, F is the flux per Angstrom at the bandpass for the standard star, A is the airmass of the observation, and E is the extinction at the bandpass. Thus, C is the ratio of the observed count rate per Angstrom corrected to some extinction curve to the expected flux expressed in magnitudes. The goal of the task is to fit the observations to the relation

$$(2) C = S(W) + AE(W)$$

where W is wavelength, S(W) is the sensitivity function, and E(W) is a residual extinction function relative to the extinction used in (1). In later discussion we will also refer to the residual sensitivity which is defined by

$$(3) R = C - S(W) - AE(W)$$

The sensitivity function S(W) is output as an one dimensional image much like the spectra. The sensitivities are in magnitude units to better judge the variations and because the interpolation is smoother in the logarithmic space ($\text{mags} = 2.5 \log_{10}[\text{sensitivity}]$). There is one sensitivity function for each aperture unless the parameter *ignoreaps* is set. In the first case the image names are formed from the specified rootname with the aperture number as a four digit numerical extension. In the latter case a single sensitivity function is determined from all data, ignoring the aperture numbers, and the specified output image is created without an extension. These images are used by **calibrate** to correct observations to a absolute flux scale. If no sensitivity function image rootname is specified then the sensitivity curves are not output.

If a revised extinction function E(W) has been determined for one or more of the apertures then the functions are averaged over all apertures, added to the original extinction, and written to the specified extinction table. The format of this table is the same as the standard extinction tables and are, thus, interchangeable. If no new extinction filename is specified then no extinction table is recorded.

If a log filename is given then statistical information about the sensitivity function determinations are recorded. This includes the names of the input standard star observations and the tabulated sensitivity, extinction, and error information.

Some points to note are that if no input extinction is given then the E in (1) are zero and the E determined in (2) is the absolute extinction. If the data are not good enough to determine extinction then using one of the standard extinction curves the problem reduces to fitting

$$(4) C = S(W)$$

The sensitivity and extinction functions are determined as fitted curves. The curves are defined by a function type and order. There are four function types and the order specifies either the number of terms in the polynomial or the number of pieces in the spline. The order is automatically reduced to the largest value which produces a nonsingular result. In this case the function will attempt to pass through every calibration point. Lower orders provide for a smoother representation of the function. The latter is generally more appropriate for a detector. The initial function type and order for the sensitivity function is specified by the parameters *function* and *order*.

If the *interactive* flag is no then the default function and order is fit to equation (4) (i.e. there is no residual extinction determination or manipulation of the data). The sensitivity functions are output if an image rootname is given and the log information is output if a log filename is given.

When the sensitivity is determined interactively a each aperture. The responses "no" and "yes" select fitting the sensitivity interactively or not for the specified aperture. The responses "NO" and "YES" apply to all apertures and no further queries will be given. When interactive fitting is selected the data are graphed on the specified graphics device and input is through the specified cursor list. The graphics output consists

of from one to four graphs. The user selects how many and which types of graphs to display. The graph types and their single character code used to select them are:

```
a - residual sensitivity vs airmass
c - composite residual sensitivity and error bars vs wavelength
e - input extinction and revised extinction vs wavelength
i - Flux calibrated spectrum vs wavelength
r - residual sensitivity vs wavelength
s - sensitivity vs wavelength
```

The initial graphs are selected with the parameter **graphs** and changed interactively with the colon command `:graphs types`. The ability to view a variety of graphs allows evaluating the effects of the sensitivity curve and extinction in various ways. The flux calibrated spectrum graph uses the current sensitivity function and checks for possible wiggles in the sensitivity curve which affect the shape of the continuum. The choice of graphs also allows the user to trade off plotting speed and resolution against the amount of information available simultaneously. Thus, with some graphics devices or over a slow line one can reduce the number of graphs for greater speed while on very fast devices with large screens one can look at more data. The parameter *marks* and the associated colon command `:marks types` also let the user define the symbols used to mark included, deleted, and added data points.

The list of interactive commands is given in the section on CURSOR COMMANDS. The commands include deleting, undeleting, adding, moving, and identifying individual data points, whole stars, or all points at the same wavelength. Some other commands include `'c'` to create composite points by averaging all points at the same wavelength (this requires exact overlap in the bandpasses) which then replace the individual data points in the fit. This is different than the composite point graph which displays the residual in the mean sensitivity and error in the mean but uses the input data in the fitting. The `'s'` command shifts the data so that the mean sensitivity of each star is the same as the star with the greatest mean sensitivity. This compensates for variable grey extinction due to clouds. Note that delete points are excluded from the shift calculation and a deleted star will not be used as the star of greatest sensitivity. Another useful command is `'o'` to recover the original data. This cancels all changes made due to shifting, extinction corrections, deleting points, creating composite points, etc.

The `'e'` command attempts to compute a correlations between the sensitivity points at different airmass. Note that this is not iterative so that repeating this after having added an extinction correction simply redetermines the correction. At each wavelength or wavelength regions having multiple observations at different airmass a slope with airmass is determined. This slope is the residual extinction at that wavelength. A plot of the residual extinctions at each wavelength is made using the ICFIT procedure. The user may then examine and fit a curve through the residual extinction estimates as a function of wavelength (see **icfit** for a description of the commands). The user must decide how much wavelength dependence is derivable from the data. In many cases only a constant fit to a "gray extinction" or possibly a linear fit is realistic. The fitting is exited by the key `'q'`.

To help evaluate how important the residual extinction determination is a t-statistic significance is computed. This statistic is defined by

$$(5) t = \sqrt{r^2 * (N - 2) /}$$

where the correlation coefficient

$$(6) r = \text{RMS with correction} /$$

is the fractional improvement in the RMS due to the added extinction correction and N is the number of wavelength points. For large N this approaches a gaussian sigma but a more precise significance requires the t-distribution for N-2 degrees of freedom. Basically this asks, was the improvement in the RMS significantly more than would occur with random errors? A value greater than 3 a value less than 1 is not significant. The user may then accept the revised extinction and apply it to the data.

Note that when there are multiple apertures used each aperture has an independent system sensitivity but the residual extinction is the same. Therefore, the residual extinctions from each aperture are averaged at the end. If one determines a new extinction then one may replace the original input extinction by the new extinction and rederive the sensitivity.

EXAMPLES

1. The following command generates sensitivity spectra

```
cl> sensfunc std sens
```

This command uses the data from the **standard** output file "std" to create sensitivity functions with rootname "sens". If not interactive the task will produce the output with some progress messages being printed. If it is interactive the graphics device will be used to display the data and the fit and user can change the function and order of the fit, delete bad points, shift data to correct for clouds or bandpass errors, and possibly determine a revised extinction function. The statistics of the sensitivity determination are written to the logfile ("logfile" by default).

2. The following examples illustrate the colon command syntax. Generally if no argument is given the current value is displayed. For the statistics commands an optional output file may be given to record the information.

```
:flux 1e-12 INDEF      Set lower limit for flux plots
:flux INDEF INDEF      Restore autoscaling in flux plots
:func spline3          Select cubic spline function
:g srae                Graph sensitivity, residuals, airmass,
                        and extinction
:g sii                 Graph sensitivity and two images
:i n1.0004 n1.0008     Set first two images to graph (the defaults
                        are taken from the standard star list)
:skys n1.0005          Subtract this sky image from first image
                        for flux calibrated spectrum
:m plus                Change the mark type for included points and
                        don't change the deleted or added point mark type
:stats                 Print statistics to terminal
:vstats stdstats       Print verbose statistics to file
```

REVISIONS

SENSFUNC V2.10.3+

Deleted points and stars are now ignored from the grey shift calculation.

SENSFUNC V2.10.3

A color parameter was added for graphics terminals supporting color.

SENSFUNC V2.10

The latitude parameter has been replaced by the observatory parameter. The 'i' flux calibrated graph type now shows flux in linear scaling while the new graph type 'l' shows flux in log scaling. A new colon command allows fixing the flux limits for the flux calibrated graphs.

SENSFUNC V2.8

This task has been completely rewritten from that of versions 2.5 and earlier.

1. The input standard data format is different.
2. Extinction corrections beyond a grey term are now supported.
3. Weighting by the counts is not supported.
4. Tabular input is not supported.
5. The data which can be displayed is greatly improved.
6. The fitting options have been greatly enhanced.

7. The fitting function types available have been extended.
8. One or more flux calibrated images may be displayed using the current sensitivity function.
9. Additional flexibility is provided for treating apertures.

BUGS

If the flux points do not span the wavelength range, set by the standard star observations, then the fitting may fail at some maximum order. When it fails there is no message but the highest order which can be successfully fit is used. To work around this one can either add fake points, truncate the wavelength range in the first line of each tabulated object in the file produced by **standard**, or exclude the part of the image data which cannot be uncalibrated (using **scopy** or **dispcor**).

SEE ALSO

standard, lcalib, calibrate, observatory, icfit, ranges, scopy, dispcor

NAME

setairmass -- update image headers with the effective airmass

USAGE

setairmass images

PARAMETERS**images**

The list of images for which to calculate the airmass. The image headers may optionally be updated with the airmass and the mid-UT of the exposure.

observatory = ")_.observatory"

Observatory for which airmasses are to be computed if the observatory is specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a value. The observatory may be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information. If the input consists of images then the observatory is defined by the OBSERVAT keyword if present.

intype = "beginning"

The time stamp of the observation as recorded at the telescope for the time dependent header keywords. The choices are the "beginning", "middle" or "end" of the observation.

outtype = "effective"

The output time stamp desired for the airmass. The choices are the "effective" airmass, or the airmass at the "beginning", "middle" or "end" of the observation.

ra = "ra"

The name of the keyword that contains the right ascension. The right ascension is assumed to be in hours unless ra is one of the standard CRVALn keywords in which case it is assumed to be in degrees.

dec = "dec"

The name of the keyword that contains the declination in degrees.

equinox = "epoch"

The name of the keyword that contains the equinox of the right ascension and declination coordinates in years.

st = "st"

The name of the keyword containing the sidereal time in hours.

ut = "ut"

The name of the keyword containing the ut time. This keyword can either be in date plus time format or in hours. Note that this allows setting both the "date-obs" and "ut". If no time is found then a time of 0hrs is used.

date = "date-obs"

The name of the keyword that contains the UT date of the observation. The format should be 'DD/MM/YY' (old FITS format), YYYY-MM-DD (new FITS format), or

YYYY-MM-DDTHH:MM:SS (new FITS format with time). If there is a time and no time is found in the ut keyword then it is used for the ut.

exposure = "exptime"

The name of the keyword that contains the exposure time (in seconds) of the image.

airmass = "airmass"

The name of the output keyword that will receive the computed airmass.

utmiddle = "utmiddle"

The name of the output keyword that will receive the universal time for the middle of the observation. The format of the keyword will be the same as that specifying the universal time.

scale = 750.0

The atmospheric scale height.

show = yes

Print the airmasses and mid-UT's for each image?

update = yes

Update the image headers with the airmasses and the mid-UT's?

override = yes

If updating the image headers, override values that were previously recorded ?

DESCRIPTION

SETAIRMASS will calculate the effective airmass of an astronomical image, as described below under "ALGORITHMS". The task requires the instantaneous zenith distance at the beginning, middle and end of the exposure. These are calculated using the right ascension, declination, and equinox as well as the sidereal time, exposure time, UT date, and observatory from the image header. If the observatory is not available in the image header under the keyword OBSERVAT, the observatory is defined by the *observatory* parameter. See help for *observatory* for further information.

The right ascension and declination will be precessed from the given equinox to the date of observation. The name of the right ascension, declination, equinox, sidereal time, ut time, exposure time, and date keywords can be specified as parameters. These keywords should express the right ascension in hours, the declination in degrees, the equinox in years, the sidereal time in hours, the universal time in hours, the exposure time in seconds, and the date in FITS format. If any of the required keywords are missing from the image headers, they can be added using the hedit or asthedit tasks. Note that the universal time keyword may be in either a date plus time format or in hours and any output middle universal time will be in the same format.

Before using this task, you will need to know the "time stamp" of the time varying header quantities (e.g. sidereal time). Do the recorded values represent the beginning, the middle or the end of the exposure ? be set in the **intype** parameter.

If for some reason the effective airmass is not desired, the value of the airmass at the beginning, middle or end of the exposure can be recorded in the header keyword specified by the *airmass* parameter. The **show** parameter can be used to control the output to the terminal. The **update** and **override** parameters control the header keyword output.

SETAIRMASS will also calculate the universal time of the middle of the exposure and place the value in

the header keyword specified by the *utmiddle* parameter. This assumes that the value for the UT is in the date keyword or ut keyword, with the same time stamp as the sidereal time. The mid-observation UT is useful for interpolating calibration arc dispersion solutions using REFSPECTRA, especially when the exposure time is long.

ALGORITHMS

The mean airmass is calculated using the formula described in "Some Factors Affecting the Accuracy of Stellar Photometry with CCDs" by P. Stetson, DAO preprint, September 1988. This simple formula is:

$$AM (eff) = [AM (beginning) + 4*AM (middle) + AM (end)] / 6$$

and is derived by using Simpson's 1/3 rule to approximate the integral that represents the mean airmass.

The beginning, middle and end airmasses are calculated using the relation between airmass and elevation (or zenith distance) in John Ball's book on Algorithms for the HP-45:

$$AM = \sqrt{x^2 + 2*scale + 1} - x, \text{ where}$$

$$x = scale * \sin(elevation) = scale * \cos(ZD)$$

The atmospheric scaling parameter is *scale* (see "Astrophysical Quantities" by Allen, 1973 p.125,133).

KEYWORDS

The input keywords are:

OBSERVAT

Observatory at which the data was taken. If absent the observatory is determined using the *observatory* parameter.

ra

Right ascension in hours at the beginning, middle, or end of the observation. If *ra* is one of the CRVALn keywords it is assumed to be in degrees.

dec

Declination in degrees at the beginning, middle, or end of the observation.

equinox

The equinox of the coordinates. The right ascension and declination will be precessed from this epoch to the date of the observation before being used.

st

Sidereal time in hours at the beginning, middle, or end of the observation.

ut

Universal time in hours at the beginning, middle, or end of the observation. This may be in either date plus time format or just in hours.

date

The value of the date parameter is the keyword name to be used for the date of the observation. The date must be in either the old or new FITS format.

exposure

The value of the exposure parameter is the keyword name to be used for the exposure time in seconds.

The output keywords are:

airmass

The value of the airmass parameter is the keyword name to be used for the computed airmass at either the beginning, middle, or end of the exposure, or for the weighted effective value over the exposure.

utmiddle

The value of the utmiddle parameter is the keyword name to be used for the universal time at the middle of the exposure.

EXAMPLES

1. Calculate the effective airmass of the IRAF test picture, dev\$pix.

```
cl> setairmass dev$pix exposure=itime update-
```

Note that the test picture does not have the correct coordinate epoch listed in its header, so no precession will be performed.

2. Calculate the effective airmass of the IRAF test picture dev\$ypix in two ways.

```
cl> setairmass dev$ypix exposure=itime update-
```

```
cl> setairmass dev$ypix ra=crval1 dec=crval2 equinox=equinox \
    exposure=itime update-
```

Note the first way gives the same results as example 1. The second way uses the J2000 equatorial system rather than the ra and dec at the time of observation.

REVISIONS

SETAIRMASSV2.11.4

The ut keyword now has precedence over any time in the date keyword and it can be either date plus time or hours.

SETAIRMASSV2.11.3

The right ascension, declination, equinox, st, and ut keywords were made parameters rather than being hard wired.

SETAIRMASSV2.11.2

Y2K update: This task was updated to use the new FITS date format.

SEE ALSO

airmass, hedit, refspectra, observatory

NAME

specfocus -- Determine spectral focus and alignment variations

USAGE

specfocus images

PARAMETERS**images**

List of 1D or 2D focus images. Typically the input is a 2D CCD images of arc slit spectra. The 1D image input is provided to allow use of extraction techniques beyond those provided by this task.

focus = ""

List of focus identification values to be associated with each input image or an image header keyword containing the values. The list may be an explicit list of values, a range specification, an @ file containing the values, or an image header keyword. If none of these is given the identification values are simple index values in the order of the input images. A range specification has the forms A, Ax C, A-BxC where A is a starting value, B is an ending value, and C is an increment.

corwidth= 20.

Correlation width in pixels.

level = 0.5

Percent or fraction of the correlation peak at which to measure focus widths. The default is 50% or full width at half maximum.

shifts = yes

Compute dispersion shifts across the dispersion when there are multiple samples? If yes and there are multiple samples across the dispersion (*ndisp* > 1), pixel shifts relative to the central sample are determined by crosscorrelation.

dispaxis = 2

Dispersion axis for 2D images. The image header keyword DISPAXIS has precedence over this value.

nspectra= 1, ndisp = 1

The number of spectral samples across the dispersion and the number of subpieces along the dispersion to divide the spectra into. If *nspectra* is greater than one then information about variations across the dispersion will be determined and if *ndisp* is greater than 1 information about variations along the dispersion will be determined. *Nspectra* applies only to 2D images. For 1D spectra in multispec format each line is used as a separate sample.

slit1 = INDEF, slit2 = INDEF

The lower and upper edges of the slit (or data region) in pixel coordinates (lines or columns) across the dispersion axis. A value of INDEF specifies the image edges.

logfile = "logfile"

File in which to record the results. If no file is specified no log output is produced.

CURSOR COMMANDS

All keys select an image and a sample (one of the *ndisp* samples along the dispersion and one of the *nspectra* samples across the dispersion) which is then generally highlighted.

```
? Help summary
b Best focus at each sample summary graphs
d Delete image, sample, or point
p Profiles at one sample for all images and all samples for one image
q Quit
r Redraw
s Spectra at one sample for all images and all samples for one image
u Undelete spectrum, sample, or point
w Profile widths verses focus and distribution of widths
z Zoom on a single sample showing correlation profile and spectrum
<space> Status line output for selected image and sample
```

DESCRIPTION

This task estimates the dispersion width of spectral lines in sequences of arc spectra taken at different focus settings (or with some other parameter varied). The widths can be measured at different spatial and dispersion positions, called "samples", on the detector. The width estimates are recorded and displayed graphically to investigate dependencies and determine appropriate settings for the spectrograph setup. The task may also measure dispersion shifts when multiple spectral samples are specified. This task does not measure the focus point-spread-function width across the dispersion.

The input images are specified with an image template list. The list may consist of explicit image names, wildcard templates, and @ files. A "focus" value is associated with each image. This may be any numeric quantity (integer or floating point). The focus values may be specified in several ways. If no value is given then index numbers are assigned to the images in the order in which they appear in the image list. A range list may be specified as described in the help topic **ranges**. This consists of individual values, ranges of values, a starting value and a step, and a range with a step. The elements of the list are separated by commas, ranges are separated by hyphens, and a character 'x'. Long range lists, such as a values, may be placed in a file and specified with the @<filename> convention. Finally, a parameter in the image header may be used for the focus values by simply specifying the parameter name.

Two dimensional long slit images are summed into one or more one dimensional spectra across the dispersion. The dispersion axis is defined either by the image header parameter DISPAXIS or the *dispaxis* task parameter with the image header parameter having precedence. The range of lines or columns across the dispersion to be used is specified by the parameters *slit1* and *slit2*. If specified as INDEF then the image limits are used. This range is then divided into the number of spectra given by the parameter *nspectra*. Use of more than one spectrum across the dispersion allows investigation of variations along the slit. In addition, if the parameter *shifts* is set the spectrum nearest the center is used as a reference against which shifts in the dispersion positions of the features in the other spectra are determined by crosscorrelation.

The conversion of two dimensional spectra to one dimensional spectra may also be performed separately using the tasks in the **apextract** package. This would be done typically for multifiber or echelle format spectra. If the two dimensional spectra have been extracted to one dimensional spectra in this way the task ignores the dispersion axis and number of spectra parameters. The data limits (*slit1* and *slit2*) are still used to select a range of lines in "multispec" format images. The *shifts* parameter also applies when there are multiple spectra per image. However, it does not make sense in the case of echelle spectra and so it should be set to no in that case.

In addition to dividing the spatial axis into a dispersion axis may also be divided into a set of subspectra. The number of divisions is specified by the *ndisp* parameter which applies to both long slit and 1D extracted spectra. When the dispersion axis is divided into more than one sample, the dependence of the dispersion widths and shifts along the dispersion may be investigated.

Each spectral sample has a low order continuum subtracted using a noninteractive iterative rejection algorithm to exclude the spectral lines. This technique is described further under the topic *continuum*. The continuum subtracted spectrum is then tapered with a cosine bell function and autocorrelated. The length of the taper and the range of shifts for the correlation is set by the *corwidth* parameter. This parameter should be only slightly bigger than the expected feature widths to prevent correlations between different spectral lines. The correlation profile is offset to zero at the edges of the profile and normalized to unity at the profile center. The profiles may be viewed as described below.

If there is more than one spatial sample the central spectrum is also crosscorrelated against the other spectra at the same dispersion sample. The crosscorrelation is computed in exactly the same way as the autocorrelation. The crosscorrelation profiles are only used for determining shifts between the two samples and are not used in the width determinations.

A cubic spline interpolator is fit to the profiles and this interpolation function is used to determine the profile width and center. The width is measured at a point given by the *level* parameter relative to the profile peak. It may be specified as a fraction of the peak if it is less than one or a percentage of the peak if it is greater than one. The default value of 0.5 selects the full width at half maximum. The autocorrelation width is divided by the square root of two to yield an estimate of the width of the spectral features in the spectrum in units of pixels.

Having computed the width and shift for each input image at each sample, the "best focus" values (focus, width, and shift) are estimated for each sample. As discussed later, it is possible to exclude some samples from this calculation by deleting them graphically. First the images with the smallest measured width at each distinct focus are selected since it is possible to input more than one image at same focus. The selected images are sorted by focus value and the image with the smallest width is found. If that image has the lowest or highest focus (which will always be the case if there are only one or two images) then the best focus, width, and shift are those measured for that image. If there are three or more focus values and the minimum width focus image is not an endpoint then parabolic interpolation is used to find the minimum width. The focus at this minimum width is the "best focus". The dispersion shift is the parabolic interpolation of the shifts at the best focus. The "average best focus" values are then the average of the "best focus" values over all samples.

After computing the correlation profiles, the profile widths and shifts, and the best focus values, an interactive graphics mode is entered. This is described in detail below. The graphics mode is exited with the 'q' key. At this point the results are written to the standard output (usually the terminal) and to a logfile if one is specified. The output begins with a banner identifying the task, version of IRAF, the user, and the date and time. The next line gives the best average focus and width. This banner also appears in all plots. Then each image is listed with the focus value and average width (over all samples). Finally the image with the smallest average width is identified and tables showing the width and shifts (if computed) at each sample position are printed. If there is only one sample then the tables are not output.

INTERACTIVE GRAPHICS MODE

There are five types of plot formats which are selected with the 'b', 'p', 's', 'w', and 'z' keys. The available formats and their content are modified depending on the number of images and the number of samples. If there is only one image or one sample per image some of the plot formats are not available. If there are a large number of images or a large number of samples the content of the plot formats may be abbreviated for legibility.

In all plots there is a concept of the current image and the current sample. In general there is an indication, usually a box, of which image and sample is the current one. The current image and sample are changed by pointing at a particular point, box, circle, or symbol for that image and sample and typing a key.

The 'b' key produces summary graphs of the best focus values (as described above) at each sample position. There must be more than one image and more than one sample (either along or across the

dispersion or both). This is the initial plot shown when this condition is satisfied. The central graph, which is always drawn, represents the best focus (smallest) width at each sample by circles of size proportional to the width. The position of the circle indicates the central line and column of the sample. If there are multiple samples across the dispersion and the *shifts* parameter is set then little vectors are also drawn from the center of the circle in the direction of the shift and with length proportional to the shift. If there are 5 or fewer samples in each dimension the values of the best focus and the width and shift (if computed and nonzero) at that focus, are printed on the graph next to the circles. If there are more samples this information may be obtained by pointing at the sample and typing the space key.

In addition to the spatial graph there may be graphs along the line or column axes. These graphs again show the widths as circles but one axis is either the line or column and the other axis is either the best focus value or the shift. The focus graph marks the best average focus (over all samples) by a dashed line and a solid line connects the mean focus at each column or line. The focus graphs will only appear if there is more than one sample along a particular image axis. The shift graphs will only appear if shifts are computed (*shifts* parameter is yes) and there is more than one sample along a particular dimension. Lines are drawn at zero shift and connecting the mean shift at each point along the spatial axis. Note that there is always a point at zero shift which is the reference sample.

The best focus graphs are the exception in showing a sample. When changing to one of the other plots based on a current image and sample the circle from the central spatial graph nearest the cursor is used (note that the other focus and shift graphs are ignored). The sample is defined by its spatial position and the image is the one with focus closest to the best focus value of that sample.

The 'w' key produces a graph showing the sample widths as a focus value. There must be more than one image and more than one sample for this type of graph. The top graph is a focus. The symbols are crosses except for the current image which is shown with pluses. The current sample is highlighted with a long dashed line connecting the widths for the current sample at each focus value and short dashed lines showing the best average focus and width.

The lower portion of the 'w' key are graphs showing the widths as circles with size proportional to the width and position corresponding to the spatial position of the sample in the image. If there are more than 5 samples in either dimension the graph is for the current image. Otherwise there is a box for each image with the focus value (provided there are not too many images) indicated. The circles are arranged as they would be spatially in columns and rows. The samples closest to the best focus are indicated by pluses. This allows seeing where the best focus values cluster. The current image and sample are indicated by highlighting boxes.

The 'p' key produces graphs of the autocorrelation profiles. This also requires more than one image and more than one sample. The top graph shows the profiles of all images at a particular sample and the bottom graph shows the profiles of all samples at a particular image. The bottom sample boxes are arranged in columns and rows in the same way the samples are distributed in the image. The current image and current sample are highlighted by a box.

The profiles are drawn with a solid line using the interpolator function and the actual pixel lags are indicated with pluses. The profiles are drawn shifted by the amount computed from the crosscorrelation. Note that the shift is added to the autocorrelation profile and the crosscorrelation profile is not what is plotted. The zero shift position is indicated by a vertical line. If there are less than 25 boxes the boxes are labeled by the width, shift (if nonzero), and focus.

The 's' key plot is similar to the 'p' key plot but shows the spectra rather than the profiles. The top graphs are the spectra of each image at a particular sample and the bottom graphs are the spectra of each sample for a particular image. The current image and sample are highlighted by a box.

The 'z' key graphs the autocorrelation profile and the spectrum of a single sample. This graph provides scales which are not provided with the 'p' and 's' graphs. If there is only one image and one sample then this is the only plot available.

It is possible to exclude some of the samples from the calculation of the best focus and best average focus values. This is done by deleting them using the 'd' key. When using the 'd' key you must specify the sample to be deleted in one of the graphs. You are then asked if only that sample (point) is to be deleted, if all samples from that image are to be deleted, or if the same sample from all images is to be deleted. The deleted data is no longer shown explicitly but the space occupied by the data is still present so that the data may be included again by typing the 'u' undelete key. When the task is exited with the 'q' key the printed and logged results will have the deleted data excluded.

The remaining cursor keys do the following. The '?' key gives a summary of the cursor keys. The 'r' key redraws the current plot. The space key prints information about the current sample. This is mostly used when there are too many images or samples to annotate the graphs with the focus, width, and shift. Finally the 'q' key quits the task.

EXAMPLES

1. A series of 2D focus images is obtained with focus values starting at 400 in steps of -50. The slit is between columns 50 and 130. There are 3 samples across the dispersion and 3 the dispersion.

```
cl> lpar specfocus
      images = "@imlist"      List of images
      (focus = "400x-50")    Focus values
      (corwidth = 20)         Correlation width
      (level = 0.5)           Percent or fraction of peak
      (shifts = yes)          Compute shifts across the disp?\n
      (dispaxis = 2)          Dispersion axis (long slit only)
      (nspectra = 3)          Number of spec samples (ls only)
      (ndisp = 3)             Number of dispersion samples
      (slit1 = 50)            Lower slit edge
      (slit2 = 130)          Upper slit edge\n
      (logfile = "logfile")   Logfile
      (mode = "ql")
cl> specfocus @imlist
<Interactive graphics which is exited with the 'q' key>
SPECFOCUS: NOAO/IRAF V2.10EXPORT valdes Thu 19:41:41 17-Sep-92
Best avg focus at 206.6584 with avg width of 2.91 at 50% of peak
```

-- Average Over All Samples

Image	Focus	Width
jdv011.imh	100.	3.78
jdv010.imh	150.	3.28
jdv009.imh	200.	2.95
jdv008.imh	250.	3.17
jdv007.imh	300.	3.41
jdv006.imh	350.	3.74
jdv005.imh	400.	4.16

-- Image jdv009.imh at Focus 200. --

Width at 50% of Peak:

	Columns		
	50-76	77-103	104-130
Lines	+	-----	-----
2-267	2.93	2.58	2.74
268-533	3.17	2.76	2.89
534-799	3.77	2.23	3.50

Position Shifts Relative To Central Sample:

Lines	Columns		
	50-76	77-103	104-130
2-267	0.68	0.00	0.18
268-533	0.64	0.00	0.13
534-799	0.92	0.00	0.16

SEE ALSO

imexamine, implot, ranges, splot

The IRAF/NOAO Spectral World Coordinate Systems

1. Types of Spectral Data

Spectra are stored as one, two, or three dimensional images with one axis being the dispersion axis. A pixel value is the flux over some interval of wavelength and position. The simplest example of a spectrum is a one dimensional image which has pixel values as a function of wavelength.

There are two types of higher dimensional spectral image formats. One type has spatial axes for the other dimensions and the dispersion axis may be along any of the image axes. Typically this type of format is used for long slit (two dimensional) and Fabry-Perot (three dimensional) spectra. This type of spectra is referred to as *spatial* spectra and the world coordinate system (WCS) format is called *ndspec*. The details of the world coordinate systems are discussed later.

The second type of higher dimensional spectral image consists of multiple, independent, one dimensional spectra stored in the higher dimensions with the first image axis being the dispersion axis; i.e. each line is a spectrum. This format allows associating many spectra and related parameters in a single data object. This type of spectra is referred to as *multispec* and there are two coordinate system formats, *equispec* and *multispec*. The *equispec* format applies to the common case where all spectra have the same linear dispersion relation. The *multispec* format applies to the general case of spectra with differing dispersion relations or non-linear dispersion functions. These multi-spectrum formats are important since maintaining large numbers of spectra as individual one dimensional images is very unwieldy for the user and inefficient for the software.

Examples of multispec spectral images are spectra extracted from a multi-fiber or multi-aperture spectrograph or orders from an echelle spectrum. The second axis is some arbitrary indexing of the spectra, called *apertures*, and the third dimension is used for associated quantities. The IRAF **apextract** package may produce multiple spectra from a CCD image in successive image lines with an optimally weighted spectrum, a simple aperture sum spectrum, a background spectrum, and sigma spectrum as the associated quantities along the third dimension of the image.

Many **onedspec** package tasks which are designed to operate on individual one dimensional spectra may operate on spatial spectra by summing a number of neighboring spectra across the dispersion axis. This eliminates the need to "extract" one dimensional spectra from the natural format of this type of data in order to use tasks oriented towards the display and analysis of one dimensional spectra. The dispersion axis is either given in the image header by the keyword DISPAXIS or the package *dispaxis* parameter. The summing factors across the dispersion are specified by the *nsum* package parameter. See "help onedspec .package" for information on these parameters.

One dimensional spectra, whether from multispec spatial spectra, have several associated quantities which may appear in the image header as part of the coordinate system description. The primary identification of a spectrum is an integer aperture number. This number must be unique within a single image. There is also an integer beam number used for various purposes such as discriminating object, sky, and arc spectra in multi-fiber/multi-aperture data or to identifying the order number in echelle data. For spectra summed from spatial spectra the aperture number is the central line, column, or band. In 3D images the aperture index wraps around the lowest non-dispersion axis. Since most one dimensional spectra are derived from an integration over one or more spatial axes, two additional aperture parameters record the aperture limits. These limits refer to the original pixel limits along the spatial axis. This information is primarily for record keeping but in some cases it is used for spatial interpolation during dispersion calibration. These values are set either by the **apextract** tasks or when summing neighboring vectors in spatial spectra.

An important task to be aware of for manipulating spectra between image formats is **scopy**. This task allows selecting spectra from multispec images and grouping them in various ways and also "extracts" apertures from long slit and 3D spectra simply and without resort to the more general **apextract** package.

2. World Coordinate Systems

IRAF images have three types of coordinate systems. The pixel array coordinates of an image or image section, i.e. the lines and columns, are called the *logical* coordinates. The logical coordinates of individual pixels change as sections of the image are used or extracted. Pixel coordinates are tied to the data, i.e. are fixed to features in the image, are called *physical* coordinates. Initially the logical and physical coordinates are the equivalent but differ when image sections or other tasks which modify the sampling of the pixels are applied.

The last type of coordinate system is called the *world* coordinate system. Like the physical coordinates, the world coordinates are tied to the features in the image and remain unchanged when sections of the image are used or extracted. If a world coordinate system is not defined for an image, the physical coordinate system is considered to be the world coordinate system. In spectral images the world coordinate system includes dispersion coordinates such as wavelengths. In many tasks outside the spectroscopy packages, for example the **plot**, **tv** and **images** packages, one may select the type of coordinate system to be used. To make plots and get coordinates in dispersion units for spectra with these tasks one selects the "world" system. The spectral tasks always use world coordinates.

The coordinate systems are defined in the image headers using a reserved keywords which are set, changed, and updated by various tasks. Some of the keywords consist of simple single values following the FITS convention. Others, the WAT keywords, encode long strings of information, one for each coordinate axis and one applying to all axes, into a set of sequential keywords. The values of these keywords must then be pasted together to recover the string. The long strings contain multiple pieces called WCS *attributes*. In general the WCS keywords should be left to IRAF tasks to modify. However, if one wants modify them directly some tasks which may be used are **hedit**, **hfix**, **wcsedit**, **wcsreset**, **specshift**, **dopcor**, and **sapertures**. The first two are useful for the simple keywords, the two "wcs" tasks are useful for the linear ndspec and equispec formats, the next two are for the common cases of shifting the coordinate zero point or applying a doppler correction, and the last one is the one to use for the more complex multispec format attributes.

3. Physical Coordinate System

The physical coordinate system is used by the spectral tasks when there is no dispersion coordinate information (such as before dispersion calibration), to map the physical dispersion axis to the logical dispersion axis, and in the multispec world coordinate system dispersion functions which are defined in terms of physical coordinates.

The transformation between logical and physical coordinates is defined by the header keywords LTVi, LTMi_j (where i and j are axis numbers) through the vector equation

$$l = |m| * p + v$$

where l is a logical coordinate vector, p is a coordinate vector, v is the origin translation vector specified by the LTV keywords and |m| is the scale/rotation matrix specified by the LTM keywords. For spectra rotation terms (nondiagonal matrix elements) generally do not make sense (in fact many tasks will not work if there is a rotation) so the transformations along each axis are given by the linear equation

where l is a logical coordinate vector, p is a v is the origin translation vector specified by the LTV keywords and |m| is the scale/rotation matrix specified by the LTM keywords. For spectra a rotation term (nondiagonal matrix elements) generally does not make sense (in fact many tasks will not work if there is a transformations along each axis are given by the linear equation

$$l_i = LTM_{i_i} * p_i + LTV_i.$$

If all the LTM/LTV keywords are missing they are assumed to have zero values except that the diagonal matrix terms, LTMi_i, are assumed to be 1. Note that if some of the keywords are present then a take the value zero which generally causes an arithmetic or matrix inversion error in the IRAF tasks.

The dimensional mapping between logical and physical axes is given by the keywords WCSDIM and WAXMAP01. The WCSDIM keyword gives the dimensionality of the physical and world coordinate system. There must be coordinate information for that many axes in the header (though some may be missing and take their default values). If the WCSDIM keyword is missing it assumed to be the same as the logical image dimensionality.

The syntax of the WAXMAP keyword are pairs of integer values, one for each physical axis. The first number of each pair indicates which current *logical* axis corresponds to the original *physical* axis (in order) or zero if that axis is missing. When the first number is zero the second number gives the offset to the element of the original axis which is missing. As an example consider a which the second plane is extracted (an IRAF image section of [*,2,*]). The keyword would then appear as WAXMAP01 = '1 0 0' is missing the mapping is 1:1; i.e. the dimensionality and order of the axes are the same.

The dimensional mapping is important because the dispersion axis for the nspec spatial spectra as specified by the DISPAXIS keyword or task parameter, or the axis definitions for the equispec and or multispec formats are always in terms of the original physical axes.

4. Linear Spectral World Coordinate Systems

When there is a linear or logarithmic relation between pixels and dispersion coordinates which is the same for all spectra the WCS header format is simple and uses the FITS convention (with the CD matrix keywords proposed by Hanisch and Wells 1992) for the logical pixel to world coordinate transformation. This format applies to one, two, and three dimensional data. The higher dimensional data may have either linear spatial axes or the equispec format where each one dimensional spectrum stored along the lines of the image has the same dispersion.

The FITS image header keywords describing the spectral world coordinates are CTYPEi, CRPIXi, CRVALi, and CDi_j where i and j are axis numbers. As with the physical coordinate transformation the nondiagonal or rotation terms are not expected in the spectral WCS and may cause problems if are not zero. The CTYPEi keywords will have the value LINEAR to identify the type of coordinate system. The transformation between dispersion coordinate, wi, and logical pixel coordinate, li, along axis i

$$w_i = CRVAL_i + CD_{i,i} * (l_i - CRPIX_i)$$

If the keywords are missing then the values are assumed to be zero except for the diagonal elements of the scale/rotation matrix, the CDi_i, which are assumed to be 1. If only some of the keywords are present then any missing CDi_i keywords will take the value 0 which will cause IRAF tasks to fail with arithmetic or matrix inversion errors. If the CTYPEi keyword is missing it is assumed to be "LINEAR".

If the pixel sampling is logarithmic in the dispersion coordinate, as required for radial velocity cross-correlations, the WCS coordinate values are logarithmic and wi (above) is the logarithm of the dispersion coordinate. The spectral tasks (though not other tasks) will recognize this case and automatically apply the anti-log. The two types of pixel sampling are identified by the value of the keyword DC-FLAG. A value of 0 defines a linear sampling of the dispersion and a logarithmic sampling of the dispersion. Thus, in all cases the spectral tasks will display and analyze the spectra in the same dispersion units regardless of the pixel sampling.

Other keywords which may be present are DISPAXIS for 2 spatial spectra, and the WCS attributes "system", "wtype", "label", and "units". The system attribute will usually have the value "world" for spatial spectra and "equispec" for equispec spectra. The wtype attribute will have the value

"linear". Currently the label will be either "Pixel" or "Wavelength" and the units will be "Angstroms" for dispersion corrected spectra. In the future there will be more generality in the units for dispersion calibrated spectra.

Figure 1 shows the WCS keywords for a two dimensional long slit spectrum. The coordinate system is defined to be a generic "world" system and the wtype attributes and CTYPE keywords define the axes to be linear. The other attributes define a label and unit for the second axis, which is dispersion axis as indicated by the DISPAXIS keyword. The LTM/LTV keywords in this example show that a subsection of the original image has been extracted with a factor of 2 block averaging along the dispersion axis. The dispersion coordinates are given in terms of the *logical* pixel coordinates by the FITS keywords as defined previously.

Figure 1: Long Slit Spectrum

```

WAT0_001= 'system=world'
WAT1_001= 'wtype=linear'
WAT2_001= 'wtype=linear label=Wavelength units=Angstroms'
WCSDIM   =                2
DISPAXIS =                2
DC-FLAG  =                0

CTYPE1   = 'LINEAR  '
LTV1     =                -10.
LTM1_1   =                1.
CRPIX1   =                -9.
CRVAL1   =    19.5743865966797
CD1_1    =    1.01503419876099

CTYPE2   = 'LINEAR  '
LTV2     =                -49.5
LTM2_2   =                0.5
CRPIX2   =                -49.
CRVAL2   =    4204.462890625
CD2_2    =    12.3337936401367

```

Figure 2 shows the WCS keywords for a three dimensional image where each line is an independent spectrum or associated data but where all spectra have the same linear dispersion. This type of coordinate system has the system name "equispec". The ancillary information about each aperture is found in the APNUM keywords. These give the aperture number, beam number, and extraction limits. In this example the LTM/LTV keywords have their default values; i.e. the logical and physical coordinates are the same.

Figure 2: Equispec Spectrum

```

WAT0_001= 'system=equispec'
WAT1_001= 'wtype=linear label=Wavelength units=Angstroms'
WAT2_001= 'wtype=linear'
WAT3_001= 'wtype=linear'
WCSDIM   =                3
DC-FLAG  =                0
APNUM1   = '41 3 7.37 13.48'
APNUM2   = '15 1 28.04 34.15'
APNUM3   = '33 2 43.20 49.32'

CTYPE1   = 'LINEAR  '
LTM1_1   =                1.
CRPIX1   =                1.
CRVAL1   =    4204.463
CD1_1    =    6.166897000000001

```

```

CTYPE2 = 'LINEAR'
LTM2_2 = 1.
CD2_2 = 1.

CTYPE3 = 'LINEAR'
LTM3_3 = 1.
CD3_3 = 1.

```

5. Multispec Spectral World Coordinate System

The *multispec* spectral world coordinate system applies only to one dimensional spectra; i.e. there is no analog for the spatial type spectra. It is used either when there are multiple 1D spectra with differing dispersion functions in a single image or when the dispersion functions are nonlinear.

The multispec coordinate system is always two dimensional though there may be an independent third axis. The two axes are coupled and they both have axis type "multispec". When the image is one dimensional the physical line is given by the dimensional reduction keyword WAXMAP. The second, line axis, has world coordinates of aperture number. The aperture numbers are integer values and need not be in any particular order but do need to be unique. This aspect of the WCS is not of particular user interest but applications use the inverse world to physical transformation to select a spectrum line given a specified aperture.

The dispersion functions are specified by attribute strings with the identifier *specN* where N is the *physical* image line. The attribute strings contain a series of numeric fields. The fields are indicated symbolically as follows.

```
specN = ap beam dtype w1 dw nw z aplow ahigh [functions_i]
```

where there are zero or more functions having the following fields,

```
function_i = wt_i w0_i ftype_i [parameters] [coefficients]
```

The first nine fields in the attribute are common to all the dispersion functions. The first field of the WCS attribute is the aperture number, the second field is the beam number, and the third field is the dispersion type with the same function as DC-FLAG in the *nspec* and *equispec* formats. A value of -1 indicates the coordinates are not dispersion coordinates (the spectrum is not dispersion calibrated), a value of 0 indicates linear dispersion sampling, a value of 1 indicates log-linear dispersion sampling, and a value of 2 indicates a dispersion.

The next two fields are the dispersion coordinate of the first *physical* pixel and the average dispersion interval per *physical* pixel. For linear and log-linear dispersion types the dispersion parameters are exact while for the nonlinear dispersion functions they are approximate. The next field is the number of valid pixels, hence it is possible to have spectra with varying lengths in the same image. In that case the image is as big as the biggest spectrum and the number of pixels selects the actual data in each image line. The next (seventh) field is a doppler factor. This doppler factor is applied to all dispersion coordinates by multiplying by $1/(1+z)$ (assuming wavelength dispersion units). Thus a value of 0 is no doppler correction. The last two fields are extraction aperture limits as discussed previously.

Following these fields are zero or more function descriptions. For linear or log-linear dispersion coordinate systems there are no function fields. For the nonlinear dispersion systems the function fields specify a zero point offset, the type of dispersion function, and the parameters and coefficients describing it. The function type codes, *ftype_i*, are 1 for a chebyshev polynomial, 2 for a legendre polynomial, 3 cubic spline, 4 for a linear spline, 5 for a pixel coordinate array, and 6 for a sampled coordinate array. The number of

fields before the next function and the number of functions are determined from the parameters of the preceding function until the end of the attribute is reached.

The equation below shows how the final wavelength is computed based on the nfunc individual dispersion functions $W_i(p)$. Note that this is completely general in that different function types may be combined. However, in practice when multiple functions are used they are generally of the same type and represent a calibration before and after the actual object observation with the weights based on the relative time difference between the calibration dispersion functions and the object observation.

$$w = \text{sum from } i=1 \text{ to } n\text{func} \{wt_i * (w0_i + W_i(p)) / (1 + z)\}$$

The multispec coordinate systems define a transformation between physical pixel, p , and world coordinates, w . Generally there is an intermediate coordinate system used. The following equations define these coordinates. The first one shows the transformation between logical, l , p , coordinates based on the LTM/LTV keywords. The polynomial functions are defined in terms of a normalized coordinate, n , as shown in the second equation. The normalized coordinates run between -1 and 1 range of physical coordinates, $pmin$ and $pmax$ which are parameters of the function, upon which the coefficients were defined. The spline functions map the physical range into an index over the number of evenly divided spline pieces, $npieces$, which is a parameter of the function. This mapping is shown in the third and fourth equations where s is the continuous spline coordinate and j than or equal to s .

$$\begin{aligned} p &= (l - LTV1) / LTM1_1 \\ n &= (p - pmiddle) / (prange / 2) \\ &= (p - (pmax+pmin)/2) / ((pmax-pmin) / 2) \\ s &= (p - pmin) / (pmax - pmin) * npieces \\ j &= \text{int}(s) \end{aligned}$$

5.1 Linear and Log Linear Dispersion Function

The linear and log-linear dispersion functions are described by a wavelength at the first *physical* pixel and a *physical* pixel. A doppler correction may also be applied. The equations below show the two forms. Note that the coordinates returned are always wavelength even though the pixel sampling and the dispersion parameters may be log-linear.

$$\begin{aligned} w &= (w1 + dw * (p - 1)) / (1 + z) \\ w &= 10 ** \{(w1 + dw * (p - 1)) / (1 + z)\} \end{aligned}$$

Figure 3 shows an example from a multispec image with independent linear dispersion coordinates. This is a linearized echelle spectrum where each order (identified by the beam number) is stored as a separate image line.

Figure 3: Echelle Spectrum with Linear Dispersion Function

```

WAT0_001= 'system=multispec'
WAT1_001= 'wtype=multispec label=Wavelength units=Angstroms'
WAT2_001= 'wtype=multispec spec1 = "1 113 0 4955.44287109375 0.05...'
WAT2_002= '5 256 0. 23.22 31.27" spec2 = "2 112 0 4999.0810546875...'
WAT2_003= '58854293 256 0. 46.09 58.44" spec3 = "3 111 0 5043.505...'
WAT2_004= '928358078002 256 0. 69.28 77.89"
WCSDIM   =                               2

CTYPE1   = 'MULTISPE'
LTM1_1   =                               1.
CD1_1    =                               1.

```



```

CTYPE2 = 'MULTISPE'
LTM2_2 = 1.
CD2_2 = 1.

```

5.2 Chebyshev Polynomial Dispersion Function

The parameters for the chebyshev polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, ci. The equation below shows how to evaluate the function using an iterative definition where $x_1 = x_2 = n$, and $x_i = 2 * n * x_{i-1} - x_{i-2}$.

The parameters for the chebyshev polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, c_i. The equation below shows how to evaluate the function using an iterative definition where $x_1 = 1$, $x_2 = n$, and $x_i = 2 * n * x_{i-1} - x_{i-2}$.

$$W = \text{sum from } i=1 \text{ to order } \{c_i * x_i\}$$

5.3 Legendre Polynomial Dispersion Function

The parameters for the legendre polynomial dispersion function are the order (number of coefficients) and the normalizing range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute n. Following the parameters are the order coefficients, c_i. The equation below shows how to evaluate the function using an iterative definition where $x_1 = 1$, $x_2 = x_i = ((2i-3)*n*x_{i-1} - (i-2)*x_{i-2})/(i-1)$.

$$W = \text{sum from } i=1 \text{ to order } \{c_i * x_i\}$$

Figure 4 shows an example from a multispec image with independent nonlinear dispersion coordinates. This is again from an echelle spectrum. Note that the IRAF **echelle** package determines a function, in this case a bidimensional legendre polynomial, with the independent variables being the order number and the extracted pixel coordinate. To assign and store this function in the image is simply a matter of collapsing the two dimensional dispersion function by fixing the order number and combining all the terms with the same order.

```

Figure 4: Echelle Spectrum with Legendre Polynomial Function
WAT0_001= 'system=multispec'
WAT1_001= 'wtype=multispec label=Wavelength units=Angstroms'
WAT2_001= 'wtype=multispec spec1 = "1 113 2 4955.442888635351 0.05...'
WAT2_002= '83 256 0. 23.22 31.27 1. 0. 2 4 1. 256. 4963.0163112090...'
WAT2_003= '976664 -0.3191636898579552 -0.8169352858733255" spec2 =...'
WAT2_004= '9.081188912082 0.06387049476832223 256 0. 46.09 58.44 1...'
WAT2_005= '56. 5007.401409453303 8.555959076467951 -0.176732458267...'
WAT2_006= '09935064388" spec3 = "3 111 2 5043.505764869474 0.07097...'
WAT2_007= '256 0. 69.28 77.89 1. 0. 2 4 1. 256. 5052.586239197408 ...'
WAT2_008= '271 -0.03173489817897474 -7.190562320405975E-4"
WCSDIM = 2

CTYPE1 = 'MULTISPE'
LTM1_1 = 1.

```

```

CD1_1    =          1.

CTYPE2   = 'MULTISPE'
LTM2_2   =          1.
CD2_2    =          1.

```

5.4 Linear Spline Dispersion Function

The parameters for the linear spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+1 coefficients, c_i. The two coefficients used in a linear combination are selected based on the spline coordinate, where a are the fractions of the interval in the spline piece between the spline knots, a=(j+1)-s, b=s-j, and x_0=a, and x_1=b.

$$W = \text{sum from } i=0 \text{ to } 1 \{c_{(i+j)} * x_i\}$$

5.5 Cubic Spline Dispersion Function

The parameters for the cubic spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+3 coefficients, c_i. The four coefficients used are selected based on the spline coordinate. The fractions of the interval between the integer spline knots are given by a and b, a=(j+1)-s, b=s-j, and x_0=a sup 3, x_1=(1+3*a*(1+a*b)), x_2=(1+3*b*(1+a*b)), and x_3=b**3.

The parameters for the cubic spline dispersion function are the number of spline pieces, npieces, and the range of physical coordinates, pmin and pmax, over which the function is defined and which are used to compute the spline coordinate s. Following the parameters are the npieces+3 coefficients, c_i. The four coefficients used are selected based on the spline coordinate. The fractions of the interval between the integer spline knots are given by a and b, a=(j+1)-s, b=s-j, and x_0=a**3, x_1=(1+3*a*(1+a*b)), x_2=(1+3*b*(1+a*b)), and x_3=b**3.

$$W = \text{sum from } i=0 \text{ to } 3 \{c_{(i+j)} * x_i\}$$

5.6 Pixel Array Dispersion Function

The parameters for the pixel array dispersion function consists of just the number of coordinates ncoords. Following this are the wavelengths at integer physical pixel coordinates starting with 1. To evaluate a wavelength at some physical coordinate, not necessarily an integer, a linear interpolation is used between the nearest integer physical coordinates and the desired physical coordinate where a fractional intervals k=int(p), a=(k+1)-p, b=p-k, and x_0=a, and x_1=b.

$$W = \text{sum from } i=0 \text{ to } 1 \{c_{(i+j)} * x_i\}$$

5.7 Sampled Array Dispersion Function

The parameters for the sampled array dispersion function consists of the number of coordinate pairs, ncoords, and a dummy field. Following these are the physical coordinate and wavelength pairs which are in increasing order. The nearest physical coordinates to the desired physical coordinate are located and a is computed between the two sample points.

NAME

splot -- plot and analyze spectra

USAGE

splot images [line [band]]

PARAMETERS**images**

List of images (spectra) to plot. If the image is 2D or 3D the line and band parameters are used. Successive images are plotted following each 'q' cursor command. One may use an image section to select a desired column, line, or band but the full image will be in memory and any updates to the spectrum will be part of the full image.

line, band

The image line/aperture and band to plot in two or three dimensional images. For multiaperture spectra the aperture specified by the line parameter is first sought and if not found the specified image line is selected. For other two dimensional images, such as long slit spectra, the line parameter specifies a line or column. Note that if the line and band parameters are specified on the command line it will not be possible to change them interactively.

units = ""

Dispersion coordinate units for the plot. If the spectra have known units, currently this is generally Angstroms, the units may be converted to other units for plotting as specified by this task parameter. If this parameter is the null string and the world coordinate system attribute "units_display" is defined then that will be used. If both this task parameters and "units_display" are not given then the spectrum dispersion units will be used. The units may also be changed interactively. See the units section of the **package** help for a further description and available units.

options = "auto" [auto,zero,xydraw,histogram,nosysid,wreset,flip,overplot]

A list of zero or more, possibly abbreviated, options. The options can also be toggled with colon commands. The currently defined options are "auto", "zero", "xydraw", "histogram", "nosysid", "wreset", "flip", and "overplot". Option "auto" automatically replots the graph whenever changes are made. Otherwise the graph is replotted with keystrokes 'c' or 'r'. Option "zero" makes the initial minimum y of the graphs occur at zero. Otherwise the limits are set automatically from the range of the data or the ymin parameter. Option "xydraw" changes the 'x' draw key to use both x and y cursor values for drawing rather than the nearest pixel value for the y value. Option "histogram" plots the spectra in a histogram style rather than connecting the pixel centers. Option "nosysid" excludes the system banner from the graph title. Option "wreset" resets the graph limits to those specified by the xmin, xmax, ymin, ymax parameters whenever a new spectrum is plotted. The "flip" option selects that initially the spectra be plotted with decreasing wavelengths. The options may be queried and changed interactively. The "overplot" options overplots all graphs and a screen erase only occurs with the redraw key.

xmin = INDEF, xmax = INDEF, ymin = INDEF, ymax = INDEF

The default limits for the initial graph. If INDEF then the limit is determined from the range of the data (autoscaling). These values can be changed interactively with 'w' window key options or the cursor commands ":xwindow" and ":ywindow" (see **gtools**).

save_file = "splot.log"

The file to contain any results generated by the equivalent width or deblending functions. Results are added to this file until the file is deleted. If the filename is null (""), then no results are saved.

graphics="stdgraph"

Output graphics device: one of "stdgraph", "stdplot", "stdvdm", or name.

cursor=""

Graphics cursor input. When null the standard cursor is used otherwise the specified file is used.

The following parameters are used for error estimates in the 'd', 'k', and 'e' key measurements. See the ERROR ESTIMATES section for a discussion of the error estimates.

nerrsample= 0

Number of samples for the error computation. A value less than 10 turns off the error computation. A value of ~10 does a rough error analysis, a value of ~50 does a reasonable error analysis, and a detailed error analysis. The larger this value the longer the analysis takes.

sigma0 = INDEF, **invgain** = INDEF

The pixel sigmas are modeled by the formula:

$$\text{sigma}^{**2} = \text{sigma0}^{**2} + \text{invgain} * I$$

where I is the pixel value and "**2" means the square of the quantity. If either parameter is specified as INDEF or with a value less than zero then no sigma estimates are made and so no error estimates for the measured parameters are made.

The following parameters are for the interactive curve fitting function entered with the 't' key. This function is usually used for continuum fitting. The values of these parameters are updated during the fitting. See **icfit** for additional details on interactive curve fitting.

function="spline3"

Function to be fit to the spectra. The functions are "legendre" (legendre polynomial), "chebyshev" (chebyshev polynomial), "spline1" (linear spline), and "spline3" (cubic spline). The functions may be abbreviated.

order= 1

The order of the polynomials or the number of spline pieces.

low_reject= 2., **high_reject**= 4.

Rejection limits below and above the fit in units of the residual sigma. Unequal limits are used to reject spectral lines on one side of the continuum during continuum fitting.

niterate= 10

Number of rejection iterations.

grow = 1.

When a pixel is rejected, pixels within this distance of the rejected pixel are also rejected.

markrej= yes

Mark rejected points? If there are many rejected points it desired to not mark rejected points.

The following parameters are used to overplot standard star fluxes with the 'y' key. See **standard** for more information about these parameters.

star_name

Query parameter for the standard star fluxes to be overplotted. Unrecognized names or a "?" will print a list of the available stars in the specified calibration directory.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

caldir = "_caldir"

The standard star calibration directory. The default value redirects the value to the parameter of the same name in the package parameters.

fnuzero = 3.68e-20

The absolute flux per unit frequency at a magnitude of zero used to convert the calibration magnitudes to absolute flux.

The following parameters are used for queries in response to particular keystrokes.

next_image

In response to 'g' (get next image) this parameter specifies the image.

new_image

In response to 'i' (write current spectrum) this parameter specifies the name of a new image to create or existing image to overwrite.

overwrite = no

Overwrite an existing output image? If set to yes it is possible to write back into the input spectrum or to some other existing image. Otherwise the user is queried again for a new image name.

spec2

When adding, subtracting, multiplying, or dividing by a ('+', '-', '*', '/') keys in the 'f' mode) this parameter is used to get the name of the second spectrum.

constant

When adding or multiplying by a constant ('p' or 'm' keys in the 'f' mode) the parameter is used to get the constant.

wavelength

This parameter is used to get a dispersion coordinate value during deblending or when changing the dispersion coordinates with 'u'.

linelist

During deblending this parameter is used to get a peak values, profile types, and widths.

wstart,wend, dw

In response to 'p' (convert to a linear wavelength scale) these parameters specify the starting wavelength, ending wavelength, and wavelength per pixel.

boxsize

In response to 's' (smooth) this parameter specifies the box size in pixels to be used for the boxcar smooth. The value must be odd. If an even value is specified the next larger odd value is actually used.

DESCRIPTION

Splot provides an interactive facility to display and analyze spectra. See also **bplot** for a version of this task useful for making many plots noninteractively. Each spectrum in the image list is displayed successively. To quit the current image and go on to the next the 'q' cursor command is used. If an image is two-dimensional, such as with multiple aperture or long slit spectra, the aperture or image column/line to be displayed is needed. If the image is three-dimensional, such as with the extra information produced by **apextract**, the band is needed. These parameters are queried unless specified on the command line. If given on the command line it will not be possible to change them interactively.

The plots are made on the specified graphics device which is usually to the graphics terminal. The initial plot limits are set with the parameters *xmin*, *xmax*, *ymin*, and *ymax*. If a is determined from the range of the data. The "zero" option may also be set in the *options* parameter to set the lower intensity limit to zero. Other options that may be set to control the initial plot are to exclude the system identification banner, and to select a histogram line type instead of connecting the pixel centers. The dispersion units used in the plot are set by the *units* parameter. This allows converting to units other than those in which the dispersion coordinates are defined in the spectra.

The *option* parameter, mentioned in the previous paragraph, is a list of zero or more options. As previously noted, some of the options control the initial appearance of the plots. The "auto" option determines how frequently plots are redrawn. For slow terminals or via modems one might wish to minimize the redrawing. The default, however, is to redraw when changes are made. The "xydraw" parameter is specific to the 'x' key.

After the initial graph is made an interactive cursor loop is entered. The *cursor* parameter may be reset to read from a file but generally the graphics device cursor is read. The cursor loop takes single keystroke commands and typed in commands begun with a colon, called colon commands. These commands are described below and a summary of the commands may be produced interactively with the '?' key or a scrolling help on the status line with the '/' key.

Modifications to the spectra being analyzed may be saved using the 'i' key in a new, the current, or other existing spectra. A new image is created as a new copy of the current spectrum and so if the current spectrum is part of a multiple spectrum image (including a other spectra are copied. If other spectra in the same image are then modified and saved use the overwrite option to replace then in the new output image. If the output spectrum already exists then the *overwrite* flag must be set to allow modifying the data. This includes the case when the output spectrum is the same as the input spectrum. The only odd case here is when the input spectrum is one dimensional and the output spectrum is two dimensional. In this case the user is queried for the line to be written.

The other form of output, apart from that produced on the terminal, are measurements of equivalent widths, and other analysis functions. This information will be recorded in the *save_file* if specified.

The following keystrokes are active in addition to the normal IRAF cursor facilities (available with ":.help"):

? Page help information.

/ Cycle through short status line help.

<space>

The space bar prints the cursor position and value of the nearest pixel.

- a** Expand and autoscale to the data range between two cursor positions. See also 'w', and 'z'. Selecting no range, that is the two cursor positions the same, produces an autoscale of the whole spectrum.
- b** Set the plot base level to zero rather than autoscaling.
- c** Clear all windowing and redraw the full current spectrum. This redraws the spectrum and cancels any effects of the 'a', 'z', and 'w' keys. The 'r' key is used to redraw the spectrum with the current windowing.
- d** Mark two continuum points and fit (deblend) multiple line profiles. The center, continuum at the center, core intensity, integrated flux, equivalent width, FWHMs for each profile are printed and saved in the log file. See 'k' for fitting a '-' to subtract the fitted profiles.
- e** Measure equivalent width by marking two continuum points around the line to be measured. The linear continuum is subtracted and the flux is determined by simply summing the pixels with partial pixels at the ends. Returned values are the line center, continuum at the region center, flux above or below the continuum, and the equivalent width.
- f** Enter arithmetic function mode. This mode allows arithmetic functions to be applied to the spectrum. The pixel values are modified according to the function request and may be saved as a new spectrum with the 'i' command. Operations with a second spectrum are done in wavelength space and the second spectrum is automatically resampled if necessary. If one spectrum is longer than the other, only the smaller number of pixels are affected. To exit this mode type 'q'.

The following keystrokes are available in the function mode. Binary operations with a constant or a second spectrum produce a query for the constant value or spectrum name.

- a** Absolute value
 - d** Power of base 10 (inverse log base 10)
 - e** Power of base e (inverse log base e)
 - i** Inverse/reciprocal (values equal to zero are set to 0.0 in the inverse)
 - l** Log base 10 (values less than or equal to 0.0 are set to -0.5)
 - m** Multiply by a constant (constant is queried)
 - n** Log base e (values less than or equal to 0.0 are set to -0.5)
 - p** Add by a constant (constant is queried)
 - q** Quit Function mode
 - s** Square root (values less than 0.0 are set to 0.0)
 - +** Add another spectrum
 - Subtract another spectrum
 - *** Multiply by another spectrum
 - /** Divide by another spectrum
- g** Get another spectrum. The current spectrum is replaced by the new spectrum. The aperture/line and band are queried is necessary.

- h** Measure equivalent widths assuming a gaussian profile with the width measured at a specified point. Note that this is not a gaussian fit (see 'k' to fit a gaussian)! The gaussian profile determined here may be subtracted with the '-' key. A second cursor key is requested with one of the following values:
 - a** Mark the continuum level at the line center and use the LEFT half width at the half flux point.
 - b** Mark the continuum level at the line center and use the RIGHT half width at the half flux point.
 - c** Mark the continuum level at the line center and use the FULL width at the half flux point.
 - l** Mark a flux level at the line center relative to a and use the LEFT width at that flux point.
 - r** Mark a flux level at the line center relative to a and use the RIGHT width at that flux point.
 - k** Mark a flux level at the line center relative to a and use the FULL width at that flux point.
- i** Write the current spectrum out to a new or existing image. The image name is queried and overwriting must be confirmed.
- j** Set the value of the nearest pixel to the x cursor to the y
- k + (g, l or v)** Mark two continuum points and fit a single line profile. The second key selects the type of profile: g for gaussian, l for lorentzian, and v for voigt. Any other second key defaults to gaussian. The center, continuum at the center, core intensity, integrated flux, equivalent width, and FWHMs are printed and saved in the log file. See 'd' for fitting multiple profiles and '-' to subtract the fit.
- l** Convert to flux per unit wavelength (f-lambda). The spectrum is assumed to be flux calibrated in flux per unit frequency (f-nu). See also 'n'.
- m** Compute the mean, RMS, and signal-to-noise over a x cursor positions.
- n** Convert to flux per unit frequency (f-nu). The spectrum is assumed to be flux calibrated in flux per unit wavelength (f-lambda). See also 'l'.
- o** Set overplot flag. The next plot will overplot the current plot. Normally this key is immediately followed by one of 'g', '#', '%', '(', or ')'. The ":overplot" colon command and overplot parameter option may be used to set overplotting to be permanently on.
- p** Define a linear wavelength scale. The user is queried for a wavelength and an ending wavelength. If either (though not both) are specified as INDEF a dispersion is queried for and used to compute an endpoint. A wavelength scale set this way will be used for other spectra which are not dispersion corrected.
- q** Quit and go on to next input spectrum. After the last spectrum exit.
- r** Redraw the spectrum with the current windowing. To redraw the full spectrum and cancel any windowing use the 'c' key.
- s** Smooth via a boxcar. The user is prompted for the box size.
- t** Fit a function to the spectrum using the ICFIT mode. Typically interactive rejection is used to exclude spectra lines from the fit in order to fit a smooth continuum. A second keystroke selects what to do with the fit.

- / Normalize by the fit. When fitting the continuum this continuum normalizes the spectrum.
- Subtract the fit. When fitting the continuum this continuum subtracts the spectrum.
- f** Replace the spectrum by the fit.
- c** Clean the spectrum by replacing any rejected points by the fit.
- n** Do the fitting but leave the spectrum unchanged (a NOP on the spectrum). This is useful to play with the spectrum using the capabilities of ICFIT.
- q** Quit and don't do any fitting. The spectrum is not modified.

- u** Adjust the user coordinate scale. There are three options, 'd' mark a position with the cursor and doppler shift it to a specified value, 'z' mark a position with the cursor and zeropoint shift it value, or 'l' mark two positions and enter two values to define a linear (in wavelength) dispersion scale. The units used for input are those currently displayed. A wavelength scale set this way will be used for other spectra which are not dispersion corrected.

- v** Toggle to a velocity scale using the position of the cursor as the velocity origin and back.

- w** Window the graph. For further help type '?' to the "window:" prompt or see help under **gtools**. To cancel the windowing use 'a'.

- x** "Etch-a-sketch" mode. Straight lines are drawn between successive positions of the cursor. Requires 2 cursor settings in x. The nearest pixels are used as the endpoints. To draw a line between arbitrary y values first use 'j' to adjust the endpoints or set the "xydraw" option.

- y** Overplot standard star values from a calibration file.

- z** Zoom the graph by a factor of 2 in x.

- (** In multiaperture spectra go to the spectrum in the preceding image line. If there is only one line go to the spectrum in the preceding band.

-)** In multiaperture spectra go to the spectrum in the following image line. If there is only one line go to the spectrum in the following band.

- #** Get a different line in multiaperture spectra or two dimensional images. The aperture/line/column is queried.

- %** Get a different band in a three dimensional image.

- \$** Switch between physical pixel coordinates and world (dispersion) coordinates.

- Subtract the fits generated by the 'd' (deblend), 'k' (single profile fit), and 'h' (gaussian of specified width). The region to be subtracted is marked with two cursor positions.

- ' , '** Shift the graph window to the left.

- .** Shift the graph window to the right.

- I** Force a fatal error interrupt to leave the graph. This is used because the normal interrupt character is ignored in graphics mode.

:show

Page the full output of the previous deblend and equivalent width measurements.

:log Enable logging of measurements to the file specified by the parameter *save_file*. When the program is first entered logging is enabled (provided a log file is specified). There is no way to change the file name from within the program.

:nolog

Disable logging of measurements.

:dispaxis<val>

Show or change dispersion axis for 2D images.

:nsum<val>

Show or change summing for 2D images.

:units<value>

Change the coordinate units in the plot. See below for more information.

:# <comment>

Add comment to logfile.

Labels:**:label<label> <format>**

Add a label at the cursor position.

:mabove<label> <format>

Add a tick mark and label above the spectrum at the cursor position.

:mbelow<label> <format>

Add a tick mark and label below the spectrum at the cursor position.

The label must be quoted if it contains blanks. A label beginning with % (i.e. %.2f) is treated as a format for the x cursor position. The optional format is a gtext string (see help on "cursors"). The labels are not remembered between redraws.

:auto[yes|no]

Enable/disable autodraw option

:zero[yes|no]

Enable/disable zero baseline option

:xydraw[yes|no]

Enable/disable xydraw option

:hist[yes|no]

Enable/disable histogram line type option

:nosysid[yes|no]

Enable/disable system ID option

:wreset[yes|no]

Enable/disable window reset for new spectra option

:flip [yes|no]

Enable/disable the flipped coordinates option

:overplot[yes|no]

Enable/disable the permanent overplot option

:/help

Get help on GTOOLS options.

:.help

Get help on standard cursor mode options

PROFILE FITTING AND DEBLENDING

The single profile ('k') and multiple profile deblending ('d') commands fit gaussian, lorentzian, and voigt line profiles with a linear background. The single profile fit, 'k' key, is a fitting designed to be simple to use. Two cursor positions define the region to be fit and a fixed linear continuum. The second key is used to select the type of profile to fit with 'g' for gaussian, 'l' for lorentzian, and 'v' for voigt. Any other second key will default to a gaussian profile. The profile center, peak strength, and width(s) are then determined and the results are printed on the status line and in the log file. The meaning of these quantities is described later. The fit is also overplotted and may be subtracted from the spectrum subsequently with the '-' key.

The more complex deblending function, 'd' key, defines the fitting region and initial linear continuum in the same way with two cursor positions. The continuum may be included in the fitting as an option. The lines to be fit are entered with the cursor near the line center ('g' for gaussian, 'l' for lorentzian, 'v' for voigt), by typing the wavelengths ('t'), or read from a file ('f'). The latter two methods are useful if the lines are known accurately and if fits restricting the absolute or relative positions of the lines will be used. The 't' key is restricted to gaussian fits only.

The 'f' key asks for a line list file. The format of this file has one or more columns. The columns are the wavelength, the peak value (relative to the continuum with negative values being absorption), the profile type (gaussian, lorentzian, or voigt), and the gaussian and/or lorentzian FWHM. End columns may be missing or INDEF values may be used to have values be approximated. Below are examples of the file line formats

```
wavelength
wavelength peak
wavelength peak (gaussian|lorentzian|voigt)
wavelength peak gaussian gfwhm
wavelength peak lorentzian lfwhm
wavelength peak voigt gfwhm
wavelength peak voigt gfwhm lfwhm

1234.5                <- Wavelength only
1234.5 -100           <- Wavelength and peak
1234.5 INDEF v        <- Wavelength and profile type
1234.5 INDEF g 12     <- Wavelength and gaussian FWHM
```

where peak is the peak value, gfwhm is the gaussian FWHM, and lfwhm is the lorentzian FWHM. This format is the same as used by **fitprofs** and also by **artdata.mk1dspec** (except in the latter case the peak is normalized to a continuum of 1).

There are four queries made to define the set of parameters to be fit or constrained. The positions may be held "fixed" at their input values, allowed to shift by a "single" offset from the input values, or "all"

positions may be fit independently. The widths may be constrained to a "single" value or "all" fit independently. The linear background may be included in the fit or kept fixed at that input using the cursor.

As noted above, sometimes the absolute or relative wavelengths of the lines are known a priori and this information may be entered by typing the wavelengths explicitly using the 't' option or read from a 'f' option during marking. In this case one should fix or fit a single shift for the position. The latter may be useful if the lines are known but there is a measurable doppler shift.

After the fit, the modeled lines are overplotted. The line center, flux, equivalent width, and full width half maxima are printed on the status line for the first line. The values for the other lines and the RMS of the fit may be examined by scrolling the status line using the '+', '-', and 'r' keys. To continue enter 'q'.

The fitting may be repeated with different options until exited with 'q'. For each line in the blend the line center, continuum intensity at the line center, the core intensity above or below the continuum, the FWHM for the gaussian and lorentzian parts, the flux above or below the continuum, and the equivalent width are recorded in the log file. All these parameters except the continuum are based on the fitted analytic profiles. Thus, even though the fitted region may not extend into the wings of a line the equivalent width measurements include the wings in the fitted profile. For direct integration of the flux use the 'e' key.

The fitted model may be subtracted from the data (after exiting the deblending function) using the '-' (minus) keystroke to delimit the region for which the subtraction is to be performed. This allows you to fit a portion of a line which may be contaminated by a away the entire line to examine the remaining components.

The fitting uses an interactive algorithm based on the Levenberg-Marquardt method. The iterations attempt to improve the fit by varying the parameters along the gradient of improvement in the chi square. This method requires that the initial values for the parameters be close enough that the gradient leads to the correct solution rather than an incorrect local minimum in the chi square. The initial values are determined as follows:

1. If the lines are input from a data file then those values in the file are used. Missing information is determined as below.
2. The line centers are those specified by the user either by marking with the cursor, entering the wavelengths, for read from a file.
3. The initial widths are obtained by dividing the width of the marked fitting region by the number of lines and then dividing this width by a factor depending on the profile type.
4. The initial peak intensities are the data values at the given line centers with the marked continuum subtracted.

Note that each time a new fitting option is specified the initial parameters are those from the previous fits. Thus the results do depend on the history of previous fits until the fitting is exited. Within each fit an iteration of parameters is performed as described next.

The iteration is more likely to fail if one initially attempts to fit too many parameters simultaneously. A constrained approach to the solution is obtained by iterating starting with a few parameters and then adding more parameters as the solution approaches the true chi square minimum. This is done by using the solutions from the more constrained options as the starting point for the less constrained options. In particular, the positions and a single width are fit first with fixed background. Then multiple widths and the background are added.

To conclude, here are some general comments. The most restrictive (fixed positions and single width(s)) will give odd results if the initial positions are not close to the true centers. The most general (simultaneous positions, widths, and background) can also lead to incorrect results by using unphysically different widths

to make one line very narrow and another very broad in an attempt to fit very blended lines. The algorithm works well when the lines are not severely blended and the shapes of the lines are close to the profile type.

CENTROID, FLUX, AND EQUIVALENT WIDTH DETERMINATIONS

There are currently five techniques in SPLOT to measure equivalent widths and other line profile parameters. The simplest (conceptually) is by integration of the pixel values between two marked pixels. This is invoked with the 'e' keystroke. The user marks the two edges of the line at the continuum. The measured line center, continuum value, line flux, and equivalent width are given by:

```
center = sum (w(i) * (I(i)-C(i))**3/2) / sum ((I(i)-C(i))**3/2)
continuum = C(midpoint)
flux = sum ((I(i)-C(i)) * (w(i2) - w(i1)) / (i2 - i1))
eq. width = sum (1 - I(i)/C(i))
```

where $w(i)$ is the wavelength of pixel i , $i1$ and $i2$ are the nearest integer pixel limits of the integrated wavelength range, $I(i)$ is the data value of pixel i , $C(i)$ is the continuum at pixel (i) , and the sum is over the marked range of pixels. The continuum is a linear function between the two points marked. The factor multiplying the continuum subtracted pixel values in the flux calculation is the wavelength interval per pixel so that the flux integration is done in wavelength units. (See the discussion at the end of this section concerning flux units).

The most complex method for computing line profile parameters is performed by the profile fitting and deblending commands which compute a non-linear least-squares fit to the line(s). These are invoked with the 'd' or 'k' keystroke. These were described in detail previously.

The fourth and fifth methods, selected with the 'h' key, determine the equivalent width from a gaussian profile defined by a constant continuum level "cont", a core depth "core", and the width of the line "dw" at some intermediate level "Iw".

```
I(w) = cont + core * exp (-0.5*((w-center)/sigma)**2)
sigma = dw / 2 / sqrt (2 * ln (core/Iw))
fwhm = 2.355 * sigma
flux = core * sigma * sqrt (2*pi)
eq. width = abs (flux) / cont
```

where w is wavelength.

For ease of use with a large number of lines only one cursor position is used to mark the center of the line and one flux level. Note that both the x and y cursor positions are read simultaneously. From the x position the line center and core intensity are determined. The region around the specified line position is searched for a minimum or maximum and a parabola is fit to better define the extremum.

The two methods based on the simple gaussian profile model differ in how they use the y cursor position and what part of the line is used. After typing 'h' one selects the method and whether to use the left, right, or both sides of the line by a second keystroke. The 'l', 'r', and 'k' keys require a continuum level of one. The y cursor position defines where the width of the line is determined. The 'a', 'b', and 'c' keys use the y cursor position to define the continuum and the line width is determined at the point half way between the line core and the continuum. In both cases the width at the appropriate level is determined by the interception of the y level with the data using linear interpolation between pixels. The one-sided measurements use the half-width on the appropriate side and the two-sided measurements use the full-width.

The adopted gaussian line profile is drawn over the spectrum and the horizontal and vertical lines show the measured line width and the depth of the line center from the continuum. This model may also be subtracted from the spectrum using the '-' key.

The major advantages of these methods are that only a (both the x and y positions are used) is required and they are fast. The 'l', 'r', and 'k' keys give more flexibility in adjusting the width of gaussian line at

the expense of requiring that the spectrum be normalized to a unit continuum. The 'a', 'b', and 'c' keys allow measurements at any continuum level at the expense of only using the half flux level to determine the gaussian line width.

All these methods print and record in the log file the line center, continuum intensity at the line center, the flux, and the equivalent width. For the 'e' key the flux is directly integrated while for the other methods the fitted gaussian is integrated. In addition, for the profile fitting methods the core intensity above or below the continuum, and the FWHMs are also printed. A zero value is record for the gaussian or lorentzian width if the value is not determined by profile fit. A brief line of data for each measurement is printed on the graphics status line. To get the full output and the output from previous measurements use the command ":show". This pages the output on the text output which may involve erasing the graphics.

The integrated fluxes for all the methods are in the same units as the intensities and the integration is done in the same units as the plotted scale. It is the user's responsibility to keep track of the flux units. As a caution, if the data is in flux per unit frequency, say ergs/cm2/sec/hz, and the dispersion in Angstroms then the integrated flux will not be in the usual units but will be A-ergs/cm2/sec/hz. For flux in wavelength units, ergs/cm2/sec/A and the dispersion scale in Angstroms the integrated flux will be correct; i.e. ergs/cm2/sec.

Note that one can compute integrated flux in pixel units by using the '\$' to plot in pixels. This is appropriate if the pixel values are in data numbers or photon counts to get total data number or photons.

ERROR ESTIMATES

The deblending ('d'), single profile fitting ('k'), and profile integration and equivalent width ('e') functions provide error estimates for the measured parameters. This requires a model for the pixel sigmas. Currently this model is based on a Poisson statistics model of the data. The model parameters are a constant gaussian sigma and an "inverse gain" as specified by the parameters *sigma0* and *invgain*. These parameters are used to compute the pixel value sigma from the following formula:

$$\text{sigma}^2 = \text{sigma0}^2 + \text{invgain} * I$$

where I is the pixel value and "I**2" means the square of the quantity.

If either the constant sigma or the inverse gain are specified as INDEF or with values less than zero then no noise model is applied and no error estimates are computed. Also if the number of error samples is less than 10 then no error estimates are computed. Note that for processed spectra this noise model will not generally be the same as the detector readout noise and gain. These parameters would need to be estimated in some way using the statistics of the spectrum. The use of an inverse gain rather than a direct gain was choosed to allow a value of zero for this parameters. This provides a model with constant uncertainties.

The direct profile integration error estimates are computed by error propagation assuming independent pixel sigmas. Also it is assumed that the marked linear background has no errors. The error estimates are one sigma estimates. They are given in the log output (which may also be view without exiting the program using the :show command) below the value to which they apply and in parenthesis.

The deblending and profile fit error estimates are computed by Monte-Carlo simulation. The model is fit to the data (using the sigmas) and this model is used to describe the noise-free spectrum. A number of simulations, given by the *nerrs* parameter, are created in which random gaussian noise is added to the noise-free spectrum using the pixel sigmas from the noise model. The model fitting is done for each simulation and the absolute deviation of each fitted parameter to model parameter is recorded. The error estimate for the each parameter is then the absolute deviation containing 68.3% of the parameter estimates. This corresponds to one sigma if the distribution of parameter estimates is gaussian though this method does not assume this.

The Monte-Carlo technique automatically includes all effects of parameter correlations and does not depend on any approximations. However the computation of the errors does take a amount of time. The amount of

time and the accuracy of the error estimates depend on how many simulations are done. A small number of samples (of order 10) is fast but gives crude estimates. A large number (greater than 100) is slow but gives good estimates. A compromise value of 50 is recommended for many applications.

UNITS

The dispersion units capability of **splot** allows specifying the units with the *units* parameter and interactively changing the units with the ":units" command. In addition the 'v' key allows plotting in velocity units with the zero point velocity defined by the cursor position.

The units are specified by strings having a along with the possible preceding modifiers, "inverse", to select the inverse of the unit and "log" to select logarithmic units. For example "log angstroms" to plot the logarithm of wavelength in Angstroms and "inv microns" to plot inverse microns. The various identifiers may be abbreviated as words but the syntax is not sophisticated enough to recognize standard scientific abbreviations except as noted below.

```

    angstroms - Wavelength in Angstroms
    nanometers - Wavelength in nanometers
    millimicrons - Wavelength in millimicrons
    microns - Wavelength in microns
    millimeters - Wavelength in millimeters
    centimeter - Wavelength in centimeters
    meters - Wavelength in meters
    hertz - Frequency in hertz (cycles per second)
    kilohertz - Frequency in kilohertz
    megahertz - Frequency in megahertz
    gigahertz - Frequency in gigahertz
    m/s - Velocity in meters per second
    km/s - Velocity in kilometers per second
    ev - Energy in electron volts
    kev - Energy in kilo electron volts
    mev - Energy in mega electron volts

    nm - Wavelength in nanometers
    mm - Wavelength in millimeters
    cm - Wavelength in centimeters
    m - Wavelength in meters
    Hz - Frequency in hertz (cycles per second)
    KHz - Frequency in kilohertz
    MHz - Frequency in megahertz
    GHz - Frequency in gigahertz
    wn - Wave number (inverse centimeters)

```

The velocity units require a trailing value and unit defining the velocity zero point. For example to plot velocity relative to a wavelength of 1 micron the unit string would be:

```
km/s 1 micron
```

Some additional examples of units strings are:

```

milliang
megahertz
inv mic
log hertz
m/s 3 inv mic

```

EXAMPLES

This task has a very large number of commands and capabilities which are interactive and graphical. Therefore these examples are fairly superficial. The user is encouraged to simply experiment with the task. To get some help use the '?' or '/' keys.

1. To plot a single spectrum and record any measurements in the file 'ngc7662':

```
cl> splot spectrum save_file=ngc7662
```

2. To force all plots to display zero as the minimum y

```
cl> splot spectrum options="auto, zero"
```

Note that the options auto and zero can be abbreviated to one character.

3. To successively display graphs for a set of spectra with the wavelength limits set to 3000 to 6000 angstroms:

```
cl> splot spec* xmin=3000 xmax=6000
```

4. To make batch plots create a file containing the simple cursor command

```
0 0 0 q
```

or an empty file and then execute one of the following:

```
cl> splot spec* graphics=stdplot cursor=curfile
cl> set stdvdm=splot.mc
cl> splot spec* graphics=stdvdm cursor=curfile
cl> splot spec* cursor=curfile >G splot.mc
```

The first example sends the plots to the standard plot device specified by the environment variable "stdplot". The next example sends the plots to the standard virtual display metacode file specified by the environment variable "stdvdm". The last example redirects the standard graphics to the metacode file splot.mc. To spool the metacode file the tasks **stdplot** and **gkimosaic** may be used. For a large number of plots **gkimosaic** is preferred since it places many plots on one page instead of one plot per page. The other GKI tasks in the **plot** package may be used to examine the contents of a metacode file. A simple script call **bplot** is provided which has the default cursor file given above and default device of "stdplot".

5. More complex plots may be produced both interactively using the '=' key or the ":.snap" or ":.write" commands or by preparing a script of cursor commands.

REVISIONS

SPLITV2.11

The profile fitting and deblending was expanded to include lorentzian and voigt profiles. A new parameter controls the number of Monte-Carlo samples used in the error estimates.

Added colon commands for labeling.

SPLITV2.10.3

The 'u' key now allows three ways to adjust the dispersion scale. The old method of setting a linear dispersion scale is retained as well as adding a doppler and zeropoint adjustment. The coordinates are input in the currently displayed units.

If a wavelength scale is set with either 'p' or 'u' then any other spectra which are not dispersion corrected will adopt this wavelength scale.

The '(' and ') ' keys cycle through bands if there is only one spectrum.

A new option, "flip", has been added to the options parameter to select that the spectra are plotted in decreasing wavelength.

A new options "overplot" has been added to the options parameters and colon commands to

permanently set overplotting. This allows quickly overplotting many spectra.

This task will now write out the current display units in the "units_display" WCS attribute. The default task units have been changed to "" to allow picking up the "units_display" units if defined.

The deblending and gaussian fitting code now subsamples the profile by a factor of 3 and fits the data pixels to the sum of the three subsamples. This accounts for finite sampling of the data.

Error estimates are provided for the deblending ('d'), gaussian fitting ('k'), and profile integration ('e') results.

SPLITV2.10

This is a new version with a significant number of changes. In addition to the task changes the other general changes to the spectroscopy packages also apply. In particular, long slit spectra and spectra with nonlinear dispersion functions may be used with this task. The image header or package dispaxis and nsum parameters allow automatically extracting spectra from 2D image. The task parameters have been modified primarily to obtain the desired initial graph without needing to do it interactively. In particular, the new band parameter selects the band in 3D images, the units parameter selects the dispersion units, and the new histogram, nosysid, and xydraw options select histogram line type, whether to include a system ID banner, and allow editing a spectrum using different endpoint criteria.

Because nearly every key is used there has been some shuffling, consolidating, or elimination of keys. One needs to check the run time '?' help or the help to determine the key changes.

Deblending may now use any number of components and simultaneous fitting of a linear background. A new simplified version of Gaussian fitting for a single line has been added in the 'k' key. The old 'k', 'h', and 'v' equivalent width commands are all part of the single 'h' command using a second key to select a specific option. The Gaussian line model from these modes may now be subtracted from the spectrum in the same way as the Gaussian fitting. The one-sided options, in particular, are interesting in this regard as a new capability.

The arithmetic functions between two spectra are now done in wavelength with resampling to a common dispersion done automatically. The 't' key now provides for the full power of the ICFIT package to be used on a spectrum for continuum normalization, subtraction, or line and cosmic ray removal. The 'x' editing key may now use the nearest pixel values rather than only the y cursor position to replace regions by straight line segments. The mode is selected by the task option parameter "xydraw".

Control over the graph window (plotting limits) is better integrated so that redrawing, zooming, shifting, and the GTOOLS window commands all work well together. The new 'c' key resets the window to the full spectrum allowing the 'r' redraw key to redraw the current window to clean up overplots from the Gaussian fits or spectrum editing.

The dispersion units may now be selected and changed to be from hertz to Mev and the log or inverse (for wave numbers) of units taken. As part of the units package the 'v' key or colon commands may be used to plot in velocity relative to some origin. The \$ key now easily toggles between the dispersion units (whatever they may be) and pixels coordinates.

Selection of spectra has become more complex with multiaperture and long slit spectra. New keys allow selecting apertures, lines, columns, and bands as well as quickly scrolling through the lines in multiaperture spectra. Overplotting is also more general and consistent with other tasks by using the 'o' key to toggle the next plot to be overplotted. Overplots, including those of the Gaussian line models, are now done in a different line type.

There are new colon commands to change the dispersion axis and summing parameters for 2D image, to toggle logging, and also to put comments into the log file. All the options may also be set with colon commands.

SEE ALSO

bplot, gtools, icfit, standard, package, specplot, graph, implot, fitprofs

NAME

standard -- Add standard stars to sensitivity file

USAGE

standard input [records] output

PARAMETERS**input**

List of input standard star spectra or root names if using the record number extension format. All spectra of the same aperture must be of the same standard star. In beam switch mode or when the same star parameter is set all spectra must be of the same standard star regardless of aperture number. Normally the spectra will not be extinction corrected but if then the extinction file should also be given and the same extinction file should be used with **sensfunc**.

records(imred.irsandimred.iidsonly)

List of records or ranges of records to be appended to the input spectra names when using record number extension format. The syntax of this list is comma separated record numbers or ranges of record numbers. A range consists of two numbers separated by a . A null list may be used if no record number extensions are desired. This is a positional query parameter only if the record format is specified.

output

The name of a text file which will contain the output from **standard**. Each execution of **standard** appends to this file information about the standard stars, the calibration bandpasses, and observed counts (see the DESCRIPTION section for more details). The output must be explicitly deleted by the user if the filename is to be reused.

samestar= yes

Is the same star in all apertures? If set to no then each aperture may contain a different standard star. The standard star name is queried each time a new aperture is encountered. Note that this occurs only once per aperture and multiple spectra with the same aperture number must be of the same star. If set to yes the standard star name is only queried once. When in beam switch mode this parameter is ignored since all apertures must contain the same star.

beam_switch= no

Beam switch the spectra? If yes then a beam switch mode is used for the spectra in which successive pairs of object and sky observations from the same aperture are sky subtracted. This requires that the object type flag OFLAG be present and that the spectra are appropriately ordered. All object observations must be of the same standard star and the *samestar* parameter is ignored.

apertures =""

List of apertures to be selected from the input list of spectra. If is specified then all apertures are selected. The syntax is the same as the record number extensions.

bandwidth= INDEF,bandsep= INDEF

Bandpass widths and separations in wavelength units. If INDEF then the default bandpasses are those given in the standard star calibration file. If values for these parameters are specified then a of bandpasses of equal width and separation are defined over the range of the input spectrum. In both cases the default bandpasses can be changed interactively if desired.

fnuzero= 3.68e-20

The absolute flux per unit frequency at an AB magnitude of zero. This is to convert the calibration AB magnitudes to absolute flux by the formula

$$f_{\nu} = \text{fnuzero} * 10. ** (-0.4 * m_{\text{AB}})$$

The flux units are also determined by this parameter. However, the frequency to wavelength interval conversion assumes frequency in hertz. The default value is based on a calibration of Vega at 5556 Angstroms of $3.52\text{e-}20$ ergs/cm²/s/Hz for an AB magnitude of 0.0336. This default value is that used in earlier versions of this task which did not allow the user to change this calibration.

extinction= <no default>

Extinction file used to make second order extinction corrections across the bandpasses. The default value is redirected to the package parameter of the same name. See **lcalib** for a extinctionfiles. Normally the input spectra will not be extinction corrected. But if they are this file will be used to remove the extinction and then the same file should be specified in **sensfunc**. Note that one can choose to use a null extinction file in both.

caldir =")_caldir"

Calibration directory containing standard star data. The default value of **")_caldir"** means to use the package parameter "caldir". A list of standard calibration directories may be obtained by listing the file "onedstds\$README"; for example:

```
cl> page onedstds$README
```

The user may copy or create their own calibration files and specify the directory. The directory **"** refers to the current working directory. The standard calibration directory for blackbody curves is "onedstds\$blackbody/".

observatory =")_observatory"

Observatory at which the spectra were obtained if not specified in the image header by the keyword OBSERVAT. The default is a redirection to look in the parameters for the parent package for a be one of the observatories in the observatory database, "observatory" to select the observatory defined by the environment variable "observatory" or the parameter **observatory.observatory**, or "obsvars" to select the current parameters set in the **observatory** task. See help for **observatory** for additional information.

interact= no

If set to no, then the default wavelength set (either that from the star calibration file or the set given by the *bandwidth* and *bandsep* parameters) is used to select wavelength points along the spectrum where the sensitivity is measured. If set to yes, the spectra may be plotted and the bandpasses adjusted.

graphics ="stdgraph"

Graphics output device for use with the interactive mode. Normally this is the user's graphics terminal.

cursor =""

Graphics cursor input for use with the interactive mode. When null the standard graphics cursor is used otherwise the specified file is used.

star_name

The name of the star observed in the current series of spectra. Calibration data for the star must be in the specified calibration directory "caldir". This is normally a interactive query parameter and

should not be specified on the command line unless all spectra are of the same standard star.

The following three queried parameters apply if the selected calibration file is for a blackbody.

mag The magnitude of the observed star in the band given by the *magband* parameter. If the magnitude is not in the same band as the blackbody calibration file then the magnitude may be converted to the calibration band provided the "params.dat" file containing relative magnitudes between the two bands is in the calibration directory

magband

The standard band name for the input magnitude. This should generally be the same band as the blackbody calibration file. If it is not the magnitude will be converted to the calibration band.

teff The effective temperature (deg K) or the spectral type of the star being calibrated. If a spectral type is specified a "params.dat" file must exist in the calibration directory. The spectral types are specified in the same form as in the "params.dat" file. For the standard blackbody calibration directory the spectral types are specified as A0I, A0III, or A0V, where A can be any letter OBAFGKM, the single digit subclass is between 0 and 9, and the luminosity class is one of I, III, or V. If no luminosity class is given it defaults to dwarf.

The following two parameters are queried if the image does not contain the information.

airmass,exptime

If the airmass and exposure time are not in the header nor can they be determined from other keywords in the header then these query parameters are used to request the airmass and exposure time. The values are updated in the image.

The following parameter is for the task to make queries.

answer

Interactive query parameter.

CURSOR KEYS

```
? Display help page
a Add a new band by marking the endpoints
d Delete band nearest the cursor in wavelength
r Redraw current plot
q Quit with current bandpass definitions
w Window plot (follow with '?' for help)
I Interrupt task immediately
```

```
:show Show current bandpass data
```

DESCRIPTION

Observations of standard stars are integrated over calibration bandpasses and written to an output file along with the associated calibration fluxes. The fluxes are obtained from tabulated standard star calibration files or a model flux distribution (currently just a blackbody) based on the magnitude and spectral type of the star. The output data is used by the task **sensfunc** to determine the detector sensitivity function and possibly the extinction. The spectra are required to be dispersion corrected. The input spectra may be in either "onedspec" or "echelle" format and may have many different observation apertures. The spectra may also be beam switched and use the a record number extension format.

The input spectra are specified by a list of names or root names if the record number extension format. In the latter case each name in the list has each of the specified record numbers appended. A subset of the

input spectra may be selected by their aperture numbers using the parameter *apertures*. The spectrum name, aperture number, and title are printed to the standard output. The airmass is required but if absent from the image header it may be computed from the observation header parameters and the latitude task parameter (normally obtained from the **observatory** task). If the airmass cannot be computed, due to missing keywords, then a query is made for the airmass. The airmass is then updated in the header.

The name of the standard star or blackbody curve is obtained by querying the user. If the parameter *samestar* is yes or beam switch mode is selected then all spectra are assumed to be of the same standard star and the query is made once. If the parameter is no then a each aperture. This allows each aperture to contain a different standard star. Note however that multiple observations with the same aperture number must be of the same standard star.

The standard star name is either the name of an actual standard star or of a blackbody calibration. The latter generally have a star name consisting of just the standard bandpass identifier. If the standard star name is not recognized a menu of the available standard stars in the calibration directory, the file "standards.men", is printed and then the query is repeated. Thus, to get a list you can type ?

The standard star names must map to a file containing tabulated calibration data. The calibration filename is formed from the star name with blanks, "+", and "-" removed, converted to lower case, and the extension ".dat" added. This name is appended to a calibration directory, so the directory name must have an appropriate directory delimiter such as "\$" or "/". Generally one of the system calibration directories is used but one may copy and modify or create new calibration files in a personal directory. For the current working directory the calibration directory is either null or "./".

The calibration files may include comment parameter information consisting of the comment character '#', a parameter name, and the parameter value. These elements are separated by whitespace. Any other comment where the first word does not match one of the allowed parameter names is ignored by the program. The parameter names are "type" identifying the type of calibration file, "units" identifying wavelength units, "band" identifying the band for magnitudes, and "weff" identifying the effective wavelength of the band.

There are two types of standard star calibration files as described below.

STANDARD STAR CALIBRATION FILES

This type of file is any file that does not contain the parameter "type" with a value of "blackbody". The only other parameter used by this type of calibration file is the "units" parameter for the wavelength units. If the units are not specified then the wavelengths default to Angstroms. All older calibration files will have no parameter information so they are interpreted as standard star calibration files with wavelengths in Angstroms.

The calibration files consist of lines with wavelengths, calibration magnitudes, and bandpass widths. The magnitudes are *m_AB* defined as

$$m_{AB}(\text{star}) = -2.5 * \log_{10} (f_{\nu}) - 48.60$$

where *f_nu* is in erg/cm²/s/Hz. The *m_AB* calibration magnitudes are converted to absolute flux per unit frequency using the parameter *fnuzero* defined by

$$f_{\nu} = fnuzero * 10. ** (-0.4 * m_{AB})$$

Thus, *fnuzero* is the flux at *m_AB* of zero. The flux units are determined by this number. The default value was chosen such that Vega at 5556 Angstroms has an AB magnitude of 0.0336 and a ergs/cm²/s/Hz. This is the same value that was used by all previous versions of this task.

BLACKBODY CALIBRATION FILES

This type of file has the comment parameter "type" with a "blackbody". It must also include the "band" and "weff" comment parameters. If no "units" comment parameter is given then the default units are Angstroms.

The rest of the file consists of lines with wavelengths, m_{AB} of a magnitude star (in that band magnitude system), and the bandpass widths. The m_{AB} are defined as described previously. Normally all the m_{AB} values will be the same though it is possible to adjust them to produce a departure from a pure blackbody flux distribution.

The actual m_{AB} calibration magnitudes for the star are obtained by the relation

$$m_{AB}(\text{star}) = \text{mag} + m_{AB}(m=0) - 2.5 * \log_{10} (B(\text{weff}, \text{teff})/B(w, \text{teff}))$$

where m is the magnitude of the star in the calibration band, $m_{AB}(m=0)$ is the calibration value in the calibration file representing the magnitude of a $m=0$ star (basically the m_{AB} of Vega), weff is the effective wavelength for the calibration file, and teff is the effective temperature of the star. The function $B(w,T)$ is the blackbody function in f_{ν} that provides the shape of the calibration. Note how the normalization is such that at weff the last term is zero and $m_{AB}(\text{star}) = m + m_{AB}(m=0)$.

The $m_{AB}(\text{star})$ computed using the calibration values and the blackbody function are then in the same units and form as for the standard star files. The conversion to f_{ν} and the remaining processing proceeds in the same way as for standard star calibration data.

The parameters $\backslash \text{mag}$ and teff are specified by the user for each star as described in the section BLACKBODY PARAMETERS. These parameters are queried by the task for each star (unless forced to a value on the command line).

The beam switch mode is selected with the *beam_switch* parameter. This mode requires that all apertures are of the same star, the header keyword OFLAG be present to identify object and sky spectra, and that the sequence of spectra specified are paired such that if an object spectrum is encountered first the next spectrum for that aperture (spectra from other apertures may appear in between) is a or the reverse. These restrictions are not fundamental but are made so that this mode behaves the same as with the previous version of this task. The sky spectrum is subtracted from the object spectrum and the result is then used in generating the observed intensities in the calibration bandpasses.

If the spectra have been extinction corrected (EX-FLAG = extinction correction is removed. The specified extinction file is used for this operation and so must be the same as that used when the extinction correction was made. The airmass is also required in this step and, if needed to compute the airmass, the observatory specified in the image or observatory parameter is used. The treatment of extinction in this task is subtle. The aim of this task is to produce observed integrated instrumental intensities without extinction correction. Thus, the extinction correction is removed from extinction corrected spectra. However, a correction is made for an extinction gradient across the bandpasses. This is done by applying an extinction correction, integrating across the bandpass, and then correcting the integrated intensity for the extinction at the center of the bandpass. An alternative way to look at this is that the integral is weighted by the ratio of the extinction correction at each pixel to the extinction correction at the center of the bandpass. This correction or weighting is why the extinction file and latitude are parameters in this task even though for nonextinction corrected spectra they appear not to be needed.

The observed instrumental intensities are integrated within a bandpasses by summing the pixels using partial pixels at the bandpass edges. Initial bandpasses are defined in one of two ways. A set of evenly spaced bandpasses of constant width covering the range of the input spectrum may be specified using the parameters *bandwidth* and *bandsep* in the same units as the spectrum dispersion. If these parameters have the value INDEF then the bandpasses from the calibration file which are entirely within the spectrum are

selected. Generally these bandpasses are the actual measured bandpasses though one is free to make calibration files using estimated points. The calibration bandpasses are preferable because they have been directly measured and they have been placed to avoid troubles with spectral lines. However, when the coverage or resolution is such that these bandpasses do not allow a good determination of the instrumental response the evenly spaced bandpasses may be needed. The calibration fluxes are linearly interpolated (or extrapolated) from the calibration data points to the defined bandpasses.

Each spectrum adds a line to the output file containing the spectrum image name, the sky spectrum image name if beam switching, the aperture or beam number, the number of points in the spectrum, the exposure time, airmass, wavelength range, and title. If the airmass is not found in the image header it is computed using the latitude parameter and observation information from the header. If the airmass cannot be computed, due to missing keywords, then a query is made for the airmass.

Following the spectrum information, calibration data is added for each bandpass. The bandpass wavelength, absolute flux (per Angstrom), bandpass width, and observed instrumental intensity in the bandpass are added to the output file. As discussed above, the observed intensity does not include an extinction term but does apply a small correction or weighting for the variation of the extinction across the bandpass.

The setting and editing of the bandpasses may be performed interactively if the *interact* flag is set. In this case the user is queried for each spectrum. The answers to this query may be "no" or "yes" to skip editing or edit the bandpasses for this spectrum, "NO" or "YES" to skip or not skip editing all spectra of the same aperture with no further queries for this aperture, and "NO!" or "YES!" to skip editing or edit all spectra with no further queries.

When editing the bandpasses a graph of the spectrum is made with the bandpasses plotted at the computed intensity per pixel. The cursor and colon commands available are summarized in the section CURSOR KEYS. Basically bandpasses may be added or deleted and the current bandpass data may be examined. Additional keys allow the usual windowing and cursor mode operations. When satisfied with the bandpasses exit with 'q'. The edited bandpasses for that aperture remain in effect until changed again by the user. Thus if there are many spectra from the same aperture one may reply with "NO" to queries for the next spectra to accept the current bandpasses for all other spectra of the same aperture.

BLACKBODY PARAMETERS

When a blackbody calibration is selected (the calibration file selected by the *star_name* parameter has "# type blackbody") there are two quantities needed to scale the blackbody ~~the~~ observation. These are the magnitude of the star in the same band as the observation and the effective temperature. The magnitude is used for the flux scaling and the effective temperature for the shape of the flux distribution. The values are obtained or derived from the user specified parameters *mag*, *magband*, and *teff*. This section describes how the values are derived from other parameters using the data file "params.dat" in the calibration directory.

The effective temperature in degrees Kelvin may be specified directly or it may be derived from a spectral type for the star. In the latter case the file "params.dat" is searched for the effective temperature. The file consists of lines with the first value being the spectral type and the second the effective temperature. Other columns are described later. The spectral type can be any string without whitespace that matches what is in the file. However, the program finds the last spectral type that matches the first two characters when there is no complete match. This scheme is intended for the case where the spectral types are of the form A0I, A0III, or A0V, where A can be any spectral type letter OBAFGKM, the single digit subtype is between 0 and 9, and the luminosity class is one of I, III, or V. The two character match selects the last spectral type independent of the luminosity class. The standard file "onedstds\$blackbody/params.dat" uses these spectral type identifiers with the dwarf class acting as the default.

The magnitude band is specified along with the input magnitude. If band is the same as the calibration band given in the calibration file then no further transformation is required. However if the magnitude is

specified in a different band, a conversion is performed using information from the "params.dat" file based on the spectral type of the star.

When an effective temperature is specified rather than the nearest tabulated temperature for the spectral types that have "V" as the third character is used. For the standard spectral type designations this means that when an effective temperature is specified the dwarf spectral type is used for the magnitude transformation.

As mentioned previously, the "params.dat" data file has additional columns following the spectral type and effective temperature. These columns are relative magnitudes in various bands. The standard file has V magnitudes of zero so in this case the columns are also the X-V colors (where X is appropriate magnitude). Given the spectral type the relative magnitudes for the calibration band, m_1, and the input magnitude band, m_2, are found and the calibration magnitude for the star is given by

$$m_{\text{calibration}} = m_{\text{input}} + m_1 - m_2$$

If one of the magnitudes is missing, given as "INDEF" because the transformation is not available for the spectral type, the last spectral type matching the first two characters which does specify the two magnitudes will be used. For example if there is no information for a B3III star for a M-J color then the spectral type B3V might be used.

In order for the program to determine the bands for each column in the data file there must be a comment before the data with the column names. It must begin with "# Type Teff" and then be followed by the same band identifiers used in the blackbody calibration files and as specified by the *magband* parameter. Any amount whitespace (space or tab) is used to separate the various fields in the comment and in the fields of the table. For example the file might have the comment

```
# Type      Teff      V      J      H      K      L      Lprime      M
```

identifying the third column of the file as the V magnitude and the ninth file as the M magnitude.

EXAMPLES

1. To compile observations of three standard stars using a switched instrument like the IIDS:

```
cl> standard.recformat=yes
cl> standard nite1 1001-1008 std beam_switch+ interact-
[nite1.1001][0]: HZ 44 - Night 1
[nite1.1004][0]: HZ 44 - Night 1
[nite1.1005][0]: HZ 44 - Night 1
[nite1.1008][0]: HZ 44 - Night 1
Star name in calibration list: hz 44
cl> standard nite1 1009-1016 std beam_switch+ interact-
...
cl> standard nite1 1017-1024 std beam_switch+ interact-
...
```

This will create a file "std" which will contain sensitivity measurements from the beam-switched observations of the three standard stars given. Note that **standard** is run separately for each standard star.

The spectra will be from the images: nite1.1001, nite.1002 ... nite1.1024, and the default calibration file, "onedstds\$irscal.dat" will be used.

2. For echelle spectra all apertures, the orders, are of the same star and so the samestar parameter is set. Usually the resolution is much higher than the calibration data so in order to get sufficient coverage bandpasses must be interpolated from the calibration data. Therefore the evenly spaced bandpasses are used.

```
cl> standard.recformat=no
cl> standard.samestar=yes
```

```

cl> standard ech001.ec std bandwidth=10 bandsep=15
[ech001.ec][0]: Feige 110
Star name in calibration list: feige 110
[ech001.ec][0]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): yes
[ech001.ec][1]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): yes
[ech001.ec][2]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!): NO!

```

3. To use a blackbody infrared calibration where the V magnitude of the star is known.

```

cl> standard std1.ms std caldir=onedstds$blackbody/
std1.ms(1): Standard Star
Star name in calibration list: J
Magnitude of star: 10.3
Magnitude type (|V|J|H|K|L|Lprime|M|): V
Effective temperature or spectral type: B3III
WARNING: Effective temperature for B3III not found - using B3V
Blackbody: V = 10.30, J = 10.32, Teff = 19000
std1[1]: Edit bandpasses? (no|yes|NO|YES|NO!|YES!) (yes):

```

Note the warning message and the confirmation information.

REVISIONS

STANDARD V2.10.4

The calibration files can be defined to compute blackbody values.

STANDARD V2.10.3

A query for the airmass and exposure time is now made if the information is not in the header and cannot be computed from other header keywords.

STANDARD V2.10

Giving an unrecognized standard star name will page a stars available in the calibration directory and then repeat the query.

SEE ALSO

observatory, lcalib, sensfunc

NAME

subsets -- Description of CCD subsets

DESCRIPTION

The **ccdred** package groups observation into subsets. The image header parameter used to identify the subsets is defined in the instrument translation file (see help for **instruments**). For example to select subsets by the header parameter "filters" the instrument translation file would contain the line:

```
subset filters
```

Observations are generally grouped into subsets based on a instrument configuration such as a filter, aperture mask, grating setting, etc. This allows combining images from several different subsets automatically and applying the appropriate flat field image when processing the observations. For example if the subsets are by filter then **flatcombine** will search through all the images, find the flat field images (based on the CCD type parameter), and combine the flat field images from each filter separately. Then when processing the images the flat field with the same filter as the observation is used.

Each subset is assigned a short identifier. This is listed when using **ccdlist** and is appended to a root name when combining images. Because the subset parameter in the image header may be any string there must be a mapping applied to generate unique identifiers. This mapping is defined in the file given by the package parameter *ccdred.ssfile*. The file consists of lines with two fields (except that comment lines may be included as a line by itself or following the second field):

```
'subset string' subset_id
```

where the subset string is the image header string and the subset_id is the identifier. A field must be quoted if it contains blanks. The user may create this file but generally it is created by the tasks. The tasks use the first word of the subset string as the default identifier and a number is appended if the first word is not unique. The following steps define the subset identifier:

- (1) Search the subset file, if present, for a matching subset string and use the defined subset identifier.
- (2) If there is no matching subset string use the first word of the image header subset string and, if it is not unique, add successive integers until it is unique.
- (3) If the identifier is not in the subset file create the file and add an entry if necessary.

EXAMPLES

1. The subset file is "subsets" (the default). The subset parameter is translated to "f1pos" in the image header (the old NOAO CCD parameter) which is an integer filter position. After running a "ccdlist *.imh" to cause all filters to be checked, the subset file contains:

```
'2'      2
'5'      5
'3'      3
```

The order reflects the order in which the filters were encountered. Suppose the user wants to have more descriptive names then the subset file can be created or edited to the form:

```
# Sample translation file.
'2'      U
'3'      B
'4'      V
```

(This is only an example and does not mean these are standard filters.)

2. As another example suppose the image header parameter is "filter" and contains more descriptive strings. The subset file might become:

```
'GG 385 Filter' GG  
'GG 495 Filter' GG1  
'RG 610 Filter' RG  
'H-ALPHA'      H_ALPHA
```

In this case use of the first word was not very good but it is better if the filters are encoded with the thought that the first word will be used by **ccdred**; it should be short and unique.

SEE ALSO

instruments

NAME

zerocombine -- Combine and process zero level images

USAGE

zerocombine input

PARAMETERS**input**

List of zero level images to combine. The *ccdtype* parameter may be used to select the zero level images from a list containing all types of data.

output = "Zero"

Output zero level root image name.

combine = "average" (average|median)

Type of combining operation performed on the final set of pixels (after rejection). The choices are "average" or "median". The median uses the average of the two central values when the number of pixels is even.

reject = "minmax" (none|minmax|ccdclip|crreject|sigclip|avsigclip|pclip)

Type of rejection operation. See **combine** for details.

ccdtype = "zero"

CCD image type to combine. If no image type is given then all input images are combined.

process = no

Process the input images before combining?

delete = no

Delete input images after combining? Only those images combined are deleted.

clobber = no

Clobber existing output images?

scale = "none" (none|mode|median|mean|exposure)

Multiplicative image scaling to be applied. The choices are none, scale by the mode, median, or mean of the specified statistics section, or scale by the exposure time given in the image header.

statsec = ""

Section of images to use in computing image statistics for scaling. If no section is given then the entire region of the image is sampled (for efficiency the images are sampled if they are big enough).

Algorithm Parameters**nlow = 0, nhigh = 1 (minmax)**

The number of low and high pixels to be rejected by the "minmax" algorithm.

nkeep = 1

The minimum number of pixels to retain or the maximum number to reject when using the clipping algorithms (ccdclip, crreject, sigclip, avsigclip, or pclip). When given as a positive value this is the minimum number to keep. When given as a negative value the absolute value is the maximum

number to reject. This is actually converted to a number to keep by adding it to the number of images.

mclip = yes (ccdclip, crreject, sigclip, avsigclip)

Use the median as the estimate for the true intensity rather than the average with high and low values excluded in the "ccdclip", "crreject", "sigclip", and "avsigclip" algorithms? The median is a better estimator in the presence of data which one wants to reject than the average. However, computing the median is slower than the average.

lsigma = 3., hsigma = 3. (ccdclip, crreject, sigclip, avsigclip, pclip)

Low and high sigma clipping factors for the "ccdclip", "crreject", "sigclip", "avsigclip", and "pclip" algorithms. They multiply a "sigma" factor produced by the algorithm to select a point below and above the average or median value for rejecting pixels. The lower sigma is ignored for the "crreject" algorithm.

rdnoise = "0.", gain = "1.", snoise = "0." (ccdclip, crreject)

CCD readout noise in electrons, gain in electrons/DN, and sensitivity noise as a fraction. These parameters are used with the "ccdclip" and "crreject" algorithms. The values may be either numeric or an image header keyword which contains the value.

pclip = -0.5 (pclip)

Percentile clipping algorithm parameter. If greater than one in absolute value then it specifies a number of pixels above or below the median to use for computing the clipping sigma. If less than one in absolute value then it specifies the fraction of the pixels above or below the median to use. A positive value selects a point above the median and a negative value selects a point below the median. The default of -0.5 selects approximately the quartile point. See **combine** for further details.

blank = 0.

Output value to be used when there are no pixels.

DESCRIPTION

The zero level images in the input image list are combined. The input images may be processed first if desired. The original images may be deleted automatically if desired. The output pixel datatype will be real.

This task is a script which applies **ccdproc** and **combine**. The parameters and combining algorithms are described in detail in the help for **combine**. This script has default parameters specifically set for zero level images and simplifies the combining parameters. There are other combining options not included in this task. For these additional features, such as thresholding, offsetting, masking, and projecting, use **combine**.

EXAMPLES

1. The image data contains four zero level images. To automatically select them and combine them as a using the default combining algorithm:

```
cl> zerocombine ccd*.imh&
```

SEE ALSO

ccdproc, combine